

Panduan Pengguna Panel 3X-UI

 العربية ·  English ·  Español ·  فارسی ·  Bahasa Indonesia ·  日本語 ·  Português ·  Русский · 
Türkçe ·  Українська ·  Tiếng Việt ·  简体中文 ·  繁體中文

Versi 3X-UI: 3.4.2. Panduan ini disusun berdasarkan versi tersebut dan berlaku untuk versi ini. Ringkasan perubahan 3.4.2 dibandingkan 3.4.1 tersedia di bagian [«Apa yang Baru di 3.4.2»](#).

Panduan lengkap berbahasa Indonesia untuk panel web **3X-UI** (pengelolaan Xray-core): fitur, konfigurasi, dan pengoperasian, dengan penjelasan setiap field dan pengaturan di antarmuka.

Nama dan label sesuai dengan antarmuka panel. Kata *inbound* / *outbound* tidak diterjemahkan.

Daftar Isi

- Apa yang Baru di 3.4.2
- 1. Pendahuluan, Persyaratan, dan Instalasi
 - 1.1. Apa Itu 3X-UI
 - 1.2. Sistem Operasi dan Arsitektur yang Didukung
 - 1.3. Metode Instalasi
 - 1.4. Pengaktifan Pertama dan Kredensial Default
 - 1.5. Lokasi File
 - 1.6. Perintah Manajemen `x-ui` (Menu Skrip)
 - 1.7. Subperintah `x-ui` (Tanpa Menu Interaktif)
 - 1.8. Migrasi SQLite → PostgreSQL
- 2. Login Panel dan Keamanan Akses
 - 2.1. Formulir Login
 - 2.2. Autentikasi Dua Faktor (2FA / TOTP)
 - 2.3. Pembatasan Percobaan Login (login limiter / perlindungan brute-force)
 - 2.4. Penggantian Username dan Password Administrator
 - 2.5. Jalur Rahasia (URI-path / webBasePath) dan Port Panel
 - 2.6. Masa Aktif Sesi (timeout)
 - 2.7. LDAP (Sinkronisasi dan Autentikasi)
- 3. Ikhtisar / Dasbor
 - 3.1. Prinsip Umum Pengumpulan Data
 - 3.2. CPU (CPU)
 - 3.3. Memori (RAM)
 - 3.4. Swap (Swap)
 - 3.5. Disk (Storage)
 - 3.6. Uptime Sistem (Uptime)
 - 3.7. Load Average Sistem (Load average)
 - 3.8. Jaringan: Kecepatan dan Total Volume Trafik
 - 3.9. Alamat IP Server
 - 3.10. Koneksi TCP/UDP
 - 3.11. Status Xray dan Manajemen Proses
 - 3.12. Pembaruan Panel (3X-UI)
 - 3.13. Pembaruan File Geo (GeoIP / GeoSite)
 - 3.14. Pencadangan dan Pemulihan Basis Data
 - 3.15. Elemen Antarmuka Tambahan
- 4. Inbounds: pembuatan dan parameter umum
 - 4.1. Field formulir umum
 - 4.2. Sniffing (Sniffing)
 - 4.3. Allocate (strategi alokasi port)
 - 4.4. External Proxy (Proksi eksternal)
 - 4.5. Fallbacks (Fallback)
 - 4.6. Reset lalu lintas berkala
 - 4.7. JSON inbound (lanjutan)
 - 4.8. Tindakan pada inbound: QR / Edit / Reset / Delete dan statistik

- 5. Protokol
 - 5.1. Daftar Protokol yang Didukung
 - 5.2. Protokol Mana yang Mendukung TLS / REALITY / Transport
 - 5.3. VLESS
 - 5.4. VMess
 - 5.5. Trojan
 - 5.6. Shadowsocks
 - 5.7. Dokodemo-door / Tunnel (forwarder transparan)
 - 5.8. SOCKS / HTTP (protokol `mixed`)
 - 5.9. WireGuard (inbound)
 - 5.10. Hysteria (default v2)
 - 5.11. MTPROTO (proksi untuk Telegram)
 - 5.12. Panduan singkat pemilihan protokol
- 6. Transport (Stream Settings)
 - 6.1. Pemilihan Jaringan Transmisi
 - 6.2. RAW / TCP (`tcpSettings`)
 - 6.3. mKCP (`kcpSettings`)
 - 6.4. WebSocket (`wsSettings`)
 - 6.5. gRPC (`grpcSettings`)
 - 6.6. HTTPUpgrade (`httpupgradeSettings`)
 - 6.7. XHTTP / SplitHTTP (`xhttpSettings`)
 - 6.8. Transport Hysteria (`hysteriaSettings`)
 - 6.9. Parameter Pelengkap
- 7. Keamanan Koneksi: TLS, XTLS, dan REALITY
 - 7.1. Apa Perbedaannya: TLS vs XTLS vs REALITY
 - 7.2. Mode «Tidak Ada» (`none`)
 - 7.3. Mode TLS
 - 7.4. Mode REALITY
 - 7.5. Rekomendasi Praktis untuk Konfigurasi
- 8. Klien
 - 8.1. Kolom Klien
 - 8.2. Tautan ke Inbound
 - 8.3. Operasi pada Klien
 - 8.4. Operasi Massal
 - 8.5. Pencarian, Filter, dan Pengurutan
 - 8.6. Ikon dan Status
- 9. Grup Klien
 - 9.1. Apa itu grup klien dan untuk apa
 - 9.2. Hubungan grup dengan klien, inbound, node, dan protokol
 - 9.3. Daftar referensi grup dan grup "kosong"
 - 9.4. Kolom dan bidang grup
 - 9.5. Membuat grup
 - 9.6. Mengganti nama grup
 - 9.7. Menambahkan klien ke grup
 - 9.8. Menghapus klien dari grup (tanpa menghapus klien itu sendiri)
 - 9.9. Mereset traffic grup

- 9.10. Menghapus grup dan menghapus klien grup
- 9.11. Hubungan dengan halaman "Klien"
- 9.12. Ringkasan endpoint API
- 9.13. Traffic per grup
- 10. Langganan (Subscription)
 - 10.1. Apa itu subld dan bagaimana tautan dibentuk
 - 10.2. Pengaturan server langganan
 - 10.3. Format output
 - 10.4. Halaman informasi langganan dan kode QR
 - 10.5. Template halaman langganan kustom
- 11. Xray: routing, outbounds, DNS, dan ekstensi
 - 11.1. Struktur editor: tab/mode
 - 11.2. Pengaturan Utama (General)
 - 11.3. Aturan routing (routing)
 - 11.4. Outbounds (koneksi keluar)
 - 11.5. Balancer (Balancers)
 - 11.6. DNS
 - 11.7. Fake DNS
 - 11.8. WireGuard / WARP / NordVPN
 - 11.9. Reverse-proxy dan TUN
 - 11.10. Log dan statistik (Stats, metrics)
 - 11.11. Penyimpanan, mulai ulang, dan transformasi otomatis
 - 11.12. Outbound dari langganan (dengan pembaruan otomatis)
 - 11.13. Rotasi IP di WARP
- 12. Node (multipanel, master/slave)
 - 12.1. Ringkasan di bagian atas daftar
 - 12.2. Menambah dan mengedit node
 - 12.3. Verifikasi TLS (untuk node https)
 - 12.4. Informasi yang ditampilkan untuk setiap node
 - 12.5. Tindakan pada node
 - 12.6. Riwayat metrik
 - 12.7. Cara inbound dan klien disinkronkan
 - 12.8. Rantai node (sub-node / node transitif)
 - 12.9. Node: hal baru di 3.3.0
- 13. Pengaturan Panel
 - 13.1. Menyimpan dan Me-restart Panel
 - 13.2. Pengaturan Umum (tab "Panel" / *General*)
 - 13.3. Akses ke Panel: IP, Port, Jalur, Domain, Sertifikat
 - 13.4. Sesi, Proxy Panel, dan Proxy Tepercaya (tab "Proxy dan Server" / *Proxy and Server*)
 - 13.5. Bot Telegram (tab "Bot Telegram" / *Telegram Bot*)
 - 13.6. Tanggal dan Waktu (tab "Tanggal dan Waktu" / *Date and Time*)
 - 13.7. Trafik Eksternal dan Perilaku Xray (tab "Trafik Eksternal" / *External Traffic*)
 - 13.8. Lainnya: Template Konfigurasi Xray dan URL Pengujian
 - 13.9. Akun Administrator dan Token API
 - 13.10. Perubahan API di 3.3.0 (penting untuk integrasi)

- 14. Bot Telegram
 - 14.1. Mengaktifkan dan mengonfigurasi bot
 - 14.2. Menu utama dan tombol
 - 14.3. Perintah bot
 - 14.4. Manajemen klien (hanya administrator)
 - 14.5. Notifikasi dan laporan
 - 14.6. Backup dan log
 - 14.7. Fitur operasional
- 15. Basis Geo (geoip / geosite dan kustom)
 - 15.1. Apa itu geoip.dat dan geosite.dat
 - 15.2. File geo standar dan pembaruannya
 - 15.3. Pembaruan otomatis data geo melalui Xray (Geodata Auto-Update)
 - 15.4. Validasi dan batasan
 - 15.5. Pemeriksaan otomatis saat panel dijalankan
 - 15.6. Penggunaan basis geo dalam aturan perutean
- 16. Operasional: backup, log, pembaruan, CLI
 - 16.1. Pencadangan dan pemulihan database
 - 16.2. Melihat log
 - 16.3. Level dan pengaturan logging Xray
 - 16.4. Mengelola Xray: menghentikan dan me-restart
 - 16.5. Me-restart dan memperbaiki panel
 - 16.6. Tugas berkala (cron)
 - 16.7. Menu konsol dan CLI (`x-ui`)
 - 16.8. Menghapus panel
 - 16.9. Perintah `x-ui migrateDB`

Apa yang Baru di 3.4.2

Versi 3.4.2 adalah pembaruan besar: WireGuard dialihkan ke model multi-klien, REALITY mendapat pemindai target langsung, balancer memperoleh tab Observatory/Burst Observatory, dan ditambahkan konfirmasi pengaturan sensitif dengan kode 2FA. Berikut adalah perubahan dibandingkan 3.4.1, dikelompokkan berdasarkan bagian panduan.

Perubahan di bagian 1 – Pendahuluan, Persyaratan, dan Instalasi

- Di menu samping (dan di laci geser pada perangkat seluler) muncul tombol **«Dokumentasi»** (ikon buku) – membuka dokumentasi resmi <https://docs.sanaei.dev/>.
- Versi Xray minimum yang dapat diperbarui oleh panel dinaikkan menjadi **26.6.27** (inti Xray 26.6.27 sudah disertakan).

Perubahan di bagian 2 – Login Panel dan Keamanan Akses

- Saat 2FA diaktifkan, mengganti login/password administrator dan menonaktifkan 2FA kini memerlukan **memasukkan kode saat ini** dari aplikasi autentikator (konfirmasi perubahan sensitif).
- LDAP: sakelar baru **«Lewati verifikasi sertifikat TLS»** (`ldapInsecureSkipVerify`) – menonaktifkan verifikasi sertifikat saat LDAPS; hanya tersedia jika «Gunakan TLS (LDAPS)» diaktifkan.

Perubahan di bagian 3 – Ikhtisar / Dashboard

- Tombol versi panel kini selalu membuka jendela pembaruan (lihat bagian 16 – saluran dev).
- Peningkatan **aksesibilitas** secara menyeluruh: label aria untuk ikon dan aktivasi elemen dengan Enter/Space (untuk pembaca layar dan navigasi keyboard).

Perubahan di bagian 4 – Inbounds: pembuatan dan parameter umum

- Tindakan **«Ekspor semua tautan»** kini membentuk tautan melalui mesin langganan – menerapkan template remark ke setiap klien dan memilih endpoint Host terkelola (sebelumnya remark tetap `inbound-email`).

Perubahan di bagian 5 – Protokol

- **WireGuard dialihkan ke model multi-klien.** Peer kini menjadi klien biasa (dengan penetapan alamat tunnel otomatis, dukungan langganan, batas lalu lintas/masa berlaku, dan grup); daftar inline «Peer» dari formulir inbound dihapus.
- Pada inbound WireGuard ditambahkan field **DNS** yang dapat dikonfigurasi (default `1.1.1.1, 1.0.0.1`) dan **kartu konfigurasi klien** – salin/unduh/QR `.conf` lengkap serta tautan `wireguard:// / wg://`.

Perubahan di bagian 6 – Transport (Stream Settings)

- Pada XHTTP untuk inbound baru, parameter `maxConnections` di **xmux** kini default-nya **6** (sebelumnya **0** – tanpa batas). Inbound yang sudah ada mempertahankan nilainya.

Perubahan di bagian 7 – Keamanan Koneksi: TLS, XTLS, dan REALITY

- Ditambahkan **pemindai target REALITY langsung**: tombol «**Pindai**» (memeriksa target saat ini «secara langsung») dan «**Cari target**» (memindai domain atau rentang **IP/CIDR** dan memilih target yang cocok berdasarkan sertifikatnya). Field «Target» dan SNI kini kosong saat REALITY pertama kali dipilih.

Perubahan di bagian 8 – Klien

- Perpanjangan masa berlaku/kuota melalui `bulkAdjust` kini **secara otomatis mengaktifkan** klien yang dinonaktifkan hanya karena habis (masa berlaku berakhir atau kuota terlampaui), jika perpanjangan mengembalikannya ke dalam batas. Klien yang dinonaktifkan secara manual atau yang masih habis tetap nonaktif.

Perubahan di bagian 9 – Grup Klien

- «**Reset Traffic**» pada grup kini hanya menol-kan **penghitung grup itu sendiri**; penghitung, kuota, dan status masing-masing klien tidak terpengaruh, dan restart Xray tidak diperlukan. Ini adalah perubahan dari perilaku sebelumnya (sebelumnya lalu lintas semua klien grup direset).

Perubahan di bagian 10 – Langganan (Subscription)

- Pada **host terkelola**, field **VLESS route** didefinisikan ulang: kini berupa satu nilai `0-65535` (bukan daftar port), dan nilai itu benar-benar «ditanamkan» ke UUID setiap langganan (raw/JSON/Clash).
- Variabel `{{EMAIL}}` (dan sinonimnya `{{USERNAME}}`) di template remark kini hanya ditampilkan pada **tautan pertama** klien – sama seperti blok lalu lintas/masa berlaku.

Perubahan di bagian 11 – Xray: Routing, Outbounds, DNS, dan Ekstensi

- **Balancer**: halaman dibagi menjadi tab «**Pengaturan balancer**» dan «**Observatory**»; alih-alih JSON mentah – formulir Observatory dan Burst Observatory (pada Burst ditambahkan field «**Metode HTTP**»). Balancer Random/Round-robin dengan `fallbackTag` kini otomatis membuat Burst Observatory.
- Saat menghapus outbound atau balancer, panel secara otomatis membersihkan tautan terkait di routing dan menampilkan **pratinjau dampak** di dialog konfirmasi.
- Dalam aturan routing, kriteria jaringan **L4** ditulis ke config dalam huruf kecil (`tcp / udp`), tetapi ditampilkan dalam huruf kapital di tabel.
- Kesalahan pada formulir penambahan/pengeditan balancer kini ditangguhkan hingga field pertama disentuh atau penyimpanan dicoba.

Perubahan di bagian 12 – Node (multipanel, master/slave)

- Notifikasi «disimpan secara lokal, node offline – akan disinkronkan nanti» kini hanya ditampilkan ketika node benar-benar offline atau dimatikan (sebelumnya – pada setiap penyimpanan ke node online).

Perubahan di bagian 16 – Operasional: Cadangan, Log, Pembaruan, CLI

- Nama file cadangan kini menyertakan alamat server dan **tanggal-waktu**: `{host}_YYYY-MM-DD_HHMMSS.db` (`.dump` untuk PostgreSQL), misalnya `panel.example.com_2026-06-27_000000.db` – baik saat diunduh dari panel maupun pada cadangan yang dikirim bot Telegram.
- Saluran **dev** pembaruan kini dapat diaktifkan dari build stabil: tombol versi selalu membuka jendela pembaruan, dan muncul sakelar «**Saluran Dev**» dengan peringatan tentang ketidakstabilan dan tidak adanya rollback otomatis.

1. Pendahuluan, Persyaratan, dan Instalasi

1.1. Apa Itu 3X-UI

3X-UI adalah panel manajemen web sumber terbuka untuk server [Xray-core](#). Panel ini menyediakan antarmuka web multibahasa terpadu untuk penerapan, konfigurasi, dan pemantauan berbagai protokol proksi dan VPN: mulai dari satu VPS tunggal hingga konfigurasi terdistribusi dengan beberapa node.

3X-UI adalah fork lanjutan dari proyek X-UI asli. Dibandingkan dengan pendahulunya, panel ini menambahkan dukungan lebih banyak protokol, stabilitas yang ditingkatkan, pencatatan lalu lintas per klien, dan berbagai fitur yang berguna.

Fitur-fitur utama:

- **Inbound berbagai protokol** – VLESS, VMess, Trojan, Shadowsocks, WireGuard, Hysteria2, HTTP, SOCKS (Mixed), Dokodemo-door / Tunnel, TUN, dan **MTPProto** (proksi Telegram, ditambahkan di versi 3.3.0).
- **Transportasi dan enkripsi modern** – TCP (Raw), mKCP, WebSocket, gRPC, HTTPUpgrade, dan XHTTP, dilindungi oleh TLS, XTLS, dan REALITY.
- **Fallback** – melayani beberapa protokol pada satu port (misalnya VLESS dan Trojan pada port 443) menggunakan mekanisme fallback di Xray.
- **Manajemen per klien** – kuota lalu lintas, tanggal kedaluwarsa, batas IP, tampilan status "online", tautan undangan satu klik, kode QR, dan langganan.
- **Statistik lalu lintas** – per inbound, klien, dan outbound, dengan kemampuan reset.
- **Dukungan multi-node** – manajemen dan penskalaan ke beberapa server dari satu panel.
- **Outbound dan routing** – WARP, NordVPN, aturan routing kustom, load balancer, rantai proksi.
- **Server langganan bawaan** dengan beberapa format output.
- **Bot Telegram** untuk pemantauan dan manajemen jarak jauh.
- **REST API** dengan dokumentasi Swagger bawaan.
- **Penyimpanan fleksibel** – SQLite (default) atau PostgreSQL.
- **13 bahasa antarmuka**, tema gelap dan terang.
- **Integrasi Fail2ban** untuk menerapkan batas IP per klien.

Penting: proyek ini ditujukan hanya untuk penggunaan pribadi. Tidak disarankan untuk menggunakannya untuk tujuan ilegal atau dalam lingkungan produksi.

1.2. Sistem Operasi dan Arsitektur yang Didukung

Sistem Operasi

Skrip instalasi mendeteksi distribusi berdasarkan bidang `ID` dari `/etc/os-release` (atau `/usr/lib/os-release`). Sistem yang resmi didukung:

Ubuntu, Debian, Armbian, Fedora, CentOS, RHEL, AlmaLinux, Rocky Linux, Oracle Linux, Amazon Linux, Virtuozzo, Arch, Manjaro, Parch, openSUSE (Tumbleweed / Leap), Alpine, serta Windows.

Pada sistem berbasis Alpine, layanan OpenRC (`rc-service` / `rc-update`) digunakan; pada sistem lainnya digunakan systemd. Untuk CentOS 7, paket dipasang melalui `yum` ; untuk rilis yang lebih baru digunakan `dnf` . Jika distribusi tidak dikenali, skrip akan mencoba menggunakan manajer paket `apt-get` secara default.

Arsitektur Prosesor

Arsitektur dideteksi dari output `uname -m` dan dipetakan ke salah satu nilai yang didukung:

Nilai <code>uname -m</code>	Arsitektur 3X-UI
<code>x86_64</code> , <code>x64</code> , <code>amd64</code>	<code>amd64</code>
<code>i*86</code> , <code>x86</code>	<code>386</code>
<code>armv8*</code> , <code>arm64</code> , <code>aarch64</code>	<code>arm64</code>
<code>armv7*</code> , <code>arm</code>	<code>armv7</code>
<code>armv6*</code>	<code>armv6</code>
<code>armv5*</code>	<code>armv5</code>
<code>s390x</code>	<code>s390x</code>

Jika arsitektur tidak ada dalam daftar ini, skrip akan menampilkan pesan "Unsupported CPU architecture!" dan menghentikan instalasi.

Dependensi Dasar

Sebelum memasang panel, skrip secara otomatis memasang sekumpulan paket dasar (nama bervariasi tergantung distribusi): `cron` / `cronie` / `dcron` , `curl` , `tar` , `tzdata` / `timezone` , `socat` , `ca-certificates` , `openssl` .

1.3. Metode Instalasi

Metode 1. Skrip Instalasi (Direkomendasikan)

Instalasi dilakukan dengan satu perintah sebagai root:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

Skrip ini wajib dijalankan dengan hak root: jika dijalankan bukan sebagai root, akan muncul pesan "Please run this script with root privilege" dan proses akan berhenti dengan error.

Yang dilakukan penginstal secara bertahap:

1. Mendeteksi OS dan arsitektur.
2. Memasang dependensi dasar.
3. Mengunduh arsip rilis `x-ui-linux-<arch>.tar.gz` dan mengekstraknya ke direktori `/usr/local/x-ui` .
4. Mengunduh skrip manajemen `x-ui.sh` dan memasangnya sebagai perintah `/usr/bin/x-ui` .
5. Membuat direktori log `/var/log/x-ui` .

6. Menjalankan konfigurasi awal: pemilihan database, pembuatan kredensial, pemilihan port, konfigurasi SSL opsional.
7. Memasang dan memulai layanan autostart (unit systemd `x-ui.service` atau skrip init OpenRC untuk Alpine).

Pemilihan database saat instalasi. Penginstal menawarkan:

- 1) SQLite (default, direkomendasikan jika jumlah klien < 500) – satu file `/etc/x-ui/x-ui.db`, tidak memerlukan konfigurasi.
- 2) PostgreSQL (direkomendasikan untuk jumlah klien yang besar atau beberapa node). PostgreSQL dapat dipasang secara lokal (pengguna dan database khusus bernama `xui` akan dibuat) atau Anda dapat menentukan DSN ke server yang sudah ada. Parameter koneksi ditulis ke file lingkungan layanan (`/etc/default/x-ui`, `/etc/conf.d/x-ui`, atau `/etc/sysconfig/x-ui` tergantung distribusi) dalam bentuk variabel `XUI_DB_TYPE` dan `XUI_DB_DSN`.

Contoh: penulisan parameter PostgreSQL ke file lingkungan layanan. Setelah memilih PostgreSQL dan menentukan DSN, penginstal akan menambahkan baris seperti ini ke file lingkungan:

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

Di sini `xui` adalah nama pengguna dan database, `127.0.0.1:5432` adalah alamat dan port server, `sslmode=disable` cocok untuk koneksi lokal (untuk server jarak jauh biasanya digunakan `require`).

Instalasi versi tertentu (lama). Anda dapat menentukan tag versi secara eksplisit – penginstal akan mengunduh rilis yang sesuai:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

Versi minimum yang diizinkan untuk instalasi semacam ini adalah `v2.3.5`; jika versi yang lebih lama ditentukan, akan muncul pesan "Please use a newer version (at least v2.3.5)".

Instalasi build dev. Selain tag versi, penginstal juga menerima argumen `dev-latest` (alias `dev`) – ini memasang rolling dev build berdasarkan commit terbaru dari branch `main`:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh) dev-
latest
```

Build dev adalah pre-release per-commit (tag `dev-latest`), bukan versi stabil, sehingga pemeriksaan versi minimum tidak dilakukan untuknya. Saat dijalankan, akan muncul peringatan "Installing the rolling dev build (tag: dev-latest). This is a per-commit pre-release, not a stable version.". Tanpa argumen, penginstal memasang rilis stabil terbaru. Gunakan build dev hanya untuk menguji perbaikan yang belum dirilis; dalam penggunaan normal, pasanglah versi stabil.

Metode 2. Docker

Menjalankan dengan database SQLite default:

```
docker compose up -d
```

Untuk menjalankan dengan layanan PostgreSQL bawaan, uncomment baris `XUI_DB_*` di `docker-compose.yml` dan jalankan dengan profil:

```
docker compose --profile postgres up -d
```

Image menyertakan Fail2ban (aktif secara default) untuk menerapkan batas IP per klien. Fail2ban memblokir pelanggar melalui `iptables`, yang memerlukan kapabilitas `NET_ADMIN`. Kapabilitas ini sudah disediakan di `docker-compose.yml` melalui `cap_add`. Saat menjalankan secara manual melalui `docker run`, kapabilitas perlu ditambahkan secara manual, jika tidak, pemblokiran hanya akan dicatat di log tetapi tidak diterapkan:

Contoh: perintah `docker run` lengkap. Varian minimal dengan meneruskan port panel, kapabilitas jaringan, dan volume persisten untuk database:

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

Volume `/etc/x-ui` menyimpan file `x-ui.db` di antara restart container; tanpanya, pengaturan dan akun akan hilang.

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

Dalam Docker, panel adalah proses utama container: autostart diatur oleh kebijakan restart container (misalnya `restart: unless-stopped`), bukan oleh layanan di dalam container.

1.4. Pengaktifan Pertama dan Kredensial Default

Pada instalasi pertama (ketika kredensial default masih digunakan), penginstal **menghasilkan nilai acak** untuk nama pengguna, kata sandi, dan jalur web, serta port:

Parameter	Cara Dibentuk Saat Instalasi	Catatan
Nama pengguna (Username)	string acak 10 karakter	dibuat secara otomatis
Kata sandi (Password)	string acak 10 karakter	dibuat secara otomatis
Jalur web panel (WebBasePath)	string acak 18 karakter	melindungi panel dari deteksi melalui URL root
Port panel (Port)	secara default port acak dalam rentang 1024–62000; dapat ditentukan secara manual jika diinginkan	nilai "pabrik" <code>webPort</code> adalah <code>2053</code> , tetapi penginstal akan menyimpannya

Di akhir instalasi, skrip menampilkan ringkasan: nama pengguna, kata sandi, port, jalur web, token API, dan tautan akses siap pakai (Access URL) dalam format:

```
https://<domain-atau-IP>:<port>/<jalur-web>
```

Jika sertifikat SSL belum dikonfigurasi, tautan akan menggunakan `http://`, dan skrip akan menampilkan peringatan tentang perlunya mengkonfigurasi SSL (item menu 19).

Wajib mengganti kredensial. Karena login dan kata sandi dibuat secara acak, **simpan segera setelah instalasi**. Anda dapat menggantinya kapan saja melalui item menu "Reset Username & Password" (lihat di bawah) atau dari antarmuka web di pengaturan panel. Setelah reset, skrip mengingatkan: "Please use the new login username and password to access the X-UI panel. Also remember them!".

Setelah instalasi, gunakan perintah `x-ui` untuk membuka menu manajemen (lihat bagian 1.6).

1.5. Lokasi File

Jalur	Fungsi
<code>/usr/local/x-ui/</code>	direktori instalasi panel (biner <code>x-ui</code> , skrip <code>x-ui.sh</code>)
<code>/usr/local/x-ui/bin/xray-linux-<arch></code>	biner Xray-core (pada armv5/armv6/armv7 diganti namanya menjadi <code>xray-linux-arm</code>)
<code>/usr/bin/x-ui</code>	skrip manajemen (perintah <code>x-ui</code>)
<code>/etc/x-ui/x-ui.db</code>	file database SQLite (default)
<code>/var/log/x-ui/</code>	direktori log panel
<code>/etc/systemd/system/x-ui.service</code>	unit systemd layanan (bukan untuk Alpine)
<code>/etc/init.d/x-ui</code>	skrip init OpenRC (hanya Alpine)
<code>/etc/default/x-ui</code> · <code>/etc/conf.d/x-ui</code> · <code>/etc/sysconfig/x-ui</code>	file variabel lingkungan layanan (jalur tergantung distribusi); tempat <code>XUI_DB_TYPE</code> / <code>XUI_DB_DSN</code> ditulis

Direktori database dapat diganti dengan variabel lingkungan `XUI_DB_FOLDER` (default `/etc/x-ui`), dan direktori biner Xray dengan variabel `XUI_BIN_FOLDER` (default `bin` relatif terhadap direktori panel). Nama file database adalah `x-ui.db`.

Contoh: memindahkan database ke disk terpisah. Untuk menyimpan `x-ui.db` bukan di `/etc/x-ui`, melainkan misalnya di disk yang di-mount `/data`, tentukan variabel di file lingkungan layanan dan restart panel:

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

Jalur lengkap ke database akan menjadi `/data/x-ui/x-ui.db`.

Variabel Lingkungan Utama

Variabel	Fungsi	Default
XUI_DB_TYPE	backend database: <code>sqlite</code> atau <code>postgres</code>	<code>sqlite</code>
XUI_DB_DSN	string koneksi PostgreSQL (saat <code>XUI_DB_TYPE=postgres</code>)	–
XUI_DB_FOLDER	direktori file database SQLite	<code>/etc/x-ui</code>
XUI_INIT_WEB_BASE_PATH	jalur URI awal panel web (hanya saat inisialisasi pertama)	<code>/</code>
XUI_DB_MAX_OPEN_CONNS	maksimum koneksi terbuka (pool PostgreSQL)	–
XUI_DB_MAX_IDLE_CONNS	maksimum koneksi idle (pool PostgreSQL)	–
XUI_ENABLE_FAIL2BAN	aktifkan penerapan batas IP melalui Fail2ban	<code>true</code>
XUI_LOG_LEVEL	level logging (<code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code>)	<code>info</code>
XUI_DEBUG	mode debug	<code>false</code>

Contoh: mengaktifkan logging terperinci sementara. Untuk mendiagnosis masalah, naikan level log ke `debug` dan restart layanan:

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log      # melihat log debug
```

Setelah diagnosis, kembalikan ke nilai `info` agar log tidak membengkak.

Jalur awal panel web melalui lingkungan. Variabel `XUI_INIT_WEB_BASE_PATH` menetapkan jalur URI panel web (`webBasePath`) saat inisialisasi pengaturan pertama. Ini berguna saat penerapan di Docker atau melalui `systemd` untuk langsung menetapkan jalur masuk ke panel. Nilai dinormalisasi secara otomatis – garis miring awal dan akhir ditambahkan jika diperlukan, dan nilai kosong atau yang hanya terdiri dari spasi diabaikan (dalam hal ini jalur default `/` diterapkan). Variabel ini hanya memengaruhi **inisialisasi pertama**: jika pengaturan sudah ada, jalur diubah melalui antarmuka web atau item menu "Reset Web Base Path".

1.6. Perintah Manajemen `x-ui` (Menu Skrip)

Setelah instalasi, perintah `x-ui` (dijalankan sebagai root) membuka menu interaktif "3X-UI Panel Management Script". Pilihan item dilakukan dengan memasukkan nomornya (rentang 0–27). Banyak item juga tersedia sebagai subperintah untuk digunakan dalam skrip (lihat bagian 1.7).

Menu dibagi menjadi blok-blok tematik.

Instalasi dan Pembaruan

- **1. Install** – memasang panel (menjalankan `install.sh`). Sebelum instalasi, akan diperiksa apakah panel belum terpasang.
- **2. Update** – memperbarui semua komponen `x-ui` ke versi terbaru. Data tidak akan hilang; setelah pembaruan, panel akan restart secara otomatis. Memerlukan konfirmasi.

- **3. Update Menu** – memperbarui hanya skrip manajemen (`x-ui.sh` / perintah `x-ui`) ke versi terbaru tanpa memasang ulang panel.
- **4. Legacy Version** – memasang versi panel tertentu (lama). Skrip akan meminta nomor versi (misalnya `2.4.0`) dan mengunduh rilis yang sesuai.
- **5. Uninstall** – menghapus panel sepenuhnya **beserta Xray**. Layanan dihentikan dan dinonaktifkan, direktori `/etc/x-ui/` dan `/usr/local/x-ui/` , file lingkungan layanan, serta skrip manajemen dihapus. Memerlukan konfirmasi (default "tidak").

Kredensial dan Pengaturan

- **6. Reset Username & Password** – reset nama pengguna dan kata sandi panel. Anda dapat memasukkan nilai Anda sendiri atau membiarkan kosong untuk pembuatan acak (nama acak – 10 karakter, kata sandi acak – 18 karakter). Anda juga akan ditawarkan untuk menonaktifkan autentikasi dua faktor (2FA) jika sudah dikonfigurasi. Setelah reset, panel akan restart.
- **7. Reset Web Base Path** – reset jalur web panel: jalur acak baru (18 karakter) dibuat, panel restart. Digunakan jika jalur sebelumnya dikompromikan atau terlupakan.
- **8. Reset Settings** – reset semua pengaturan panel ke nilai default. **Kredensial (nama pengguna dan kata sandi) serta data akun tidak hilang**. Memerlukan konfirmasi; setelah reset, panel restart.
- **9. Change Port** – mengubah port panel web. Diminta nomor port (1–65535); setelah ditetapkan, restart diperlukan agar port berlaku.
- **10. View Current Settings** – melihat pengaturan saat ini (`x-ui setting -show`). Menampilkan backend database yang digunakan (SQLite atau PostgreSQL dengan kata sandi disamarkan dalam DSN) serta tautan akses siap pakai (Access URL). Jika SSL belum dikonfigurasi, menawarkan untuk menerbitkan sertifikat Let's Encrypt untuk alamat IP.

Manajemen Layanan

- **11. Start** – memulai layanan panel. Jika panel sudah berjalan, akan muncul pesan bahwa restart tidak diperlukan.
- **12. Stop** – menghentikan layanan panel.
- **13. Restart** – me-restart layanan panel.
- **14. Restart Xray** – me-restart hanya inti Xray-core tanpa me-restart panel itu sendiri (melalui `systemctl reload x-ui` , dalam Docker – dengan sinyal `USR1` ke proses panel).
- **15. Check Status** – memeriksa status layanan (`systemctl status x-ui` atau `rc-service x-ui status`).
- **16. Logs Management** – manajemen log: melihat log debug (Debug Log, melalui `journalctl`) dan, kecuali Alpine, menghapus semua log (Clear All logs).

Autostart

- **17. Enable Autostart** – mengaktifkan autostart panel saat OS booting (`systemctl enable x-ui` atau `rc-update add`).
- **18. Disable Autostart** – menonaktifkan autostart saat OS booting.

Dalam Docker, autostart diatur oleh kebijakan restart container, sehingga item-item ini hanya menampilkan petunjuk yang sesuai.

Keamanan dan Jaringan

- **19. SSL Certificate Management** – manajemen sertifikat SSL melalui acme.sh: menerbitkan sertifikat untuk domain, mencabut, memperbarui paksa, melihat domain yang ada, menentukan jalur sertifikat untuk panel, serta menerbitkan sertifikat berumur pendek (~6 hari, dengan pembaruan otomatis) untuk alamat IP.
- **20. Cloudflare SSL Certificate** – menerbitkan sertifikat SSL melalui validasi DNS Cloudflare.
- **21. IP Limit Management** – manajemen batas jumlah IP per klien (berbasis Fail2ban): melihat dan menghapus pemblokiran, dll.
- **22. Firewall Management** – manajemen firewall (membuka/menutup port dan melihat aturan).
- **23. SSH Port Forwarding Management** – mengonfigurasi penerusan port SSH untuk membuka panel dari mesin lokal melalui tunnel SSH.

Performa dan Pemeliharaan

- **24. Enable BBR** – mengaktifkan/menonaktifkan algoritma manajemen kemacetan TCP BBR (submenu dengan item Enable BBR / Disable BBR).
- **25. Update Geo Files** – memperbarui database geo (file `.dat`) dengan pilihan sumber: Loyalsoldier (`geoip.dat`, `geosite.dat`), chocolate4u (`geoip_IR.dat`, `geosite_IR.dat`), runetfreedom (`geoip_RU.dat`, `geosite_RU.dat`), atau All (semuanya sekaligus). Setelah pembaruan, panel restart.
- **26. Speedtest by Ookla** – menjalankan uji kecepatan jaringan melalui Speedtest by Ookla.
- **27. PostgreSQL Management** – manajemen instans PostgreSQL bawaan/terhubung (mengaktifkan dan operasi terkait).
- **0. Exit Script** – keluar dari menu.

1.7. Subperintah `x-ui` (Tanpa Menu Interaktif)

Untuk digunakan dalam skrip, perintah `x-ui` mendukung subperintah langsung (menjalankan `x-ui` tanpa argumen akan membuka menu):

Perintah	Aksi
<code>x-ui</code>	membuka menu manajemen
<code>x-ui start</code>	memulai panel
<code>x-ui stop</code>	menghentikan panel
<code>x-ui restart</code>	me-restart panel
<code>x-ui restart-xray</code>	me-restart Xray
<code>x-ui status</code>	status layanan saat ini
<code>x-ui settings</code>	pengaturan saat ini
<code>x-ui enable</code>	mengaktifkan autostart saat OS booting
<code>x-ui disable</code>	menonaktifkan autostart
<code>x-ui log</code>	melihat log
<code>x-ui banlog</code>	melihat log pemblokiran Fail2ban

Perintah	Aksi
<code>x-ui update</code>	memperbarui panel
<code>x-ui update-all-geofiles</code>	memperbarui semua file geo
<code>x-ui migrateDB [file]</code>	konversi <code>.db</code>  <code>.dump</code> (SQLite)
<code>x-ui legacy</code>	memasang versi lama
<code>x-ui install</code>	memasang panel
<code>x-ui uninstall</code>	menghapus panel

1.8. Migrasi SQLite → PostgreSQL

Instalasi yang ada pada SQLite dapat dipindahkan ke PostgreSQL:

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"
# kemudian tetapkan XUI_DB_TYPE dan XUI_DB_DSN di /etc/default/x-ui dan restart:
systemctl restart x-ui
```

File SQLite asli tetap tidak tersentuh – hapus secara manual hanya setelah memverifikasi bahwa backend baru berfungsi dengan benar.

Contoh: memverifikasi peralihan ke PostgreSQL. Setelah migrasi, pastikan panel benar-benar berjalan di backend baru dengan perintah melihat pengaturan – outputnya harus menunjukkan PostgreSQL (kata sandi dalam DSN disamarkan):

```
x-ui settings | grep -i -E 'db|dsn'
```

Jika panel terbuka dan akun ada di tempatnya, file `x-ui.db` asli dapat dihapus.

2. Login Panel dan Keamanan Akses

Bagian ini menjelaskan semua hal yang berkaitan dengan autentikasi administrator panel 3X-UI: formulir login, autentikasi dua faktor (TOTP), perlindungan dari serangan brute-force, penggantian kredensial, perubahan jalur rahasia dan port panel, masa aktif sesi, serta sinkronisasi/autentikasi melalui LDAP.

2.1. Formulir Login

Halaman login disajikan di root jalur rahasia panel (`webBasePath`). Jika pengguna sudah terautentikasi, ia akan secara otomatis diarahkan ke `.../panel/`. Halaman ini memiliki pengalih tema, pemilih bahasa antarmuka, dan formulir login itu sendiri.

Kolom formulir:

Kolom	Label/Judul	Wajib diisi	Deskripsi
Username	«Username»	Ya	Login administrator. Nilai kosong ditolak di sisi klien, dan di sisi server dengan pesan «Username wajib diisi».
Password	«Password»	Ya	Kata sandi administrator. Nilai kosong ditolak dengan pesan «Password wajib diisi».
Kode 2FA	«Kode 2FA»	Hanya jika 2FA diaktifkan	Kolom ini muncul hanya jika autentikasi dua faktor diaktifkan di panel. Kode 6 digit dari aplikasi autentikator.

Tombol «**Login**» mengirimkan formulir ke `POST /login`.

Perilaku dan pesan:

- Jika login berhasil, ditampilkan pesan «Login berhasil» dan pengguna diarahkan ke `.../panel/`.
- Jika ada kesalahan kredensial atau kode 2FA yang salah, server mengembalikan **satu** pesan yang sama: «Nama pengguna, kata sandi, atau kode dua faktor tidak valid.» (bahasa Inggris: *Invalid username or password or two-factor code.*). Ini dilakukan dengan sengaja — panel tidak memberi tahu apa yang salah (login, kata sandi, atau kode) agar tidak memudahkan serangan brute-force.
- Kolom «Kode 2FA» ditampilkan atau disembunyikan panel berdasarkan permintaan `POST /getTwoFactorEnable`, yang mengembalikan status 2FA saat ini sebelum autentikasi.
- Jika sesi server telah kedaluwarsa, pada permintaan berikutnya akan ditampilkan «Sesi telah kedaluwarsa. Silakan login kembali», dan pengguna diarahkan ke halaman login.

Catatan tentang CSRF: sebelum mengirimkan formulir, klien mendapatkan token CSRF (`GET /csrf-token`); permintaan `/login` dan `/logout` dilindungi oleh pemeriksaan CSRF.

Contoh: login melalui API. Jika 2FA dinonaktifkan, cukup gunakan username dan password; jika 2FA diaktifkan, tambahkan kolom `twoFactorCode` :

```
# Tanpa 2FA
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль'

# Dengan 2FA diaktifkan – tambahkan kode 6 digit
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль&twoFactorCode=123456'
```

Jika berhasil, server akan mengembalikan `Set-Cookie` dengan cookie sesi – cookie ini perlu disertakan dalam permintaan berikutnya ke `/panel/api/...`.

2.2. Autentikasi Dua Faktor (2FA / TOTP)

2FA di 3X-UI diimplementasikan sesuai standar **TOTP** dan kompatibel dengan aplikasi autentikator apa pun (Google Authenticator, Aegis, FreeOTP, dan sejenisnya). Parameter ditetapkan secara tetap: algoritma **SHA1**, 6 digit, periode **30** detik, issuer `3x-ui`, label `Administrator`.

Contoh: URI otpauth yang dikodekan dalam kode QR. Jika aplikasi autentikator tidak dapat memindai kamera, token dapat ditambahkan secara manual menggunakan tautan berikut (ganti `JBSWY3DPEHPK3PXP` dengan rahasia Base32 Anda):

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPEHPK3PXP&issuer=3x-
ui&algorithm=SHA1&digits=6&period=30
```

Parameter `algorithm=SHA1`, `digits=6`, `period=30` sesuai dengan nilai tetap panel – tidak perlu mengubahnya.

Pengaturan terdapat di bagian **Pengaturan** → **Akun**, tab «**Autentikasi Dua Faktor**».

Elemen	Teks	Deskripsi
Toggle	«Aktifkan 2FA»	Mengaktifkan/menonaktifkan autentikasi dua faktor.
Deskripsi	«Menambahkan lapisan autentikasi tambahan untuk meningkatkan keamanan.»	Petunjuk di bawah toggle.

Cara Mengaktifkan 2FA

Saat toggle diaktifkan, panel **menghasilkan rahasia baru secara lokal** – string acak dalam encoding Base32 (alfabet `A–Z` dan `2–7`). Jendela «Aktifkan Autentikasi Dua Faktor» akan terbuka dengan panduan langkah demi langkah:

1. «**Pindai kode QR ini di aplikasi autentikator atau salin token di sebelah kode QR dan tempelkan ke aplikasi**». Di bawah kode QR, rahasia ditampilkan dalam bentuk teks – dengan mengklik kode QR, rahasia disalin ke clipboard (muncul pesan «Disalin»).
2. «**Masukkan kode dari aplikasi**» – perlu memasukkan kode 6 digit yang dihasilkan aplikasi. Kode diverifikasi **di sisi browser**: panel sendiri menghitung TOTP saat ini berdasarkan rahasia yang baru dihasilkan dan

membandingkannya dengan kode yang dimasukkan. Jika kode salah – «Kode tidak valid»; kolom hanya menerima tepat 6 digit.

Hanya setelah konfirmasi berhasil, rahasia dan flag aktivasi disimpan. Setelah disimpan, ditampilkan pesan «Autentikasi dua faktor berhasil diaktifkan».

Penting: perubahan di bagian pengaturan diterapkan dengan tombol umum **«Simpan»**, setelah itu biasanya diperlukan restart panel («Simpan perubahan dan restart panel untuk menerapkannya»). Saat 2FA pertama kali diaktifkan, server juga akan **menginvalidasi semua sesi aktif** (menaikkan «login epoch»), sehingga setelah menerapkan pengaturan, login ulang diperlukan – kali ini dengan kode 2FA.

Cara Menonaktifkan 2FA

Menekan toggle kembali akan membuka jendela «Nonaktifkan Autentikasi Dua Faktor» dengan petunjuk «Masukkan kode dari aplikasi untuk menonaktifkan autentikasi dua faktor». Setelah memasukkan kode yang benar (mulai 3.4.2, kode diverifikasi **di sisi server**), flag dan rahasia dihapus, dan ditampilkan pesan «Autentikasi dua faktor berhasil dihapus».

Verifikasi Kode saat Login

Saat login, server mengambil rahasia yang tersimpan dan membandingkan TOTP saat ini dengan kode 2FA yang dikirimkan. Ketidakcocokan dianggap sebagai login yang gagal, namun pengguna melihat pesan terpadu yang sama «Nama pengguna, kata sandi, atau kode dua faktor tidak valid.».

Pemulihan Akses (recovery)

3X-UI **tidak** memiliki mekanisme «kode pemulihan» tersendiri. Jika akses ke aplikasi autentikator hilang, pemulihan login melalui antarmuka panel tidak dapat dilakukan. Satu-satunya cara adalah menonaktifkan 2FA langsung di database di server: setel kunci `twoFactorEnable` menjadi `false` (dan jika perlu hapus `twoFactorToken`) di tabel pengaturan, lalu restart panel. Oleh karena itu, disarankan untuk menyimpan rahasia (token Base32) di tempat yang aman saat mengaktifkan 2FA.

Contoh: menonaktifkan 2FA secara darurat di server. Setelah mendapatkan akses ke server melalui SSH, hentikan panel, setel ulang kunci di tabel pengaturan, dan jalankan panel kembali:

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE
key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

Setelah itu, login dilakukan hanya dengan username dan password, dan 2FA dapat dikonfigurasi ulang jika diperlukan.

Kaitan dengan penggantian kredensial: saat mengganti username/password (lihat 2.4), 2FA **secara otomatis dinonaktifkan** di server, agar rahasia lama tidak memblokir akses dengan akun baru.

2.3. Pembatasan Percobaan Login (login limiter / perlindungan brute-force)

Panel memiliki pembatas percobaan login gagal bawaan (setara fail2ban pada level aplikasi). Parameter ditetapkan dalam kode dan **tidak dapat dikonfigurasi** melalui antarmuka:

Parameter	Nilai	Fungsi
Maksimum kegagalan	5	Berapa banyak percobaan gagal yang diizinkan dalam satu jendela.
Jendela perhitungan	5 menit	Jendela geser tempat kegagalan terakumulasi (yang lebih lama dibuang).
Durasi blokir (cooldown)	15 menit	Berapa lama kunci diblokir setelah melebihi ambang batas.

Cara kerjanya:

- Kunci blokir dibentuk dari **pasangan «IP + username»** (username diubah ke huruf kecil, spasi dipangkas). Artinya, blokir diterapkan pada pasangan alamat + nama pengguna tertentu, bukan pada seluruh panel.
- Setiap percobaan gagal (username/password salah atau kode 2FA salah) menaikkan penghitung. Setelah mencapai **5 kegagalan dalam 5 menit**, kunci diblokir selama **15 menit**. Selama pemblokiran, setiap percobaan dari pasangan tersebut langsung ditolak dengan pesan yang sama «Nama pengguna, kata sandi, atau kode dua faktor tidak valid.», meskipun datanya benar.
- **Login yang berhasil langsung mereset** penghitung dan mencabut blokir untuk pasangan tersebut.
- Alamat IP klien ditentukan dengan mempertimbangkan proxy tepercaya (lihat `trustedProxyCIDRs`): header `X-Real-IP` dan `X-Forwarded-For` diterima hanya jika permintaan berasal dari alamat tepercaya. Jika tidak, digunakan alamat koneksi nyata, dan jika tidak dapat diekstrak – string `unknown`.

Semua percobaan dicatat dalam log. Untuk percobaan gagal, peringatan ditulis ke log server dengan username, IP, alasan, dan jika diblokir – waktu `blocked_until`. Jika notifikasi login melalui bot Telegram diaktifkan (`tgNotifyLogin` – «Notifikasi Login»), administrator juga menerima username, IP, dan waktu untuk percobaan yang berhasil, gagal, maupun diblokir.

Contoh: notifikasi login di Telegram. Dengan `tgNotifyLogin` diaktifkan, setelah setiap percobaan administrator menerima pesan yang kira-kira seperti ini:

```
Уведомление о входе
Пользователь: admin
IP: 203.0.113.45
Время: 2026-06-10 14:32:07
Статус: успешно
```

Untuk pasangan «IP + username» yang diblokir, statusnya akan menunjukkan bahwa percobaan ditolak oleh pembatas.

2.4. Penggantian Username dan Password Administrator

Bagian **Pengaturan** → **Akun**, tab «Kredensial Administrator». Kolom:

Kolom	Teks	Deskripsi
Username saat ini	«Username saat ini»	Nama pengguna yang aktif. Harus cocok dengan username saat ini, jika tidak perubahan ditolak.
Password saat ini	«Password saat ini»	Password aktif untuk konfirmasi identitas.
Username baru	«Username baru»	Nama pengguna baru. Tidak boleh kosong.
Password baru	«Password baru»	Password baru. Tidak boleh kosong.

Perubahan diterapkan dengan tombol **«Konfirmasi»** dan dikirim ke `POST /panel/setting/updateUser`.

Logika dan pesan server:

- Jika «Username saat ini» tidak cocok dengan yang sebenarnya atau «Password saat ini» salah – «Terjadi kesalahan saat mengubah kredensial administrator.» dengan penjelasan «Nama pengguna atau kata sandi tidak valid».
- Jika username baru atau password baru kosong – penjelasan «Username baru dan password baru harus diisi».
- Jika berhasil – «Anda berhasil mengubah kredensial administrator.» Password disimpan sebagai bcrypt hash.

Konfirmasi dengan kode 2FA (mulai 3.4.2). Jika autentikasi dua faktor diaktifkan pada panel, mengganti login/password juga memerlukan **memasukkan kode saat ini** dari aplikasi autentikator. Akan terbuka jendela **«Ganti kredensial»** dengan petunjuk «Masukkan kode dari aplikasi untuk mengubah kredensial administrator.»; kode (`twoFactorCode`) diverifikasi di sisi server, dan jika kode salah atau kosong, perubahan tidak diterapkan. Konfirmasi yang sama kini juga diperlukan saat menonaktifkan 2FA (lihat [2.2](#)).

Contoh: penggantian kredensial melalui API. Permintaan memerlukan cookie sesi yang valid (diperoleh saat login) dan konfirmasi username/password saat ini:

```
curl -X POST https://panel.example.com:2053/мой-секрет/panel/setting/updateUser \
  -b 'session=BAWA_CECCИОНHAYя_COOKIE' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=СтарыйПароль&newUsername=root&newPassword=НовыйСложныйПароль'
```

Setelah berhasil, sesi saat ini dihapus – perlu login ulang dengan kredensial baru.

Efek penting dari penggantian kredensial:

- **Semua sesi yang ada dihapus** (penghitung `login_epoch` pengguna dinaikkan), sehingga setelah penggantian panel secara otomatis melakukan logout dan mengarahkan ke halaman login – perlu login ulang.
- Jika **2FA diaktifkan** pada saat penggantian, **2FA secara otomatis dinonaktifkan** (flag dan rahasia dihapus). Autentikasi dua faktor perlu dikonfigurasi ulang setelah mengganti username/password.

Jika 2FA diaktifkan, sebelum mengirimkan formulir akan muncul jendela «Ubah Kredensial» dengan petunjuk «Masukkan kode dari aplikasi untuk mengubah kredensial administrator.» – perubahan kredensial hanya dapat dilakukan setelah mengonfirmasi kode 2FA saat ini.

2.5. Jalur Rahasia (URI-path / webBasePath) dan Port Panel

Parameter ini berada di bagian **Pengaturan** → **Panel** dan secara langsung memengaruhi «ketidakterlihat-an» dan aksesibilitas panel. Diterapkan setelah menyimpan dan **merestart panel**.

Kolom	Teks	Nilai default	Deskripsi
Port panel	«Port panel» (<code>panelPort</code>), petunjuk «Port tempat panel berjalan»	2053	Port TCP antarmuka web.
URI-path	«URI-path» (<code>panelUrlPath</code>), petunjuk «Harus dimulai dengan '/' dan diakhiri dengan '/'»	/	Jalur dasar rahasia (<code>webBasePath</code>). Panel hanya dapat diakses melalui jalur ini (misalnya, <code>/jalur-rahasia-saya/</code>).
Alamat IP untuk manajemen panel	«Alamat IP untuk manajemen panel» (<code>panelListeningIP</code>), petunjuk «Kosongkan untuk menerima koneksi dari IP mana pun»	kosong	Alamat tempat panel mendengarkan. Kosong = semua antarmuka.
Domain panel	«Domain panel» (<code>panelListeningDomain</code>), petunjuk «Kosongkan untuk menerima koneksi dari domain dan IP mana pun.»	kosong	Pembatasan akses berdasarkan domain (Host).
Path public key sertifikat panel	<code>publicKeyPath</code> , petunjuk «Masukkan path lengkap yang dimulai dengan '/'»	kosong	Sertifikat TLS untuk akses HTTPS ke panel.
Path private key sertifikat panel	<code>privateKeyPath</code> , petunjuk yang sama	kosong	Private key TLS.

Perilaku jalur dasar (`webBasePath`):

- Nilai dinormalisasi secara otomatis: jika tidak dimulai dengan `/` , karakter tersebut ditambahkan di awal; jika tidak diakhiri dengan `/` , ditambahkan di akhir. Artinya, jalur selalu berbentuk `/.../` .
- Jalur dasar diterapkan pada panel itu sendiri, pada aset, dan pada cookie sesi (cookie hanya diterbitkan untuk jalur ini).

Rekomendasi keamanan (bagian «Peringatan Keamanan»): panel sendiri menampilkan peringatan jika konfigurasi «terlalu publik»:

- «Panel berjalan dengan HTTP biasa — konfigurasi TLS untuk produksi.»
- «Port default 2053 sudah dikenal luas — ubah ke port acak.»
- «Jalur dasar default `/` sudah dikenal luas — ubah ke jalur acak.»

Dengan kata lain, untuk server produksi perlu menetapkan **port non-standar**, **URI-path yang tidak mudah ditebak**, dan **sertifikat TLS**.

Contoh: konfigurasi panel «tersembunyi» untuk produksi. Di bagian **Pengaturan** → **Panel**, tetapkan nilai seperti berikut:

Kolom	Nilai
Port panel	34571 (acak, bukan 2053)
URI-path	/aXf9Qm2/ (tidak mudah ditebak, dimulai dan diakhiri dengan /)
Path public key sertifikat panel	/etc/letsencrypt/live/panel.example.com/fullchain.pem
Path private key sertifikat panel	/etc/letsencrypt/live/panel.example.com/privkey.pem

Setelah menyimpan dan merestart, panel hanya dapat diakses melalui `https://panel.example.com:34571/aXf9Qm2/`, dan peringatan keamanan akan hilang.

2.6. Masa Aktif Sesi (timeout)

Kolom «Durasi Sesi» (`sessionMaxAge`) terdapat di antara pengaturan panel/interval.

Kolom	Teks	Nilai default	Satuan	Deskripsi
Durasi Sesi	«Durasi Sesi», petunjuk «Durasi sesi dalam sistem (nilai: menit)»	360	menit	Masa aktif cookie sesi administrator.

Perilaku:

- Nilai dimasukkan dalam **menit** (default 360 menit = 6 jam) dan dikonversi ke detik saat cookie dikonfigurasi.
- Jika nilainya **lebih dari 0**, `MaxAge` yang sesuai diatur pada cookie sesi. Setelah jangka waktu ini, cookie tidak lagi berlaku dan pada permintaan berikutnya pengguna mendapatkan «Sesi telah kedaluwarsa. Silakan login kembali».
- Sesi juga menjadi tidak valid lebih awal saat penggantian kredensial atau saat 2FA pertama kali diaktifkan (melalui mekanisme `login_epoch`, lihat 2.4 dan 2.2) dan saat logout eksplisit (`POST /logout`).
- Cookie sesi ditandai `HttpOnly`, dengan kebijakan `SameSite=Lax`; flag `Secure` diatur saat akses langsung HTTPS ke panel.

Selain timeout itu sendiri, ada notifikasi terkait: «Perbedaan Waktu Notifikasi Kedaluwarsa Sesi» (`expireTimeDiff`, petunjuk «Menerima notifikasi kedaluwarsa sesi sebelum mencapai nilai ambang batas (nilai: hari)», default 0) – memungkinkan menerima peringatan lebih awal.

2.7. LDAP (Sinkronisasi dan Autentikasi)

Bagian LDAP menyediakan dua kemampuan: (1) mengautentikasi login administrator melalui LDAP jika password lokal tidak cocok, dan (2) menyinkronkan status klien secara berkala (flag `VLESS` diaktifkan/dinonaktifkan) dari direktori.

Cara digunakan saat login: server pertama-tama memeriksa bcrypt hash password lokal. Jika **tidak cocok** dan LDAP diaktifkan, panel mencoba mengautentikasi pengguna di direktori: dengan `Bind DN` yang ditetapkan, dilakukan service bind, kemudian entri pengguna dicari menggunakan filter dan atribut, lalu dilakukan percobaan bind dengan DN yang ditemukan menggunakan password yang dimasukkan. Jika berhasil, login diterima. (Setelah autentikasi LDAP berhasil, jika 2FA diaktifkan, kode TOTP tetap diperiksa.)

Kolom bagian ini:

Kolom	Teks	Nilai default	Deskripsi
Aktifkan Sinkronisasi LDAP	«Aktifkan Sinkronisasi LDAP» (<code>enable</code>)	false	Sakelar utama integrasi LDAP.
Host LDAP	«Host LDAP» (<code>host</code>)	kosong	Alamat server LDAP.
Port LDAP	«Port LDAP» (<code>port</code>)	389	Port. Untuk LDAPS biasanya 636.
Gunakan TLS (LDAPS)	«Gunakan TLS (LDAPS)» (<code>useTls</code>)	false	Jika diaktifkan, skema <code>ldaps://</code> digunakan (secara default – dengan verifikasi sertifikat server).
Lewati verifikasi sertifikat TLS	«Lewati verifikasi sertifikat TLS» (<code>ldapInsecureSkipVerify</code>)	false	Ditambahkan di 3.4.2. Menonaktifkan verifikasi sertifikat server saat LDAPS – untuk CA internal/sertifikat yang ditandatangani sendiri. Petunjuk: «Tidak aman – menonaktifkan verifikasi sertifikat server. Gunakan hanya dengan CA internal/tidak tepercaya». Sakelar hanya tersedia jika «Gunakan TLS (LDAPS)» diaktifkan.
Bind DN	«Bind DN» (<code>bindDn</code>)	kosong	DN akun layanan untuk bind/pencarian awal. Jika kosong – bind tidak dilakukan (pencarian anonim).
Password bind	petunjuk: «Dikonfigurasi; kosongkan untuk mempertahankan password saat ini.» / «Belum dikonfigurasi.» / «Dikonfigurasi – masukkan nilai baru untuk mengganti»	kosong	Password untuk <code>Bind DN</code> . Disimpan terpisah; untuk mempertahankan yang lama, kolom dibiarkan kosong.
Base DN	«Base DN» (<code>baseDn</code>)	kosong	Root subpohon tempat pencarian dilakukan (pencarian rekursif, seluruh subpohon).
Filter pengguna	«Filter pengguna» (<code>userFilter</code>)	(<code>objectClass=person</code>)	Filter LDAP untuk memilih akun. Saat autentikasi, username dimasukkan ke filter dengan escaping.
Atribut pengguna (username/email)	«Atribut pengguna (username/email)» (<code>userAttr</code>)	<code>mail</code>	Atribut yang dicocokkan dengan username/pengidentifikasi klien (misalnya, <code>mail</code> atau <code>uid</code>).

Kolom	Teks	Nilai default	Deskripsi
Atribut flag VLESS	«Atribut flag VLESS» (<code>vlessField</code>)	<code>vless_enabled</code>	Atribut yang menentukan apakah akses VLESS klien harus diaktifkan.
Atribut flag umum (opsional)	«Atribut flag umum (opsional)» (<code>flagField</code>), petunjuk «Jika diatur, menggantikan flag VLESS – mis. <code>shadowInactive</code> .»	kosong	Jika diatur, digunakan sebagai pengganti <code>vless_enabled</code> .
Nilai truthy	«Nilai truthy» (<code>truthyValues</code>), petunjuk «Dipisahkan koma; default: <code>true,1,yes,on</code> »	<code>true,1,yes,on</code>	Daftar nilai atribut flag yang dianggap sebagai «diaktifkan».
Balik flag	«Balik flag» (<code>invertFlag</code>), petunjuk «Aktifkan jika atribut berarti «dinonaktifkan» (mis. <code>shadowInactive</code>).»	<code>false</code>	Membalik arti flag.
Jadwal sinkronisasi	«Jadwal sinkronisasi» (<code>syncSchedule</code>), petunjuk «String seperti cron, mis. <code>@every 1m</code> »	<code>@every 1m</code>	Frekuensi sinkronisasi dalam format seperti cron.
Tag inbound	«Tag inbound» (<code>inboundTags</code>), petunjuk «Inbound tempat sinkronisasi LDAP dapat membuat atau menghapus klien secara otomatis.»	kosong	Membatasi inbound mana yang mengizinkan operasi otomatis. Jika tidak ada inbound: «Inbound tidak ditemukan. Buat inbound terlebih dahulu.»
Pembuatan klien otomatis	«Pembuatan klien otomatis» (<code>autoCreate</code>)	<code>false</code>	Membuat klien di inbound yang ditentukan jika klien tersebut muncul di direktori.
Penghapusan klien otomatis	«Penghapusan klien otomatis» (<code>autoDelete</code>)	<code>false</code>	Menghapus klien jika klien tersebut tidak lagi ada di direktori.
Volume default (GB)	«Volume default (GB)» (<code>defaultTotalGb</code>)	<code>0</code>	Batas lalu lintas untuk klien yang dibuat secara otomatis (0 = tanpa batas).
Masa berlaku default (hari)	«Masa berlaku default (hari)» (<code>defaultExpiryDays</code>)	<code>0</code>	Masa berlaku untuk klien yang dibuat secara otomatis (0 = tidak terbatas).
Batas IP default	«Batas IP default» (<code>defaultIpLimit</code>)	<code>0</code>	Batas jumlah IP simultan (0 = tanpa batas).

Kekhasan logika flag sinkronisasi: saat membaca atribut flag (`flagField` , default `vless_enabled`), nilai dianggap «diaktifkan» jika termasuk dalam daftar nilai truthy; jika inversi diaktifkan, hasilnya dibalik. Atribut pengguna (`userAttr`) digunakan sebagai kunci pencocokan (email/nama) – entri tanpa nilai atribut ini dilewati.

Keamanan: disarankan untuk mengaktifkan **TLS (LDAPS)** agar password bind dan password yang diperiksa tidak dikirimkan dalam teks biasa, dan untuk Bind DN gunakan akun dengan hak baca minimal yang diperlukan.

Contoh: konfigurasi sinkronisasi LDAP yang umum (Active Directory). Pengisian kolom bagian ini untuk direktori di mana status akses disimpan dalam atribut mirip flag `userAccountControl`, dan pencocokan dilakukan berdasarkan email:

Kolom	Nilai
Host LDAP	<code>ldap.example.com</code>
Port LDAP	<code>636</code>
Gunakan TLS (LDAPS)	diaktifkan
Bind DN	<code>CN=svc-3xui,OU=Service,DC=example,DC=com</code>
Base DN	<code>OU=Users,DC=example,DC=com</code>
Filter pengguna	<code>(objectClass=person)</code>
Atribut pengguna (username/email)	<code>mail</code>
Atribut flag VLESS	<code>vless_enabled</code>
Nilai truthy	<code>true,1,yes,on</code>
Jadwal sinkronisasi	<code>@every 5m</code>

Dengan konfigurasi ini, setiap 5 menit panel akan menelusuri subpohon `OU=Users`, mencocokkan klien berdasarkan `mail`, dan mengaktifkan/menonaktifkan akses VLESS berdasarkan nilai `vless_enabled`.

3. Ikhtisar / Dasbor

Dasbor («Dasbor», dalam antarmuka bahasa Inggris – *Overview*) adalah halaman awal panel. Halaman ini menampilkan status server dan proses Xray secara real-time. Semua metrik dikirim dari sisi server. Penjadwal latar belakang membangun ulang snapshot **setiap 2 detik** dan mendistribusikannya ke semua tab yang terbuka melalui WebSocket; setiap satu menit, baris metrik yang terkumpul disimpan ke disk. HTTP endpoint `GET / status` mengembalikan snapshot terakhir yang di-cache.

Di bawah ini dijelaskan setiap metrik dan setiap elemen kontrol pada halaman.

Di menu samping panel (dan di laci geser pada perangkat seluler) mulai versi 3.4.2 terdapat tombol «**Dokumentasi**» (ikon buku) – membuka dokumentasi resmi <https://docs.sanaei.dev/>. Selain itu, di 3.4.2 dilakukan peningkatan **aksesibilitas** secara menyeluruh: ikon-tombol dan elemen yang dapat diklik memperoleh label aria dan aktivasi dengan Enter/Space (untuk pembaca layar dan navigasi keyboard).

3.1. Prinsip Umum Pengumpulan Data

- Snapshot dikumpulkan oleh library `gopsutil`. Jika pengukuran tertentu gagal, field tetap bernilai nol, dan peringatan ditulis ke log (`get cpu percent failed`, `get uptime failed`, dll.) – ini tidak merusak seluruh dasbor, hanya tile yang bersangkutan yang akan menampilkan 0/N-A.
- Kecepatan «instan» (CPU %, jaringan, disk I/O) dihitung sebagai selisih antara snapshot saat ini dan sebelumnya, dibagi dengan interval dalam detik. Oleh karena itu, saat pertama kali halaman dimuat, nilai kecepatan bisa nol sampai pengukuran kedua terkumpul.
- Riwayat dapat dilihat di bagian «Riwayat Sistem» (*System History*) – grafik dibangun berdasarkan baris data yang sama yang dijelaskan di bawah (lihat poin 3.12).

3.2. CPU (CPU)

Tile «CPU» (*CPU*) menampilkan beban prosesor saat ini dalam persen, serta parameter prosesor itu sendiri.

Metrik	Deskripsi
Beban CPU, %	Proporsi waktu prosesor yang digunakan dalam interval terakhir. Diperhalus dengan rata-rata eksponensial (EMA, koefisien <code>alpha = 0.3</code>) agar lonjakan tidak membuat indikator bergetar. Nilai selalu dibatasi dalam rentang 0–100 %. Pada pengukuran pertama, dikembalikan 0 (inisialisasi titik dasar).
Prosesor logis	Jumlah core logis – yaitu dengan memperhitungkan Hyper-Threading.
Core fisik	Jumlah core fisik.
Frekuensi	Frekuensi dasar prosesor dalam MHz. Diminta secara lazy dan di-cache: pengukuran pertama yang berhasil disimpan, percobaan ulang dilakukan tidak lebih sering dari sekali dalam 5 menit, dan permintaan itu sendiri dibatasi dengan timeout 1,5 s (pada beberapa sistem, permintaan frekuensi merespons lambat).

Beban CPU dihitung secara algoritmik: jika tersedia implementasi platform native, maka digunakan itu, jika tidak – perhitungan berdasarkan delta counter waktu prosesor (`busy / total`). Waktu Guest dan GuestNice dikecualikan agar tidak dihitung dua kali.

3.3. Memori (RAM)

Tile «Memori» (*RAM*) menampilkan penggunaan dan total. Ditampilkan dalam format «digunakan / total» dan/atau persentase pengisian. Persentase disimpan ke riwayat.

3.4. Swap (Swap)

Tile «Swap» (*Swap*) menampilkan penggunaan dan total. Jika file/partisi swap tidak dikonfigurasi (total = 0), metrik bernilai nol; jika tidak ada swap, nilai 0 ditulis ke baris riwayat.

3.5. Disk (Storage)

Tile «Disk» (*Storage*) menampilkan penggunaan dan total, dengan memperhitungkan **hanya partisi root** /. Persentase pengisian ditulis ke riwayat «Penggunaan Disk» (*Disk Usage*). Disk I/O (baca / tulis, byte/s) dikumpulkan secara terpisah sebagai delta counter per interval – ditampilkan pada tab «Disk I/O» di riwayat.

3.6. Uptime Sistem (Uptime)

Metrik «Uptime Sistem» (*Uptime*). Ini adalah waktu sejak boot **seluruh server** (dalam detik), bukan waktu kerja panel atau Xray. Uptime proses Xray disimpan secara terpisah (lihat poin 3.9), begitu pula jumlah thread panel (dalam terjemahan – «Thread» / *Threads*).

Memori yang Digunakan Panel

Di samping metrik proses panel, ditampilkan jumlah RAM yang digunakan oleh proses 3X-UI itu sendiri. Nilai ini diambil dari RSS proses aktual (seperti yang dilihat oleh sistem operasi) dan sesuai dengan apa yang ditampilkan oleh utilitas sistem. Angka turun seiring dibebaskannya memori. Sebelumnya, panel menampilkan counter internal Go yang membesar-besarkan konsumsi memori (misalnya, ~300 MB pada server idle dengan satu klien) dan tidak pernah berkurang – sekarang artefak ini tidak ada. Selain itu, proses latar belakang periodik mengembalikan memori yang tidak digunakan ke sistem operasi agar metrik mencerminkan konsumsi aktual.

3.7. Load Average Sistem (Load average)

Blok «Beban Sistem» (*System Load*) – array tiga angka [Load1, Load5, Load15]. Tooltip: «Rata-rata beban sistem dalam 1, 5, dan 15 menit terakhir» (*System load average for the past 1, 5, and 15 minutes*). Grafik riwayat disebut «Rata-rata Beban Sistem (1 / 5 / 15 mnt)». Nilai ditulis ke baris riwayat secara terpisah: load1, load5, load15.

Ini adalah metrik Unix standar: rata-rata jumlah proses yang berada dalam antrian eksekusi. Acuan – bandingkan dengan jumlah core: beban yang secara konsisten melebihi jumlah core fisik menunjukkan kelebihan beban.

3.8. Jaringan: Kecepatan dan Total Volume Trafik

Hanya **antarmuka fisik** yang diperhitungkan. Antarmuka virtual dan tunnel dikecualikan: yaitu `lo` / `lo0`, serta semua yang dimulai dengan `loopback`, `docker`, `br-`, `veth`, `virbr`, `tun`, `tap`, `wg`, `tailscale`, `zt`. Nilai dijumlahkan di semua antarmuka yang tersisa.

Kecepatan Keseluruhan (*Overall Speed*) – kecepatan instan, delta counter per interval:

Metrik	Deskripsi
Upload / kirim (label «Upload» / <i>Upload</i>)	Kecepatan keluar, byte/s.
Download / terima (label «Download» / <i>Download</i>)	Kecepatan masuk, byte/s.

Total Volume Trafik (*Total Data*) – counter kumulatif sejak sistem dimulai:

Metrik	Deskripsi
Terkirim (label «Terkirim» / <i>Sent</i>)	Total byte yang dikirim.
Diterima (label «Diterima» / <i>Received</i>)	Total byte yang diterima.

Selain itu, kecepatan paket (paket/s) dan counter paket total dikumpulkan – ditampilkan pada tab «Paket Jaringan» (*Network Packets*) di riwayat. Baris riwayat jaringan: `netUp`, `netDown`, `pktUp`, `pktDown`.

3.9. Alamat IP Server

Blok «Alamat IP Server» (*IP Addresses*) menampilkan `IPv4` dan `IPv6`. Alamat eksternal ditentukan melalui layanan pihak ketiga (`api4.ipify.org`, `ipv4.icanhazip.com`, `v4.api.ipinfo.io/ip`, `ipv4.myexternalip.com/raw`, `4.ident.me`, `check-host.net/ip` untuk `IPv4` dan yang serupa untuk `IPv6`). Daftar dicoba secara berurutan hingga respons pertama yang berhasil; timeout setiap permintaan – 3 s.

Keistimewaan:

- Hasilnya **di-cache** selama masa hidup proses: alamat yang berhasil ditentukan tidak diminta ulang.
- Jika tidak ada layanan yang merespons, field tetap `N/A`. Untuk `IPv6`, pada `N/A` pertama, permintaan `IPv6` dinonaktifkan sama sekali agar tidak membuang waktu di jaringan tanpa `IPv6`.
- Di sampingnya ada tombol «mata» untuk menyembunyikan/menampilkan alamat – tooltip «Sembunyikan atau tampilkan alamat IP server» (*Toggle visibility of the IP*). Ini hanya penyembunyian visual di antarmuka (misalnya, untuk screenshot), tidak mempengaruhi alamat itu sendiri.

3.10. Koneksi TCP/UDP

Blok «Jumlah Koneksi» (*Connection Stats*) menampilkan total jumlah koneksi TCP dan UDP aktif di server (di seluruh sistem, bukan hanya Xray). Grafik riwayat – «Koneksi Aktif (TCP / UDP)» (*Active Connections*), baris `tcpCount`, `udpCount`.

3.11. Status Xray dan Manajemen Proses

Kartu «Xray» menampilkan status proses Xray-core dan memungkinkan pengelolaannya.

Status

Nilai	Label	Terjemahan	Kapan ditetapkan
running	«Berjalan»	<i>Running</i>	Proses Xray berjalan.
stop	«Berhenti»	<i>Stopped</i>	Proses tidak berjalan, dan tidak ada kesalahan startup yang tercatat.
error	«Kesalahan»	<i>Error</i>	Proses tidak berjalan, tetapi kesalahan startup tercatat. Teks kesalahan ditampilkan di jendela pop-up dengan judul «Terjadi kesalahan saat menjalankan Xray» (<i>An error occurred while running Xray</i>).
–	«Tidak diketahui»	<i>Unknown</i>	Ditampilkan selama status belum diterima.

Versi Xray ditampilkan di samping status.

Tombol Kontrol

- **Stop** (*Stop*). Memanggil `POST /stopXrayService`. Jika berhasil, panel mendistribusikan status baru `stop` melalui WebSocket dan notifikasi «Xray berhasil dihentikan» (*Xray service has been stopped*); jika error – status `error` beserta teks. Penting: jika panel diakses *melalui* Xray itu sendiri, menghentikan Xray dapat memutus koneksi ke panel – tidak ada masalah jika terhubung langsung ke panel.
- **Restart** (*Restart*). Memanggil `POST /restartXrayService`. Sebelum tindakan, ditampilkan konfirmasi «Restart xray?» dengan penjelasan «Memuat ulang layanan xray dengan konfigurasi yang tersimpan». Jika berhasil – status `running` dan notifikasi «Xray berhasil di-restart» (*Xray service has been restarted successfully*). Restart menerapkan konfigurasi tersimpan saat ini – gunakan setelah mengubah pengaturan.

Catatan. Dalam fork ini, kontrol Start / Stop / Restart yang lengkap ditambahkan ke dasbor untuk semua jenis otorisasi; di UI 3x-ui asli, tidak ada tombol «start» terpisah – startup dilakukan melalui restart.

Tombol Lihat Log Xray

Di kartu Xray terdapat tombol untuk melihat log Xray (*Logs*). Tombol ini muncul hanya ketika access-log dikonfigurasi dalam konfigurasi Xray: viewer bawaan membaca file tersebut, sehingga tanpa access-log tombol disembunyikan. Visibilitas tombol terikat pada flag terpisah `accessLogEnable` dan tidak lagi bergantung pada batas IP – daftar online dan batas alamat IP terus bekerja bahkan tanpa access-log (lihat poin 8).

Pemilihan Versi Xray

Bagian «Pemilihan Versi» (*Version*) memungkinkan untuk beralih Xray-core ke rilis lain. Daftar versi dimuat melalui `GET /getXrayVersion`:

- Sumbernya adalah GitHub API repositori `XTLS/Xray-core` (`/releases`). Permintaan di-cache selama **15 menit**; jika GitHub gagal, daftar yang terakhir berhasil didapat dikembalikan agar picker tidak kosong.
- Hanya rilis dengan format `X.Y.Z` dan **tidak lebih lama dari 26.4.25** yang masuk dalam daftar.

Tooltip: «Pilih versi yang ingin Anda beralih» (*Choose the version you want to switch to.*) dan peringatan «Penting: versi lama mungkin tidak mendukung pengaturan saat ini» (*Choose carefully, as older versions may not be compatible with current configurations.*).

Peralihan: `POST /installXray/:version` . Skenario:

Contoh. Beralih ke versi Xray-core tertentu (cookie sesi harus sudah diperoleh melalui otorisasi):

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v25.6.8' \
  -b cookie.txt
```

Di sini `v25.6.8` adalah tag dari daftar yang dikembalikan oleh `GET /getXrayVersion` . Versi harus ada dalam daftar ini, jika tidak, panel akan menolak permintaan. Mulai versi 3.4.2, versi Xray minimum yang diizinkan untuk instalasi dinaikkan menjadi **26.6.27** (inti Xray 26.6.27 sudah disertakan), sehingga build yang lebih lama tidak tersedia untuk pembaruan.

1. Versi yang dipilih diverifikasi keberadaannya dalam daftar rilis terkini (jika tidak – ditolak).
2. Xray dihentikan.
3. Arsip `Xray-<os>-<arch>.zip` diunduh dari GitHub untuk OS dan arsitektur saat ini (mendukung amd64/64, arm64-v8a, arm32-v7a/v6/v5, 386/32, s390x; untuk Windows – `xray.exe`). Ukuran arsip dan binary dibatasi 200 MB.
4. Binary diganti secara atomik (melalui file sementara + rename) dan ditandai sebagai executable.
5. Xray dijalankan kembali.

Sebelum peralihan, dialog «Alihkan Versi Xray» (*Do you really want to change the Xray version?*) ditampilkan dengan deskripsi «Ini akan mengubah versi Xray ke #version#». Jika berhasil – notifikasi «Xray berhasil diperbarui» (*Xray updated successfully*).

3.12. Pembaruan Panel (3X-UI)

Blok pengecekan pembaruan panel. Data diterima melalui `GET /getPanelUpdateInfo` :

Field	Deskripsi
Versi panel saat ini	Versi panel yang terinstal.
Versi panel terbaru	Rilis 3x-ui terbaru yang diperoleh dari GitHub.
Pembaruan tersedia	Flag bahwa versi terbaru lebih baru dari yang saat ini. Jika tidak diperlukan pembaruan – ditampilkan «Panel sudah diperbarui» / «Diperbarui».

Tombol «**Perbarui Panel**» (*Update Panel*) memulai `POST /updatePanel` . Tooltip: «Ini akan memperbarui 3X-UI ke rilis terbaru dan me-restart layanan panel». Sebelum memulai – konfirmasi «Apakah Anda benar-benar ingin memperbarui panel?» dengan teks «Ini akan memperbarui 3X-UI ke versi #version# dan me-restart layanan panel».

Keistimewaan dan keterbatasan:

- Pembaruan diri sendiri hanya didukung **di Linux** (pada OS lain, error dikembalikan).

- Skrip pembaruan diunduh dari repositori resmi (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh` , batas 2 MB) dan dijalankan melalui `bash` , sedapat mungkin diisolasi melalui `systemd-run` .
- Jika berhasil dimulai, ditampilkan «Pembaruan panel dimulai» (*Panel update started*); jika pengecekan pembaruan gagal – «Pengecekan pembaruan panel gagal». Selama instalasi, peringatan ditampilkan «Instalasi sedang berlangsung. Jangan muat ulang halaman».

3.13. Pembaruan File Geo (GeoIP / GeoSite)

Tombol/dialog pembaruan basis geo memanggil `POST /updateGeofile` (semua file) atau `POST /updateGeofile/:fileName` (satu file). Pembaruan bekerja berdasarkan daftar putih nama dan sumber yang ketat:

File	Sumber
<code>geoip.dat</code> , <code>geosite.dat</code>	<code>Loyalsoldier/v2ray-rules-dat</code> (latest)
<code>geoip_IR.dat</code> , <code>geosite_IR.dat</code>	<code>chocolate4u/iran-v2ray-rules</code> (latest)
<code>geoip_RU.dat</code> , <code>geosite_RU.dat</code>	<code>runetfreedom/russia-v2ray-rules-dat</code> (latest)

Perilaku:

- Nama file divalidasi: `..` , slash, path absolut dilarang; hanya `[a-zA-Z0-9._-]+.dat` yang diizinkan. File di luar daftar putih tidak diunduh.
- Permintaan kondisional `If-Modified-Since` digunakan: jika file di server sumber tidak berubah (HTTP 304), file tidak diunduh ulang, hanya timestamp yang diperbarui.
- Setelah pengunduhan, Xray **di-restart** (untuk memuat basis baru).

Contoh. Memperbarui hanya basis geo Rusia, tanpa menyentuh file lain:

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoip_RU.dat' -b cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b cookie.txt
```

Untuk memperbarui semua file dari daftar putih sekaligus – panggil `POST /updateGeofile` tanpa nama file.

- Dialog: «Apakah Anda benar-benar ingin memperbarui file geo?» dengan «Ini akan memperbarui file #filename#» untuk satu file dan «Ini akan memperbarui semua file geo» untuk tombol «Perbarui Semua». Sukses – «File geo berhasil diperbarui».

3.14. Pencadangan dan Pemulihan Basis Data

Blok «Cadangan & Pemulihan» (*Backup & Restore*). Perilaku bergantung pada DBMS yang digunakan (SQLite secara default atau PostgreSQL).

Ekspor Basis Data (Cadangan)

Tombol «Ekspor Basis Data» / «Cadangan» (*Back Up*) memanggil `GET /getDb`. File dikembalikan sebagai lampiran:

- **SQLite**: pertama dilakukan checkpoint (flush WAL), kemudian file `x-ui.db` diunduh. Tooltip: «Klik untuk mengunduh file .db yang berisi cadangan basis data Anda saat ini...».
- **PostgreSQL**: `pg_dump` diunduh dalam format kustom (`pg_dump --format=custom --no-owner --no-privileges`). Alat klien PostgreSQL harus terinstal di server; jika tidak – error tentang tidak adanya `pg_dump`.

Impor Basis Data (Pemulihan)

Tombol «Impor Basis Data» / «Pemulihan» (*Restore*) mengunggah file melalui `POST /importDB` (field form `db`). Tooltip: «Klik untuk memilih dan mengunggah file .db... untuk memulihkan basis data dari cadangan».

Skenario untuk **SQLite** aman, dengan rollback:

1. File diverifikasi formatnya sebagai SQLite dan disimpan ke file sementara, kemudian integritas diperiksa.
2. Xray dihentikan, basis data saat ini ditutup dan diubah namanya menjadi `*.backup` (fallback).
3. File baru menggantikan basis data kerja, inisialisasi dan migrasi dilakukan. Jika terjadi kesalahan – fallback dipulihkan.
4. Xray dijalankan kembali.

Untuk **PostgreSQL**, `.dump` diunggah (tanda tangan `PGDMP` diverifikasi) dan diterapkan melalui `pg_restore --clean --if-exists --single-transaction ...`. Tooltip secara eksplisit memperingatkan: «Ini akan menggantikan semua data saat ini».

Pesan: «Basis data berhasil diimpor», «Terjadi kesalahan saat mengimpor basis data», «...saat membaca basis data», «...saat mengambil basis data».

File Migrasi (antara SQLite dan PostgreSQL)

Tombol «Unduh File Migrasi» (*Download Migration*) memanggil `GET /getMigration` dan membentuk ekspor portabel untuk menjalankan panel pada DBMS lain:

- Pada **SQLite**, `x-ui.dump` (dump SQL teks) diunduh.
- Pada **PostgreSQL**, `x-ui.db` diunduh – basis data SQLite siap pakai yang dikumpulkan dari data PostgreSQL.

3.15. Elemen Antarmuka Tambahan

- **Indikator klien online**. Dasbor memelihara baris `online` (*Online Clients* / «Klien Online») – jumlah klien dengan koneksi aktif. Dihitung saat Xray berjalan (jika tidak – 0) dan dicatat ke riwayat pada interval 2 detik yang sama. Grafik – tab «Online».
- **Riwayat Sistem (grafik)**. Tombol/bagian «Grafik» → «Riwayat Sistem» dengan tab: «Bandwidth», «Paket», «Disk I/O», «Online», «Beban», «Koneksi», «Penggunaan Disk». Data diambil melalui `GET /history/:metric/:bucket`; interval agregasi yang diizinkan (bucket, detik): **2, 30, 60, 180, 360, 720**,

1440, 2880, 10080, hingga 60 titik dikirimkan per tab. Dalam pemilih rentang di halaman tersedia tombol **2m, 1h, 3h, 6h, 12h, 24h, 2d, 7d** (bucket **2, 60, 180, 360, 720, 1440, 2880, 10080** secara berurutan). Untuk rentang panjang **2d** dan **7d**, label waktu pada sumbu dilengkapi dengan tanggal dalam format **MM-DD HH:MM**. Penyimpanan diorganisir dengan rollup tiga tingkat: data baru disimpan dengan langkah 2 s selama **satu jam** terakhir, kemudian dirata-ratakan ke langkah 1 mnt selama **48 jam** dan ke langkah 10 mnt selama **7 hari**. Oleh karena itu, grafik (CPU, RAM, trafik, paket, koneksi, disk, online, beban) dapat dilihat untuk periode **hingga 7 hari** (sebelumnya – hingga 48 jam), dengan semakin jauh ke masa lalu, semakin kasar detailnya. Metrik yang diizinkan: **cpu, mem, swap, netUp, netDown, pktUp, pktDown, diskRead, diskWrite, diskUsage, tcpCount, udpCount, online, load1, load5, load15**. Label «2 menit terakhir» sesuai dengan bucket = 2 (mode real-time).

Contoh. Mendapatkan baris beban CPU untuk ~2 menit terakhir (bucket = 2 s, hingga 60 titik) dan baris yang sama diagregasi per 5 menit (bucket = 300 s):

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

Metrik dapat diganti dengan yang diizinkan lainnya (**mem, netUp, tcpCount, load1**, dll.). Bucket di luar daftar putih **2, 30, 60, 180, 360, 720, 1440, 2880, 10080** akan ditolak.

- **Metrik Xray** – blok terpisah dengan konsumsi memori dan garbage collection Xray (baris **xrAlloc, xrSys, xrHeapObjects, xrNumGC, xrPauseNs**) dan «Observatory» (status koneksi outbound). Berfungsi hanya jika blok **metrics** dikonfigurasi dalam konfigurasi Xray (**listen 127.0.0.1:11111, tag metrics_out**); jika tidak, ditampilkan «Endpoint metrik Xray tidak dikonfigurasi». Di jendela metrik Xray terdapat pemilih rentang tersendiri dengan tombol **2m, 1h, 3h, 6h, 12h** (bucket **2, 60, 180, 360, 720**).

Contoh blok yang mengaktifkan tile metrik Xray. Di bagian pengaturan Xray, **metrics** (dengan tag) dan inbound yang mendengarkan tag tersebut harus ada secara bersamaan:

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

Alamat **127.0.0.1:11111** sengaja tidak diekspos ke luar – panel mengaksesnya secara lokal.

- **Pengalih tema gelap.** Terletak di menu/header umum, bukan di dasbor itu sendiri. Pilihan: «Tema» (*Theme*) dengan opsi «Gelap» dan «Sangat Gelap» (*Ultra Dark*). Ini adalah pengaturan tampilan visual semata, tidak mempengaruhi fungsi panel.

- **Tautan lainnya** di sekitar dasbor (dari menu/panel bawah): «Log», «Konfigurasi» – melihat JSON Xray akhir (GET /getConfigJson), «Dokumentasi».
-

4. Inbounds: pembuatan dan parameter umum

Bagian **«Incoming»** (Inbounds) adalah daftar semua titik masuk Xray yang digunakan klien untuk terhubung. Setiap inbound menyimpan baik field "panel" (catatan, batas lalu lintas, jadwal reset) maupun blok JSON konfigurasi Xray mentah (`settings` , `streamSettings` , `sniffing`).

Pembuatan dilakukan dengan tombol **«Buat Koneksi»** (*Add Inbound*), pengeditan dengan **«Ubah Koneksi»** (*Modify Inbound*). Kedua operasi dikirimkan ke endpoint API `POST /add` dan `POST /update/:id`.

Di bawah ini dijelaskan semua field formulir yang **tidak** berkaitan dengan pengaturan protokol tertentu (klien, enkripsi, REALITY/TLS) dan **tidak** berkaitan dengan transport/stream (tab **«Stream»**, **«Security»**) – ini adalah topik bagian terpisah.

4.1. Field formulir umum

Remark (Catatan)

Parameter	Nilai
Field	<code>remark</code>
Tipe	string
Default	kosong

Nama inbound yang dapat dibaca manusia, ditampilkan dalam daftar dan di judul dialog (**«Hapus koneksi "{remark}"?»** dan sejenisnya). Label field – **«Catatan»**. Tidak mempengaruhi cara kerja Xray, hanya diperlukan untuk kemudahan administrasi; disarankan menggunakan nama unik yang bermakna karena nama tersebut digunakan dalam nama file yang diekspor dan dalam konfirmasi operasi massal.

Protocol (Protokol)

Parameter	Nilai
Field	<code>protocol</code>
Label	«Protokol»
Validasi	<code>required,oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun</code>

Daftar dropdown protokol inbound. Nilai yang diizinkan:

Nilai	Catatan
<code>vmess</code>	
<code>vless</code>	
<code>trojan</code>	

Nilai	Catatan
shadowsocks	
wireguard	
hysteria	Hysteria v2 – ini adalah hysteria dengan streamSettings.version = 2 ,tidak ada protokol terpisah
http	
mixed	socks/http pada satu port
tunnel	
tun	diterima oleh validator,tidak ada konstanta protokol tersendiri

Field ini wajib diisi (required). Pemilihan protokol menentukan field pengaturan klien dan transport mana yang akan tersedia (lihat bagian spesifik protokol).

Penting: saat menyimpan, layanan menormalkan streamSettings . Pengaturan transport hanya dipertahankan untuk protokol vmess , vless , trojan , shadowsocks , hysteria ; untuk yang lain (http , mixed , tunnel , wireguard , tun) field streamSettings **dihapus paksa**.

Untuk inbound bertipe tunnel /TPProxy yang blok streamSettings -nya tidak mengandung kunci security (varian tanpa transport), formulir dibuka dan disimpan tanpa error validasi streamSettings.security Invalid input .

Listen IP (IP yang didengarkan)

Parameter	Nilai
Field	listen
Tipe	string
Default	kosong → Xray mendengarkan pada 0.0.0.0 (semua IP)

Alamat IP tempat inbound menerima koneksi. Petunjuk field:

«Biarkan kosong untuk mendengarkan semua alamat IP».

Saat membuat konfigurasi Xray, nilai kosong diganti dengan 0.0.0.0 . Selain IP, field ini juga menerima **jalur Unix socket** – petunjuk:

«Anda juga dapat menentukan jalur Unix socket (misalnya, /run/xray/in.sock) atau nama abstract socket dengan awalan @ (misalnya, @xray/in.sock) untuk mendengarkan socket alih-alih port TCP – dalam hal ini tetapkan port 0».

Dengan demikian, field menerima dua bentuk Unix socket: jalur di sistem file (`/run/xray/in.sock`) dan nama abstract socket dengan awalan `@` (`@xray/in.sock`). Dalam kedua kasus, tetapkan `Port` ke `0` .

Field ini diubah ketika inbound perlu dibatasi pada satu antarmuka (misalnya, `127.0.0.1` untuk inbound yang hanya berfungsi sebagai target fallback di belakang Nginx) atau ketika inbound mendengarkan Unix socket.

Contoh. Inbound yang hanya mendengarkan antarmuka lokal (target fallback tipikal di belakang Nginx) dan Unix socket:

```
listen = 127.0.0.1    port = 8443
listen = /run/xray/in.sock    port = 0
```

Port (Port)

Parameter	Nilai
Field	<code>port</code>
Label	«Port»
Validasi	<code>gte=0,lte=65535</code>
Default	– (ditentukan oleh pengguna)

Port TCP/UDP yang didengarkan. Nilai yang diizinkan dari `0` hingga `65535` . Nilai `0` hanya digunakan berpasangan dengan mendengarkan pada Unix socket (lihat di atas).

Saat menyimpan, layanan memeriksa konflik port: dua inbound tidak dapat secara bersamaan menempati `listen:port` yang tumpang tindih untuk transport yang sama (TCP/UDP). Transport dihitung dari protokol dan `streamSettings / settings` : misalnya, `hysteria` dan `wireguard` selalu menempati UDP, `kcp / quic` – UDP, dan sebagian besar lainnya – TCP. Jika terjadi konflik, penyimpanan ditolak dengan error.

Secara terpisah, panel tidak mengizinkan penggunaan **port yang dicadangkan untuk API Xray internal** (tag `api` , default `62789` pada `127.0.0.1`): inbound TCP lokal yang alamat dengarnya tumpang tindih dengan port ini pada loopback ditolak dengan error konflik port yang sama. Port API aktual dibaca dari template konfigurasi Xray (dengan nilai fallback `62789`). Pada node, pembatasan ini tidak berlaku – node memiliki Xray sendiri.

Tag Xray (Tag , unik) dibuat secara otomatis dari port dan transport dalam format `in-<port>-<tcp|udp|tcpudp|any>` ; untuk inbound yang di-deploy pada node, ditambahkan awalan `n<nodeId>-` . Jika terjadi tabrakan, `-2` , `-3` , dan seterusnya ditambahkan ke tag. Pengguna biasanya tidak mengedit tag.

Total traffic (Total lalu lintas, GB)

Parameter	Nilai
Field	<code>total</code> (dalam byte)

Parameter	Nilai
Label	«Total penggunaan»
Default	0

Batas lalu lintas total inbound. Dalam formulir, nilai dimasukkan dalam gigabyte; di database disimpan dalam byte. Petunjuk field:

«= Tanpa batas. (satuan: GB)».

Artinya, **0 berarti tanpa batas**. Ini adalah batas di tingkat seluruh inbound (bukan klien individual); lalu lintas yang sebenarnya dikonsumsi disimpan di field `up` (dikirim) dan `down` (diterima) dan dibandingkan dengan `total`.

Expiry date / Duration (Tanggal kedaluwarsa / durasi)

Parameter	Nilai
Field	<code>expiryTime</code> (Unix timestamp)
Label	«Tanggal kedaluwarsa» (<i>Duration</i>)
Default	kosong / 0

Masa berlaku inbound. Petunjuk:

«Biarkan kosong agar tidak terbatas».

Nilai kosong (0) berarti inbound tanpa batas waktu. Nilai disimpan sebagai Unix timestamp; formulir memungkinkan pengaturan baik tanggal tertentu maupun durasi dalam hari (penghitungan relatif dari saat ini – label field *Duration*).

Enabled (Aktifkan)

Parameter	Nilai
Field	<code>enable</code>
Label	«Aktifkan» (<i>Enabled</i>)
Default	ditentukan saat pembuatan

Tanda aktifitas inbound. Peralihan flag ini dalam daftar diproses oleh endpoint "ringan" terpisah `POST /setEnable/:id`, bukan pembaruan penuh – ini dilakukan secara khusus agar tidak harus melakukan serialisasi ulang seluruh blok `settings` (semua klien) setiap kali toggle diklik pada inbound dengan ribuan klien. Saat dinonaktifkan, inbound dihapus dari Xray yang berjalan; saat diaktifkan – ditambahkan kembali.

Node / Deploy to (Node / Deploy ke)

Parameter	Nilai
Field	nodeId
Label	«Deploy ke», «Panel lokal»
Default	kosong (panel lokal)

Pilihan di mana inbound beroperasi secara fisik: pada panel lokal atau pada salah satu node yang terdaftar. Fitur implementasi: `nodeId = 0` dinormalisasi menjadi `nil`, karena `0` bukan id node yang valid, melainkan artefak binding formulir; `nil / 0` berarti panel lokal. Saat menyimpan inbound pada node yang offline, mungkin muncul toast «perubahan akan disinkronkan saat node terhubung kembali». Mulai versi 3.4.2, toast ini hanya ditampilkan ketika node benar-benar offline atau dimatikan (sebelumnya, toast dapat muncul juga saat menyimpan ke node online).

Strategi alamat untuk tautan (Share address strategy)

Parameter	Nilai
Field	strategi + (opsional) alamat kustom
Label	«Strategi alamat untuk tautan» (<i>Share address strategy</i>)
Default	«Alamat dengar inbound» (<i>Inbound listen</i>)

Daftar dropdown menentukan alamat mana yang dimasukkan ke dalam **tautan berbagi dan kode QR yang diekspor** dari inbound ini. Nilai:

Nilai	Label	Yang dimasukkan
node	«Alamat node» (<i>Node address</i>)	alamat node tempat inbound beroperasi
listen	«Alamat dengar inbound» (<i>Inbound listen</i>)	alamat dengar inbound itu sendiri
custom	«Kustom» (<i>Custom</i>)	alamat sendiri dari field «Alamat berbagi kustom» (<i>Custom share address</i>)

Saat memilih «Kustom», muncul field «Alamat berbagi kustom»; di sini dimasukkan host atau IP **tanpa skema dan port** (nilai divalidasi). Opsi «Alamat node» ditampilkan dalam daftar hanya jika ada node aktif yang dapat menjalankan inbound ini; jika tidak, opsi disembunyikan dan nilainya dikembalikan ke «Alamat dengar inbound».

Strategi ini mempengaruhi **hanya** tautan berbagi langsung dan kode QR. Ini **tidak** mempengaruhi output langganan – di sana alamat masih ditentukan oleh logika panel biasa.

4.2. Sniffing (Sniffing)

Tab «Sniffing» mengedit blok `sniffing` konfigurasi Xray, yang disimpan sebagai JSON mentah. Sniffing memungkinkan Xray "mengintip" nama domain/protokol nyata di dalam koneksi untuk keperluan routing.

Subfield	Label	Tujuan
<code>enabled</code>	(toggle tab)	Mengaktifkan/menonaktifkan sniffing untuk inbound
<code>destOverride</code>	–	Daftar protokol yang alamat tujuannya dicegat: <code>http</code> , <code>tls</code> , <code>quic</code> , <code>fakedns</code>
<code>metadataOnly</code>	«Hanya metadata»	Gunakan hanya metadata koneksi, tanpa membaca payload
<code>routeOnly</code>	«Hanya routing»	Terapkan hasil sniffing hanya untuk routing, tanpa menimpa alamat tujuan
<code>domainsExcluded</code>	«Domain yang dikecualikan»	Domain yang dikecualikan dari sniffing
(IP yang dikecualikan)	«IP yang dikecualikan»	Alamat IP yang dikecualikan dari sniffing

- **`destOverride`** – kumpulan sniffer: `http` (menentukan domain dari header HTTP Host), `tls` (dari SNI), `quic` (dari QUIC ClientHello), `fakedns` (pencocokan dengan pool FakeDNS). Biasanya `http` dan `tls` diaktifkan untuk menentukan domain.

Contoh blok sniffing (menentukan domain melalui HTTP dan TLS, gunakan hasil hanya untuk routing, jangan menyentuh jaringan lokal):

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **`metadataOnly`** – saat diaktifkan, Xray tidak membaca isi paket pertama dan hanya mengandalkan metadata; berguna agar tidak mengganggu protokol yang datanya tidak bisa "diintip".
- **`routeOnly`** – hasil sniffing hanya digunakan oleh aturan routing; alamat koneksi dalam outbound tidak ditimpa dengan domain yang terdeteksi.

Catatan: panel menyimpan `sniffing` sebagai blok JSON opak dan tidak menambahkan apapun saat menyimpan – semua nilai default untuk checkbox ini dibentuk di sisi aplikasi klien. Dalam bentuk mentah, blok dapat diedit melalui bagian «JSON inbound» (lihat di bawah).

4.3. Allocate (strategi alokasi port)

Blok `allocate` dalam `streamSettings` mengontrol bagaimana Xray mendistribusikan port yang didengarkan. Ini adalah bagian dari konfigurasi Xray; panel menyimpan dan meneruskannya sebagai bagian dari `streamSettings` /JSON inbound. Parameter (menurut terminologi Xray-core):

Subfield	Tujuan	Nilai / default
<code>strategy</code>	Strategi alokasi port	<code>always</code> – selalu dengarkan port yang ditentukan (default); <code>random</code> – ganti port yang didengarkan secara berkala dalam rentang

Subfield	Tujuan	Nilai / default
refresh	Interval pergantian port (menit) saat random	bilangan bulat menit (disarankan 5; minimum – 2)
concurrency	Berapa port yang dibuka secara bersamaan saat random	bilangan bulat (default 3; tidak lebih dari sepertiga lebar rentang port)

strategy: **always** membuat inbound tetap pada satu port (mode standar). strategy: **random** diperlukan untuk skenario anti-blocking ketika inbound secara berkala "melompat" melalui rentang port; dalam hal ini refresh dan concurrency menjadi relevan. Ubah nilai ini hanya saat menggunakan mode port acak secara sadar.

Contoh blok allocate dalam streamSettings (mode port acak: pertahankan 3 port terbuka, ganti setiap 5 menit):

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

Agar ini berfungsi, port inbound ditentukan sebagai rentang (misalnya, 20000–20100).

4.4. External Proxy (Proksi eksternal)

Field **«External Proxy»** berkaitan dengan pengaturan pembuatan tautan undangan dan disimpan dalam streamSettings inbound. Field ini menentukan daftar alamat eksternal alternatif (host/port, bila perlu dengan TLS paksa – **«TLS Paksa»**) yang dimasukkan ke dalam tautan klien alih-alih listen:port inbound yang sebenarnya.

Digunakan ketika klien harus terhubung bukan langsung ke server, tetapi melalui proksi/reverse/CDN eksternal: dalam hal ini tautan bersama menentukan alamat publik frontend tersebut. Ini tidak mempengaruhi proses penerimaan koneksi Xray itu sendiri – ini hanya "kosmetik" untuk tautan yang dihasilkan. Field formulir terkait: **«TLS Paksa»**, **«Fingerprint»**, label setiap entri.

4.5. Fallbacks (Fallback)

Bagian **«Fallback»** menentukan aturan pengalihan koneksi yang tidak cocok dengan klien inbound manapun. Tersedia untuk master inbound pada transport TLS (VLESS/Trojan TCP-TLS). Dikelola melalui endpoint GET /:id/fallbacks / POST /:id/fallbacks .

Petunjuk bagian:

«Ketika koneksi pada inbound ini tidak cocok dengan klien manapun, koneksi tersebut dialihkan ke tempat lain. Pilih inbound anak di bawah ini agar field routing (SNI / ALPN / Path / xver) terisi otomatis dari transportnya, atau biarkan pilihan kosong dan tentukan Dest secara langsung (misalnya, 8080 atau

127.0.0.1:8080) untuk mengalihkan ke server eksternal seperti Nginx. Setiap inbound anak harus mendengarkan pada 127.0.0.1 dengan security=none».

Bagian fallback hanya ditampilkan untuk inbound VLESS/Trojan di atas RAW (TCP) dengan keamanan TLS atau REALITY. Inbound baru dimulai dengan security=none, sehingga bagian ini mungkin terlihat tidak ada pada awalnya. Dalam kondisi ini (VLESS/Trojan, RAW/TCP, keamanan belum dikonfigurasi) alih-alih bagian tersebut ditampilkan petunjuk bawaan: fallback akan tersedia setelah memilih TLS atau Reality pada tab **«Security»**.

Field baris fallback

Field	Default	Deskripsi
(inbound anak)	—	Pilihan inbound anak (label «Pilih inbound»). Jika dipilih, field Name/Alpn/Path/Dest dapat terisi otomatis dari transportnya
Name	kosong (= sembarang)	Kondisi pencocokan berdasarkan nama (SNI/nama). Label "sembarang" — «sembarang»
Alpn	kosong	Kondisi pencocokan berdasarkan ALPN
Path	kosong	Kondisi pencocokan berdasarkan path (untuk transport WS/HTTP dari inbound anak)
Dest	otomatis	Ke mana dialihkan. Placeholder «otomatis (listen:port anak)» . Dapat menentukan port (8080) atau host:port (127.0.0.1:8080)
Xver	0	Versi PROXY protocol («Xver»): 0 — nonaktif, 1 atau 2 — versi PROXY protocol yang sesuai
(urutan)	berdasarkan posisi	Urutan penerapan aturan; ditentukan dengan tombol «Naik»/«Turun»

Logika penyimpanan: seluruh daftar fallback master diganti secara atomik. Baris yang tidak memiliki inbound anak yang dipilih (`childId <= 0`) maupun Dest yang ditentukan, **dilewati**. Jika inbound anak yang dipilih sama dengan id master, nilainya direset. Saat membuat JSON akhir: jika Dest kosong, dihitung dari inbound anak sebagai listen:port, di mana 0.0.0.0 / :: / ::0 diganti dengan 127.0.0.1; field name / alpn / path yang kosong tidak dimasukkan ke JSON output; xver hanya ditambahkan jika nilainya lebih dari 0.

Contoh settings.fallbacks akhir (lalu lintas dengan alpn=h2 diarahkan ke target WS pada path /ws, semua lainnya ke Nginx lokal pada port 8080):

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

Baris terakhir tanpa name / alpn / path adalah aturan "default" yang menangkap semua lainnya.

Tombol dan petunjuk bagian fallback

- **«Tambah fallback»** – tambah baris; **«Belum ada fallback»** – kondisi kosong.
- **«Tambah semua yang sesuai dengan cepat»** / **«Tambah semua»** – menambahkan baris fallback untuk setiap inbound yang sesuai yang belum terhubung. Hasilnya: «{n} fallback ditambahkan» atau «Tidak ada inbound baru yang sesuai».
- **«Isi dari anak»** – ambil ulang field routing (SNI/ALPN/Path/xver) dari transport inbound anak yang dipilih; setelah selesai – «Diisi dari anak».
- **«Ubah field routing»** / **«Sembunyikan lanjutan»** – tampilkan/sembunyikan field detail baris.
- Label **«Merutekan ketika»** dan **«Default – menangkap semua lainnya»** menjelaskan kondisi pemicuan setiap baris.

Setelah menyimpan fallback, server memanggil restart Xray agar `settings.fallbacks` baru berlaku.

4.6. Reset lalu lintas berkala

Blok **«Reset Lalu Lintas»** mengonfigurasi reset otomatis penghitung lalu lintas inbound sesuai jadwal. Deskripsi:

«Reset otomatis penghitung lalu lintas pada interval yang ditentukan».

Parameter	Nilai
Field	<code>trafficReset</code>
Validasi	<code>omitempty,oneof=never hourly daily weekly monthly</code>
Default	<code>never</code>
Field terkait	<code>lastTrafficResetTime</code> – timestamp reset terakhir (label «Reset terakhir»)

Daftar dropdown:

Nilai	Label
<code>never</code>	«Tidak pernah»
<code>hourly</code>	«Setiap jam»
<code>daily</code>	«Setiap hari»
<code>weekly</code>	«Setiap minggu»
<code>monthly</code>	«Setiap bulan»

Untuk setiap periode, cron job terdaftar yang berjalan sesuai jadwal yang sesuai (`@hourly` , `@daily` , `@weekly` , `@monthly`). Job memilih semua inbound dengan `trafficReset` yang ditentukan dan untuk setiap inbound mereset penghitung inbound itu sendiri (`up=0` , `down=0`) **dan** lalu lintas semua kliennya. Artinya, reset berkala mempengaruhi inbound dan klien-kliennya.

Contoh nilai field. Agar penghitung direset pada hari pertama setiap bulan, pilih «**Setiap bulan**» dalam formulir, yang disimpan sebagai:

```
{ "trafficReset": "monthly" }
```

Nilai `never` (default) sepenuhnya menonaktifkan auto-reset.

4.7. JSON inbound (lanjutan)

Bagian «**Bagian JSON inbound**» memberikan akses langsung ke blok JSON mentah inbound. Deskripsi:

«JSON lengkap inbound dan editor terpisah untuk settings, sniffing, dan streamSettings».

Editor yang tersedia:

Tab	Label	Yang diedit
Semua	«Objek inbound lengkap dengan semua field dalam satu editor»	seluruh objek Inbound
Pengaturan	«Wrapper blok settings Xray»	field <code>settings</code>
Sniffing	«Wrapper blok sniffing Xray»	field <code>sniffing</code>
Stream	«Wrapper blok stream Xray»	field <code>streamSettings</code>

Field-field ini diserialisasi sebagai objek JSON bersarang: blok kosong dikembalikan sebagai `null`, dan teks yang bukan JSON valid dibungkus dalam string agar data tidak hilang. Error parsing saat menyimpan ditampilkan dengan awalan «**JSON Lanjutan**».

Jendela tampilan «JSON inbound», seperti jendela impor inbound, menggunakan editor kode lengkap dengan penyorotan sintaks JSON (bukan field teks biasa): tampilan konfigurasi dalam mode baca saja dengan sorotan, sedangkan impor dalam mode yang dapat diedit, yang memudahkan membaca dan mengedit.

4.8. Tindakan pada inbound: QR / Edit / Reset / Delete dan statistik

Dalam daftar dan kartu inbound, tindakan berikut tersedia (menu «**Menu**»):

Statistik lalu lintas

Ditampilkan lalu lintas agregat inbound: «**Dikirim/diterima**» (field `up / down`), «**Total lalu lintas**», «**Total koneksi**». Dalam kartu juga — «**Dibuat**», «**Diperbarui**», «**Tanggal kedaluwarsa**».

Dalam daftar inbounds terdapat kolom **Speed** dengan kecepatan lalu lintas saat ini untuk setiap inbound (kirim/unduh), dihitung dari selisih penghitung antara polling; kecepatan langsung yang sama ditampilkan di jendela statistik inbound. Ketika polling berikutnya tidak menghasilkan selisih, nilai kecepatan direset.

Dalam ringkasan klien pada halaman inbounds, status ditentukan berdasarkan prioritas «habis/berakhir»: klien yang masa berlakunya habis atau lalu lintasnya habis (dan yang tugas auto-nya mencabut `enable`) masuk ke status «**Habis/Berakhir**» (*Depleted/Ended*), bukan ke «**Dinonaktifkan**» (*Disabled*) abu-abu, dan tidak dihitung dua

kali. Klasifikasi sesuai dengan yang ditampilkan di kartu klien itu sendiri, dan dengan benar memperhitungkan klien yang terikat ke beberapa inbound.

Kode QR dan penyalinan tautan

- **«Detail»** – membuka tautan koneksi dan langganan.
- Kode QR klien: petunjuk **«Klik kode QR untuk menyalin»**.
- **«Salin tautan»** (*Copy URL*), **«Ekspor tautan»**. Mulai versi 3.4.2, tindakan tingkat halaman **«Ekspor semua tautan»** (GET /panel/api/inbounds/allLinks ; jendela «Ekspor semua tautan inbound», file «All-Inbounds») membentuk tautan melalui mesin langganan – menerapkan template remark ke setiap klien dan memilih endpoint Host terkelola (sebelumnya digunakan remark tetap inbound-email).

Edit (Ubah)

«Ubah koneksi» – membuka formulir pengeditan (POST /update/:id). Saat memperbarui, layanan membaca ulang baris yang ada, mentransfer field yang diubah, bila perlu membuat ulang tag (jika tag lama dibuat otomatis) dan menyinkronkan runtime Xray. Sukses – toast **«Koneksi berhasil diperbarui»**.

Reset Traffic (Reset lalu lintas)

«Reset lalu lintas» – mereset penghitung up / down inbound ini (POST /:id/resetTraffic , menetapkan up=0, down=0). Konfirmasi:

«Reset lalu lintas "{remark}"?» / «Mereset penghitung pengiriman/penerimaan koneksi ini ke 0».

Reset lalu lintas inbound **tidak** menyentuh penghitung klien-kliennya (untuk itu ada tindakan «Reset lalu lintas klien» terpisah). Setelah reset, restart Xray dimulai. Sukses – toast **«Lalu lintas masuk direset»**. Ada juga varian massal – **«Reset lalu lintas semua koneksi»** (POST /resetAllTraffics).

Delete (Hapus)

«Hapus koneksi» (POST /del/:id). Konfirmasi:

«Hapus koneksi "{remark}"?» / «Koneksi dan semua kliennya akan dihapus. Tindakan ini tidak dapat dibatalkan».

Penghapusan melepas inbound dari Xray yang berjalan (bila perlu dengan restart). Sukses – toast **«Koneksi berhasil dihapus»**. Penghapusan massal – POST /bulkDel , dengan pelaporan per elemen dan tidak lebih dari satu restart Xray.

Tindakan lain pada klien inbound

Dalam menu juga tersedia: **«Klon»** (salinan inbound dengan port baru dan daftar klien kosong), **«Hapus semua klien»** (POST /:id/deleteAllClients – menghapus semua klien, inbound itu sendiri dipertahankan), **«Hapus**

klien yang dinonaktifkan», «Ikat/Lepas ikat klien», «Impor»/«Ekspor koneksi» (POST /import). Detail operasi klien termasuk dalam bagian tentang klien.

5. Protokol

Saat membuat inbound, langkah pertama adalah memilih **Protokol** («Protocol»). Protokol menentukan metode autentikasi dan enkripsi lalu lintas yang akan diterapkan Xray-core pada inbound tersebut, kumpulan field di `settings` yang perlu diisi, serta transport (`network`) dan jenis keamanan (TLS / REALITY) yang tersedia untuknya.

Field protokol ditetapkan sekali saat pembuatan inbound dan **tidak dapat diubah saat pengeditan** (daftar dropdown dinonaktifkan di formulir pengeditan). Untuk mengganti protokol, perlu membuat inbound baru.

5.1. Daftar Protokol yang Didukung

Server menerima kumpulan nilai field `Protocol` berikut:

```
oneof=vless vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto
```

Mulai versi **3.3.0**, nilai `mtproto` (proksi Telegram) ditambahkan ke daftar.

Nilai dalam config	Tujuan	Model klien
<code>vless</code>	Protokol proksi utama (default saat membuat inbound)	Klien dengan UUID, dukungan flow dan enkripsi pasca-kuantum
<code>vmess</code>	Protokol proksi klasik Xray	Klien dengan UUID dan parameter <code>security</code>
<code>trojan</code>	Proksi yang menyamar sebagai HTTPS biasa	Klien dengan kata sandi
<code>shadowsocks</code>	Proksi Shadowsocks (termasuk SIP022 / 2022-blake3)	Satu pengguna atau beberapa (2022)
<code>wireguard</code>	Inbound WireGuard	Peer (bukan klien)
<code>hysteria</code>	Inbound Hysteria (default versi 2)	Klien dengan token <code>auth</code>
<code>http</code>	Proksi HTTP klasik (forward proxy)	Akun user/pass, tanpa pencatatan lalu lintas
<code>mixed</code>	Proksi SOCKS + HTTP gabungan	Akun user/pass
<code>tunnel</code>	Forwarder transparan (xray <code>dokodemo-door</code>)	Tanpa klien
<code>tun</code>	Antarmuka TUN (hanya rendering yang sudah ada)	Tanpa klien
<code>mtproto</code>	Proksi Telegram (MTPROTO), ditambahkan di 3.3.0; dilayani oleh proses <code>mtg</code> terpisah, bukan Xray	Tanpa klien (akses melalui secret)

Catatan tentang `tun` : nilai ini dipertahankan dalam daftar untuk kompatibilitas dan **penampilan** inbound yang tersimpan sebelumnya, namun pada versi terkini backend tidak merekomendasikan

pembuatannya – dukungannya dianggap usang. Membuat inbound baru bertipe ini tidak memiliki makna.

Catatan tentang Hysteria 2: tidak ada protokol «hysteria2» yang terpisah. Ini adalah protokol `hysteria` dengan field `streamSettings.version = 2`. Skema link `hysteria2://` saat pembuatan share-link dipilih secara otomatis ketika versi stream sama dengan 2.

Tidak semua protokol mendukung distribusi ke node. Hanya protokol berikut yang dapat dideploy ke node: `vless`, `vmess`, `trojan`, `shadowsocks`, `hysteria`, `wireguard`. Protokol `http`, `mixed`, `tunnel`, `tun`, `mtproto` hanya berfungsi di panel lokal.

5.2. Protokol Mana yang Mendukung TLS / REALITY / Transport

Kemampuan untuk mengaktifkan lapisan keamanan atau transport tertentu bergantung pada protokol dan jaringan yang dipilih (`streamSettings.network`):

Kemampuan	Tersedia untuk protokol	Jaringan yang diizinkan (network)
TLS	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> (serta selalu untuk <code>hysteria</code>)	<code>tcp</code> , <code>ws</code> , <code>http</code> , <code>grpc</code> , <code>httpupgrade</code> , <code>xhttp</code>
REALITY	<code>vless</code> , <code>trojan</code>	<code>tcp</code> , <code>http</code> , <code>grpc</code> , <code>xhttp</code>
flow (xtls-rprx-vision)	hanya <code>vless</code>	hanya <code>tcp</code> , dengan <code>security = tls</code> atau <code>reality</code>
Stream / transport (tab «Aliran»)	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> , <code>hysteria</code>	–

Untuk protokol `http`, `mixed`, `tunnel`, `tun`, `wireguard`, tab transport tidak tersedia – protokol-protokol ini tidak memiliki pengaturan stream Xray.

5.3. VLESS

Tujuan: protokol proksi modern utama. Mendukung XTLS-Vision (`flow`), REALITY, serta enkripsi pasca-kuantum pada level VLESS itu sendiri (field `decryption` / `encryption`). Digunakan secara default untuk inbound baru.

Field blok `settings` :

Field	Nilai default	Deskripsi
<code>clients</code>	<code>[]</code>	Daftar klien. Setiap klien memiliki: <code>id</code> (UUID), <code>email</code> (wajib), <code>flow</code> , batas (<code>limitIp</code> , <code>totalGB</code> , <code>expiryTime</code>), <code>enable</code> , <code>tgId</code> , <code>subId</code> , <code>comment</code> , <code>reset</code>
<code>decryption</code>	<code>none</code>	Parameter dekripsi di sisi server. Label di UI: «Расшифрование» (bhs. Inggris «Decryption»)

Field	Nilai default	Deskripsi
encryption	none	Parameter enkripsi berpasangan (masuk ke link klien). Label: «Шифрование» (bhs. Inggris «Encryption»)
fallbacks	[]	Daftar fallback (lihat bagian tentang fallback); tersedia ketika <code>network = tcp</code> dan <code>security = TLS</code> atau REALITY
testseed	(4 angka: 900, 500, 900, 256)	«Vision testseed» – 4 bilangan bulat positif untuk XTLS-Vision padding. Hanya diterapkan pada klien dengan flow <code>xtls-rprx-vision</code> , jika tidak diabaikan

flow (xtls-rprx-vision)

flow ditetapkan **pada klien**, bukan pada inbound, dan menerima salah satu dari tiga nilai:

Nilai	Makna
"" (kosong)	Tanpa XTLS-flow (default)
xtls-rprx-vision	XTLS-Vision – mode yang direkomendasikan untuk VLESS di atas TCP+TLS/REALITY
xtls-rprx-vision-udp443	Vision yang sama, tetapi dengan pemrosesan UDP/443 (QUIC)

Field `flow` hanya dapat dipilih ketika semua kondisi terpenuhi: protokol `vless`, `network = tcp`, dan `security = tls` atau `reality`. Field **Vision testseed** di formulir hanya ditampilkan pada kondisi yang sama.

Pengecualian untuk XHTTP: pada VLESS di atas `network = xhttp` dengan autentikasi pasca-kuantum VLESS yang diaktifkan (`encryption / decryption, vlessenc`), `flow xtls-rprx-vision` juga diizinkan – terlepas dari lapisan keamanan, termasuk dengan REALITY. Dalam kasus ini panel meneruskan `xtls-rprx-vision` dengan benar ke share-link dan ke langganan (termasuk format Clash/Mihomo), sehingga klien mendapatkan konfigurasi dengan Vision.

Dekripsi / Enkripsi (autentikasi pasca-kuantum VLESS)

Field `decryption` dan `encryption` adalah autentikasi pada level VLESS itu sendiri (terpisah dari transport TLS/REALITY). Secara default keduanya bernilai `none`. Di formulir di bawah field-field ini terdapat blok «**Pembuatan Kunci**» – daftar dropdown mode dan tombol «**Buat**» (di sampingnya – tombol «**Hapus**»). Daftar dropdown berisi enam pilihan: **X25519 (native)**, **X25519 (xorpub)**, **X25519 (random)**, **ML-KEM-768 (native)**, **ML-KEM-768 (xorpub)**, **ML-KEM-768 (random)** – yaitu dua jenis kunci (X25519 klasik dan ML-KEM-768 pasca-kuantum), masing-masing dalam tiga mode:

- **native** – pasangan kunci dasar dari jenis yang dipilih;
- **xorpub** – mode turunan dengan pemrosesan tambahan pada bagian publik;
- **random** – mode turunan dengan komponen acak.

Pilih mode yang diperlukan dari daftar dan klik «**Buat**»: panel akan mengisi **kedua** field (`decryption` dan `encryption`) dengan pasangan nilai siap pakai untuk mode tersebut. Tombol «**Hapus**» menyetel ulang kedua field kembali ke `none`.

Di bawah blok ditampilkan baris status «**Dipilih: ...**», yang mengenali dari string yang dihasilkan baik jenis kunci (X25519 atau ML-KEM-768) maupun mode (native / xorpub / random) dan menampilkannya. Field kosong atau none ditampilkan sebagai «None».

Secara teknis tombol-tombol memanggil `GET /panel/api/server/getNewVlessEnc` (pembuatan kunci melalui `xray vlessenc`) dan mengisi **kedua** field dengan nilai berpasangan dalam bentuk `mlkem768x25519plus.native.<rtt>.<role>` (misalnya, `decryption = mlkem768x25519plus.native.600s.server-x25519`, `encryption = mlkem768x25519plus.native.0rtt.client-x25519`). Parameter `decryption` tetap di server, `encryption` dikirim ke link klien.

Penting: saat menghasilkan konfigurasi inbound untuk Xray, panel menghapus yang berlebihan: jika `encryption` (yang merupakan sisi klien) tersisa di `settings`, maka **dipotong** dari konfigurasi server. Di server sendiri hanya `decryption` yang tersisa.

Kapan memilih VLESS: ini adalah pilihan default yang direkomendasikan untuk inbound baru, terutama dalam kombinasi dengan REALITY (tanpa sertifikat sendiri) atau dengan TLS + XTLS-Vision.

Contoh: blok `settings` VLESS-inbound dengan satu klien dan XTLS-Vision. Field `flow` ada pada klien, `decryption` tetap di server:

```
{
  "clients": [
    {
      "id": "d342d11e-d424-4583-b36e-524ab1f0afa4",
      "email": "user1",
      "flow": "xtls-rprx-vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

Untuk kombinasi REALITY, blok `streamSettings` yang sesuai (tab «Transport» → Security: REALITY) terlihat seperti ini:

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<kunci privat X25519>",
    "shortIds": [ "", "6ba85179e30d4fc2" ]
  }
}
```

5.4. VMess

Tujuan: protokol proksi klasik Xray. Autentikasi menggunakan UUID, dan pada klien dikonfigurasi tambahan metode enkripsi payload (security).

Field blok settings :

Field	Nilai default	Deskripsi
clients	[]	Daftar klien

Setiap klien VMess (selain field umum email , batas , enable , tgId , subId , comment , reset):

Field klien	Nilai default	Deskripsi
id	–	UUID klien
security	auto	Metode enkripsi payload VMess. Nilai yang diizinkan: aes-128-gcm , chacha20-poly1305 , auto , none , zero

Nilai security :

- auto – Xray memilih cipher sendiri tergantung platform (direkomendasikan);
- aes-128-gcm , chacha20-poly1305 – cipher AEAD yang tetap;
- none – tanpa enkripsi payload (hanya masuk akal di atas TLS);
- zero – tanpa enkripsi dan tanpa autentikasi payload.

Kompatibilitas historis: catatan lama mungkin menyimpan security: "" – saat dibaca, string kosong dikonversi ke auto . Saat menghasilkan konfigurasi server, field security pada klien VMess **dihapus** dari settings , karena tidak diperlukan untuk inbound.

Kapan memilih VMess: untuk kompatibilitas dengan klien lama atau konfigurasi yang sudah ada. Untuk deployment baru, VLESS biasanya lebih disukai.

5.5. Trojan

Tujuan: proksi yang meniru lalu lintas HTTPS biasa. Autentikasi menggunakan kata sandi. Seperti VLESS, mendukung fallback dan (dengan network = tcp) REALITY/TLS.

Field blok settings :

Field	Nilai default	Deskripsi
clients	[]	Daftar klien
fallbacks	[]	Daftar fallback (tersedia dengan network = tcp dan TLS/REALITY)

Pada setiap klien Trojan, field utamanya adalah:

Field klien	Nilai default	Deskripsi
password	–	Kata sandi klien (wajib, minimal 1 karakter)
email	–	Identifikasi unik klien

Field klien lainnya bersifat umum (`limitIp` , `totalGB` , `expiryTime` , `enable` , `tgId` , `subId` , `comment` , `reset`).

Kapan memilih Trojan: ketika diperlukan penyamaran sebagai HTTPS di port 443, termasuk dengan fallback ke web server (Nginx) untuk koneksi yang tidak diminta.

Contoh: blok `settings` Trojan dengan fallback ke web server lokal. Koneksi yang tidak diminta (tanpa kata sandi yang valid) diarahkan ke Nginx yang mendengarkan di `127.0.0.1:8080` :

```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

Untuk fallback diperlukan `network = tcp` dan Security = TLS atau REALITY; jika tidak, field fallbacks tidak tersedia.

5.6. Shadowsocks

Tujuan: proksi Shadowsocks ringan. Mendukung cipher AEAD lama maupun metode SIP022 baru (`2022-blake3-*`). Dapat beroperasi dalam mode pengguna tunggal atau multi-pengguna.

Field blok `settings` :

Field	Nilai default	Deskripsi
method	<code>2022-blake3-aes-256-gcm</code>	Metode enkripsi inbound. Label di UI: «Метод шифрования» (bhs. Inggris «Encryption method»)
password	""	Kata sandi inbound (untuk metode 2022 dihasilkan secara otomatis sesuai metode yang dipilih)
network	<code>tcp,udp</code>	Transport. Label: «Сеть» (bhs. Inggris «Network»). Pilihan: <code>tcp,udp</code> (TCP, UDP), <code>tcp, udp</code>
clients	<code>[]</code>	Daftar klien
ivCheck	<code>false</code> (nonaktif)	Toggle «ivCheck» – perlindungan terhadap penggunaan ulang IV

Metode enkripsi (`method`)

Kumpulan yang diizinkan:

Metode	Kategori
aes-256-gcm	AEAD lama
chacha20-poly1305	AEAD lama
chacha20-ietf-poly1305	AEAD lama
xchacha20-ietf-poly1305	AEAD lama
2022-blake3-aes-128-gcm	SS 2022 (direkomendasikan)
2022-blake3-aes-256-gcm	SS 2022 (default)
2022-blake3-chacha20-poly1305	SS 2022, pengguna tunggal

Logika panel berdasarkan metode:

- **Metode 2022** (2022-blake3-*) dianggap «SS 2022». Metode 2022-blake3-chacha20-poly1305 – **pengguna tunggal** (multi-user tidak didukung); metode 2022 lainnya mengizinkan beberapa klien. Field kata sandi (dengan tombol generate yang menyesuaikan panjang kunci dengan metode) ditampilkan dalam formulir khusus untuk metode 2022.
- **Cipher lama** (aes-* , chacha20-*) beroperasi dengan skema klasik «satu metode + satu kata sandi».

Normalisasi sebelum menjalankan Xray: untuk cipher lama, setiap klien harus memiliki `method` yang cocok dengan metode inbound (jika tidak, Xray gagal dengan «unsupported cipher method»). Untuk metode 2022 sebaliknya – field `method` pada klien harus **kosong** (jika tidak, Xray menolak inbound dengan «users must have empty method»). Panel secara otomatis merapikan data saat beralih metode.

Regenerasi kunci klien saat ukuran kunci berubah: untuk Shadowsocks-2022, saat metode enkripsi diganti dengan metode yang memiliki ukuran kunci berbeda (misalnya antara 2022-blake3-aes-256-gcm dan 2022-blake3-aes-128-gcm), panel secara otomatis meregenerasi PSK klien untuk panjang baru saat menyimpan inbound. Jika tidak, kunci lama akan tetap berukuran panjang yang lama, dan Xray akan menolaknya. Konsekuensinya: klien yang terdampak perlu mendapatkan langganan baru – link sebelumnya tidak akan bisa terhubung.

Kapan memilih Shadowsocks: untuk deployment sederhana tanpa penyamaran TLS; pilihan modern – metode 2022-blake3-* .

Contoh: blok `settings` Shadowsocks untuk metode 2022-blake3 (mode multi-pengguna). Inbound memiliki kata sandinya sendiri (kunci base64 dengan panjang yang sesuai), setiap klien memiliki kata sandinya sendiri, field `method` klien **kosong**:

```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlNjQta2V5LWlcmU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zMilieXRlcy1iYXNlNjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

Untuk cipher lama (`aes-256-gcm` dan sejenisnya) – kebalikannya: satu kata sandi untuk inbound, dan `method` klien harus cocok dengan metode inbound.

5.7. Dokodemo-door / Tunnel (forwarder transparan)

Tujuan: forwarder transparan (dalam panel – protokol `tunnel` , yang mengimplementasikan perilaku `dokodemo-door`). Menerima lalu lintas dan meneruskannya ke alamat/port tertentu, tanpa autentikasi dan klien.

Field blok `settings` :

Field	Nilai default	Deskripsi
<code>rewriteAddress</code>	(tidak ada)	«Tulis ulang alamat» (bhs. Inggris «Rewrite address») – alamat tujuan kemana lalu lintas diarahkan
<code>rewritePort</code>	(tidak ada)	«Tulis ulang port» (bhs. Inggris «Rewrite port») – port tujuan (0–65535)
<code>allowedNetwork</code>	<code>tcp,udp</code>	«Jaringan yang diizinkan» (bhs. Inggris «Allowed network»). Pilihan: <code>tcp,udp</code> , <code>tcp</code> , <code>udp</code>
<code>portMap</code>	<code>{}</code>	«Pemetaan port» – peta port→port (Record<string,string>)
<code>followRedirect</code>	<code>false</code> (nonaktif)	«Ikuti redirect» (bhs. Inggris «Follow redirect») – gunakan alamat tujuan asli dari koneksi yang dicegat

Tab «Transport» untuk Tunnel: pada inbound jenis ini, tab **«Transport»** tersedia, terbatas pada pengaturan `sockopt` – ini cukup untuk mode **TProxy** (transparent proxying/redirect melalui `sockopt.tproxy`). Daftar dropdown pemilihan transport (`network`) dan tab «Security» untuk Tunnel disembunyikan, karena TLS/REALITY tidak didukung oleh jenis ini.

Kapan memilih: untuk transparent proxying/pengalihan port ke layanan internal.

Field «Tulis ulang port» (`rewritePort`) dapat dikosongkan: saat dihapus, nilainya hanya dikecualikan dari pengaturan inbound, dan tidak menyebabkan kesalahan penyimpanan. (Sebelumnya, mengosongkan field ini menyebabkan error validasi `settings.rewritePort` dan memblokir penyimpanan, termasuk melalui tab JSON.)

5.8. SOCKS / HTTP (protokol `mixed`)

Dalam build ini tidak ada protokol `socks` terpisah – SOCKS dan HTTP-proksi digabungkan dalam protokol `mixed` (SOCKS + HTTP gabungan). Selain itu, ada `http` -proksi murni yang terpisah.

5.8.1. Mixed (SOCKS + HTTP)

Field blok `settings` :

Field	Nilai default	Deskripsi
<code>auth</code>	<code>password</code>	«Auth» – mode autentikasi. Pilihan: <code>password</code> (dengan login/kata sandi) atau <code>noauth</code> (tanpa otorisasi)
<code>accounts</code>	(opsional)	«Akun» – daftar pasangan user/pass. Dengan <code>auth = noauth</code> , field tidak ditulis ke config
<code>udp</code>	<code>false</code> (nonaktif)	Toggle «UDP» – dukungan UDP melalui SOCKS
<code>ip</code>	<code>127.0.0.1</code>	«UDP IP» – alamat lokal untuk asosiasi UDP. Field hanya ditampilkan saat <code>udp</code> diaktifkan

Akun ditambahkan dengan tombol «Tambah»; saat penambahan, login acak (8 karakter) dan kata sandi (12 karakter) dihasilkan, yang dapat diedit.

5.8.2. HTTP (proksi murni)

Tujuan: forward proxy HTTP klasik. Pada level Xray tidak melacak klien sebagai «billing» (tidak ada email/batas) – hanya ada daftar akun.

Field blok `settings` :

Field	Nilai default	Deskripsi
<code>accounts</code>	<code>[]</code>	«Akun» – daftar pasangan user/pass (kedua field wajib)
<code>allowTransparent</code>	<code>false</code> (nonaktif)	«Izinkan transparan» (bhs. Inggris «Allow transparent») – teruskan permintaan dengan header Host asli

Kapan memilih SOCKS/HTTP: untuk akses proksi lokal atau layanan tanpa penyamaran yang rumit. `mixed` praktis karena satu port melayani klien SOCKS maupun HTTP.

5.9. WireGuard (inbound)

Tujuan: inbound WireGuard – tunnel VPN WireGuard, bukan proksi yang disamarkan. Transport dan TLS/REALITY tidak berlaku untuknya.

Mulai versi 3.4.2, WireGuard dialihkan ke model multi-klien (seperti VLESS/VMess/Trojan/Shadowsocks/Hysteria). Inbound itu sendiri hanya menyimpan bagian server; perangkat-peer kini ditambahkan sebagai **klien** biasa (lihat bagian 8) – dengan penetapan alamat tunnel otomatis, dukungan langganan, batas lalu lintas/ masa berlaku, dan grup. Daftar inline «Peer / Tambah peer» yang lama dari formulir inbound dihapus: inbound baru dibuat tanpa peer, dan setiap klien diberi alamat secara otomatis.

Field inbound (blok `settings`):

Field	Nilai default	Deskripsi
<code>secretKey</code>	–	Kunci privat server (wajib). Di sampingnya ada tombol generate; kunci publik (<code>Public Key</code>) ditampilkan secara otomatis darinya (hanya baca; <code>pubKey</code> yang sebelumnya disimpan terpisah telah dihapus)
<code>mtu</code>	1420	MTU antarmuka
<code>dns</code>	1.1.1.1, 1.0.0.1	Baru di 3.4.2. DNS yang ditulis ke baris <code>DNS =</code> pada <code>.conf</code> klien
<code>noKernelTun</code>	false (nonaktif)	«TUN tanpa kernel» – userspace-TUN alih-alih kernel
<code>domainStrategy</code>	(opsional)	«Domain Strategy»: <code>ForceIP</code> , <code>ForceIPv4</code> , <code>ForceIPv4v6</code> , <code>ForceIPv6</code> , <code>ForceIPv6v4</code>

Parameter WireGuard pada klien – tab «**Kredensial**» di jendela penambahan/pengeditan klien (ditampilkan ketika inbound yang terikat adalah WireGuard):

Field	Deskripsi
«Kunci privat WireGuard»	Kunci privat klien (dapat diedit, ada tombol «Hasilkan ulang»); saat dimasukkan, kunci publik dihitung otomatis darinya
«Kunci publik WireGuard»	Hanya baca; dihitung dari kunci privat
«Kunci bersama WireGuard» (PSK)	Kunci bersama tambahan yang opsional
«IP yang diizinkan WireGuard»	Hanya baca (dalam mode edit): alamat tunnel yang diberikan kepada klien, misalnya <code>10.0.0.2/32</code>

Alamat tunnel diberikan oleh server secara otomatis dari subnet inbound (default `10.0.0.0/24` : server menempati `.1`, klien – mulai dari `.2`); jika klien yang ada menggunakan `/24` lain, alamat baru diberikan di subnet yang sama.

Domain Strategy dan tab Transport

Selain yang disebutkan di atas, inbound WireGuard memiliki field **Domain Strategy** (strategi resolusi domain: `ForceIP` , `ForceIPv4` , `ForceIPv4v6` , `ForceIPv6` , `ForceIPv6v4`). Field ini opsional dan ditulis ke config hanya jika ditetapkan.

Field **Workers** (`workers` , jumlah thread pekerja) telah dihapus dari formulir WireGuard (baik inbound maupun outbound): mulai dari xray-core v26.6.22, engine tidak lagi menggunakannya dan mengandalkan

mekanisme internal wireguard-go. Konfigurasi yang tersimpan sebelumnya berfungsi tanpa perubahan – saat parsing, field hanya diabaikan, tidak perlu migrasi.

Untuk WireGuard juga tersedia tab **«Transport»** – tetapi dalam bentuk terbatas: hanya `sockopt` dan obfuskasi **Finalmask** yang dikonfigurasi di dalamnya. Daftar dropdown pemilihan transport (`network`) disembunyikan, karena WireGuard selalu mendengarkan via UDP. Dalam catatan noise (noise), Finalmask memiliki field terpisah **Rand Range** (rentang byte 0–255, dengan validasi), dan sebagai metode obfuskasi untuk WireGuard dan Hysteria tersedia **Salamander**.

Konfigurasi dan sharing klien WireGuard

Pada klien WireGuard, di jendela «Informasi»/QR tersedia **kartu konfigurasi yang dapat dilipat**: `.conf` lengkap ([Interface] dengan PrivateKey / Address / DNS / MTU dan [Peer] dengan PublicKey / PresharedKey / AllowedIPs = `0.0.0.0/0, ::/0` / Endpoint / PersistentKeepalive) dengan tombol **Salin**, **Unduh**, dan **QR**. Didukung pula tautan berbentuk `wireguard:// / wg://`, yang diurai oleh aplikasi klien menjadi `.conf` siap pakai.

Kapan memilih WireGuard: ketika diperlukan tunnel VPN WireGuard yang sesungguhnya, bukan proksi yang disamarkan.

5.10. Hysteria (default v2)

Tujuan: inbound Hysteria di atas QUIC. Panel secara default beroperasi dengan versi 2. Setiap klien diautentikasi dengan token `auth` alih-alih UUID/kata sandi. TLS untuk Hysteria selalu tersedia (lihat tabel kemampuan di 5.2).

Field blok `settings` :

Field	Nilai default	Deskripsi
<code>version</code>	<code>2</code>	Versi protokol (minimal 1; panel default 2)
<code>clients</code>	<code>[]</code>	Daftar klien

Pada setiap klien, field utamanya adalah `auth` (token, wajib) ditambah field umum (`email` , `batas` , `enable` , `tgId` , `subId` , `comment` , `reset`).

Parameter tambahan ditetapkan di `streamSettings.hysteriaSettings` :

Field	Nilai / pilihan	Deskripsi
<code>version</code>	ditetapkan 2 (field dikunci)	«Versi» (bhs. Inggris «Version»)
<code>udpIdleTimeout</code>	(bilangan bulat ≥ 1 , detik)	«UDP idle timeout (s)» – timeout tidak aktif UDP
<code>masquerade</code>	dinonaktifkan secara default	«Masquerade» – penyamaran sebagai web server biasa saat menerima permintaan «yang tidak diminta»

Saat mengaktifkan `masquerade`, tersedia pilihan tipe (`type`):

- `""` – default (halaman 404);
- `proxy` – reverse proxy (field «Upstream URL», «Tulis ulang Host», «Lewati TLS verify»);
- `file` – melayani direktori (field «Direktori», misalnya `/var/www/html`);
- `string` – respons tetap (field «Kode status», «Body», «Header»).

Kapan memilih Hysteria: ketika diperlukan transport QUIC dan ketahanan pada saluran yang tidak stabil/mobile; `masquerade` meningkatkan kerahasiaan titik masuk.

5.11. MTProto (proksi untuk Telegram)

Ditambahkan dalam versi **3.3.0**. Nilai protokol – `mtproto`.

MTProto adalah protokol proksi bawaan Telegram. Di 3X-UI, inbound semacam ini **dilayani bukan oleh Xray, melainkan oleh proses `mtg` terpisah**, yang dikelola oleh panel itu sendiri. Panel secara berkala mencocokkan inbound MTProto yang diaktifkan dengan proses `mtg` yang berjalan: menjalankan yang kurang, menghentikan yang berlebih, dan mengambil penghitung lalu lintas dari metrik `mtg`. Oleh karena itu **pencatatan lalu lintas** pada inbound semacam ini berfungsi seperti protokol biasa.

Petunjuk resmi dalam formulir:

«MTProto dilayani oleh proses `mtg` terpisah, bukan Xray. Pengaturan transport dan klien tidak berlaku di sini – bagikan link di bawah ini di Telegram.»

Konsekuensinya:

- Tab «**Transport**» (**Stream Settings**) dan «**Klien**» tidak berlaku untuk inbound ini – akses ditetapkan oleh satu secret, bukan daftar klien.
- Inbound MTProto berjalan **hanya di panel utama**; ke node anak tidak dideploy (inbound dengan `NodeID` tertentu dilewati).
- Tab «**Sniffing**» untuk MTProto disembunyikan – protokol ini dilayani oleh proses `mtg`, bukan Xray, sehingga sniffing tidak berlaku untuknya.

Field. Disimpan di `settings` inbound:

Field di UI	Kunci	Deskripsi
Remark	<code>remark</code>	Label inbound.
Listen IP	<code>listen</code>	IP untuk mendengarkan (kosong = semua antarmuka).
Port	<code>port</code>	Port proksi.
Secret	<code>settings.secret</code>	Secret akses dalam format FakeTLS .

Field di UI	Kunci	Deskripsi
Domain penyamaran (FakeTLS)	<code>settings.fakeTlsDomain</code>	Domain yang lalu lintasnya ditiru oleh proksi sebagai HTTPS.

Format secret (FakeTLS). Panel secara otomatis menyesuaikan secret ke bentuk yang benar: hasilnya = `ee` + 32 karakter hex + kode hex domain penyamaran, yaitu `ee<hex32><hex(fakeTlsDomain)>`. Prefix `ee` mengaktifkan mode FakeTLS, dan domain (misalnya, situs terkenal) digunakan untuk menyamarkan lalu lintas sebagai HTTPS biasa. Cukup tentukan domain – sisanya akan dibangun oleh panel secara otomatis.

Domain-fronting dan opsi lanjutan mtg

Inbound MTProto memiliki parameter tambahan untuk proses `mtg`. Field **Domain fronting IP**, **Domain fronting port**, dan **Domain fronting PROXY protocol** menentukan kemana `mtg` mengirim lalu lintas non-Telegram (misalnya, ke situs NGINX palsu): jika IP dikosongkan, domain FakeTLS digunakan melalui DNS, port default – 443. Selain itu tersedia **Accept PROXY protocol** (untuk listener), **IP preference** (`prefer-ipv6` / `prefer-ipv4` / `only-ipv6` / `only-ipv4`) dan **Debug logging**. Setiap nilai ditulis ke file `mtg-<id>.toml` hanya jika ditetapkan.

Perutean lalu lintas Telegram melalui Xray

Toggle «**Route through Xray**» (dinonaktifkan secara default) dan field opsional **Outbound** memungkinkan egress Telegram tunduk pada router Xray. Saat diaktifkan, panel menyisipkan bridge SOCKS lokal dengan tag inbound itu sendiri ke konfigurasi Xray, dan `mtg` mengirim lalu lintas Telegram melaluinya. Setelah itu, lalu lintas dapat dicocokkan dengan aturan di tab «Routing» atau diarahkan paksa ke outbound atau load balancer yang dipilih melalui field **Outbound** (jika field kosong, aturan routing yang menentukan).

Cara mendistribusikan ke pengguna. Untuk inbound MTProto, panel menghasilkan link undangan:

Contoh: secret FakeTLS dan link siap pakai. Jika domain penyamaran adalah `www.cloudflare.com`, secret dirangkai sebagai `ee` + 32 karakter hex + kode hex domain, misalnya:

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

Link undangan siap pakai (link ini dan kode QR dikirim ke pengguna di Telegram):

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

```
tg://proxy?server=<alamat>&port=<port>&secret=<secret>
```

(setara – `https://t.me/proxy?server=...&port=...&secret=...`). Link ini dan kode QR perlu dikirimkan ke pengguna Telegram – saat dibuka, proksi langsung ditambahkan ke aplikasi. Link juga tersedia melalui server langganan.

Kapan menggunakan. Cara standar untuk melewati pemblokiran Telegram; penyamaran FakeTLS (domain penyamaran) membuat lalu lintas terlihat seperti kunjungan biasa ke situs yang ditentukan.

5.12. Panduan singkat pemilihan protokol

- **VLESS** – pilihan default; opsi terbaik dengan REALITY atau TLS + XTLS-Vision, mendukung autentikasi pasca-kuantum.
 - **Trojan** – penyamaran sebagai HTTPS dengan fallback ke web server.
 - **VMess** – kompatibilitas dengan klien lama.
 - **Shadowsocks** – proksi sederhana tanpa TLS; pilihan modern – metode `2022-blake3-*`.
 - **Hysteria** – QUIC, ketahanan pada saluran yang buruk.
 - **mixed / http** – proksi SOCKS/HTTP layanan.
 - **WireGuard** – tunnel VPN penuh.
 - **tunnel** – pengalihan port transparan.
 - **MTPROTO** – proksi untuk melewati pemblokiran Telegram (FakeTLS); proses `mtg` terpisah.
-

6. Transport (Stream Settings)

Transport (di antarmuka panel – kolom **«Transport»**, Ingg. *Transmission*) menentukan cara Xray-core mengirimkan data di dalam inbound: protokol jaringan apa yang digunakan di atas TLS/Reality dan bagaimana lalu lintas dibingkai. Parameter ini disimpan dalam objek `streamSettings` konfigurasi Xray dan diatur pada tab transport di editor inbound. Enkripsi (TLS / Reality) dibahas di bagian terpisah – di sini hanya dijelaskan pemilihan jaringan dan parameternya.

6.1. Pemilihan Jaringan Transmisi

Jaringan dipilih dari daftar dropdown **«Transport»** (`streamSettings.network`). Nilai default adalah `tcp` (ditampilkan dalam daftar sebagai **RAW**). Opsi yang tersedia:

Nilai dalam daftar	Kolom network	Transport
RAW	<code>tcp</code>	TCP biasa (diganti nama menjadi RAW pada versi Xray terbaru), opsional dengan obfuskasi HTTP
mKCP	<code>kcp</code>	Transport UDP andal mKCP
WebSocket	<code>ws</code>	WebSocket di atas HTTP(S)
gRPC	<code>grpc</code>	Terowongan gRPC (HTTP/2)
HTTPUpgrade	<code>httpupgrade</code>	HTTP Upgrade
XHTTP	<code>xhttp</code>	XHTTP / SplitHTTP – transport bermultipleks modern

Saat nilai diubah, panel mengosongkan blok pengaturan jaringan sebelumnya dan mengisi blok jaringan baru dengan nilai default dari skemanya, sehingga setiap kolom sub-form selalu memiliki nilai awal yang bermakna.

Penting. Pada build panel ini **transport HTTP/2 (h2) tidak tersedia dalam daftar** – transport tersebut telah dihapus dari kumpulan jaringan; untuk terowongan dua arah mirip HTTP/2 digunakan gRPC, sedangkan untuk transport modern berbalut HTTP digunakan XHTTP. Transport **Hysteria** (`hysteria`) tidak dipilih melalui daftar ini: transport tersebut terikat erat dengan protokol Hysteria dan muncul secara otomatis ketika inbound dibuat dengan protokol Hysteria (lihat poin 6.8).

Di bawah ini setiap jaringan dan setiap kolomnya dibahas secara terpisah.

6.2. RAW / TCP (`tcpSettings`)

Transport TCP dasar. Secara default lalu lintas dikirim apa adanya; secara opsional dapat disamarkan sebagai pertukaran HTTP/1.1 biasa.

Kolom	Nilai default	Deskripsi
Proxy Protocol (<code>acceptProxyProtocol</code>)	<code>false</code> (nonaktif)	Menerima header PROXY protocol dari load balancer/ proksi hulu
Obfuskasi HTTP (<code>header.type</code>)	<code>none</code> (nonaktif)	Mengaktifkan penyamaran lalu lintas sebagai HTTP/1.1

Proxy Protocol

Sakelar **«Proxy Protocol»** (`acceptProxyProtocol`). Saat diaktifkan, Xray mengharapkan header PROXY protocol pada koneksi masuk dan mengekstrak IP asli klien darinya. Aktifkan hanya jika terdapat reverse proxy/load balancer di depan panel (misalnya HAProxy atau nginx dengan `send-proxy`) yang menambahkan header tersebut. Dinonaktifkan secara default.

Obfuskasi HTTP (camouflage)

Sakelar **«HTTP Obfuskasi»**. Mengontrol kolom `header` :

- **Nonaktif** → `header.type = "none"` (kolom `header` tidak ada pada paket). TCP murni.
- **Aktif** → `header.type = "http"` . Lalu lintas dibingkai menyerupai permintaan dan respons HTTP/1.1. Saat diaktifkan, panel langsung mengisi sub-objek `request` dan `response` dengan nilai default.

Saat obfuskasi HTTP diaktifkan, kolom pengaturan permintaan dan respons yang ditiru menjadi tersedia.

Header permintaan (`header.request`):

Kolom	Kunci	Nilai default	Deskripsi
Versi permintaan	<code>request.version</code>	<code>1.1</code>	Versi HTTP dalam baris awal permintaan
Metode permintaan	<code>request.method</code>	<code>GET</code>	Metode HTTP permintaan yang ditiru
Path permintaan	<code>request.path</code>	<code>/</code>	Path. Dimasukkan sebagai daftar nilai yang dipisahkan koma; pada paket berupa array string. Jika dikosongkan, maka digunakan <code>/</code>
Header permintaan	<code>request.headers</code>	<code>{}</code> (kosong)	Tabel «Nama/Nilai» header HTTP. Disimpan sebagai peta <code>nama → [nilai]</code> (satu nama dapat memiliki beberapa nilai)

Header respons (`header.response`):

Kolom	Kunci	Nilai default	Deskripsi
Versi respons	<code>response.version</code>	<code>1.1</code>	Versi HTTP dalam baris awal respons
Status respons	<code>response.status</code>	<code>200</code>	Kode status HTTP respons yang ditiru
Alasan respons	<code>response.reason</code>	<code>OK</code>	Deskripsi teks status (reason-phrase)
Header respons	<code>response.headers</code>	<code>{}</code> (kosong)	Tabel «Nama/Nilai» header respons (peta <code>nama → [nilai]</code>)

Kolom header diedit baris per baris — setiap baris menentukan nama header (**Nama**) dan nilainya (**Nilai**). Parameter ini hanya digunakan untuk menyamarkan tampilan lalu lintas; tidak memengaruhi kriptografi. Nilai default (GET / HTTP/1.1 , respons 200 OK) cocok untuk sebagian besar skenario — ubah hanya jika perlu meniru situs/layanan tertentu.

Contoh streamSettings untuk RAW dengan obfuskasi HTTP:

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
      "response": {
        "version": "1.1",
        "status": "200",
        "reason": "OK"
      }
    }
  }
}
```

Perhatikan: path pada paket adalah array string, dan setiap header adalah array nilai (Host → ["www.example.com"]).

6.3. mKCP (kcpSettings)

mKCP adalah transport andal di atas UDP. Berguna pada saluran dengan kehilangan paket dan latensi tinggi, tetapi menghasilkan overhead lalu lintas yang lebih besar. Semua nilai default sesuai dengan yang direkomendasikan oleh xray-core.

Kolom	Kunci	Default	Nilai yang diizinkan	Deskripsi
MTU	mtu	1350	576–1460	Ukuran paket maksimum (byte). Kurangi jika terjadi masalah fragmentasi
TTI (ms)	tti	20	10–100	Interval transmisi (ms). Lebih kecil = latensi lebih rendah, tetapi overhead lebih besar
Uplink (MB/s)	uplinkCapacity	5	≥ 0	Estimasi kapasitas bandwidth upload (MB/s)
Downlink (MB/s)	downlinkCapacity	20	≥ 0	Estimasi kapasitas bandwidth download (MB/s)

Kolom	Kunci	Default	Nilai yang diizinkan	Deskripsi
Pengali CWND	<code>cwndMultiplier</code>	<code>1</code>	≥ 1	Pengali congestion window
Maks. jendela pengiriman	<code>maxSendingWindow</code>	<code>2097152</code>	≥ 0	Ukuran maksimum jendela pengiriman

Catatan kolom:

- **Uplink / Downlink capacity** menentukan seberapa agresif mKCP menggunakan saluran. Sesuaikan dengan lebar pita saluran yang sebenarnya: nilai terlalu tinggi menyebabkan lalu lintas berlebih, nilai terlalu rendah menyebabkan saluran tidak dimanfaatkan secara optimal.
- **TTI** secara langsung memengaruhi kompromi «latensi [?] overhead»: nilai lebih kecil mengurangi latensi tetapi meningkatkan volume paket overhead.
- **MTU** membatasi ukuran satu paket mKCP; pengurangan membantu pada saluran di mana paket UDP besar dipotong atau hilang.

Pada build ini kolom «seed» (kata sandi obfuskasi mKCP) dan daftar dropdown **jenis header/obfuskasi** (none , srtp , utp , wechat-video , dtls , wireguard) di sub-form mKCP **tidak tersedia sebagai kolom terpisah** – obfuskasi lapisan transport dipindahkan ke mekanisme umum «FinalMask» (termasuk mode mKCP-legacy), yang dijelaskan di bagian tersendiri. Parameter «congestion» sebagai kotak centang terpisah juga tidak ditampilkan; kontrol kemacetan diatur melalui `cwndMultiplier` dan `maxSendingWindow`.

6.4. WebSocket (`wsSettings`)

Transport WebSocket di atas HTTP(S). Melewati CDN dan reverse proxy dengan baik, menyamar sebagai lalu lintas web biasa.

Kolom	Kunci	Default	Deskripsi
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Menerima header PROXY protocol dari proksi hulu (lihat poin 6.2)
Host	<code>host</code>	<code>""</code> (kosong)	Nilai header HTTP <code>Host</code> . Tentukan saat bekerja melalui CDN/domain fronting
Path	<code>path</code>	<code>/</code>	Path dalam string permintaan handshake WebSocket
Periode heartbeat	<code>heartbeatPeriod</code>	<code>0</code>	Interval pengiriman frame heartbeat (dalam detik). <code>0</code> menonaktifkan heartbeat
Header	<code>headers</code>	<code>{}</code> (kosong)	Header HTTP tambahan untuk handshake. Peta «Nama → Nilai» (hanya nilai string, tanpa array)

Catatan:

- **Path** harus cocok antara server (inbound) dan klien. Sering kali titik masuk disamakan di balik path ini pada sisi web server.

- **Host** perlu ditentukan jika inbound berada di balik CDN atau menggunakan domain fronting; jika tidak, bisa dibiarkan kosong.
- **Periode heartbeat** menjaga koneksi «tetap hidup» saat melewati proksi/CDN yang secara agresif memutus sesi tidak aktif. Secara default (0) heartbeat dinonaktifkan.
- Berbeda dengan RAW, tabel header WebSocket menggunakan format «datar» `nama → nilai` (satu baris nilai per header).

Contoh `streamSettings` untuk WebSocket di balik CDN:

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

Nilai `host` dan `path` harus cocok di sisi klien; berbeda dengan RAW, nilai header di sini adalah string biasa, bukan array.

6.5. gRPC (`grpcSettings`)

Transport dengan jumlah parameter paling sedikit. Melakukan tunneling lalu lintas di dalam panggilan gRPC (di atas HTTP/2); kompatibel dengan CDN yang mendukung gRPC. Tidak ada obfuskasi header.

Kolom	Kunci	Default	Deskripsi
Nama layanan (Service Name)	<code>serviceName</code>	"" (kosong)	Nama layanan gRPC (secara efektif – «path» terowongan). Harus cocok antara server dan klien
Authority	<code>authority</code>	"" (kosong)	Nilai pseudo-header <code>:authority</code> (setara <code>Host</code> untuk HTTP/2). Tentukan saat bekerja melalui CDN/domain
Multi Mode	<code>multiMode</code>	<code>false</code> (nonaktif)	Mengaktifkan multipleksing beberapa aliran gRPC paralel dalam satu koneksi

Catatan:

- **Service Name** adalah pengenal utama saluran gRPC; harus sama di kedua sisi. Nilai kosong diizinkan, tetapi biasanya digunakan string tidak jelas untuk penyamaran.
- **Authority** memengaruhi `:authority` yang dikirim dalam frame HTTP/2; terutama diperlukan saat melakukan proxy melalui CDN.
- **Multi Mode** memungkinkan beberapa aliran logis melewati satu koneksi fisik; aktifkan untuk meningkatkan performa ketika server dan klien mendukungnya.

Contoh `streamSettings` untuk gRPC:

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

`serviceName` (di sini `GunService`) berperan sebagai «path» terowongan dan harus cocok antara server dan klien.

6.6. HTTPUpgrade (`httpupgradeSettings`)

Transport berdasarkan mekanisme HTTP Upgrade (seperti WebSocket, tetapi tanpa protokol WebSocket itu sendiri). Juga melewati proksi dan CDN dengan baik. Kumpulan kolom menyerupai WebSocket, tetapi **tanpa** periode heartbeat.

Kolom	Kunci	Default	Deskripsi
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Menerima header PROXY protocol dari proksi hulu
Host	<code>host</code>	<code>""</code> (kosong)	Nilai header HTTP <code>Host</code>
Path	<code>path</code>	<code>/</code>	Path permintaan HTTP dengan header <code>Upgrade</code>
Header	<code>headers</code>	<code>{}</code> (kosong)	Header HTTP tambahan. Peta «datar» <code>nama → nilai</code> (seperti WebSocket)

Fungsi kolom **Host**, **Path**, dan **Header** sama dengan WebSocket (poin 6.4). Heartbeat tidak tersedia untuk HTTPUpgrade – itu adalah fitur khusus WebSocket.

6.7. XHTTP / SplitHTTP (`xhttpSettings`)

XHTTP (alias SplitHTTP) adalah transport HTTP bermultipleks modern dari xray-core. Memisahkan aliran uplink dan downlink menjadi permintaan HTTP terpisah, yang sangat cocok untuk CDN dan lingkungan dengan batasan durasi koneksi. Tidak semua kolom ditampilkan sekaligus di editor: sebagian muncul tergantung pada mode yang dipilih (`mode`) dan sakelar yang diaktifkan.

Kolom dasar (selalu terlihat)

Kolom	Kunci	Default	Deskripsi
Host	<code>host</code>	<code>""</code> (kosong)	Nilai header HTTP <code>Host</code>
Path	<code>path</code>	<code>/</code>	Path dasar permintaan HTTP

Kolom	Kunci	Default	Deskripsi
Mode (Mode)	mode	auto	Mode transmisi (lihat di bawah)
Server Max Header Bytes	serverMaxHeaderBytes	0	Batas ukuran header permintaan di server (byte). 0 – nilai default xray-core
Padding Bytes	xPaddingBytes	100–1000	Rentang padding acak (dalam byte, format min–maks) untuk mempersulit analisis ukuran
Header	headers	{ } (kosong)	Header HTTP tambahan. Peta «datar» nama → nilai
Metode HTTP Uplink	uplinkHTTPMethod	"" (Default = POST)	Metode HTTP untuk permintaan uplink. Pilihan: kosong (default POST), POST , PUT , GET (terakhir hanya tersedia dalam mode packet–up)
Padding Obfs Mode	xPaddingObfsMode	false (nonaktif)	Mengaktifkan obfuskasi padding lanjutan dan membuka kolom tambahan (lihat di bawah)
No SSE Header	noSSEHeader	false (nonaktif)	Tidak mengirim header Content–Type: text/event–stream (SSE). Aktifkan jika header tersebut mengganggu penerusan melalui node perantara

Kolom «Mode» (mode)

Daftar dropdown dengan nilai:

Nilai	Deskripsi
auto	Pemilihan mode otomatis (default)
packet–up	Aliran uplink dipecah menjadi permintaan HTTP terpisah (satu paket per permintaan)
stream–up	Aliran uplink dikirim dalam satu permintaan streaming panjang
stream–one	Satu permintaan streaming dua arah bersama

Pilihan mode menentukan kolom tambahan mana yang menjadi terlihat.

Kolom yang terlihat hanya saat mode = packet–up :

Kolom	Kunci	Default	Deskripsi
Maks. upload yang di-buffer	scMaxBufferedPosts	30	Jumlah maksimum permintaan POST uplink yang di-buffer secara bersamaan
Maks. ukuran upload (byte)	scMaxEachPostBytes	1000000	Ukuran maksimum satu permintaan POST uplink (byte)
Uplink Data Placement	uplinkDataPlacement	"" (Default = body)	Tempat menempatkan data uplink: body , header , cookie , query
Uplink Data Key	uplinkDataKey	""	Nama kunci/header untuk data uplink. Muncul hanya jika uplinkDataPlacement ditentukan dan bukan body

Kolom yang terlihat hanya saat `mode = stream-up` :

Kolom	Kunci	Default	Deskripsi
Stream-Up Server	<code>scStreamUpServerSecs</code>	<code>20-80</code>	Rentang waktu mempertahankan koneksi streaming server (dalam detik, format <code>min-maks</code>)

Kolom obfuskasi padding (terlihat saat `xPaddingObfsMode = aktif`)

Kolom	Kunci	Default	Deskripsi
Padding Key	<code>xPaddingKey</code>	<code>""</code> (placeholder <code>x_padding</code>)	Nama kunci untuk padding
Padding Header	<code>xPaddingHeader</code>	<code>""</code> (placeholder <code>X-Padding</code>)	Nama header HTTP tempat padding dikirim
Padding Placement	<code>xPaddingPlacement</code>	<code>""</code> (Default = <code>queryInHeader</code>)	Tempat menempatkan padding: <code>queryInHeader</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Padding Method	<code>xPaddingMethod</code>	<code>""</code> (Default = <code>repeat-x</code>)	Metode pembuatan padding: <code>repeat-x</code> atau <code>tokenish</code>

Penempatan sesi dan urutan (selalu terlihat)

Kolom	Kunci	Default	Deskripsi
Session ID Placement	<code>sessionIDPlacement</code>	<code>""</code> (Default = <code>path</code>)	Tempat mengirimkan ID sesi: <code>path</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Session ID Key	<code>sessionIDKey</code>	<code>""</code> (placeholder <code>x_session</code>)	Nama kunci sesi. Muncul hanya jika <code>sessionIDPlacement</code> ditentukan dan bukan <code>path</code>
Session ID Table	<code>sessionIDTable</code>	<code>""</code> (placeholder <code>Base62</code>)	Kumpulan karakter untuk menghasilkan ID sesi. Dapat dipilih dari daftar dropdown autocomplete yang telah ditentukan (<code>ALPHABET</code> , <code>Alphabet</code> , <code>BASE36</code> , <code>Base62</code> , <code>HEX</code> , <code>alphabet</code> , <code>base36</code> , <code>hex</code> , <code>number</code>) atau dimasukkan string ASCII arbitrer. Kosong – nilai default xray-core
Session ID Length	<code>sessionIDLength</code>	<code>""</code> (kosong)	Panjang atau rentang (misalnya <code>8-16</code>) ID yang dihasilkan. Ditampilkan hanya saat <code>Session ID Table</code> ditentukan; nilai minimum harus lebih dari 0
Sequence Placement	<code>seqPlacement</code>	<code>""</code> (Default = <code>path</code>)	Tempat mengirimkan nomor urutan paket: <code>path</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Sequence Key	<code>seqKey</code>	<code>""</code> (placeholder <code>x_seq</code>)	Nama kunci urutan. Muncul hanya jika <code>seqPlacement</code> ditentukan dan bukan <code>path</code>

Kolom sesi diganti namanya sesuai xray-core v26.6.22: sebelumnya disebut **Session Placement / Session Key** (`sessionPlacement` / `sessionKey`) – sekarang menjadi **Session ID Placement / Session ID Key**

(`sessionIDPlacement` / `sessionIDKey`); nama lama tidak lagi dipahami oleh core. Inbound yang disimpan sebelum pembaruan dimigrasikan ke kunci baru secara otomatis – tidak perlu disimpan ulang.

Rekomendasi:

- Untuk sebagian besar instalasi, cukup biarkan **Mode = auto**, tentukan **Path/Host**, dan (saat bekerja melalui CDN) selaraskan dengan klien.
- Kolom penempatan (`*Placement` / `*Key`) dan obfuskasi padding hanya diperlukan untuk penyesuaian halus pada skenario anti-DPI/CDN tertentu; saat nilai dikosongkan, nilai default xray-core yang ditunjukkan dalam tanda kurung akan digunakan.
- Parameter yang berkaitan dengan sisi klien/outbound (misalnya, interval pengiriman ulang POST, ukuran chunk) tidak ditampilkan dalam form inbound – server listener mengabaikannya. Sebaliknya, multipleksor XMUX tersedia dalam form inbound (lihat di bawah).
- **Default layanan tidak diterapkan.** Panel tidak lagi menulis nilai default layanan `scMaxEachPostBytes` dan `scMinPostsIntervalMs` ke konfigurasi XHTTP – nilai internal xray-core yang digunakan. Ini menghilangkan tanda tangan DPI tetap (literal `scMinPostsIntervalMs=30`) yang sebelumnya dapat menyebabkan pemblokiran lalu lintas. Untuk inbound yang sudah disimpan, nilai yang cocok dengan default xray-core tidak ditampilkan dalam tautan dan subscription (tidak perlu menyimpan ulang inbound); nilai yang diatur secara manual tetap tersimpan.

Contoh `streamSettings` untuk XHTTP (mode `auto`):

```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

Untuk sebagian besar instalasi, keempat kolom ini sudah cukup; kolom penempatan sesi/urutan dan obfuskasi padding dibiarkan kosong – nilai default xray-core yang akan digunakan.

XMUX (multipleksing koneksi)

Sakelar **XMUX** (`enableXmux`) mengaktifkan lapisan multipleksing yang mendistribusikan permintaan paralel ke sejumlah kecil koneksi fisik. Saat diaktifkan, enam kolom konfigurasi tersedia (disimpan dalam `xhttpSettings.xmux`):

Kolom	Kunci	Default	Deskripsi
Max Concurrency	<code>maxConcurrency</code>	16-32	Jumlah maksimum permintaan bersamaan per koneksi (rentang <code>min-maks</code>)

Kolom	Kunci	Default	Deskripsi
Max Connections	<code>maxConnections</code>	6	Jumlah maksimum koneksi fisik (0 – tanpa batas). Mulai versi 3.4.2, inbound baru dibuat dengan nilai 6 ; inbound yang dibuat sebelumnya mempertahankan nilainya.
Max Reuse Times	<code>cMaxReuseTimes</code>	"" (kosong)	Berapa kali koneksi dapat digunakan kembali
Max Request Times	<code>hMaxRequestTimes</code>	600–900	Jumlah maksimum permintaan per koneksi (rentang)
Max Reusable Secs	<code>hMaxReusableSecs</code>	1800–3000	Waktu koneksi dapat digunakan kembali (detik, rentang)
Keep Alive Period	<code>hKeepAlivePeriod</code>	"" (kosong)	Periode keep-alive untuk mempertahankan koneksi

Penting. Tidak boleh menetapkan **Max Connections** dan **Max Concurrency** secara bersamaan – xray-core akan menolak konfigurasi tersebut. Secara default saat XMUX diaktifkan, panel menetapkan **Max Concurrency** = 16–32 ; jika Anda menetapkan **Max Connections** (nilai lebih dari 0), panel akan menghapus nilai default **Max Concurrency** untuk menghindari konflik.

6.8. Transport Hysteria (`hysteriaSettings`)

Transport **Hysteria** tidak dipilih dari daftar «Transport»: transport ini diaktifkan secara otomatis ketika inbound dibuat dengan protokol Hysteria, dan disembunyikan untuk protokol lain (saat beralih dari protokol Hysteria, jaringan dipaksa kembali ke `tcp`). Parameter:

Kolom	Kunci	Default	Deskripsi
Versi	<code>version</code>	2 (tetap, kolom dikunci)	Versi Hysteria. Hanya Hysteria 2 yang didukung
UDP idle timeout (s)	<code>udpIdleTimeout</code>	60	Timeout tidak aktif sesi UDP (detik). Rentang yang diizinkan – 2–600; xray-core menolak nilai di luar rentang ini saat startup
Masquerade	<code>masquerade</code>	nonaktif (tidak ada)	Mengaktifkan penyamaran listener sebagai server HTTP/3 saat dilakukan probe

Saat **Masquerade** diaktifkan, pilihan tipe (`type`) dan kolom terkaitnya muncul:

- **"" – default (404 page):** mengembalikan halaman 404 standar (tidak memerlukan kolom tambahan).
- **proxy (reverse proxy):** proxy terbalik ke situs eksternal.
 - `url` (**Upstream URL**, placeholder `https://www.example.com`) – alamat tujuan;
 - `rewriteHost` (**Timpa Host**, default `false`) – mengganti header `Host` ;
 - `insecure` (**Lewati TLS verify**, default `false`) – tidak memverifikasi sertifikat TLS upstream.
- **file (serve directory):** melayani file dari direktori.
 - `dir` (**Direktori**, placeholder `/var/www/html`).

- **string (fixed body)**: respons HTTP tetap.
 - `statusCode` (**Kode status**, default `0`, rentang 0–599);
 - `content` (**Body**) – isi respons;
 - `headers` (**Header**) – peta nama → nilai.

Masquerade memungkinkan inbound berbasis Hysteria tampak seperti server HTTP/3 biasa saat dilakukan probe aktif, sehingga meningkatkan penyamaran. Secara default masquerade dinonaktifkan.

Contoh `hysteriaSettings` dengan reverse proxy (`masquerade` → `proxy`):

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

Di sini saat dilakukan probe, listener mengembalikan respons dari `https://www.example.com`, menyamar sebagai situs HTTP/3 biasa.

6.9. Parameter Pelengkap

Selain pemilihan jaringan, pada tab yang sama terdapat dua blok umum yang tidak bergantung pada transport tertentu (detail – di bagian masing-masing):

- **External Proxy** (`externalProxy`) – daftar alamat/port eksternal yang menggantikan alamat panel itu sendiri dalam tautan subscription.
- **Sockopt** (`sockopt`) – opsi socket tingkat rendah (TCP Fast Open, mark, strategi domain, proxy transparan, dll.).

Real client IP (menentukan IP asli di balik CDN/relay)

Ketika inbound berada di balik perantara (CDN seperti Cloudflare, terowongan L4/relay, atau panel lain), Xray melihat alamat perantara, bukan pengunjung asli. Alamat ini masuk ke daftar klien online dan dihitung sebagai batas IP per klien, sehingga keduanya menjadi tidak berguna di balik proksi. Untuk memulihkan IP asli, di bagian **Sockopt** pada form inbound terdapat pilihan preset **Real client IP** yang menggabungkan pengaturan `acceptProxyProtocol` dan `trustedXForwardedFor`:

Preset	Yang dilakukan	Kapan digunakan
Off / direct	Mengosongkan kedua kolom.	Inbound dapat diakses klien secara langsung
Cloudflare CDN	Menetapkan <code>sockopt.trustedXForwardedFor = ["CF-Connecting-IP"]</code> .	WebSocket / HTTPUpgrade / XHTTP / gRPC di balik CDN Cloudflare (ikon awan oranye)

Preset	Yang dilakukan	Kapan digunakan
L4 relay / Spectrum (PROXY)	Mengaktifkan <code>acceptProxyProtocol = true</code> .	Terowongan L4/relay di depan inbound atau Cloudflare Spectrum

Preset bersifat saling eksklusif: memilih satu akan mengosongkan kolom yang lain, sehingga `trustedXForwardedFor` yang sudah usang tidak menimpa IP yang dipulihkan melalui PROXY protocol. Di bawah preset, sakelar **Proxy Protocol** dan daftar **Trusted X-Forwarded-For** tetap terlihat – preset hanya mengisinya untuk Anda, dan jika perlu dapat diedit secara manual. Jika preset yang dipilih tidak didukung oleh transport saat ini (misalnya, PROXY protocol pada mKCP), form menampilkan peringatan. Kolom-kolom ini hanya berlaku untuk sisi server dan **tidak pernah dikirim ke klien dalam subscription**.

Gunakan salah satu saja. `acceptProxyProtocol` membaca IP asli dari header PROXY protocol L4, sedangkan `trustedXForwardedFor` dari header HTTP permintaan; menggabungkan keduanya secara manual hanya perlu dilakukan jika rantai perantara Anda memang memerlukannya.

- **FinalMask** (`finalmask`) – mekanisme umum obfuskasi lapisan transport (termasuk obfuskasi legacy mKCP) yang menggantikan kolom terpisah «seed»/«header type» di dalam sub-form jaringan.

7. Keamanan Koneksi: TLS, XTLS, dan REALITY

Setiap inbound yang mendukung transmisi melalui transport stream (VMess, VLESS, Trojan, Shadowsocks, Hysteria) memiliki tab «**Keamanan**» di editor. Di sini dikonfigurasi bagaimana tepatnya saluran transport dienkripsi dan disamarkan. Tersedia tiga mode yang dapat diaktifkan dengan tombol radio:

Mode	Label di UI	Kapan tersedia
none	Tidak Ada	Selalu (kecuali Hysteria, di mana TLS wajib)
tls	TLS	Untuk VMess/VLESS/Trojan/Shadowsocks pada jaringan <code>tcp</code> , <code>ws</code> , <code>http</code> , <code>grpc</code> , <code>httpupgrade</code> , <code>xhttp</code> ; untuk Hysteria – selalu
reality	Reality	Hanya untuk VLESS/Trojan pada jaringan <code>tcp</code> , <code>http</code> , <code>grpc</code> , <code>xhttp</code>

Tombol **Tidak Ada** tidak ditampilkan jika protokolnya adalah Hysteria (karena TLS wajib untuknya). Tombol **Reality** muncul hanya pada kombinasi protokol dan jaringan yang diizinkan (lihat tabel di atas).

Saat mode diubah, panel sepenuhnya merakit ulang blok `streamSettings`: `tlsSettings` dan `realitySettings` dari mode sebelumnya dihapus dan nilai default untuk mode yang dipilih dimasukkan. Secara khusus, saat memilih **Reality**, panel secara otomatis langsung: memasukkan pasangan `target` + `serverNames` (SNI) secara acak dari daftar domain populer bawaan, menghasilkan `shortIds` acak, serta membuat permintaan ke server untuk mendapatkan pasangan kunci X25519 baru (`privateKey/publicKey`).

7.1. Apa Perbedaannya: TLS vs XTLS vs REALITY

- **TLS** – enkripsi transport klasik menggunakan protokol TLS 1.2/1.3. Server harus memiliki sertifikat yang valid (domain sendiri + rantai sertifikat). Lalu lintas terlihat seperti HTTPS biasa, namun bagi sensor aktif, TLS-handshake ke domain Anda mudah dikenali; jika diblokir berdasarkan SNI atau jika sertifikat tidak terpercaya, koneksi diblokir/menampilkan kesalahan.
- **XTLS (Vision)** – bukan mode terpisah dalam daftar «Keamanan», melainkan mekanisme *flow* di atas TLS atau Reality. Diaktifkan di sisi klien inbound melalui field **Flow** = `xtls-rprx-vision` (atau `xtls-rprx-vision-udp443`). Vision tersedia untuk VLESS pada jaringan `tcp` dengan `security = tls` atau `security = reality`, serta untuk VLESS di atas transport `xhttp` dengan enkripsi VLESS diaktifkan (`vlessenc` / ML-KEM) – dalam kasus ini field **Flow** juga dapat diatur ke `xtls-rprx-vision`, dan nilainya secara tepat masuk ke tautan `vless://` (`flow=xtls-rprx-vision`). Vision mengurangi «enkripsi ganda» (TLS-in-TLS) dengan mengirimkan payload secara langsung setelah handshake, yang mempercepat transmisi dan meningkatkan penyamaran. Oleh karena itu kombinasi **VLESS + Reality + Flow xtls-rprx-vision** dianggap sebagai konfigurasi modern yang direkomendasikan.

Pemulihan otomatis flow Vision. Jika enkripsi inbound VLESS/XHTTP (ML-KEM, field decryption/encryption) diaktifkan setelah klien sudah ditambahkan, inbound menjadi layak untuk flow. Dalam situasi ini, panel sendiri memulihkan `flow = xtls-rprx-vision` pada klien yang seharusnya mendapatkannya, tetapi field **Flow**-nya kosong. Sebelumnya, dalam skenario seperti ini, Vision diam-diam menghilang dari konfigurasi, tautan undangan, dan langganan (terutama terlihat pada inbound yang

berfungsi sebagai node). Tidak diperlukan tindakan manual: perbaikan diterapkan secara otomatis saat menyimpan inbound dan sekali saat pembaruan panel. Perilakunya konservatif – panel tidak menciptakan flow secara sewenang-wenang dan tidak menerima nilai yang ditetapkan secara eksplisit oleh klien.

- **REALITY** – mekanisme penyamaran tanpa sertifikat sendiri. Server «meminjam» TLS-handshake dari situs pihak ketiga yang nyata (`target / serverNames`), sehingga bagi pengamat koneksi tidak dapat dibedakan dari akses ke situs tersebut, dan sertifikat tidak diperlukan sama sekali. Autentikasi dibangun atas pasangan kunci X25519 dan sekumpulan `shortIds` . REALITY tahan terhadap active probing dan pemblokiran berdasarkan SNI, karena SNI menunjuk pada domain eksternal nyata. Harganya adalah persyaratan konfigurasi yang lebih ketat (nilai `target` yang benar dengan port, sinkronisasi kunci dengan klien).

Aturan singkat untuk memilih:

- memiliki domain sendiri dan sertifikat valid, perlu tampilan HTTPS sederhana → **TLS** (jika memungkinkan dengan Vision);
- tidak ada domain/sertifikat atau diperlukan ketersembunyian maksimum dari DPI → **REALITY** (dengan Vision untuk VLESS/TCP).

7.2. Mode «Tidak Ada» (`none`)

Transport dikirimkan tanpa pembungkus TLS: blok `tlsSettings` dan `realitySettings` dikeluarkan dari `streamSettings` . Mode ini tidak memiliki field tambahan. Cocok digunakan saat:

- inbound hanya mendengarkan pada `127.0.0.1` dan berfungsi sebagai target fallback (sesuai aturan panel, inbound anak untuk fallback harus mendengarkan pada `127.0.0.1` dengan `security=none`);
- enkripsi/penyamaran disediakan oleh lapisan eksternal (misalnya, reverse proxy Nginx di depan panel);
- digunakan protokol dengan enkripsi bawaan (Shadowsocks) pada jaringan internal.

Untuk inbound yang dapat diakses dari luar, mode «Tidak Ada» tidak direkomendasikan.

Contoh: blok `streamSettings` untuk TLS pada jaringan `tcp` (VLESS/Trojan/VMess). Inilah tampilan hasilnya setelah memilih mode **TLS** dan mengisi SNI serta path ke sertifikat:

```

"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}

```

7.3. Mode TLS

Field blok `tlsSettings`. Nilai default diambil dari skema panel.

Parameter Utama

Field (label)	Nilai default	Deskripsi
SNI (<code>serverName</code>)	<code>""</code> (kosong)	Server Name Indication – nama domain yang ditunjukkan dalam TLS-handshake. Harus sesuai dengan domain sertifikat. Placeholder dalam bahasa Inggris: «Server Name Indication».
Cipher Suites (<code>cipherSuites</code>)	<code>""</code> → Otomatis	Daftar cipher suite yang diizinkan. Secara default kosong – pilihan diserahkan kepada Xray/Go (opsi Otomatis). Ubah hanya jika perlu membatasi cipher secara eksplisit.
Versi Min/Maks (<code>minMaxVersion</code>)	min = <code>1.2</code> , max = <code>1.3</code>	Versi TLS minimum dan maksimum. Nilai yang tersedia: <code>1.0</code> , <code>1.1</code> , <code>1.2</code> , <code>1.3</code> . Disarankan untuk mempertahankan <code>1.2</code> – <code>1.3</code> ; menurunkan minimum ke 1.0/1.1 tidak dianjurkan (versi lama, tidak aman).
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code> (dalam form – opsi None = <code>""</code> tersedia)	Sidik jari TLS klien yang diimitasi (uTLS fingerprint), agar handshake tampak seperti browser populer. Lihat daftar di bawah. Pada TLS, opsi pertama dalam daftar adalah None (<code>""</code>), yang menonaktifkan imitasi.
ALPN (<code>alpn</code>)	<code>["h2", "http/1.1"]</code>	Daftar protokol lapisan aplikasi yang dinegosiasikan dalam TLS (pilihan ganda). Nilai yang diizinkan: <code>h3</code> , <code>h2</code> , <code>http/1.1</code> . Secara default <code>h2</code> dan <code>http/1.1</code> disediakan.

Nilai yang mungkin untuk **uTLS fingerprint** (sama untuk TLS dan REALITY): `chrome` , `firefox` , `safari` , `ios` , `android` , `edge` , `360` , `qq` , `random` , `randomized` , `randomizednoalpn` , `unsafe` . Pada form TLS, opsi kosong **None** (tanpa imitasi sidik jari) juga tersedia.

Nilai yang tersedia untuk **Cipher Suites** (TLS 1.3 dan suite ECDHE): `TLS_AES_128_GCM_SHA256` , `TLS_AES_256_GCM_SHA384` , `TLS_CHACHA20_POLY1305_SHA256` , `TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA` , `TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA` , `TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA` , `TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA` , `TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256` , `TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384` , `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256` , `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384` , `TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256` , `TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256` .

Sakelar TLS

Sakelar	Default	Deskripsi
Tolak SNI Tidak Dikenal (<code>rejectUnknownSni</code>)	nonaktif (<code>false</code>)	Jika diaktifkan, server memutus koneksi ketika SNI yang ditunjukkan klien tidak sesuai dengan nama dalam sertifikat. Meningkatkan ketersembunyian (server tidak merespons permintaan «asing»), tetapi memerlukan kecocokan SNI yang tepat di sisi klien.
Nonaktifkan System Root (<code>disableSystemRoot</code>)	nonaktif (<code>false</code>)	Menonaktifkan penggunaan penyimpanan sertifikat root tepercaya sistem.
Resumption Sesi (<code>enableSessionResumption</code>)	nonaktif (<code>false</code>)	Mengaktifkan resumption sesi TLS (session resumption / session tickets).

Parameter Tambahan TLS (vcn, kurva, log kunci, ECH Sockopt)

Di bawah pengaturan TLS utama terdapat field tambahan.

Field (label)	Default	Deskripsi
Verify Peer Cert By Name (<code>settings.verifyPeerCertByName</code>)	""	Nama-nama (dipisahkan koma) yang digunakan klien untuk memverifikasi sertifikat server sebagai pengganti SNI. Ini adalah pengganti modern dari field <code>allowInsecure</code> yang dihapus dari Xray setelah 2026-06-01. Nilai ini hanya digunakan oleh panel: tidak ditulis ke konfigurasi xray server, tetapi diteruskan dalam tautan undangan dan langganan (<code>vcn=...</code>) agar klien dapat menerapkannya sendiri. Placeholder: <code>example.com</code> .
Curve Preferences (<code>curvePreferences</code>)	""	Pembatasan dan urutan kurva pertukaran kunci TLS berdasarkan preferensi (misalnya <code>X25519MLKEM768</code> , <code>X25519</code>). Kosong – menggunakan nilai default Xray-core.
Master Key Log (<code>masterKeyLog</code>)	""	Path untuk menyimpan TLS master keys dalam format <code>SSLKEYLOGFILE</code> (untuk mendekripsi lalu lintas di Wireshark saat debugging). Placeholder: <code>/path/to/sslkeylog.txt</code> . Di production, biarkan kosong – file ini memungkinkan dekripsi semua lalu lintas.

Field (label)	Default	Deskripsi
ECH Sockopt (echSockopt)	nonaktif	Sakelar dengan parameter socket untuk koneksi yang digunakan Xray untuk meminta ECH config list. Saat diaktifkan, tersedia: Dialer Proxy (dialerProxy) – arahkan permintaan melalui outbound yang ditentukan berdasarkan tag), Domain Strategy (domainStrategy), TCP Fast Open (tcpFastOpen), Multipath TCP (tcpMptcp). Biarkan nonaktif jika tidak diperlukan.

Field `verifyPeerCertByName`, `curvePreferences`, `masterKeyLog`, dan `echSockopt` berada di level atas `tlsSettings` dan bertahan saat field panel dipangkas pada penyimpanan konfigurasi.

Sertifikat

Bagian **Sertifikat SSL** (judul di UI «Sertifikat SSL») dikonfigurasi sebagai daftar: tombol **+** menambahkan entri sertifikat baru, tombol **- Hapus** menghapusnya (tombol hapus hanya tersedia jika ada lebih dari satu entri). Secara default, satu entri kosong dibuat saat TLS diaktifkan.

Untuk setiap entri, terdapat sakelar mode input (`useFile`):

- **Path ke Sertifikat** (nilai `useFile = true`, default) – path ke file di server ditentukan:
 - **Kunci Publik** (`certificateFile`) – path ke file sertifikat (`.crt` / `.pem`);
 - **Kunci Privat** (`keyFile`) – path ke file kunci privat (`.key`).
- **Isi Sertifikat** (nilai `useFile = false`) – isi disisipkan langsung ke dalam field (area teks multi-baris):
 - **Kunci Publik** (`certificate`) – isi sertifikat dalam format PEM;
 - **Kunci Privat** (`key`) – isi kunci dalam format PEM.

Di bawah field mode «Path ke Sertifikat» terdapat dua tombol:

- **Pasang Sertifikat Panel** – mengisi field dengan path ke sertifikat SSL panel itu sendiri. Untuk inbound pada panel pusat, sertifikatnya digunakan (`POST /panel/setting/all` → `webCertFile` / `webKeyFile`); untuk inbound yang ditetapkan ke node – sertifikat node itu sendiri (`GET /panel/api/nodes/webCert/{nodeId}`), karena path panel pusat tidak ada pada node. Jika sertifikat tidak dikonfigurasi, peringatan ditampilkan: «*Sertifikat tidak dikonfigurasi untuk panel. Silakan atur terlebih dahulu di Pengaturan.*» (sertifikat panel itu sendiri dikonfigurasi di bagian «Pengaturan → Keamanan»).
- **Hapus** – mengosongkan kedua path.

Field tambahan setiap entri sertifikat:

Field	Default	Deskripsi
OCSP Stapling (ocsStapling)	0 (nonaktif)	Interval pembaruan OCSP stapling dalam detik (minimum 0). Untuk inbound baru, secara default dinonaktifkan (0): ini menghilangkan kesalahan dalam log xray untuk sertifikat tanpa OCSP responder (misalnya, Let's Encrypt yang menghentikan OCSP). Aktifkan hanya untuk sertifikat yang mendukung stapling.
Muat Sekali (oneTimeLoading)	nonaktif (false)	Jika diaktifkan, sertifikat dibaca dari disk sekali saat startup dan tidak dibaca ulang saat file berubah.

Field	Default	Deskripsi
Opsi Penggunaan (<code>usage</code>)	<code>encipherment</code>	Tujuan sertifikat. Nilai yang diizinkan: <code>encipherment</code> (enkripsi – sertifikat server biasa), <code>verify</code> (verifikasi), <code>issue</code> (penerbitan – server menandatangani/menerbitkan sertifikat sendiri).
Build Chain (<code>buildChain</code>)	nonaktif (<code>false</code>)	Ditampilkan hanya saat <code>usage = issue</code> . Membangun rantai sertifikat.

Tidak ada tombol terpisah untuk sertifikat self-signed di editor inbound: panel tidak menghasilkan sertifikat self-signed secara langsung untuk inbound. Sertifikat ditentukan melalui path/isi, atau diambil dari pengaturan panel menggunakan tombol «Pasang Sertifikat Panel». Penerbitan/perolehan sertifikat SSL panel itu sendiri (termasuk pengunggahan file dan pengikatan ke domain) dilakukan di bagian **Pengaturan** → **Keamanan**; tidak ada endpoint ACME/Let's Encrypt untuk inbound di sini.

ECH dan Pinning Sertifikat (Field TLS Lanjutan)

Field	Default	Deskripsi
ECH key (<code>echServerKeys</code>)	<code>""</code>	Kunci server Encrypted Client Hello.
ECH config (<code>settings.echConfigList</code>)	<code>""</code>	ECH config list (bagian klien, masuk ke tautan).
SHA-256 Sertifikat Peer (<code>settings.pinnedPeerCertSha256</code>)	<code>[]</code>	Hash SHA-256 sertifikat peer (string hex, dipisahkan koma). Tooltip kata demi kata: «Hash SHA-256 sertifikat peer dalam bentuk string heksadesimal (misalnya e8e2d3...), dipisahkan koma. Hanya untuk panel – tidak ditulis ke konfigurasi xray server, tetapi disertakan dalam tautan undangan agar klien dapat melakukan pinning sertifikat.»

Tombol: Di sebelah field **SHA-256 Sertifikat Peer** terdapat dua tombol pengisian otomatis:

- **Fill from this inbound's certificate** (ikon perisai) – mengisi hash SHA-256 sertifikat inbound ini sendiri (diambil secara lokal melalui endpoint `getCertHash`).
- **Fetch the hash by pinging the SNI (xray tls ping)** (ikon unduh) – mendapatkan hash sertifikat server yang aktif dengan melakukan koneksi TLS ke SNI yang ditentukan (pada server dijalankan `getRemoteCertHash`). Field **SNI** (`serverName`) harus diisi – jika tidak, ditampilkan petunjuk «Set the SNI (serverName) first to ping the remote certificate.»

Hash yang diperoleh ditambahkan ke field (dipisahkan koma) dan masuk ke tautan undangan agar klien dapat melakukan pinning sertifikat.

- **Dapatkan Sertifikat ECH Baru** – meminta pasangan ECH baru dari server untuk SNI saat ini (`POST /panel/api/server/getNewEchCert` , pada server dijalankan `xray tls ech --serverName <SNI>`); mengisi field **ECH key** dan **ECH config**.
- **Hapus** – mengosongkan kedua field ECH.

7.4. Mode REALITY

Field blok `realitySettings`. REALITY tidak menggunakan sertifikat SSL: sebagai gantinya – TLS-handshake yang dipinjam dari domain eksternal dan pasangan kunci X25519.

Parameter Penyamaran

Field (label)	Nilai default	Deskripsi
Tampilkan (<code>show</code>)	nonaktif (<code>false</code>)	Output debug REALITY ke log Xray. Biasanya dibiarkan nonaktif.
Xver (<code>xver</code>)	<code>0</code>	Versi protokol PROXY yang diteruskan ke backend (<code>0</code> – nonaktif). Minimum <code>0</code> .
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code>	Sidik jari TLS yang diimitasi (daftar yang sama seperti pada TLS, tetapi tanpa opsi None kosong).
Target (<code>target</code>)	<code>""</code> (mulai 3.4.2 tetap kosong saat REALITY diaktifkan)	Field wajib. Domain nyata yang TLS-handshake-nya dipinjam oleh REALITY. Tooltip kata demi kata: « <i>Wajib. Harus mengandung port (misalnya example.com:443). Tanpa port, Xray-core tidak dapat dijalankan.</i> » Validasi panel memeriksa keberadaan dan kebenaran port; jika tidak, ditampilkan kesalahan «Target REALITY wajib diisi» / «Target REALITY harus mengandung port...» / «Target REALITY memiliki port yang tidak valid». Di sebelah field terdapat tombol « Pindai » (memeriksa target saat ini «secara langsung») dan « Cari target » (membuka pemindai target REALITY); lihat di bawah.
SNI (<code>serverNames</code>)	<code>[]</code> (dimasukkan bersama target)	Daftar SNI yang diizinkan (input ganda dengan tag). Harus sesuai dengan domain dari Target . Saat pemindaian target berhasil, SNI diisi berdasarkan sertifikatnya.
Maks. Perbedaan Waktu (ms) (<code>maxTimediff</code>)	<code>0</code>	Perbedaan jam maksimum yang diizinkan antara klien dan server dalam milidetik (<code>0</code> – tanpa batas). Minimum <code>0</code> .
Versi Klien Min. (<code>minClientVer</code>)	<code>""</code>	Versi klien Xray minimum (placeholder <code>25.9.11</code>). Kosong – tanpa batas.
Versi Klien Maks. (<code>maxClientVer</code>)	<code>""</code>	Versi klien Xray maksimum. Kosong – tanpa batas.
Short IDs (<code>shortIds</code>)	<code>[]</code> (dihasilkan saat diaktifkan)	Daftar identifikasi pendek (hex) yang membedakan klien. Input ganda dengan tag; tombol refresh menghasilkan kumpulan acak.
SpiderX (<code>settings.spiderX</code>)	<code>/</code>	Path «spider» (bagian klien dari REALITY), digunakan saat mengimitasi akses ke situs eksternal. Masuk ke tautan undangan.

Mulai versi 3.4.2, **Target** (`target`) dan **SNI** (`serverNames`) saat REALITY diaktifkan **tidak** lagi diisi secara otomatis – kedua field tetap kosong, dan target dipilih oleh pemindai langsung (lihat di bawah). Pilih situs HTTPS pihak ketiga yang besar dan stabil dengan dukungan TLS 1.3 dan HTTP/2, yang tidak berada di belakang server Anda sendiri.

Pencarian target REALITY (pemindai langsung)

Mulai versi 3.4.2, tombol «target acak» yang lama (dengan daftar bawaan statis) digantikan oleh dua tindakan di sebelah field **Target**:

- **«Pindai»** – memeriksa target saat ini «secara langsung»: membuat koneksi TLS dan, jika target cocok, mengisi SNI dari sertifikatnya. Toast: «Target cocok – target dan SNI telah diisi.» / «Target tersedia, tetapi tidak cocok untuk REALITY.» / «Gagal memindai target REALITY.».
- **«Cari target»** – membuka jendela **«Pemindai target REALITY»**: Anda dapat menentukan domain, **IP atau rentang CIDR** (panel kemudian menemukan target berdasarkan sertifikat host yang aktif), atau membiarkan field kosong (kandidat bawaan diperiksa). Hasil diberi peringkat; kolom – «Status»/«Cocok», «Pertukaran kunci», «Sertifikat», TLS, ALPN, «Latensi». Dengan tombol **«Gunakan»**, baris yang dipilih dimasukkan ke field inbound.

Target dianggap **cocok** jika server menegosiasikan **TLS 1.3, HTTP/2 (ALPN h2)**, pertukaran kunci **X25519/X25519MLKEM768**, dan menyajikan sertifikat **tepercaya** (bukan wildcard). Pemindaian dilakukan di sisi server panel (POST /panel/api/server/scanRealityTarget dan .../scanRealityTargets).

Contoh: blok streamSettings untuk REALITY pada jaringan tcp (VLESS). Sertifikat tidak diperlukan – sebagai gantinya adalah domain yang dipinjam dan pasangan kunci X25519:

```
"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": [ "", "6ba85179e30d4fc2" ],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
```

Di sini field **Target** (`target`) panel sesuai dengan `dest` dalam konfigurasi Xray yang dihasilkan. Jika inbound REALITY dibuat dengan destination di kunci `dest` (oleh versi panel lama, melalui API, atau alat eksternal), panel saat parsing menormalisasi `dest` → `target` ketika `target` kosong – sehingga inbound seperti ini dimuat dengan benar, field **Target** tidak kosong, dan penyimpanan ulang tidak merusak REALITY yang berfungsi.

Kunci REALITY (X25519)

Field	Default	Deskripsi
Kunci Publik (<code>settings.publicKey</code>)	""	Kunci publik X25519 (yang dimasukkan klien ke dalam konfigurasi/tautannya).
Kunci Privat (<code>privateKey</code>)	""	Kunci privat X25519 (hanya disimpan di server).

Tombol di bawah kunci:

- **Dapatkan Sertifikat Baru** – meminta pasangan kunci baru dari server (`GET /panel/api/server/getNewX25519Cert` ; pada server dijalankan `xray x25519`), mengisi **Kunci Privat** dan **Kunci Publik**. Pasangan yang sama dihasilkan secara otomatis saat mode REALITY pertama kali diaktifkan.

Contoh: mendapatkan pasangan kunci X25519 melalui API (di luar form, misalnya untuk skrip). Permintaan mengembalikan kunci privat dan publik:

```
curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# Respons:
# {"success":true,"obj":{"privateKey":"...","publicKey":"..."}}
```

`cookie.txt` – file cookie sesi yang diperoleh setelah login melalui `POST /login` .

- **Hapus** – mengosongkan kedua kunci.

Tanda Tangan Post-Kuantum ML-DSA-65 (mldsa65)

Lapisan autentikasi post-kuantum REALITY tambahan (opsional):

Field	Default	Deskripsi
mldsa65 Seed (<code>mldsa65Seed</code>)	""	Seed kunci ML-DSA-65 server.
mldsa65 Verify (<code>settings.mldsa65Verify</code>)	""	Nilai verifikasi (bagian klien, masuk ke tautan).

Tombol:

- **Dapatkan Seed Baru** – meminta pasangan baru (`GET /panel/api/server/getNewmldsa65` ; pada server dijalankan `xray mldsa65`), mengisi **mldsa65 Seed** dan **mldsa65 Verify**.
- **Hapus** – mengosongkan kedua field.

Pembatasan Kecepatan Fallback dan Log Kunci REALITY

Dalam pengaturan REALITY tersedia pembatasan kecepatan lalu lintas fallback – ini mencegah probe aktif menggunakan server sebagai saluran gratis ke domain yang dipinjam. Pengaturan dikonfigurasi secara terpisah untuk dua arah – **Limit Fallback Upload** dan **Limit Fallback Download** (`limitFallbackUpload` / `limitFallbackDownload`), masing-masing dengan set field yang sama:

Field (label)	Default	Deskripsi
After Bytes (afterBytes)	0	Berapa banyak byte yang diteruskan dengan kecepatan penuh sebelum pembatasan dimulai. 0 – membatasi sejak byte pertama.
Bytes Per Sec (bytesPerSec)	0	Batas kecepatan lalu lintas fallback dalam byte per detik setelah ambang batas. 0 – tanpa batas (menonaktifkan arah ini).
Burst Bytes Per Sec (burstBytesPerSec)	0	Cadangan untuk lonjakan singkat di atas kecepatan konstan (ukuran token-bucket). Jika lebih kecil dari Bytes Per Sec , dinaikkan ke nilainya.

Di sana juga ditambahkan field **Master Key Log** (masterKeyLog) – path untuk menyimpan TLS master keys dalam format `SSLKEYLOGFILE` untuk debugging di Wireshark; di production biarkan kosong.

7.5. Rekomendasi Praktis untuk Konfigurasi

1. **VLESS + Reality (direkomendasikan)**: buat inbound VLESS pada jaringan `tcp`, di tab «Keamanan» pilih **Reality** – panel akan secara otomatis memasukkan `target /SNI` acak, `shortIds`, dan menghasilkan kunci X25519. Jika diperlukan, klik «Dapatkan Sertifikat Baru» untuk pasangan kunci Anda sendiri. Untuk klien VLESS, aktifkan **Flow** = `xtls-rprx-vision` (XTLS Vision) – ini memberikan performa dan ketersembunyian maksimum.

Contoh: tautan klien akhir VLESS + Reality + Vision. Beginilah tampilan tautan undangan yang dihasilkan panel untuk inbound seperti ini (nilai kunci/ID bersifat ilustratif):

```
vless://uuid-klien@1.2.3.4:443?
type=tcp&security=reality&pbk=KUNCI_PUBLIK&fp=chrome&sni=www.nvidia.com&sid=6ba85179e30d4fc2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

Di sini `pbk` – kunci publik X25519, `sni` – domain yang dipinjam dari **Target**, `sid` – salah satu **Short IDs**, `flow=xtls-rprx-vision` – XTLS Vision yang diaktifkan. 2. **TLS dengan domain sendiri**: pilih **TLS**, isi **SNI** dengan nama domain, tambahkan sertifikat (melalui path ke file atau isi), atau klik «Pasang Sertifikat Panel» jika domain dan sertifikat sudah dikonfigurasi di «Pengaturan → Keamanan». Biarkan **Versi Min/Maks** = `1.2 – 1.3` dan **uTLS** = `chrome` untuk menyamakan sebagai browser biasa. 3. Jangan biarkan mode **Tidak Ada** untuk inbound yang terbuka ke luar – gunakan hanya untuk target fallback lokal (`127.0.0.1`) atau saat TLS disediakan oleh proxy eksternal. 4. Saran dari antarmuka: untuk sebagian besar field lanjutan, terdapat tooltip «Disarankan untuk membiarkan pengaturan pada nilai default» – ubah hanya jika Anda memahami konsekuensinya.

8. Klien

Klien adalah akun pengguna VPN: sekumpulan kredensial (UUID atau kata sandi) yang terikat ke satu atau beberapa inbound, dengan kuota lalu lintas, masa berlaku, dan batas koneksi simultan tersendiri. Dalam fork ini klien merupakan entitas mandiri (tabel `clients`): satu klien yang sama dapat ditautkan ke beberapa inbound sekaligus, dengan UUID/kata sandi yang sama dan penghitung lalu lintas yang sama. Bagian **Klien** menampilkan semua akun panel terlepas dari inbound, dilengkapi pencarian, filter, pengurutan, dan operasi massal.

8.1. Kolom Klien

Di bawah ini setiap kolom editor **Tambah klien** / **Ubah klien** diuraikan satu per satu.

Formulir klien dibagi menjadi dua tab: **Umum** (email, tautan ke inbound, batas, masa berlaku, grup, komentar, tag balik) dan **Kredensial** (UUID/kata sandi/auth, Flow, VMess Security). Pada label kolom, kuota ditampilkan sebagai **Batas Lalu Lintas (GB)**, dan masa berlaku sebagai **Durasi (hari)** dan **Perpanjangan Otomatis (hari)**; kolom **Batas Lalu Lintas (GB)** dan **Batas IP** memiliki petunjuk yang menjelaskan bahwa `0` berarti "tanpa batasan". Saat mengedit klien yang sudah ada, tombol pembuatan email acak disembunyikan, dan tombol log IP ditampilkan langsung di samping kolom **Batas IP** serta menampilkan jumlah alamat yang tercatat.

Kolom	Kunci JSON	Default	Deskripsi
Email	<code>email</code>	– (wajib)	Pengenal unik klien
UUID	<code>id</code>	dibuat otomatis	Pengenal untuk VMess/VLESS
Kata Sandi	<code>password</code>	dibuat otomatis	Kata sandi untuk Trojan/Shadowsocks
Otorisasi	<code>auth</code>	dibuat otomatis	Kata sandi untuk Hysteria
Flow	<code>flow</code>	kosong	Flow control (XTLS), hanya VLESS
VMess Security	<code>security</code>	<code>auto</code>	Metode enkripsi VMess
Batas IP	<code>limitIp</code>	<code>0</code> (tanpa batas)	Maksimum IP simultan
Total Dikirim/Diterima (GB)	<code>totalGB</code>	<code>0</code> (tanpa batas)	Kuota lalu lintas
Masa Berlaku	<code>expiryTime</code>	<code>0</code> (tidak terbatas)	Tanggal kedaluwarsa
Perpanjangan Otomatis	<code>reset</code>	<code>0</code> (nonaktif)	Periode reset lalu lintas, hari
ID Pengguna Telegram	<code>tgId</code>	<code>0</code> (tidak ada)	ID Telegram numerik
ID Langganan	<code>subId</code>	dibuat otomatis	Pengenal langganan
Grup	<code>group</code>	kosong	Label pengelompokan logis
Komentar	<code>comment</code>	kosong	Catatan bebas
Aktif	<code>enable</code>	<code>true</code>	Apakah akun aktif

Email (pengenal)

Kolom **Email** adalah pengenal utama dan wajib bagi klien. Meski bernama demikian, ini tidak harus berupa alamat email: label teks apa pun dapat digunakan (nama pengguna, nomor). Nilainya harus **unik** dalam panel; upaya membuat klien kedua dengan email yang sudah digunakan akan ditolak (`email already in use`), kecuali `subId` -nya juga cocok (ini diinterpretasikan sebagai tautan ke klien yang sama).

Email **tidak boleh dikosongkan** (`client email is required`) dan **tidak boleh mengandung spasi, karakter / , \ , maupun karakter kontrol** ("Email tidak boleh mengandung spasi, '/', ' ', atau karakter kontrol"). Email berperan dalam pencatatan lalu lintas, log IP, daftar online, dan nama operasi – mengubahnya setelah dibuat tidak disarankan.

UUID / Kata Sandi / Otorisasi (kredensial)

Kolom kredensial yang digunakan bergantung pada protokol inbound tempat klien ditautkan. Nilai diisi otomatis jika kolom dikosongkan:

- **UUID** (kolom `id`) – untuk protokol **VMess** dan **VLESS**. Jika tidak ditentukan, UUID v4 acak akan dibuat.
- **Kata Sandi** (kolom `password`) – untuk **Trojan** dan **Shadowsocks**. Untuk Trojan, secara default dibuat UUID tanpa tanda hubung. Untuk Shadowsocks, kunci dengan panjang yang sesuai dalam Base64 dibuat tergantung metode enkripsi inbound: 16 byte untuk `2022-blake3-aes-128-gcm` , 32 byte untuk `2022-blake3-aes-256-gcm` dan `2022-blake3-chacha20-poly1305` ; untuk metode lainnya – UUID tanpa tanda hubung. Jika kunci yang dimasukkan secara manual tidak sesuai dengan metode `2022-blake3`, kunci tersebut akan digantikan oleh kunci yang dibuat otomatis.
- **Otorisasi** (kolom `auth`) – kata sandi untuk **Hysteria**. Defaultnya adalah UUID tanpa tanda hubung.

Karena satu klien dapat ditautkan ke inbound dari berbagai protokol, rekaman klien dapat memiliki UUID, kata sandi, dan auth secara bersamaan – setiap protokol menggunakan kolomnya masing-masing.

Contoh: tampilan kredensial klien dalam settings berbagai inbound. Klien yang sama diidentifikasi berdasarkan `id` pada inbound VLESS, berdasarkan `password` pada Trojan, dan berdasarkan `password` (kunci Base64) pada Shadowsocks:

```
// fragmen settings.clients pada inbound VLESS
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-vision" }

// klien yang sama pada inbound Trojan
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// klien yang sama pada inbound Shadowsocks (metode 2022-blake3-aes-256-gcm)
{ "password": "GPy0aA3f7C00az53eaQ8eqMfRDjmBl0h7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow (kolom `flow`) – kontrol aliran XTLS. Berlaku **hanya untuk VLESS** dan hanya jika inbound dikonfigurasi untuk XTLS Vision: VLESS melalui transport **TCP** dengan security **tls** atau **reality** . Nilai yang diizinkan

adalah `xtls-rprx-vision` (dan secara historis `xtls-rprx-vision-udp443`); nilai kosong berarti tidak ada flow.

Jika inbound tidak mendukung XTLS-flow, flow yang ditentukan **akan dihapus secara diam-diam** saat klien disimpan: untuk klien yang sama yang terikat ke beberapa inbound, flow hanya diterapkan di tempat yang diizinkan. Ubah hanya jika Anda sengaja menggunakan VLESS-Vision.

VMess Security

VMess Security (kolom `security`) – metode enkripsi payload untuk VMess. Nilai default adalah `auto` (Xray memilih cipher sendiri). Nilai yang diizinkan adalah standar VMess: `auto`, `aes-128-gcm`, `chacha20-poly1305`, `none`, `zero`. Untuk protokol lain, kolom ini tidak digunakan.

Batas IP (koneksi simultan)

Batas IP (kolom `limitIp`) – jumlah maksimum **alamat IP berbeda** yang dapat terhubung ke klien secara bersamaan. Nilai default adalah `0`, yang berarti **tanpa batasan**. Dengan nilai positif, panel memantau IP aktif klien dan, jika batas terlampaui, menonaktifkan akun melalui tugas latar belakang. (Mulai dari **3.3.1** penghitungan IP dilakukan melalui API online-stats inti Xray dan **tidak memerlukan** log akses; pada versi inti yang lebih lama, panel kembali membaca log akses, yang harus diaktifkan.) Gunakan untuk mencegah berbagi satu langganan ke banyak perangkat: misalnya, `2` – izinkan dua perangkat.

Batas IP diterapkan menggunakan **Fail2ban**, sehingga kolom **Batas IP** hanya aktif jika Fail2ban terinstal dan berfungsi (panel memeriksa statusnya melalui `GET /panel/api/server/fail2banStatus`). Jika Fail2ban tidak terinstal, kolom editor klien (dan formulir penambahan massal) diblokir, dan saat diarahkan ke sana muncul petunjuk untuk menginstal Fail2ban dari menu bash `x-ui` ("Fail2ban is not installed, so the IP limit cannot be enforced. Install Fail2ban from the x-ui bash menu to enable this option."); di Windows, petunjuk menyatakan bahwa Fail2ban tidak tersedia ("Fail2ban is not available on Windows, so the IP limit cannot be enforced."), dan jika fitur dinonaktifkan di server – "The IP limit feature is disabled on this server". Saat memperbarui panel, batas IP klien di server tanpa Fail2ban di-nol-kan oleh migrasi satu kali, karena memang tidak diterapkan di sana.

Contoh nilai. `limitIp: 0` – tanpa batasan; `limitIp: 1` – tepat satu perangkat dalam satu waktu; `limitIp: 3` – hingga tiga IP berbeda. Pada IP aktif keempat, tugas latar belakang akan menonaktifkan klien (`enable = false`) sampai Anda menjalankan **Reset Batas IP**.

Operasi terkait: **Log IP** menampilkan daftar IP klien yang tercatat; setiap entri berisi, selain IP itu sendiri, waktu akses terakhir dan label node (`@ nama_node`), tempat koneksi dicatat – dalam konfigurasi multi-panel terlihat melalui node mana klien terhubung. **Reset Batas IP** membersihkan log IP yang terkumpul agar klien dapat terhubung kembali tanpa menunggu entri kedaluwarsa secara alami.

Total Dikirim/Diterima (GB) – kuota lalu lintas

Total Dikirim/Diterima (GB) (kolom `totalGB`) – kuota lalu lintas total (kirim + terima). Nilai default `0` berarti **tanpa batas**. Setelah kuota tercapai (`up + down >= total`), klien dianggap **habis** (depleted) dan dinonaktifkan. Di UI biasanya dimasukkan dalam gigabyte; dalam basis data disimpan dalam byte.

Dalam daftar klien, kolom **Lalu Lintas** menampilkan bilah penggunaan berwarna: jumlah lalu lintas yang terpakai, label batas (atau tanda ∞ untuk tanpa batas) dan petunjuk saat diarahkan dengan rincian kirim/terima dan sisa. Indikator ringkas yang sama ditampilkan pada kartu klien di ponsel.

Masa Berlaku (Expiry)

Masa Berlaku (kolom `expiryTime`) menentukan saat kedaluwarsa akun. Kolom ini memiliki tiga mode:

- **Tidak terbatas** — `0`. Klien tidak pernah kedaluwarsa berdasarkan waktu.
- **Tanggal tertentu** — Unix-timestamp positif (dalam milidetik). Saat tiba (`expiryTime <= sekarang`), klien dianggap kedaluwarsa (expired) dan dinonaktifkan. Di UI biasanya ditentukan dengan memilih tanggal atau durasi dalam hari (**Durasi**, satuan — **Hari**).
- **Mulai setelah penggunaan pertama** — nilai **negatif** yang mengodekan durasi. Selama klien belum mengirimkan satu byte pun, masa berlaku tetap negatif ("mulai tertunda"). Pada tick penghitungan lalu lintas pertama, panel mengonversinya menjadi tanggal absolut: `sekarang + |durasi|`. Ini memungkinkan penjualan, misalnya, "30 hari sejak koneksi pertama", tanpa mengetahui terlebih dahulu kapan klien akan aktif. Konversi dilakukan satu kali per email, sehingga semua inbound yang terikat mendapat masa berlaku yang sama.

Contoh pengkodean masa berlaku. Tanggal tetap 1 Maret 2026, 00:00 UTC → `expiryTime: 1772323200000` (timestamp positif dalam milidetik). "30 hari sejak koneksi pertama" → `expiryTime: -2592000000` (nilai negatif, $30 \times 24 \times 60 \times 60 \times 1000$); pada byte lalu lintas pertama, panel akan menggantinya dengan `sekarang + 2592000000`. Tidak terbatas → `expiryTime: 0`.

Perpanjangan Otomatis (periode reset lalu lintas klien)

Kolom **Perpanjangan Otomatis** (kolom `reset`) adalah periode perpanjangan/reset otomatis dalam hari. Petunjuk: "Perpanjangan otomatis setelah berakhir. (0 = nonaktif) (satuan: hari)".

- `0` — perpanjangan otomatis **dinonaktifkan** (nilai default). Setelah masa berlaku berakhir, klien hanya menjadi habis.
- `> 0` — tugas latar belakang saat masa berlaku habis **mereset penghitung lalu lintas ke nol** (`up = down = 0`), **menggeser masa berlaku ke depan** sebesar `reset` hari (jika perlu — beberapa periode, sampai masa berlaku baru berada di masa depan) dan jika perlu **mengaktifkan kembali** klien. Ini mengimplementasikan langganan berkala (misalnya, bulanan). Perpanjangan otomatis **tidak diterapkan ke inbound di node** (`node_id IS NOT NULL`).

Konsekuensi penting: klien dengan `reset > 0` **dikecualikan** dari konsep "habis" dalam operasi penghapusan massal — lalu lintas/masa berlaku mereka memang di-nol-kan oleh perpanjangan otomatis, bukan menjadikan akun sebagai kandidat penghapusan.

ID Pengguna Telegram

ID Pengguna Telegram (kolom `tgId`) — pengenalan Telegram numerik pengguna untuk tautan ke bot Telegram panel bawaan (notifikasi, melihat statistik secara mandiri). Petunjuk: "ID pengguna Telegram numerik (0 = tidak ada)". Nilai default `0` — tidak ada tautan. Kolom ini tersedia untuk filter (**Ada / Tidak Ada**).

ID Langganan (subId)

ID Langganan (kolom `subId`) – pengenal yang digunakan klien untuk dimasukkan dalam **langganan** (subscription). Semua klien dengan `subId` yang sama disajikan melalui satu tautan langganan. Jika kolom dikosongkan saat membuat, panel akan **secara otomatis membuat** `subId` acak (UUID). Nilainya harus **unik** di antara klien dengan email berbeda (`subId already in use`) dan tunduk pada batasan karakter yang sama dengan email ("ID Langganan tidak boleh mengandung spasi, '/', atau karakter kontrol").

Tanpa `subId`, tautan langganan untuk klien tidak tersedia ("Klien ini tidak memiliki subId, tautan berbagi tidak tersedia").

Tab Links (tautan eksternal dan langganan)

Selain tab **Umum** dan **Kredensial**, editor klien memiliki tab ketiga **Links** (petunjuk: "Add third-party share links and remote subscription URLs to include in this client's subscription."). Di sini tombol **Add External Link** menambahkan tautan berbagi pihak ketiga (`vless://`, `vmess://`, `trojan://`, `ss://`, `hysteria2://`, `wireguard://`), dan tombol **Add External Subscription** menambahkan URL langganan jarak jauh (misalnya, `https://provider.example/sub/...`).

Semua ini disertakan dalam output langganan klien tersebut (format raw, JSON, dan Clash): tautan ditambahkan apa adanya, sementara langganan jarak jauh diunduh panel secara berkala (dengan cache dan batas waktu singkat) dan konfigurasinya digabungkan dengan konfigurasi sendiri. Dengan cara ini, dalam satu tautan langganan klien, server eksternal dapat disajikan bersama server sendiri.

Grup

Grup (kolom `group`) – label logis untuk mengelompokkan klien terkait. Petunjuk: "Label logis untuk mengelompokkan klien terkait (misalnya, tim, pelanggan, wilayah). Dapat difilter dari toolbar.", placeholder – "misalnya, customer-a". Kolom ini opsional (default kosong). Daftar dapat difilter berdasarkan grup dan operasi massal dapat dilakukan; untuk menghapus label dari klien, gunakan tindakan **Pisah dari Grup**.

Label grup juga dapat dihapus langsung dari editor satu klien: jika mengosongkan kolom **Grup** dan menyimpan, label akan dihapus dengan benar dan klien tidak lagi ditampilkan di bawah grup sebelumnya.

Komentar

Komentar (kolom `comment`) – catatan teks bebas untuk administrator (default kosong). Isinya termasuk dalam pencarian dan tersedia untuk filter (**Ada / Tidak Ada** komentar).

Aktif

Aktif (kolom `enable`) – flag keaktifan akun. Default **aktif** (`true`); saat membuat, bahkan jika flag tidak dikirimkan, panel secara paksa menetapkan `true`. Klien yang dinonaktifkan (`enable = false`) tidak dapat terhubung dan dalam ringkasan termasuk kategori **tidak aktif** (deactive). Panel sendiri menonaktifkan klien yang kehabisan kuota, kedaluwarsa, atau melampaui batas IP.

Kolom Hanya Baca

Kartu klien juga menampilkan kolom layanan: **Dibuat** (`created_at`) dan **Diperbarui** (`updated_at`) – stempel waktu pembuatan dan perubahan terakhir, diisi otomatis dan tidak dapat diedit. Kolom **Tag Balik** (`reverse`) – Reverse tag opsional untuk reverse proxy VLESS sederhana ("Tag Balik Opsional").

8.2. Tautan ke Inbound

Setiap klien harus ditautkan ke setidaknya satu inbound – saat membuat diperlukan minimal satu (`at least one inbound is required`). Di editor ini adalah kolom **Inbound Terikat** dengan petunjuk **Pilih satu atau beberapa inbound**.

- **Tautkan** – menambahkan klien ke inbound yang dipilih (UUID/kata sandi yang sama dan lalu lintas bersama). Tautan yang sudah ada tetap dipertahankan.
- **Lepas Tautan** – menghapus klien dari inbound yang dipilih. Rekaman klien itu sendiri tetap disimpan (untuk penghapusan penuh, gunakan **Hapus**). Pasangan di mana klien tidak tertaut diabaikan secara diam-diam.

Saat menyimpan klien yang terikat ke beberapa inbound, kolom yang tidak kompatibel dengan protokol/transport tertentu (misalnya, Flow di luar VLESS-Vision) secara otomatis disesuaikan ke nilai yang diizinkan untuk setiap inbound.

Di atas daftar pilihan inbound (dalam formulir klien, saat penambahan massal klien, dan di jendela pemasangan/pelepasan massal) terdapat tombol **Pilih Semua** dan **Bersihkan**. Dalam daftar ini, setiap inbound diberi label keterangannya (remark) jika ada, atau tag inbound jika tidak ada.

8.3. Operasi pada Klien

Untuk klien individual (melalui kartu **Informasi Klien** atau menu konteks **Tindakan**) tersedia:

Melihat Informasi, Kode QR, dan Tautan

- **Informasi Klien** – kartu dengan semua kolom, lalu lintas terpakai/tersisa (**Sisa**), masa berlaku, dan inbound yang terikat.

Permintaan klien melalui API (`GET /panel/api/clients/get/:email`) di samping kolom `client` dan `inboundIds` juga mengembalikan `usedTraffic` – lalu lintas yang benar-benar terpakai (kirim + terima, termasuk data dari node), yang mempermudah perbandingan penggunaan dengan kuota `totalGB`.

- **Kode QR dan Tautan** – tautan konfigurasi klien untuk diimpor ke aplikasi klien. Dibentuk berdasarkan semua inbound yang terikat dengan protokol yang didukung (`GET /links/:email`). Jika tidak ada tautan yang sesuai: "Tidak ada tautan berbagi – tautkan klien ke inbound dengan protokol yang didukung terlebih dahulu."
- **Tautan Langganan** – URL langganan berdasarkan `subId` (`GET /subLinks/:subId`). Tersedia hanya jika klien memiliki `subId` dan layanan langganan diaktifkan di **Pengaturan Panel** → **Langganan** (jika tidak, "Layanan langganan dinonaktifkan."). Selain itu, **URL Langganan JSON** juga disediakan.

Reset Lalu Lintas

Reset Lalu Lintas (`POST /resetTraffic/:email`) mengosongkan penghitung kirim/terima (`up` , `down`) klien tertentu. Kuota (`totalGB`) dan masa berlaku **tidak terpengaruh** – hanya jumlah yang terpakai yang di-nol-kan. Toast: "Lalu lintas direset". Jika klien tidak terikat ke inbound mana pun: "Tautkan klien ini ke inbound terlebih dahulu".

Tombol **Reset Lalu Lintas** juga tersedia dari formulir edit klien – di panel bawah, di samping **Batal / Simpan** (konfirmasi diminta sebelum reset). Jika klien dinonaktifkan karena kehabisan lalu lintas, reset (baik individual maupun massal) secara otomatis **mengaktifkan kembali** klien (`enable = true`) dan langsung menyebarkan perubahan ini ke node – tidak perlu mengaktifkan ulang klien secara manual di master dan node.

Reset Batas IP

Membersihkan log IP klien yang terkumpul (`POST /clearIps/:email`) untuk menghapus pemblokiran sementara karena melampaui batas koneksi simultan. Toast: "Log telah dibersihkan".

Hapus

Hapus (`POST /del/:email`) – penghapusan klien secara penuh. Dialog konfirmasi: judul "Hapus klien {email}?", teks "Klien akan dihapus dari semua inbound yang terikat, dan rekaman lalu lintasnya akan dihancurkan. Tindakan ini tidak dapat dibatalkan.". Penghapusan melepas klien dari **semua** inbound dan menghancurkan rekaman lalu lintasnya. Toast: "Klien dihapus".

8.4. Operasi Massal

Dalam daftar klien, beberapa rekaman dapat dipilih (**Pilih Semua**, **Hapus Semua**); penghitung – "{count} dipilih". Untuk yang dipilih, tersedia:

- **Hapus ({count})** (`POST /bulkDel`) – penghapusan grup. Konfirmasi: "Hapus {count} klien?", "Setiap klien yang dipilih dihapus dari semua inbound yang terikat, rekaman lalu lintasnya dihancurkan. Tindakan ini tidak dapat dibatalkan.". Toast: "Klien dihapus: {count}", jika ada kegagalan sebagian – "Berhasil: {ok}, gagal: {failed}".
- **Ubah ({count}) / Penyesuaian** (`POST /bulkAdjust`) – perubahan massal masa berlaku dan/atau kuota. Dialog "Ubah {count} klien" dengan petunjuk "Nilai positif menambah, nilai negatif mengurangi. Klien dengan masa berlaku atau lalu lintas tidak terbatas dilewati untuk kolom yang sesuai.". Kolom: **Tambah Hari**, **Tambah Lalu Lintas (GB)**, dan **Set flow**. Logika:
 - **Masa Berlaku:** klien dengan masa berlaku tidak terbatas (`expiryTime == 0`) dilewati ("unlimited expiry"); untuk klien dengan tanggal, masa berlaku digeser sebesar jumlah hari yang ditentukan; untuk klien dalam mode "setelah penggunaan pertama" (masa berlaku negatif), durasi tunggu disesuaikan. Pengurangan yang melebihi sisa dilewati ("reduction exceeds remaining time/delay window").
 - **Lalu Lintas:** klien tanpa batas (`totalGB == 0`) dilewati ("unlimited traffic"); jika tidak, kuota diubah sebesar jumlah yang ditentukan, tidak kurang dari nol.
 - **Flow:** daftar dropdown **Set flow** memungkinkan pengaturan atau penghapusan XTLS flow sekaligus untuk semua klien yang dipilih. Secara default dipilih **No change** (tanpa perubahan). Opsi **Disable (clear flow)** menghapus flow, sedangkan nilai `xtls-rprx-vision` dan `xtls-rprx-vision-udp443`

menetapkan vision-flow yang sesuai. Pengaturan vision-flow hanya diterapkan ke inbound yang mendukung flow; inbound yang tidak sesuai dibiarkan tidak berubah dan ditandai sebagai dilewati, sementara penghapusan flow selalu diizinkan.

- **Aktifkan otomatis (mulai 3.4.2):** klien yang dinonaktifkan **hanya karena habis** (masa berlaku berakhir atau kuota terlampaui), saat diperpanjang akan otomatis diaktifkan kembali – secara lokal dan pada node-nya – jika penyesuaian mengembalikannya ke dalam batas. Klien yang dinonaktifkan secara manual atau yang masih habis tetap nonaktif (field `enable` kini ditulis secara eksplisit, sehingga status tersimpan bahkan untuk klien tanpa inbound).
- Jika hari, lalu lintas, maupun flow tidak ditentukan: "Tentukan hari, lalu lintas, atau flow sebelum menerapkan.". Toast: "Diubah: {count}" / "Diubah: {ok}, dilewati: {skipped}."

Contoh: perpanjang klien yang dipilih selama 30 hari dan tambahkan 50 GB. Dalam dialog **Ubah**, masukkan **Tambah Hari** = `30`, **Tambah Lalu Lintas (GB)** = `50`. Untuk sebaliknya, mengurangi seminggu dan memotong kuota sebesar 10 GB, masukkan nilai negatif: **Tambah Hari** = `-7`, **Tambah Lalu Lintas (GB)** = `-10` (klien dengan masa berlaku tidak terbatas atau tanpa batas lalu lintas untuk kolom yang sesuai akan dilewati).

- **Tautkan ({count}) / Lepas Tautan ({count})** (`POST /bulkAttach / bulkDetach`) – pemasangan/pelepasan massal klien yang dipilih ke inbound yang dipilih. Target hanya inbound multi-pengguna. Hasil pelepasan: "Dilepas {detached}, dilewati {skipped}."
- **Tautan Sub ({count})** – tabel ringkasan URL langganan dan langganan JSON klien yang dipilih dengan tombol **Salin Semua**. Jika tidak ada yang memiliki subId: "Tidak ada satu pun klien yang dipilih memiliki ID Langganan."
- **Tambah ke Grup dan Pisah dari Grup** – penetapan dan penghapusan label grup.
- **Aktifkan ({count}) / Nonaktifkan ({count})** (`POST /bulkEnable / bulkDisable`) – pengaktifan dan penonaktifan massal klien yang dipilih. **Aktifkan** mengaktifkan setiap klien yang dipilih pada semua inbound yang terikat; klien dengan kuota lalu lintas habis atau masa berlaku kedaluwarsa akan dinonaktifkan kembali secara otomatis. **Nonaktifkan** langsung mencabut akses klien, tetapi rekaman dan lalu lintas yang terkumpul tetap disimpan. Sebelum menjalankan, panel meminta konfirmasi, dan setelah operasi menampilkan notifikasi dengan jumlah klien yang diproses serta, jika ada, jumlah yang gagal.

Reset Lalu Lintas dan Penghapusan Berdasarkan Status

- **Reset lalu lintas semua klien** (`POST /resetAllTraffics`) – mengosongkan penghitung `up / down` pada **semua** klien panel. Konfirmasi: "Reset lalu lintas semua klien?" dan "Penghitung kirim/terima semua klien di-nol-kan. Kuota dan masa berlaku tidak terpengaruh. Tindakan ini tidak dapat dibatalkan.". Toast: "Lalu lintas semua klien direset".
- **Hapus yang Habis** (`POST /delDepleted`) – menghapus semua klien yang **kuotanya habis** (`total > 0 and up + down >= total`) **atau masa berlakunya kedaluwarsa** (`expiry_time > 0 and expiry_time <= sekarang`), dengan syarat `reset = 0` (klien dengan perpanjangan otomatis tidak disentuh). Konfirmasi: "Hapus klien yang habis?", "Semua klien yang kuota lalu lintasnya habis atau masa berlakunya kedaluwarsa akan dihapus. Tindakan ini tidak dapat dibatalkan.". Toast: "Klien habis yang dihapus: {count}".

Ekspor, Impor, dan Penghapusan Klien Tanpa Tautan

Saat tidak ada yang dipilih, di menu **Lainnya** pada halaman **Klien** tersedia tiga operasi.

Ekspor Klien (GET /clients/export) membuka penampil dengan daftar JSON semua klien dalam format {client, inboundIds} dengan tombol salin dan unduh (file clients-export.json). **Impor Klien** (POST /clients/import) membuka editor tempat JSON tersebut ditempel dan **Import** diklik: klien dengan inboundIds dibuat dan ditautkan ke inbound, klien tanpa tautan dipulihkan sebagai rekaman "kosong" terpisah, sedangkan email yang sudah ada **tidak pernah ditimpa** – mereka masuk ke daftar yang dilewati. Toast: "{count} clients imported", "{ok} imported, {failed} skipped".

Hapus Klien Tanpa Inbound (POST /clients/deleteOrphans) – operasi berbahaya: menghapus semua klien yang tidak terikat ke inbound mana pun, beserta rekaman lalu lintas, log IP, dan tautan eksternalnya. Konfirmasi: "Delete clients without an inbound?", "Removes every client that is not attached to any inbound, along with its traffic record. This cannot be undone.". Toast: "{count} unattached clients deleted". Tindakan ini tidak dapat dibatalkan.

8.5. Pencarian, Filter, dan Pengurutan

Di atas daftar terdapat kolom pencarian ("Cari email, komentar, sub ID, UUID, kata sandi, auth...") – pencarian dilakukan berdasarkan email, komentar, subId, UUID, kata sandi, dan auth. Penghitung hasil: "Ditampilkan {shown} dari {total}".

Daftar klien diperbarui secara otomatis: panel secara berkala mengambil halaman saat ini, sehingga klien yang baru terhubung dan urutan pengurutan yang berubah muncul tanpa pembaruan manual (indikator pemuatan tidak berkedip saat polling latar belakang).

Panel **Filter Klien** memungkinkan pemfilteran berdasarkan status (kategori), protokol, inbound yang terikat, rentang masa berlaku, rentang lalu lintas terpakai, keberadaan perpanjangan otomatis (**Ada/Tidak Ada**), keberadaan Telegram ID dan komentar, serta grup. Pada panel dengan node, muncul multi-select **Node**: daftar dapat dibatasi ke klien dari node yang dipilih; item terpisah **Panel Lokal** memfilter klien inbound tanpa tautan ke node (filter hanya terlihat jika ada node). Pengurutan: **Terlama/Terbaru, Baru Diperbarui, Baru Online, Email A→Z / Z→A, Lalu Lintas Terbanyak, Sisa Terbanyak, Hampir Kedaluwarsa**.

8.6. Ikon dan Status

Prioritas status: habis/kedaluwarsa → tidak aktif → hampir kedaluwarsa → aktif.

- **Online / Offline** – klien dengan koneksi aktif (ada dalam daftar online saat ini) dan **aktif**. Daftar online diperbarui oleh permintaan terpisah (/onlines , /onlinesByGuid).
- **Habis** (depleted) – kuota terpakai (up + down >= totalGB) **atau** masa berlaku kedaluwarsa (expiryTime <= sekarang). Klien tersebut dinonaktifkan secara otomatis dan terkena tindakan **Hapus yang Habis**.
- **Hampir Kedaluwarsa / Hampir Habis** (expiring) – klien yang diaktifkan yang masa berlakunya tersisa kurang dari interval ambang batas **atau** kuotanya tersisa kurang dari jumlah ambang batas (ambang batas ditentukan dalam pengaturan panel). Tidak dihitung jika klien sudah habis/dinonaktifkan.
- **Tidak Aktif** (deactive) – klien dengan enable = false (dininonaktifkan secara manual atau oleh tugas latar belakang).
- **Aktif** (active) – diaktifkan, tidak habis, masa berlaku tidak kedaluwarsa, dan masih jauh dari ambang batas.

9. Grup Klien

Ini adalah fitur fork 3X-UI ini. Pada proyek 3x-ui asli (MHSanaei) tidak ada konsep "grup klien" – di sini ditambahkan tabel grup tersendiri, halaman **Grup** di menu panel, dan metode API yang sesuai. Jika Anda memindahkan konfigurasi ke 3x-ui asli, label grup hanya tidak akan diperhitungkan di mana pun.

9.1. Apa itu grup klien dan untuk apa

Grup adalah label logis bernama (label) yang dapat ditempelkan pada satu atau beberapa klien. Grup tidak membuat cara koneksi baru dan bukan merupakan inbound maupun node – ini murni label organisasi yang memudahkan penyaringan dan pemrosesan massal klien.

Ide utama model klien di fork ini: **klien adalah entitas tingkat atas yang diidentifikasi berdasarkan email** (kolom `email` di tabel `clients` memiliki indeks unik). Klien yang sama (satu email dengan kredensial yang sama) dapat terdaftar di beberapa inbound sekaligus dan bahkan di beberapa node, termasuk dengan protokol yang berbeda. Label grup disimpan **satu kali per klien**, sehingga secara otomatis berlaku untuk semua binding klien ke inbound sekaligus.

Label grup adalah label logis untuk pengelompokan:

Lapisan	Tempat penyimpanan	Kolom
Catatan klien (DB)	tabel <code>clients</code>	<code>group_name</code> (defaultnya string kosong <code>' '</code>)
Daftar referensi grup (DB)	tabel <code>client_groups</code>	<code>name</code> (indeks unik, tidak boleh kosong)
Pengaturan inbound (Xray)	JSON <code>settings.clients[].group</code>	diduplikasi ke setiap objek klien di setiap inbound tempat klien terdaftar

Mengapa ini diperlukan dalam praktik:

- **Satu klien di beberapa inbound/node.** Jika klien "dijual" sebagai akses ke beberapa inbound sekaligus (misalnya protokol berbeda atau node berbeda), grup memungkinkan pengelolaan sebagai satu kesatuan: mereset traffic, menghapus, mengganti nama label – dalam satu operasi untuk semua inbound-nya.
- **Operasi massal dan penyaringan.** Di halaman **Klien**, daftar dapat difilter berdasarkan grup; di halaman **Grup** tersedia tindakan massal terhadap semua anggota grup.
- **Pengorganisasian kumpulan klien yang besar.** Label seperti `vip`, `trial`, `team-A` membantu menempatkan ribuan klien ke dalam keranjang logis tanpa harus membuat banyak inbound terpisah.

9.2. Hubungan grup dengan klien, inbound, node, dan protokol

Ini adalah subbagian terpenting untuk dipahami, karena sinkronisasi label tidaklah sepele.

Grup mendeskripsikan klien, bukan inbound. Label hidup di catatan klien (`clients.group_name`). Ketika klien terdaftar di beberapa inbound, setiap kali grup berubah, panel menelusuri **semua** inbound tempat klien tersebut terdaftar dan menetapkan/menghapus kolom `group` di dalam pengaturan Xray-nya (`settings.clients[]`). Secara teknis ini dilakukan sebagai berikut: berdasarkan email klien, semua inbound

tempat klien terdaftar ditemukan, lalu objek klien dengan email tersebut diperbaiki di JSON pengaturan masing-masing inbound. Oleh karena itu:

- Grup **tidak bergantung pada protokol**. Satu email bisa menjadi klien VLESS di satu inbound dan klien Hysteria di inbound lain – label grup tetap satu dan akan diterapkan ke keduanya (kredensial untuk setiap protokol berbeda dan disimpan secara terpisah).
- Grup **mencakup node**. Inbound yang dimiliki node berbeda dari inbound panel utama berdasarkan kolom `nodeId` (pada inbound panel utama nilainya `null / 0`). Label grup disebarkan ke objek klien di inbound terlepas dari apakah itu inbound utama atau inbound node – selama klien dengan email tersebut ada di sana.

Label grup tahan terhadap sinkronisasi dari node dan terhadap pembangunan ulang pengaturan. Perilaku ini dirancang secara khusus:

- Ketika node mengirimkan snapshot traffic, datanya **tidak mengisi** `group_name` dan `comment` lokal klien di DB panel. Grup dan komentar dianggap sebagai kolom "lokal panel" – node tidak mengelolanya.
- Saat pembangunan ulang pengaturan inbound, nilai `group` kosong dalam data yang masuk **tidak mereset** label yang sudah tersimpan. Grup dikelola khusus melalui halaman **Grup** (bukan melalui pengeditan pengaturan Xray inbound), sehingga "grup kosong" pada pembangunan ulang pengaturan biasa diartikan sebagai "jangan diubah", bukan "hapus".

Kesimpulan praktis: satu-satunya operasi yang **secara sengaja menghapus** label adalah penghapusan grup dan penghapusan klien dari grup secara eksplisit (lihat di bawah). Pengeditan inbound biasa atau sinkronisasi latar belakang dengan node tidak akan menghilangkan grup.

9.3. Daftar referensi grup dan grup "kosong"

Daftar grup di halaman dibentuk dari penggabungan dua sumber:

1. **Grup turunan (derived)** – semua nilai `group_name` tidak kosong yang benar-benar ditemukan pada klien, dengan hitungan jumlah klien.
2. **Grup tersimpan (stored)** – catatan dari tabel `client_groups`.

Penggabungan ini menghasilkan efek penting: grup dapat ada **tanpa satu pun klien**. Grup seperti itu dibuat melalui tombol "Tambah Grup" yang eksplisit (catatan di `client_groups`) dan ditampilkan dalam daftar dengan penghitung `0`. Catatan inilah yang disebut **grup kosong**. Daftar selalu diurutkan berdasarkan nama tanpa memperhatikan huruf kapital.

Penghitung ringkasan di halaman:

Kolom (RU)	Yang ditampilkan
Всего групп	Jumlah total grup (tersimpan dan turunan digabungkan).
Клиенты с группой	Berapa banyak klien yang memiliki label grup tidak kosong.
Пустые группы	Berapa banyak grup yang ada tanpa klien (penghitung <code>0</code>).
Клиентов в группе	Jumlah klien di grup tertentu (kolom tabel).

9.4. Kolom dan bidang grup

Catatan grup di tabel `client_groups` berisi:

Kolom	Tipe	Default	Deskripsi
<code>Id</code>	int	auto-increment	Kunci primer catatan grup.
<code>Name</code>	string	– (wajib diisi)	Nama grup. Indeks unik, tidak boleh kosong. Di UI – kolom Nama .
<code>CreatedAt</code>	int64 (ms)	waktu pembuatan	Waktu pembuatan catatan grup.
<code>UpdatedAt</code>	int64 (ms)	waktu perubahan	Waktu perubahan terakhir.

Tabel di halaman menampilkan setidaknya kolom **Nama** dan **Jumlah Klien di Grup**, serta tombol tindakan (lihat di bawah).

9.5. Membuat grup

Tombol **Tambah Grup**.

Langkah-langkah:

1. Klik **Tambah Grup**.
2. Masukkan nama grup.
3. Konfirmasi.

Perilaku backend (`POST /panel/api/clients/groups/create` , body `{"name": "..."}:`

- Nama dipangkas spasi di kedua ujung. Nama kosong ditolak dengan error "group name is required".
- Jika grup dengan nama tersebut sudah ada – error "group already exists".
- Jika berhasil, catatan dibuat di `client_groups` (awalnya tanpa klien – ini adalah grup kosong).

Pesan sukses: **«Grup «{name}» telah dibuat.»**

Contoh: membuat grup kosong melalui API. Siapkan sekumpulan label terlebih dahulu, sebelum diisi klien:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"name": "vip"}'
```

Respons jika berhasil:

```
{ "success": true, "msg": "Группа «vip» создана.", "obj": null }
```

Pemanggilan ulang dengan nama yang sama akan mengembalikan `"success": false` dan pesan `group already exists`.

Membuat grup kosong terlebih dahulu berguna ketika Anda ingin menyiapkan sekumpulan label dan kemudian mengisinya dengan klien melalui "Tambahkan klien...".

9.6. Mengganti nama grup

Tombol **Ganti Nama**, judul dialog – «**Ganti nama {name}**».

Perilaku (`POST /panel/api/clients/groups/rename` , body `{"oldName": "...", "newName": "..."}:`

- Kedua nama dipangkas spasi. Nama lama kosong – error "old group name is required", nama baru kosong – "new group name is required".
- Jika nama baru sama dengan nama lama – tidak ada yang dilakukan (0 klien terpengaruh).
- Selain itu, penggantian nama dilakukan secara atomik:
 - catatan di `client_groups` diubah namanya;
 - pada semua klien dengan `group_name = oldName` , kolom diperbarui ke `newName` ;
 - di **semua inbound** tempat klien yang terpengaruh terdaftar (termasuk node), nilai `group` di pengaturan Xray diubah dari yang lama ke yang baru.
- Setelah penggantian nama, panel menandai Xray sebagai memerlukan restart dan mengirimkan notifikasi perubahan klien.

Pesan:

- Sukses: «**Grup diganti nama untuk {count} klien.**»
- Konflik nama di UI: «**Grup dengan nama «{name}» sudah ada.**»

9.7. Menambahkan klien ke grup

Tombol **Tambahkan klien...**, judul – «**Tambahkan klien ke grup «{name}»**».

Keterangan persis di dialog:

«Pilih klien untuk ditambahkan ke grup ini. Binding yang ada ke inbound tetap dipertahankan; hanya label grup yang berubah. Klien yang sudah termasuk dalam grup ini tidak ditampilkan.»

Jika tidak ada yang bisa ditambahkan, ditampilkan «**Tidak ada klien lain untuk ditambahkan.**»

Perilaku (`POST /panel/api/clients/groups/bulkAdd` , body `{"emails": [...], "group": "..."}:`

- Nama grup wajib diisi (jika tidak, error "group name is required"); daftar email kosong – operasi tidak melakukan apa pun.
- Jika grup tersebut belum ada di `client_groups` maupun di antara grup turunan – grup akan dibuat secara otomatis.
- Untuk email yang dipilih, `group_name = group` ditetapkan pada klien; **binding klien ke inbound tidak berubah** – hanya label yang terpengaruh. Kemudian kolom `group` ditetapkan di semua inbound klien tersebut.
- Mengembalikan jumlah catatan klien yang terpengaruh; Xray ditandai untuk di-restart.

Pesan sukses: «**{count}** klien ditambahkan ke **{name}**.»

Contoh: menandai beberapa klien dengan grup dalam satu permintaan. Klien ditentukan berdasarkan email, binding ke inbound tidak berubah:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

Jika grup `vip` belum ada, grup akan dibuat secara otomatis. Setelah permintaan, klien-klien ini akan memiliki `group_name = "vip"` di catatannya, dan objek klien di pengaturan Xray setiap inbound mereka akan mendapat kolom `"group": "vip"`:

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. Menghapus klien dari grup (tanpa menghapus klien itu sendiri)

Tombol **Hapus klien...**, judul – «**Hapus klien dari grup «{name}»**».

Keterangan persis:

«Pilih anggota untuk dihapus dari grup ini. Klien itu sendiri tetap dipertahankan (gunakan "Hapus klien grup" untuk penghapusan penuh).»

Perilaku (`POST /panel/api/clients/groups/bulkRemove`, body `{"emails": [...]}`): secara teknis ini sama seperti "Tambahkan ke grup" dengan grup kosong. Pada klien yang dipilih, `group_name` dihapus, dan kolom `group` dihapus dari pengaturan Xray inbound mereka. Klien itu sendiri dan binding mereka ke inbound tetap dipertahankan.

Pesan sukses: «**{count}** klien dihapus dari **{name}**.»

9.9. Mereset traffic grup

Tombol **Reset Traffic**.

Dialog konfirmasi:

- Judul: «**Reset traffic grup {name}?**»
- Teks: «**Hanya penghitung lalu lintas grup yang direset. Penghitung masing-masing klien tidak terpengaruh.**»

Perilaku: hanya **penghitung yang ditampilkan untuk grup itu sendiri** yang dinol-kan – panel mengingat jumlah `up/down` grup saat ini sebagai penanda dasar dan selanjutnya menampilkan `max(0, jumlah - penanda dasar)` di daftar grup. Penghitung `up / down`, kuota, dan field `enable` masing-masing klien **tidak diubah**, dan restart Xray tidak diperlukan. Ini adalah perubahan perilaku dari versi sebelumnya, di mana reset menol-kan lalu lintas semua klien grup.

Permintaan: `POST /panel/api/clients/groups/resetTraffic` dengan body `{"name": "<nama grup>"}`.

Pesan sukses: «Traffic grup {name} telah direset.»

9.10. Menghapus grup dan menghapus klien grup

Di halaman terdapat dua operasi penghapusan yang pada dasarnya berbeda — mudah tertukar, sehingga perbedaannya sangat penting.

9.10.1. Hapus grup (pertahankan klien)

Tombol «Hapus grup (pertahankan klien)».

Dialog:

- Judul: «Hapus grup {name}?»
- Teks: «Ini menghapus grup dan menghapus labelnya dari {count} klien. Klien itu sendiri tidak dihapus.»

Perilaku (`POST /panel/api/clients/groups/delete`, body `{"name": "..."}`): catatan grup dihapus dari `client_groups`, `group_name` semua kliennya dihapus, dan kolom `group` dihapus dari inbound mereka.

Klien, koneksi, dan traffic mereka tetap dipertahankan. Xray ditandai untuk di-restart.

Pesan sukses: «Grup dihapus dari {count} klien.»

9.10.2. Hapus klien grup (penghapusan penuh)

Tombol «Hapus klien grup».

Dialog:

- Judul: «Hapus semua klien di {name}?»
- Teks: «Ini menghapus {count} klien secara permanen beserta catatan traffic mereka. Label grup juga dihapus. Tindakan ini tidak dapat dibatalkan.»

Ini adalah operasi destruktif: operasi ini menghapus klien itu sendiri (melalui penghapusan massal berdasarkan email, endpoint `POST /panel/api/clients/bulkDel`), termasuk catatan traffic mereka, sehingga menghapus mereka dari semua inbound.

Pesan:

- Sukses: «{count} klien dihapus.»
- Hasil parsial: «{ok} dihapus, {failed} dilewati»

Jika grup kosong, tindakan terhadap anggotanya tidak tersedia — ditampilkan «Grup ini belum memiliki klien.»

9.11. Hubungan dengan halaman "Klien"

Label grup terlihat dan digunakan di luar halaman **Grup** juga:

- Dalam catatan kompak klien terdapat kolom `group`, sehingga keanggotaan grup ditampilkan dalam daftar klien.
- Daftar klien (`/panel/api/clients/list/paged`) menerima parameter filter `group`: dapat diteruskan satu nama atau beberapa nama yang dipisahkan koma. Pencocokan dilakukan berdasarkan prinsip "ATAU" di dalam kolom, tanpa memperhatikan huruf kapital. Kasus khusus: elemen kosong dalam daftar grup filter berarti "klien tanpa grup" (yang `group`-nya kosong).
- Dalam respons halaman klien, array `groups` dikembalikan – daftar lengkap nama grup yang ada, agar UI dapat membuat dropdown filter.

Contoh: memfilter klien berdasarkan grup. Permintaan mengembalikan hanya klien dengan label `vip` atau `trial` (beberapa nama – dipisahkan koma, "ATAU"):

```
GET /panel/api/clients/list/paged?group=vip,trial
```

Untuk mendapatkan klien **tanpa** grup, teruskan elemen kosong dalam daftar – misalnya, nilai filter `group=` (string kosong) atau `group=vip,` (label `vip` ditambah klien tanpa grup).

9.12. Ringkasan endpoint API

Semua rute grup dipasang di bawah `/panel/api/clients`:

Metode dan path	Tujuan	Body permintaan
GET <code>/panel/api/clients/groups</code>	Daftar grup dengan penghitung klien	–
GET <code>/panel/api/clients/groups/:name/emails</code>	Email semua anggota grup (diurutkan berdasarkan email)	–
POST <code>/panel/api/clients/groups/create</code>	Membuat grup kosong	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/rename</code>	Mengganti nama grup	<code>{"oldName", "newName"}</code>
POST <code>/panel/api/clients/groups/delete</code>	Menghapus grup, mempertahankan klien (menghapus label)	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/bulkAdd</code>	Menambahkan klien ke grup (berdasarkan email)	<code>{"emails": [...], "group"}</code>
POST <code>/panel/api/clients/groups/bulkRemove</code>	Menghapus klien dari grup (menghapus label)	<code>{"emails": [...]}</code>
POST <code>/panel/api/clients/bulkDel</code>	Penghapusan penuh klien (digunakan oleh "Hapus klien grup")	<code>{"emails": [...], "keepTraffic"}</code>

Contoh: skenario tipikal siklus hidup grup melalui API.

```
# 1. Buat label trial
curl -s .../panel/api/clients/groups/create -d '{"name":"trial"}'

# 2. Tempelkan ke dua klien
curl -s .../panel/api/clients/groups/bulkAdd -d '{"emails":
["u1@example.com","u2@example.com"],"group":"trial"}'

# 3. Nalkan traffic semua anggota (berdasarkan email dari /groups/trial/emails)
curl -s .../panel/api/clients/groups/bulkRemove -d '{"emails":["u2@example.com"]}'

# 4. Hapus grup, tapi pertahankan klien (hanya menghapus label)
curl -s .../panel/api/clients/groups/delete -d '{"name":"trial"}'
```

Langkah 4 menghapus catatan grup dan menghapus `group_name` pada kliennya, tetapi klien itu sendiri, koneksi, dan traffic mereka tetap ada. Untuk penghapusan permanen klien itu sendiri, gunakan `bulkDel`.

Operasi yang mengubah label pada klien (`rename`, `delete`, `bulkAdd`, `bulkRemove`) menandai Xray sebagai memerlukan restart dan mengirimkan notifikasi perubahan klien.

9.13. Traffic per grup

Fitur baru versi 3.3.0: di bagian **Grup** (halaman "Klien", tab manajemen grup) tabel grup kini menampilkan tidak hanya jumlah klien di setiap grup, tetapi juga total traffic yang telah digunakan oleh grup. Kolom diberi label «**Traffic yang Digunakan**».

Apa yang ditampilkan kolom ini

Untuk setiap baris grup ditampilkan jumlah traffic dari semua klien yang termasuk dalam grup tersebut – yaitu `up + down` (traffic terkirim + diterima) yang dijumlahkan dari semua anggotanya. Ini memberikan jawaban cepat atas pertanyaan "berapa total yang diunduh/diunggah oleh seluruh grup", tanpa perlu membuka klien satu per satu dan menjumlahkan secara manual.

Di samping itu, dalam tabel grup ditampilkan:

Kolom	Artinya
Nama	Nama grup
Klien	Berapa banyak klien yang ditandai dengan grup ini (sebelumnya kolom disebut "Jumlah Klien di Grup")
Terkirim	Total <code>up</code> (traffic terkirim) dari semua klien grup
Diterima	Total <code>down</code> (traffic diterima) dari semua klien grup
Traffic yang Digunakan	Total <code>up + down</code> dari semua klien grup

Traffic terkirim dan diterima ditampilkan dalam kolom terpisah **Terkirim** dan **Diterima**, sedangkan kolom **Traffic yang Digunakan** menampilkan jumlahnya. Kolom jumlah klien cukup disebut **Klien**.

Ringkasan di atas tabel juga menampilkan agregat dari semua grup – «**Total grup**» dan «**Klien dengan grup**», sementara total traffic dibagi menjadi dua kartu: «**Total terkirim / diterima**» (dengan panah atas/bawah – traffic terkirim dan diterima semua grup secara terpisah) dan «**Total traffic**» (dengan ikon diagram – total gabungannya).

Cara penghitungan

Penghitungan dilakukan dengan satu kueri SQL ke tabel klien dengan join (`LEFT JOIN`) tabel pencatatan traffic:

- berdasarkan kolom label grup (`group_name`), klien dikelompokkan dan jumlahnya dihitung – itulah "Jumlah Klien di Grup";
- traffic diambil sebagai jumlah `up + down` dari tabel `client_traffics` yang di-join. Artinya, byte terkirim (`up`) dan byte diterima (`down`) dijumlahkan untuk setiap klien;
- karena email unik di tabel klien maupun di tabel traffic, join tidak menduplikasi traffic satu klien.

Kekhususan nilai:

- **Klien tanpa catatan traffic** dihitung dalam penghitung anggota, tetapi menambahkan 0 ke jumlah, sehingga grup yang baru dibuat menampilkan traffic `0`.
- **Grup kosong** (dibuat tetapi tanpa klien) juga ada dalam daftar dengan penghitung nol dan traffic nol: selain grup yang "diturunkan" dari label klien, grup yang tersimpan secara eksplisit dimasukkan ke dalam hasil, setelah itu daftar diurutkan berdasarkan nama tanpa memperhatikan huruf kapital.
- Klien tanpa label grup (`group_name` kosong) tidak masuk dalam penghitungan.

Tindakan terkait

Dari tabel grup, tindakan terhadap grup secara keseluruhan tetap tersedia, termasuk «**Reset Traffic**» – menolak **hanya penghitung grup itu sendiri** (penanda dasar), tanpa menyentuh lalu lintas masing-masing klien (lihat [9.9](#)). Setelah reset tersebut, kolom "Traffic yang Digunakan" untuk grup ini menampilkan `0`.

10. Langganan (Subscription)

Langganan (subscription) adalah mekanisme yang memungkinkan Anda memberikan satu tautan permanen (URL) kepada klien, di mana aplikasi VPN secara otomatis mengunduh dan memperbarui secara berkala seluruh kumpulan konfigurasi. Alih-alih meneruskan tautan terpisah untuk setiap inbound secara manual, pengguna cukup mendapatkan satu alamat seperti `https://domain:port/sub/<subId>`. Melalui alamat ini, panel secara dinamis mengumpulkan semua konfigurasi yang terikat pada klien tersebut dan mengirimkannya dalam format yang diinginkan klien. Ketika pengaturan server berubah (alamat baru, rotasi kunci Reality, penambahan inbound), klien mendapatkan konfigurasi terbaru saat pembaruan otomatis berikutnya, tanpa memerlukan tindakan apa pun dari pengguna.

Langganan dilayani oleh server HTTP/HTTPS terpisah di dalam panel, yang berjalan secara independen dari panel web dan mendengarkan pada portnya sendiri. Ini dilakukan demi keamanan: port langganan dapat dibuka ke luar tanpa membuka port panel itu sendiri.

10.1. Apa itu subId dan bagaimana tautan dibentuk

Setiap klien dalam inbound memiliki field `subId` (dalam antarmuka – «ID Langganan»). Nilai inilah yang menjadi kunci langganan: panel mencari di semua inbound untuk klien yang `subId`-nya cocok dengan yang diminta, dan menggabungkan konfigurasi mereka menjadi satu respons.

- Jika beberapa klien (dalam satu atau berbagai inbound) memiliki `subId` yang sama, konfigurasi mereka akan masuk ke dalam satu langganan. Ini adalah cara standar untuk memberikan kepada satu pengguna beberapa server/protokol sekaligus melalui satu tautan.

Contoh: satu pengguna – dua server dalam satu tautan. Misalkan ada dua inbound (VLESS di server A dan Trojan di server B). Untuk memberikan pengguna kedua konfigurasi dalam satu tautan, tetapkan `subId` yang sama pada kedua kliennya:

```
Inbound 1 (VLESS): email = ivan@vpn, subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn, subId = ivan2025
```

Maka melalui alamat `https://sub.example.com:2096/sub/ivan2025`, panel akan mengirimkan kedua konfigurasi sekaligus. Jika kemudian Anda menambahkan inbound ketiga dengan `subId` yang sama – konfigurasi tersebut akan muncul pada pengguna saat pembaruan langganan otomatis berikutnya, tanpa perlu mengirimkan tautan baru.

- Jika field `subId` klien kosong, tautan untuk berbagi umum tidak tersedia. Antarmuka menunjukkan petunjuk: «Klien ini tidak memiliki subId, tautan berbagi umum tidak tersedia.»

Tautan eksternal dan langganan klien (tab «Links»)

Dalam formulir klien terdapat tab «Links», di mana untuk klien tertentu Anda dapat melampirkan sumber konfigurasi tambahan yang dicampurkan khusus ke langganannya (format RAW, JSON, dan Clash):

- **Add External Link** – tautan berbagi pihak ketiga (`vless://`, `trojan://`, `ss://`, dll.). Ditambahkan ke output apa adanya, dan untuk JSON/Clash juga diurai menjadi konfigurasi.

- **Add External Subscription** — alamat langganan eksternal. Panel sendiri mengunduhnya (dengan caching dan timeout singkat) dan menggabungkan konfigurasi yang diperoleh ke dalam daftar umum klien.

Ini berguna untuk memberikan klien server tambahan di atas inbound Anda melalui satu tautan yang sama. Jika respons langganan jarak jauh terlalu besar, respons tersebut tidak lagi dipotong secara diam-diam: panel mengembalikan error dan terus menggunakan nilai terakhir yang berhasil di-cache.

- Nilai `subId` tidak dapat ditetapkan secara sembarangan: saat menyimpan, sistem memverifikasi bahwa nilai tersebut tidak mengandung spasi, karakter `/`, `\`, atau karakter kontrol. Petunjuk validasi yang sesuai: «ID Langganan tidak boleh mengandung spasi, '/', '\', atau karakter kontrol».

Tautan akhir dibentuk sebagai `<basis>/<subPath>/<subId>` (lihat bagian tentang pengaturan server langganan dan field «URI proksi terbalik»). Jika tidak ditemukan klien apa pun berdasarkan `subId` (klien dihapus, `subId` tidak ada), server mengembalikan HTTP 404 tanpa isi. Pada error internal — HTTP 500. Klien VPN hanya memperhatikan kode respons, oleh karena itu isi error sengaja dibiarkan kosong.

Urutan tautan inbound dalam langganan

Setiap inbound memiliki field «**Urutan dalam langganan**» (`subSortIndex`) — angka dari 1 yang menentukan posisi tautan inbound tersebut dalam output langganan. Nilai yang lebih kecil muncul lebih dulu; untuk nilai yang sama, urutan pembuatan asli (berdasarkan id) dipertahankan. Urutan ini berlaku untuk semua format output — teks mentah, halaman langganan, JSON, dan Clash. Field ini tidak mempengaruhi urutan inbound di panel itu sendiri.

Field ini dapat diedit dalam formulir inbound di sebelah pengaturan alamat tautan (share address) dan disinkronkan ke node mengikuti aturan biasa. Jika setidaknya satu inbound memiliki urutan yang berbeda dari 1, kolom kompak «**Urutan**» akan muncul di daftar Inbounds.

10.2. Pengaturan server langganan

Semua parameter langganan berada di bagian pengaturan panel pada tab «**Langganan**». Di bawah ini setiap parameter dijelaskan; dalam tanda kurung ditunjukkan kunci pengaturan internal dan nilai default-nya.

Bagian ini dibagi menjadi beberapa tab: «**Pengaturan Panel**», «**Informasi**», «**Profil**», «**Sertifikat**», «**Happ**», dan «**Clash / Mihomo**». Field judul langganan, URL dukungan, halaman profil, pengumuman, dan direktori tema berada di tab «**Profil**»; aturan perutean Happ dan Clash/Mihomo — di tab dengan nama yang sama; interval pembaruan langganan — di tab «**Informasi**».

Parameter utama

Field (UI)	Kunci	Nilai default	Deskripsi
Aktifkan langganan	<code>subEnable</code>	<code>true</code> (aktif)	Menjalankan server langganan terpisah. Petunjuk: «Fitur langganan dengan konfigurasi terpisah». Jika dinonaktifkan — server langganan tidak berjalan, dan tidak satu pun tautan berfungsi.
IP yang didengarkan	<code>subListen</code>	kosong	Alamat IP di mana server langganan menerima koneksi. Petunjuk: «Biarkan kosong secara default untuk memantau semua alamat IP».

Field (UI)	Kunci	Nilai default	Deskripsi
Port langganan	subPort	2096	Port TCP server langganan. Petunjuk: «Nomor port untuk melayani layanan langganan tidak boleh digunakan di server» – port harus bebas dan tidak bertentangan dengan panel atau Xray.
URI path	subPath	/sub/	Path tempat langganan biasa dikirimkan. Petunjuk: «Harus dimulai dengan '/' dan diakhiri dengan '/'».
Domain yang didengarkan	subDomain	kosong	Domain yang diizinkan mengakses langganan (validasi Host). Petunjuk: «Biarkan kosong secara default untuk mendengarkan semua domain dan alamat IP». Jika ditetapkan – permintaan dengan Host lain akan ditolak.

Penting untuk keamanan: path default `/sub/` (dan `/json/` untuk JSON) sudah dikenal luas dan mudah ditebak. Panel menampilkan peringatan: «Path langganan default `/sub/` sudah dikenal luas – ubah.» dan serupa untuk JSON. Disarankan untuk menetapkan path kustom yang tidak jelas.

TLS / sertifikat

Field (UI)	Kunci	Default	Deskripsi
Path file kunci publik sertifikat langganan	subCertFile	kosong	Path lengkap ke file sertifikat (<code>.crt</code> / <code>fullchain</code>). Petunjuk: «Masukkan path lengkap yang dimulai dengan '/'».
Path file kunci privat sertifikat langganan	subKeyFile	kosong	Path lengkap ke file kunci privat. Petunjuk: «Masukkan path lengkap yang dimulai dengan '/'».

Jika kedua path ditetapkan dan sertifikat berhasil dimuat, server langganan berjalan melalui **HTTPS**. Jika field kosong atau sertifikat gagal dibaca – server kembali ke **HTTP** (error ditulis ke log). Adanya TLS yang valid juga mempengaruhi pembentukan URL dasar: pada port 443 dengan TLS dan port 80 tanpa TLS, nomor port dihilangkan dari tautan.

Interval pembaruan

Field (UI)	Kunci	Default	Deskripsi
Interval pembaruan langganan	subUpdates	12	Seberapa sering (dalam jam) aplikasi klien harus meminta ulang langganan. Petunjuk: «Interval antara pembaruan di aplikasi klien (dalam jam)».

Nilai ini dikirimkan ke klien melalui header HTTP `Profile-Update-Interval` ; klien modern menggunakannya sebagai periode pembaruan otomatis konfigurasi.

Format dan informasi dalam respons

Field (UI)	Kunci	Default	Deskripsi
Enkripsi	subEncrypt	true	Petunjuk: «Enkripsi konfigurasi yang dikembalikan dalam langganan». Secara teknis ini bukan enkripsi, melainkan enkoding Base64 dari seluruh isi langganan biasa (format yang diharapkan sebagian besar klien). Jika dinonaktifkan, tautan dikirimkan sebagai teks biasa, satu per baris.
Tampilkan informasi penggunaan	subShowInfo	true	Petunjuk: «Tampilkan sisa traffic dan tanggal kedaluwarsa setelah nama konfigurasi». Jika diaktifkan, penanda sisa traffic () dan masa berlaku (misalnya, 5D, 3H) ditambahkan ke nama (remark) setiap konfigurasi; jika klien kedaluwarsa/tidak tersedia, ditampilkan N/A .
Sertakan Email dalam nama	subEmailInRemark	true	Petunjuk: «Sertakan email klien dalam nama profil langganan.». Menambahkan email klien ke dalam remark profil.

Template remark (Remark Template)

Nama tampilan (remark) setiap konfigurasi dalam langganan dibentuk berdasarkan **template remark** — field «**Template catatan**» (remarkTemplate) di tab «**Informasi**» dalam pengaturan langganan. Konstruktor model catatan lama (pilihan terpisah untuk bagian inbound/email/external proxy dan karakter pemisah) telah dihapus dari antarmuka; sekarang Anda menulis format nama yang bebas dan menyisipkan variabel ke dalamnya. Nilai default adalah `{{INBOUND}}-{{EMAIL}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D` (artinya secara default nama profil menyertakan email klien). Jika field dibiarkan kosong, model remark lama (yang tidak dapat dikonfigurasi melalui antarmuka) akan digunakan sebagai fallback.

Variabel dikelompokkan berdasarkan bagian **Client**, **Traffic**, dan **Time & status** dan ditampilkan di sebelah field sebagai chip yang dapat diklik `{{VAR}}` dengan petunjuk saat diarahkan; klik menyisipkan token ke dalam template, dan pratinjau langsung tersedia. Setiap variabel disubstitusikan secara individual untuk klien tertentu pada saat pembuatan langganan. Notasi sederhana dalam tanda kurung tunggal (`{DATA_LEFT}` , `{EXPIRE_DATE}` , `{PROTOCOL}` , `{TRANSPORT}` , dll.) juga didukung — panel secara otomatis mengubahnya ke format internal `{{...}}` .

Variabel yang tersedia:

- **Identifikasi klien:** `{{EMAIL}}` (sinonim — `{{USERNAME}}`), `{{INBOUND}}` (remark inbound itu sendiri), `{{HOST}}` (remark host), `{{ID}}` (UUID), `{{SHORT_ID}}` (8 karakter pertama UUID), `{{SUB_ID}}` , `{{COMMENT}}` , `{{TELEGRAM_ID}}` , `{{PROTOCOL}}` , `{{TRANSPORT}}` .
- **Traffic:** `{{TRAFFIC_USED}}` , `{{TRAFFIC_LEFT}}` , `{{TRAFFIC_TOTAL}}` (dan varian `*_BYTES` -nya dalam byte yang tepat), `{{UP}}` , `{{DOWN}}` , `{{USAGE_PERCENTAGE}}` .
- **Masa berlaku dan status:** `{{DAYS_LEFT}}` , `{{TIME_LEFT}}` , `{{EXPIRE_DATE}}` (YYYY-MM-DD), `{{JALALI_EXPIRE_DATE}}` (tanggal dalam kalender Jalali), `{{EXPIRE_UNIX}}` , `{{CREATED_UNIX}}` , `{{RESET_DAYS}}` , `{{STATUS}}` (active / expired / disabled / depleted), `{{STATUS_EMOJI}}` .
- **Koneksi (Connection):** `{{PROTOCOL}}` — protokol (VLESS, VMess, Trojan, dll.), `{{TRANSPORT}}` — jaringan transport (tcp, ws, grpc, dll.), `{{SECURITY}}` — keamanan transport (TLS, REALITY, NONE; ditampilkan dengan huruf kapital). Seperti variabel penggunaan dan masa berlaku, ketiga variabel ini hanya berfungsi

di isi langganan dan secara otomatis dihapus dari remark pada tautan yang ditampilkan di panel (QR/«Informasi») dan di halaman informasi langganan.

Template dapat dibagi menjadi segmen dengan tanda garis vertikal `|`. Segmen di mana variabel menghasilkan nilai «tak terbatas» (∞) – misalnya `{{TRAFFIC_LEFT}}` atau `{{DAYS_LEFT}}` untuk klien tanpa batasan – akan disembunyikan secara otomatis. Selain itu, blok penggunaan traffic dan masa berlaku ditampilkan sekali, pada tautan pertama klien, agar tidak terduplikasi di setiap konfigurasi.

Contoh. Template `{{EMAIL}}|{{TRAFFIC_LEFT}}|{{DAYS_LEFT}}` untuk klien dengan sisa 42 GB dan 7 hari akan menghasilkan nama seperti `ivan@vpn 42.00GB 7D`, sedangkan untuk klien tanpa batas – cukup `ivan@vpn` (segmen dengan ∞ dihilangkan).

Mulai versi 3.4.2, variabel `{{EMAIL}}` (dan sinonim `{{USERNAME}}`) hanya ditampilkan pada tautan **pertama** klien di badan langganan – sama seperti blok sisa lalu lintas/masa berlaku; pada tautan lain klien yang sama, variabel ini dihilangkan.

Pada tautan yang ditampilkan di panel (kode QR dan jendela «Informasi» di halaman «Klien») dan di halaman informasi langganan, email klien ada dalam nama profil: dalam format «inbound-host-email» jika host ditetapkan, atau «inbound-email» tanpa host. Informasi traffic dan masa berlaku (serta variabel kelompok «Koneksi») tidak disubstitusikan ke dalam nama yang ditampilkan ini – variabel tersebut hanya berfungsi di isi langganan yang diterima klien VPN.

Jika baris statistik traffic klien menjadi «yatim piatu» setelah penghapusan dan pembuatan ulang inbound, variabel `{{TRAFFIC_USED}}` (dan indikator penggunaan lainnya) tidak lagi menampilkan `0.00B`: panel juga mencari statistik berdasarkan email klien dan mensubstitusikan traffic yang telah digunakan dengan benar. | Template remark | `remarkTemplate | {{INBOUND}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}` | Template bebas untuk nama tampilan (remark) setiap konfigurasi dengan substitusi variabel `{{VAR}}`. Disubstitusikan secara individual untuk setiap klien saat pembuatan langganan. Konstruktor «model catatan» lama (pilihan inbound/email/external proxy dan pemisah) telah dihapus dari antarmuka dan hanya digunakan sebagai fallback jika field dibiarkan kosong. Detail – lihat «Template remark (Remark Template)» di bawah. |

Metadata profil (header respons)

String-string ini dikirimkan ke klien dalam header respons HTTP dan ditampilkan di aplikasi VPN sebagai metadata profil. Semuanya kosong secara default.

Field (UI)	Kunci	Header	Deskripsi
Judul langganan	<code>subTitle</code>	<code>Profile-Title</code> (dalam Base64)	«Nama langganan yang dilihat klien di aplikasi VPN». Untuk Clash juga digunakan sebagai nama profil yang diimpor melalui <code>Content-Disposition</code> .
URL dukungan	<code>subSupportUrl</code>	<code>Support-Url</code>	«Tautan dukungan teknis yang ditampilkan di aplikasi VPN».
URL profil	<code>subProfileUrl</code>	<code>Profile-Web-Page-Url</code>	«Tautan ke situs Anda yang ditampilkan di aplikasi VPN». Jika tidak ditetapkan, URL permintaan langganan yang sebenarnya akan digunakan.
Pengumuman	<code>subAnnounce</code>	<code>Announce</code> (dalam Base64)	«Teks pengumuman yang ditampilkan di aplikasi VPN».

Selain itu, header `Subscription-Userinfo` dikirimkan dalam setiap respons dengan data traffic klien yang teragregasi: `upload`, `download`, `total`, dan `expire` (waktu kedaluwarsa dalam detik). Klien menggunakannya untuk menampilkan sisa traffic dan masa berlaku.

Perutean (hanya untuk klien Happ)

Field (UI)	Kunci	Default	Deskripsi
Aktifkan perutean	<code>subEnableRouting</code>	<code>false</code>	«Pengaturan global untuk mengaktifkan perutean di aplikasi VPN klien. (Hanya untuk Happ)». Dikirimkan melalui header <code>Routing-Enable</code> .
Aturan perutean	<code>subRoutingRules</code>	kosong	«Aturan perutean global untuk aplikasi VPN klien. (Hanya untuk Happ)». Dikirimkan melalui header <code>Routing</code> .

| Sembunyikan pengaturan server | `subHideSettings` | `false` | «Sembunyikan pengaturan server dalam langganan (hanya untuk Happ)». Jika diaktifkan, klien Happ menyembunyikan kemampuan untuk melihat dan mengubah parameter server. Opsi ini hanya berlaku untuk klien Happ. |

Perutean Incy (hanya untuk klien Incy)

Untuk klien VPN **Incy**, pengaturan langganan memiliki tab terpisah **«Incy»** dengan dua field: sakelar **«Aktifkan perutean»** (`subIncyEnableRouting`, dinonaktifkan secara default) dan field teks **«Aturan perutean»** (`subIncyRoutingRules`) dengan format `incy://routing/onadd/<base64>`. Ketika perutean diaktifkan dan field diisi, string ini ditambahkan sebagai baris terpisah ke isi langganan (format raw) – sehingga profil perutean dikirimkan ke klien Incy tanpa bertentangan dengan header `Routing` klien Happ. Pengaturan ini hanya berlaku untuk klien Incy.

URI proksi terbalik

Field (UI)	Kunci	Default	Deskripsi
URI proksi terbalik	<code>subURI</code>	kosong	«Ubah URI dasar URL langganan untuk digunakan di belakang server proksi».

Jika field kosong, alamat dasar tautan dibentuk secara otomatis oleh panel dari domain dan port langganan (dengan mempertimbangkan TLS). Namun jika langganan didistribusikan melalui reverse proxy/CDN eksternal di domain atau path yang berbeda, URI dasar akhir ditetapkan di field ini, dan semua tautan akan dibentuk berdasarkan. Field terpisah yang serupa tersedia untuk JSON (`subJsonURI`) dan Clash (`subClashURI`).

Jika hanya `subURI` umum yang ditetapkan, sedangkan field terpisah untuk JSON dan Clash dibiarkan kosong, tautan format tersebut di halaman langganan mewarisi skema dan host dari `subURI` (bukan dari port sub-server dan `http`) – sehingga sesuai dengan alamat reverse proxy.

Contoh: langganan di belakang reverse proxy. Langganan sendiri mendengarkan pada `2096`, tetapi dapat diakses dari luar melalui nginx/CDN di `https://cfg.example.com/u/`. Agar tautan dalam respons dibentuk dari alamat eksternal, bukan internal `domain:2096`, URI dasar akhir ditetapkan di field «Reverse proxy URI»:

Reverse proxy URI: `https://cfg.example.com/u`

Maka tautan akhir akan berbentuk `https://cfg.example.com/u/ivan2025`. Untuk format JSON dan Clash, jika diperlukan, field terpisah `subJsonURI` dan `subClashURI` diisi dengan cara yang sama.

10.3. Format output

Langganan dapat dikirimkan dalam tiga format independen, masing-masing dengan endpoint-nya sendiri yang dapat diaktifkan/dinonaktifkan secara terpisah.

Alamat server dan node dalam output

Alamat server dalam tautan langganan disubstitusikan menggunakan strategi alamat tautan yang sama seperti tautan dan kode QR biasa di panel: «listen» – alamat binding yang dapat dirutekan, «custom» – alamat kustom yang ditetapkan pengguna (`shareAddr`), «node» (default) – alamat node. Untuk inbound tanpa strategi yang ditetapkan secara eksplisit, output langganan tidak berubah. Ini memungkinkan inbound node yang terikat ke IP publik tertentu untuk mengirimkan alamat yang dapat dijangkau kepada klien. Strategi diterapkan pada format raw, JSON, dan Clash.

Nama node (Node) tidak ditambahkan ke nama (remark) profil dalam langganan: di aplikasi klien hanya ditampilkan remark inbound yang ditetapkan oleh administrator, tanpa sufiks internal seperti `@nama-node`. Untuk membedakan entri dengan nama yang sama dalam langganan multi-node, tetapkan remark yang berbeda secara manual atau gunakan host terkelola (Hosts) dengan Remark tersendiri.

Jika karena ketidaksinkronan antar node klien yang sama masuk ke inbound JSON layanan dua kali, output langganan secara otomatis menghilangkan duplikat tersebut berdasarkan email di ketiga format, sehingga profil yang berulang tidak muncul dalam output.

Host terkelola (Hosts)

Bagian **Hosts** (item menu samping; halaman ringkasan dengan jumlah Total/Enabled/Disabled dan daftar) menentukan penggantian alamat untuk tautan langganan. Untuk setiap inbound, Anda dapat menambahkan satu atau beberapa **host** – endpoint yang disubstitusikan ke dalam tautan langganan yang dikirimkan ke klien **menggantikan alamat, port, dan parameter TLS inbound itu sendiri**. Ini berguna untuk mendistribusikan traffic melalui CDN atau relay tanpa mengubah inbound itu sendiri.

Setiap host memiliki:

- **Remark** dan deskripsi (Description), pengikatan ke **Inbound** tertentu, sakelar **Enable**, dan penetapan ke node (**Nodes**).
- **Address** (kosong – mewarisi alamat inbound) dan **Port** (`0` – mewarisi port inbound); **Tags** (hanya diperhitungkan dalam langganan RAW).
- Tab **Security** – `same` / `tls` / `none` / `reality` dengan SNI, fingerprint, ALPN, pinned-cert, `allowInsecure`, dan ECH.
- Tab **Advanced** – Host header, Path, rute VLESS, Mux, Sockopt, Final Mask, dan pengecualian host dari format langganan tertentu (raw / json / clash).
- Tab **Clash (mihomo)** – versi IP, Mihomo X25519, pengacakan host (Shuffle host).

Host diurutkan dalam inbound mereka dan mendukung pengaktifan, penonaktifan, dan penghapusan massal. Host terkelola menggantikan array External Proxy yang lama.

Route VLESS (VLESS route). Mulai versi 3.4.2, ini berupa satu angka 0–65535 (bukan daftar port; petunjuk – «satu nilai VLESS route (0-65535) yang ditanamkan ke UUID, misalnya 443; kosong – tanpa itu», placeholder 443). Nilai yang ditetapkan benar-benar «ditanamkan» ke UUID setiap langganan yang dihasilkan (raw / JSON / Clash): Xray membaca byte 6-7 UUID dan menyamakannya sebelum autentikasi, sehingga klien tetap cocok. Nilai yang kosong atau tidak valid tidak mengubah UUID.

Tautan biasa (SUB) – Base64 / teks biasa

Format dasar, endpoint `subPath` (default `/sub/`). Selalu aktif (ketika langganan secara keseluruhan diaktifkan). Mengembalikan daftar tautan Xray (`vless://`, `vmess://`, `trojan://`, `ss://`, dll.) – satu per baris. Jika opsi «Enkripsi» (`subEncrypt`) diaktifkan, seluruh daftar dikodekan dalam Base64; jika dinonaktifkan – dikirimkan sebagai teks biasa. Format ini dipahami oleh hampir semua klien (v2rayNG, V2RayTun, Sing-box, NekoBox, Streisand, Shadowrocket, Happ, dll.).

Contoh: isi respons dengan «Enkripsi» dinonaktifkan. Dengan `subEncrypt = false`, endpoint `/sub/` mengirimkan teks biasa – satu tautan per baris:

```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tls&...#srvB-ivan
```

Dengan `subEncrypt = true` (default), daftar yang sama dikodekan sepenuhnya dalam Base64 dan dikirimkan dalam satu baris – inilah format yang diharapkan sebagian besar klien.

Langganan JSON (sing-box dan yang kompatibel)

Endpoint `subJsonPath` (default `/json/`), diaktifkan dengan centang terpisah.

Field (UI)	Kunci	Default	Deskripsi
Langganan JSON	<code>subJsonEnable</code>	<code>false</code>	«Aktifkan/nonaktifkan endpoint langganan JSON secara independen.»

Mengembalikan konfigurasi JSON lengkap (format yang dipahami sing-box dan klien turunannya – Podkop, OpenWRT sing-box, Karing, NekoBox). Parameter tambahan tersedia untuk format ini (tab `subFormats`):

- **Mux** (`subJsonMux`, default kosong) – pengaturan multipleksing JSON (Mux) yang dimasukkan ke dalam outbound setiap stream langganan JSON. «Transmisi beberapa stream data independen dalam satu koneksi.»
- **Final Mask** (`subJsonFinalMask`, default kosong) – «Mask finalmask xray (TCP/UDP) dan pengaturan QUIC yang ditambahkan ke setiap stream langganan JSON. Memerlukan versi xray terbaru di klien.»
Dikonfigurasi melalui sub-field: «Paket» (`packets`), «Panjang» (`length`), «Interval» (`interval`), «Maks. pembagian» (`maxSplit`), «Noise» (`noises`: «Tipe»/ `type`, «Paket»/ `packet`, «Penundaan (ms)»/ `delayMs`, «Terapkan ke»/ `applyTo`, tombol «+ Noise»), serta «Konkurensi» (`concurrency`), «Konkurensi xudp» (`xudpConcurrency`), dan «xudp UDP 443» (`xudpUdp443`).
- **Aturan perutean** (`subJsonRules`, default kosong) – aturan global yang ditambahkan ke konfigurasi JSON.

Langganan Clash / Mihomo (YAML)

Endpoint `subClashPath` (default `/clash/`), diaktifkan dengan centang terpisah.

Field (UI)	Kunci	Default	Deskripsi
Langganan Clash / Mihomo	<code>subClashEnable</code>	<code>false</code>	Mengaktifkan pembuatan konfigurasi YAML untuk klien Clash dan Mihomo.
Aktifkan perutean	<code>subClashEnableRouting</code>	<code>false</code>	«Tambahkan aturan perutean Clash/Mihomo global ke langganan YAML yang dibuat.».
Aturan perutean global	<code>subClashRules</code>	kosong	«Aturan Clash/Mihomo yang ditambahkan ke awal setiap langganan YAML sebelum MATCH,PROXY.».

Respons dikirimkan dengan tipe `application/yaml; charset=utf-8`. Jika «Judul langganan» (`subTitle`) ditetapkan, judul tersebut juga dikirimkan di header `Content-Disposition (attachment; filename*=UTF-8' '<title>)`, agar klien Clash memberi nama profil yang diimpor dengan nama tersebut.

Format tautan dan YAML yang dihasilkan dipertahankan dalam kondisi terkini untuk klien modern: Shadowsocks-2022 (SS2022) tidak lagi mengkodekan userinfo dalam Base64; tautan Shadowsocks dengan obfuskasi http dikirimkan dalam format SIP002 dengan plugin `obfs-local`; untuk langganan Clash/Mihomo tersedia set lengkap field XHTTP. Ini tidak memerlukan pengaturan terpisah – tautan hanya lebih tepat dikenali oleh klien.

Catatan: build ini mendukung tepat tiga format – tautan biasa (Base64/teks), JSON (kompatibel sing-box), dan Clash/Mihomo (YAML). Tidak ada format Outline terpisah di server langganan.

10.4. Halaman informasi langganan dan kode QR

Jika Anda membuka tautan langganan di browser (atau secara eksplisit menambahkan parameter `?html=1` atau `?view=html` ke URL, atau mengirimkan header `Accept: text/html`), server alih-alih respons «mentah» akan mengirimkan **halaman informasi langganan** visual («Informasi langganan»). Klien VPN tetap mendapatkan respons mesin, karena mereka tidak meminta HTML.

Halaman (aplikasi satu halaman yang dibuild dengan Vite) menampilkan:

- **Informasi langganan** (blok Descriptions):
 - «ID langganan» – nilai `subId`;
 - «Status» – «Aktif», «Tidak aktif», atau «Tak terbatas». Status «tidak aktif» ditetapkan jika klien dinonaktifkan, telah menghabiskan batas traffic, atau masa berlakunya telah berakhir;
 - «Diunduh» dan «Diunggah» – volume traffic;
 - «Batas total» – batas traffic atau ∞ jika tidak dibatasi;
 - «Masa berlaku» – tanggal kedaluwarsa atau «Tanpa batas»;
 - sisa traffic dan waktu online terakhir.
 - Tanggal ditampilkan berdasarkan kalender Gregorian atau Jalali tergantung pada pengaturan «Calendar Type» panel (`datepicker`, default `gregorian`).
- **Tautan langganan**: untuk setiap format yang diaktifkan – baris terpisah dengan tag berwarna (hijau **SUB**, ungu **JSON**, emas **CLASH**), tombol salin, dan tombol **kode QR** (jendela pop-up, ukuran 240 px). Baris dengan JSON dan CLASH hanya muncul jika format yang bersangkutan diaktifkan dalam pengaturan.

- **Tautan individual** («Salin tautan»): daftar lengkap konfigurasi individual yang termasuk dalam langganan, masing-masing dengan tag protokol, tombol salin, dan kode QR (untuk tautan post-quantum, kode QR tidak dibuat).
- **Tombol «Salin semua konfigurasi»** (di atas daftar tautan individual): dengan satu klik menyalin semua tautan konfigurasi ke clipboard sekaligus (masing-masing pada baris baru), tanpa perlu menyalinnya satu per satu; setelah selesai, notifikasi «Semua konfigurasi disalin» ditampilkan.
- **Tombol impor cepat ke aplikasi** (menu dropdown berdasarkan platform): untuk Android – v2box, v2rayNG (deep-link `v2rayng://install-config?url=...`), Sing-box, V2RayTun, NPV Tunnel, Happ (`happ://add/...`), Incy (`incy://add/...`); untuk iOS – Shadowrocket (melalui parameter `flag=shadowrocket`), v2box (`v2box://install-sub?url=...&name=...`), Streisand (`streisand://import/...`), V2RayTun, NPV Tunnel, Happ, Incy. Tombol-tombol ini membuka deep-link aplikasi yang diperlukan dengan alamat langganan yang sudah diisi, atau menyalin tautan ke clipboard.

Halaman informasi dikirimkan dengan header pelarangan cache (`Cache-Control: no-cache`), agar klien selalu melihat data traffic dan masa berlaku yang terkini.

10.5. Template halaman langganan kustom

Mulai dari 3.3.0, halaman landing langganan standar dapat diganti dengan template HTML kustom Anda sendiri. Secara default, halaman bawaan dikirimkan di alamat langganan, tetapi jika Anda menentukan direktori dengan template Anda sendiri, panel akan me-render-nya dan mengisi data klien terkini ke dalamnya (traffic, masa berlaku, tautan, dll.).

Penting: panel **tidak menyediakan** template siap pakai – tema Anda sendiri harus dibuat secara mandiri. Petunjuk pembuatan dan daftar variabel yang tersedia ada di [docs/custom-subscription-templates.md](#).

Di mana diaktifkan

Direktori tema ditetapkan dalam pengaturan panel:

Pengaturan → Langganan → bagian «Informasi langganan», field **«Direktori tema langganan»** (`subThemeDir`).

Deskripsi field dalam antarmuka: «Path absolut ke folder dengan template kustom (index.html/sub.html) untuk halaman langganan (misalnya, /etc/3x-ui/sub_templates/my-theme/). Biarkan kosong untuk menggunakan halaman default.»

Di bagian yang sama terdapat pengaturan terkait yang nilainya tersedia di template:

Di deskripsi field «Direktori tema langganan» terdapat tautan **«Panduan template ↗»** ke dokumentasi pembuatan template tampilan halaman langganan kustom.

- **«Judul langganan»** (`subTitle`) – nama yang terlihat oleh klien;
- **«URL dukungan»** (`subSupportUrl`) – tautan ke dukungan teknis.

Parameter pengaturan

Parameter	Nilai default	Tujuan
<code>subThemeDir</code>	"" (kosong)	Path absolut ke direktori dengan template HTML Anda. Kosong = halaman bawaan default.

Cara menyisipkan template kustom

1. Buat folder untuk tema di server (di mana saja), misalnya `/etc/3x-ui/sub_templates/my-theme/`.
2. Letakkan file HTML bernama `index.html` atau `sub.html` di dalamnya.

Contoh: path ke tema. Susunan akhir di server dan nilai field dalam pengaturan:

```
/etc/3x-ui/sub_templates/my-theme/
└─ index.html      (atau sub.html – memiliki prioritas)
```

```
Pengaturan → Langganan → «Direktori tema langganan»:
/etc/3x-ui/sub_templates/my-theme/
```

Path harus **absolut** (dimulai dengan `/`). Jika folder tidak mengandung `index.html` atau `sub.html`, panel akan mengirimkan halaman bawaan. 3. Di panel, buka **Pengaturan** → **Langganan** dan masukkan path **absolut** ke folder tersebut di field «Direktori tema langganan». 4. Simpan pengaturan.

Perilaku pemilihan file dan rendering:

- Jika direktori berisi `sub.html`, file tersebut yang digunakan; jika tidak ada, `index.html` yang diambil. Artinya `sub.html` memiliki prioritas atas `index.html`.
- Template di-render menggunakan engine standar Go `html/template`.
- Template yang telah diurai **di-cache** dan hanya dibaca ulang dari disk jika waktu modifikasi file berubah. Oleh karena itu, perubahan template langsung berlaku tanpa restart panel, tetapi tanpa overhead pembacaan/parsing pada setiap permintaan.
- Respons dibentuk sepenuhnya dalam buffer dan baru kemudian dikirimkan ke klien: jika template gagal selama eksekusi, halaman yang sebagian dibuat (rusak) tidak akan dikirimkan ke pengguna.

Perilaku default dan fallback

- Field kosong → halaman SPA bawaan dikirimkan (data dimasukkan ke `window.__SUB_PAGE_DATA__`).
- Path tidak ada atau bukan direktori → halaman default digunakan.
- Direktori tidak mengandung `index.html` atau `sub.html` → peringatan «subThemeDir set but no usable template found» ditulis ke log, halaman default dikirimkan.
- File template ada, tetapi gagal diurai → error «custom template parse failed» ditulis ke log, halaman default dikirimkan.
- Error saat eksekusi template → «custom template execution failed» ditulis ke log, halaman default dikirimkan.

Artinya, masalah apa pun dengan template kustom tidak «merusak» langganan – panel selalu kembali ke halaman bawaan. Semua halaman langganan (baik kustom maupun standar) dikirimkan dengan header pelarangan cache (`Cache-Control: no-cache, no-store, must-revalidate`), agar klien selalu mendapatkan data traffic dan masa berlaku yang segar.

Variabel template yang tersedia

Kumpulan data klien langganan diteruskan ke konteks template. Akses melalui `{{ .nama }}` :

Variabel	Tipe	Deskripsi
<code>{{ .sId }}</code>	string	ID langganan (UUID).
<code>{{ .enabled }}</code>	bool	Apakah klien/langganan diaktifkan.
<code>{{ .download }}</code>	string	Volume unduhan yang diformat (mis. «2.5 GB»).
<code>{{ .upload }}</code>	string	Volume unggahan yang diformat.
<code>{{ .total }}</code>	string	Batas traffic total yang diformat.
<code>{{ .used }}</code>	string	Traffic yang telah digunakan yang diformat (download + upload).
<code>{{ .remained }}</code>	string	Sisa traffic yang diformat.
<code>{{ .expire }}</code>	int64	Masa berlaku – Unix-time dalam detik (<code>0</code> = tanpa batas). Untuk <code>Date JS</code> , kalikan dengan 1000.
<code>{{ .lastOnline }}</code>	int64	Waktu online terakhir – Unix-time dalam milidetik (<code>0</code> = belum pernah).
<code>{{ .downloadByte }}</code>	int64	Unduhan dalam byte yang tepat.
<code>{{ .uploadByte }}</code>	int64	Unggahan dalam byte yang tepat.
<code>{{ .totalByte }}</code>	int64	Batas total dalam byte yang tepat.
<code>{{ .subUrl }}</code>	string	URL halaman langganan.
<code>{{ .subJsonUrl }}</code>	string	URL konfigurasi JSON langganan.
<code>{{ .subClashUrl }}</code>	string	URL konfigurasi Clash/Mihomo.
<code>{{ .subTitle }}</code>	string	Judul langganan dari pengaturan (bisa kosong).
<code>{{ .subSupportUrl }}</code>	string	URL dukungan dari pengaturan (bisa kosong).
<code>{{ .links }}</code>	[]string	Daftar string konfigurasi (VMess, VLESS, dll). Iterasi: <code>{{ range .links }}</code> ... <code>{{ end }}</code> .
<code>{{ .emails }}</code>	[]string	Daftar email yang terkait dengan langganan.
<code>{{ .datepicker }}</code>	string	Format kalender panel saat ini: <code>gregorian</code> atau <code>jalali</code> (diambil dari pengaturan «Tipe Kalender»; jika kosong – <code>gregorian</code>).

Contoh minimal isi template yang menggunakan sebagian variabel:

```
<h1>{{ .subTitle }}</h1>
<p>Telah digunakan: {{ .used }} dari {{ .total }} (tersisa {{ .remained }})</p>
{{ range .links }}<div>{{ . }}</div>{{ end }}
```

Contoh: tanggal kedaluwarsa dari `expire`. Field `{{ .expire }}` adalah Unix-time dalam **detik**, oleh karena itu untuk JavaScript dikalikan dengan 1000; nilai `0` berarti «tanpa batas»:

```
<script>
  var exp = {{ .expire }};
  document.write(exp === 0
    ? 'Tanpa batas'
    : 'Hingga ' + new Date(exp * 1000).toLocaleDateString());
</script>
```

Perhatikan: `{{ .lastOnline }}` sudah dalam **milidetik** – tidak perlu dikalikan dengan 1000.

11. Xray: routing, outbounds, DNS, dan ekstensi

Bagian "**Pengaturan Xray**" adalah editor template konfigurasi Xray-core, yang menjadi dasar panel untuk menghasilkan `config.json` akhir yang digunakan untuk menjalankan inti. Keterangan untuk bagian template: *"File konfigurasi Xray dibuat berdasarkan template ini."* Berbeda dengan inbounds (yang disimpan secara terpisah di database dan dimasukkan ke dalam template saat konfigurasi dirakit), semua hal lainnya – log, routing, outbounds, DNS, kebijakan, statistik – ditentukan di sini.

Penting: nilai template disimpan di database dengan kunci `xrayTemplateConfig`. Saat disimpan, panel memprosesnya melalui serangkaian transformasi otomatis (lihat 11.11). JSON yang tidak valid secara sintaksis akan ditolak dengan kesalahan *"xray template config invalid"*.

Lokasi di menu: "Outbounds" dan "Routing"

"**Outbounds**" dan "**Routing**" adalah item menu samping yang terpisah (tepat di bawah "Hosts", di atas "Pengaturan Panel"), masing-masing memiliki alamat sendiri: `/outbound` dan `/routing`. Tautan langsung ke halaman-halaman ini dan pemuatan ulang halaman berfungsi sebagaimana mestinya. Di submenu "**Konfigurasi Xray**", yang tersisa hanyalah: Utama, Balancer, DNS, dan Template Lanjutan. Dalam deskripsi di bawah ini, bagian 11.3 dan 11.4 sesuai dengan halaman "Routing" dan "Outbounds".

11.1. Struktur editor: tab/mode

Editor menawarkan beberapa mode tampilan template (filter berdasarkan bagian JSON):

Mode	Yang ditampilkan
Utama	Bagian dasar (Log, routing dasar, pengaturan utama)
Template Lanjutan	Template JSON Xray lengkap
Semua	Semua bagian secara bersamaan

Kelompok logis pengaturan di dalam editor:

- **Pengaturan Utama** (keterangan: *"Parameter ini menjelaskan pengaturan umum"*).
- **Log** (lihat 11.10).
- **Koneksi Dasar**: pemblokiran dan rute langsung.
- **Inbounds** (keterangan: *"Mengubah template konfigurasi untuk menghubungkan klien tertentu"*).
- **Outbounds** (lihat 11.4).
- **Balancer** (lihat 11.5).
- **Routing** (keterangan: *"Prioritas setiap aturan sangat penting!"*, lihat 11.3).
- **DNS / Fake DNS** (lihat 11.6).

11.2. Pengaturan Utama (General)

Freedom Protocol Strategy

Field	Label	Deskripsi	Default
FreedomStrategy	Pengaturan strategi protokol Freedom	Strategi output jaringan untuk outbound langsung (freedom). Keterangan: "Mengatur strategi output jaringan pada protokol Freedom". Mengontrol field <code>domainStrategy</code> di dalam <code>settings</code> outbound dengan protokol <code>freedom</code> .	Dalam template referensi, <code>domainStrategy</code> untuk <code>freedom-outbound direct</code> adalah AsIs (alamat tidak di-resolve, diteruskan dalam bentuk aslinya).

`domainStrategy` untuk freedom (nilai Xray-core): **AsIs** (tidak me-resolve domain di sisi server), serta keluarga `UseIP` / `UseIPv4` / `UseIPv6` dan varian "paksa" `ForceIP*`, yang memaksa server keluar untuk me-resolve domain dan terhubung melalui IP yang diperoleh. Ubah ke `UseIPv4` jika server keluar tidak memiliki IPv6 atau Anda perlu memaksa penggunaan IPv4 saja.

Freedom Happy Eyeballs (IPv4/IPv6)

Field	Label	Deskripsi
FreedomHappyEyeballs	Freedom Happy Eyeballs (IPv4/IPv6)	Keterangan: "Dual-stack dial untuk outbound langsung (freedom) – berguna pada server keluar dengan IPv4 dan IPv6." Mengaktifkan algoritma Happy Eyeballs (percobaan simultan pada kedua keluarga alamat) untuk freedom-outbound.
try delay	(keterangan)	"Milidetik sebelum mencoba keluarga alamat lain. 150–250 md adalah titik awal yang baik." Penundaan sebelum beralih ke keluarga alamat alternatif. Rentang yang disarankan adalah 150–250 md.

Overall Routing Strategy

Field	Label	Deskripsi	Default
RoutingStrategy	Pengaturan routing domain	Strategi resolusi DNS umum untuk routing. Keterangan: "Mengatur strategi routing resolusi DNS secara keseluruhan". Mengontrol field <code>routing.domainStrategy</code> .	Dalam template referensi, <code>routing.domainStrategy</code> = AsIs .

`routing.domainStrategy` menentukan bagaimana aturan routing IP dicocokkan dengan permintaan domain: **AsIs** (hanya aturan domain, tanpa resolve), **IPIfNonMatch** (jika domain tidak cocok dengan aturan – resolve dan periksa aturan IP), **IPOnDemand** (resolve segera saat aturan IP ditemui). Agar aturan IP (misalnya `geoip:*`) berfungsi untuk permintaan domain, biasanya diperlukan **IPIfNonMatch**.

Outbound Test URL

Field	Label	Deskripsi	Default
<code>outboundTestUrl</code>	URL untuk pengujian outbound	URL untuk memeriksa konektivitas saat menguji outbound. Keterangan: "URL untuk memeriksa koneksi outbound". Disimpan secara terpisah dari template, dengan kunci <code>xrayOutboundTestUrl</code> .	<code>https://www.google.com/generate_204</code>

Nilai ini disanitasi. Saat pengujian outbound, nilai ini juga divalidasi sebagai URL publik – ini adalah perlindungan terhadap SSRF: pengguna tidak dapat menyisipkan URL sembarang (termasuk internal) melalui klien, URL pengujian selalu diambil dari pengaturan server. Nilai kosong saat penyimpanan/pengujian diganti dengan `generate_204` default.

Block BitTorrent

Field	Label	Deskripsi
<code>Torrent</code>	Blokir BitTorrent	Menambahkan aturan ke <code>routing.rules</code> yang mengarahkan lalu lintas dengan <code>protocol: ["bittorrent"]</code> ke outbound <code>blocked</code> . Dalam template referensi, aturan ini hadir secara default.

Batasan Koneksi (Connection Limits)

Keterangan: "Kebijakan tingkat koneksi untuk pengguna level 0. Biarkan field kosong untuk menggunakan nilai default Xray." Parameter ini ditulis ke `policy.levels.0`.

Field	Label	Deskripsi	Default
<code>connIdle</code>	Timeout idle (detik)	"Menutup koneksi setelah tidak aktif selama jumlah detik yang ditentukan. Mengurangi nilai ini membebaskan memori dan deskriptor file lebih cepat pada server yang sibuk (default Xray: 300)."	kosong → default Xray 300
<code>bufferSize</code>	Ukuran buffer (KB)	"Ukuran buffer internal per koneksi dalam KB. Atur ke 0 untuk meminimalkan penggunaan memori pada server dengan RAM kecil (nilai default Xray bergantung pada platform)." Placeholder: "otomatis".	kosong → bergantung pada platform; 0 – meminimalkan

Contoh (`policy.levels.0`). Field dari grup ini ditulis ke kebijakan level 0. Pada server sibuk dengan RAM kecil, Anda dapat mempercepat pembebasan sumber daya seperti ini:

```
"policy": {
  "levels": {
    "0": {
      "connIdle": 120,
      "bufferSize": 0
    }
  }
}
```

Di sini koneksi ditutup setelah 120 d tidak aktif (alih-alih default 300), dan `bufferSize: 0` meminimalkan konsumsi memori buffer. Field yang dibiarkan kosong di formulir tidak akan masuk ke JSON – dan Xray akan menerapkan nilai defaultnya sendiri.

11.3. Aturan routing (routing)

Daftar aturan `routing.rules`. **Urutannya sangat penting** ("Prioritas setiap aturan sangat penting!"): aturan dievaluasi dari atas ke bawah, aturan pertama yang cocok akan diterapkan. Keterangan: "Seret untuk mengubah urutan". Tombol kontrol urutan: **Pertama**, **Terakhir**, **Pindah ke atas**, **Pindah ke bawah**.

Setiap aturan memiliki `type: "field"`. Tombol: **Buat aturan**, **Edit aturan**. Keterangan untuk field daftar: "Elemen yang dipisahkan koma".

Di halaman "Routing", tombol "**Import aturan**" dan "**Export aturan**" dikelompokkan ke menu dropdown "**lainnya**" (more) – sama seperti di halaman "Outbounds". Tombol "**Export aturan**" tidak langsung mengunduh file, melainkan membuka jendela modal dengan pratinjau JSON dan tombol "**Salin**" dan "**Unduh**": konten dapat ditinjau sebelum disimpan. Ekspor outbounds di halaman "Outbounds" bekerja dengan cara yang sama.

Route Tester (penguji rute)

Di tab Routing terdapat sub-tab **Route Tester** – ia bertanya kepada Xray yang berjalan, outbound mana yang akan menangani koneksi tertentu, tanpa mengirimkan lalu lintas nyata. Tentukan domain atau IP, port, jaringan (TCP/UDP) dan, jika perlu, inbound dan protokol yang ditangkap (`http / tls / quic / bittorrent`), lalu tekan **Test Route**. Keputusan diambil langsung dari mesin routing yang berjalan.

Respons menampilkan outbound yang dipilih, dan saat menggunakan balancer – tag balancer juga ditampilkan. Jika tidak ada aturan yang cocok, penguji memberi tahu bahwa lalu lintas menuju outbound default (yang pertama dalam daftar `outbounds`). Ini berguna untuk memverifikasi urutan aturan sebelum mengandalkannya.

Mengaktifkan dan menonaktifkan aturan individual

Aturan routing individual dapat **dinonaktifkan** sementara menggunakan sakelar, tanpa menghapusnya. Tabel aturan memiliki kolom "**Aktifkan**" dengan sakelar (Switch), dan dalam formulir aturan field "**Aktifkan**" juga merupakan sakelar. Aturan yang dinonaktifkan tidak masuk ke konfigurasi Xray akhir, tetapi disimpan dalam template dan dapat diaktifkan kembali kapan saja.

Aturan statistik layanan (`inboundTag: ["api"]` → `outboundTag: "api"`) tidak dapat dinonaktifkan – saklarnya diblokir agar tidak merusak pencatatan lalu lintas panel (lihat [11.11](#)).

Field formulir aturan

Field formulir	Label	Field JSON	Deskripsi
Sumber	Sumber	<code>source</code>	Alamat IP/subnet sumber. Daftar yang dipisahkan koma.

Field formulir	Label	Field JSON	Deskripsi
Port sumber	Port sumber	<code>sourcePort</code>	Port sumber.
Tujuan	Tujuan	<code>domain + ip + port</code>	Domain target, IP, dan port. Domain mendukung awalan <code>domain:</code> , <code>full:</code> , <code>regexp:</code> , <code>keyword:</code> , serta <code>geosite:*</code> ; IP – <code>geoip:*</code> dan CIDR.
Jaringan	–	<code>network</code>	<code>tcp</code> , <code>udp</code> , atau <code>tcp,udp</code> . Mulai 3.4.2, ditulis ke config dalam huruf kecil (<code>tcp / udp</code>), tetapi ditampilkan dalam huruf kapital di tabel routing (<code>TCP / UDP</code>); kosong – jaringan apa pun.
Protokol	–	<code>protocol</code>	<code>http</code> , <code>tls</code> , <code>bittorrent</code> (ditentukan melalui sniffing).
Pengguna	Pengguna	<code>user</code>	Filter berdasarkan email/identifier pengguna.
Atribut / Nilai	Atribut / Nilai	<code>attrs</code>	Atribut header HTTP untuk dicocokkan.
VLESS route	VLESS route	–	Routing berdasarkan field route untuk VLESS.
Tag inbounds	Tag inbounds	<code>inboundTag</code>	Satu atau lebih tag inbound yang berlaku untuk aturan ini (termasuk bawaan <code>api</code> , dan tag DNS dari pengaturan DNS). Dalam daftar inbounds ditampilkan sebagai "tag (remark)" jika inbound memiliki catatan terpisah, jika tidak – hanya tag; dalam aturan yang disimpan, hanya tag yang disimpan.
Tag outbound	Tag outbound / Koneksi keluar	<code>outboundTag</code>	Ke mana mengarahkan lalu lintas yang cocok.
Tag balancer	Tag balancer / Balancer	<code>balancerTag</code>	Keterangan: "Mengarahkan lalu lintas melalui salah satu balancer beban yang dikonfigurasi".

Mutually exclusive antara `outboundTag` dan `balancerTag`: *"Tidak mungkin menggunakan `balancerTag` dan `outboundTag` secara bersamaan. Jika digunakan bersamaan, hanya `outboundTag` yang akan berfungsi."*
 Dalam satu aturan, tentukan tag outbound atau tag balancer.

Aturan bawaan template referensi

Dalam `config.json` standar, bagian `routing` berisi tiga aturan (dalam urutan ini):

1. `inboundTag: ["api"]` → `outboundTag: "api"` – aturan layanan untuk gRPC-API statistik panel.
2. `ip: ["geoip:private"]` → `outboundTag: "blocked"` – memblokir rentang privat.
3. `protocol: ["bittorrent"]` → `outboundTag: "blocked"` – memblokir BitTorrent.

Aturan `api` → `api` selalu secara otomatis dipindahkan ke posisi 0 saat disimpan (lihat [11.11](#)), agar permintaan statistik tidak "dimakan" oleh aturan catch-all di atasnya.

Contoh aturan. Mengirim semua lalu lintas ke situs Rusia dan jaringan privat secara langsung (melewati proxy), dan sisanya ke balancer. Urutan penting: aturan "arahkan langsung" harus berada di atas catch-all. Dalam `routing.rules` :

```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

Agar aturan IP (`geoip:ru`) juga berfungsi untuk permintaan domain, biasanya diperlukan `routing.domainStrategy: "IPIfNonMatch"` di tingkat atas routing (lihat [11.2](#)).

Grup routing yang telah dikonfigurasi sebelumnya (Koneksi Dasar)

Dalam mode "Koneksi Dasar", panel membantu menyusun aturan tipikal dari daftar yang sudah jadi:

Grup	Field	Keterangan
Blokir berdasarkan protokol/situs	—	"Konfigurasikan agar klien tidak memiliki akses ke protokol tertentu"
Blokir berdasarkan negara	IP yang diblokir, Domain yang diblokir	"Parameter ini akan memblokir lalu lintas berdasarkan negara tujuan."
Koneksi langsung	IP langsung, Domain langsung	"Koneksi langsung berarti lalu lintas tertentu tidak akan diteruskan melalui server lain."
Aturan IPv4	—	"Parameter ini akan memungkinkan klien melakukan routing ke domain tujuan hanya melalui IPv4"
Aturan WARP	—	"Ops ini akan mengarahkan lalu lintas berdasarkan tujuan tertentu melalui WARP"
Routing NordVPN	—	"Ops ini akan mengarahkan lalu lintas berdasarkan tujuan tertentu melalui NordVPN."

MTPROTO-inbound: routing lalu lintas Telegram melalui Xray

MTPROTO-inbound memiliki sakelar "**Route through Xray**" (dinonaktifkan secara default) dan pilihan **Outbound** opsional. Saat diaktifkan, panel menambahkan jembatan SOCKS loopback dengan tag inbound itu sendiri ke konfigurasi Xray, dan mtg mengarahkan lalu lintas Telegram melaluinya. Setelah itu, lalu lintas keluar Telegram dikelola oleh router: dapat dicocokkan dengan aturan biasa di tab Routing berdasarkan tag inbound atau dipaksa ke outbound atau balancer yang dipilih melalui field **Outbound**. Biarkan **Outbound** kosong agar keputusan diambil oleh aturan routing.

11.4. Outbounds (koneksi keluar)

Daftar `outbounds` . Tombol: **Buat koneksi keluar**, **Edit koneksi keluar**. Keterangan: "Mengubah template konfigurasi untuk menentukan koneksi keluar untuk server ini".

Dalam template referensi terdapat dua outbound wajib:

- `protocol: "freedom"`, `tag: "direct"` – keluar langsung ke internet (dengan `domainStrategy: "AsIs"` dan `finalRules: [{action: "allow"}]`);
- `protocol: "blackhole"`, `tag: "blocked"` – "lubang hitam" untuk lalu lintas yang diblokir.

Field formulir outbound umum

Field	Label	Deskripsi
Tag	Tag (keterangan: "Tag unik")	Identifer unik outbound. Placeholder: "tag-unik". Validasi: "Tag wajib diisi", "Tag sudah digunakan oleh outbound lain".
Protokol	—	Jenis outbound (lihat di bawah).
Alamat / Port	Alamat / Port	Target koneksi. Alamat dan port wajib diisi.
Kirim melalui	Kirim melalui	Alamat IP lokal antarmuka keluar (<code>sendThrough</code>). Placeholder: "IP lokal".
Dialer proxy (rantai)	—	Keterangan: "Hubungkan outbound ini melalui outbound lain (berdasarkan tag) untuk membangun rantai proxy. Biarkan kosong untuk koneksi langsung." Placeholder: "Pilih outbound untuk rantai". Diimplementasikan melalui <code>streamSettings.sockopt.dialerProxy</code> .

Dropdown **Dialer Proxy** menampilkan tidak hanya outbounds lokal, tetapi juga tag outbounds dari langganan – sehingga rantai dapat dibangun melalui exit yang diperoleh dari langganan. Blackhole-outbound dan outbound yang sedang diedit tetap dikecualikan dari daftar. Biarkan field kosong untuk koneksi langsung.

Protokol outbound yang didukung

Protokol yang didukung oleh formulir:

- **freedom** – keluar langsung. Field `settings.domainStrategy`, `finalRules` (lihat di bawah), Happy Eyeballs. Tidak dapat diuji ("Outbound has no testable endpoint").
- **blackhole** – membuang lalu lintas. Field **Jenis respons**. Tidak dapat diuji.
- **socks**, **http** – daftar `settings.servers[]` dengan `address / port`; field **Kata sandi otorisasi**. Untuk protokol **http**, di bawah field **Username/Password** terdapat editor **Headers** (Header) – pasangan kunci/nilai untuk header CONNECT yang dikirim ke proxy HTTP upstream. Header ini dipertahankan saat membuka kembali dan menyimpan outbound (sebelumnya hilang). Perlu diperhatikan: hanya header pada level pengaturan (`settings.headers`) yang diterapkan; header pada level server individual diabaikan oleh xray-core.
- **vmess** – `settings.vnext[]` (`address / port`).
- **vless** – `settings.address / settings.port`.
- **trojan**, **shadowsocks** – `settings.servers[]`.
- **wireguard** – `settings.peers[]` dengan `endpoint`, ditambah kunci (lihat 11.8).
- **hysteria** – `settings.address / settings.port` (transport UDP).

Untuk outbound bertipe **loopback**, blok **Sniffing** tersedia dengan parameter yang sama seperti pada inbound: aktifkan, **destOverride**, **Metadata Only**, **Route Only**, dan daftar **domain yang dikecualikan**.

Dalam mask **UDP** (FinalMask) untuk **Hysteria2**, mode tambahan tersedia. Mask **Salamander** memiliki selektor **Mode** dengan nilai **Salamander** dan **Gecko**: mode Gecko menambahkan padding paket acak dengan field ukuran **Min/Max** (`packetSize` , rentang 1–2048, default 512–1200) – ini melindungi dari fingerprinting berdasarkan panjang paket. Mask **Realm** (UDP hole-punching) mendapatkan blok **TLS Config** opsional dengan field **Server Name** (SNI), **ALPN** (`h3 / h2 / http/1.1`), **Fingerprint** (uTLS), dan sakelar **Allow Insecure**.

Contoh: rantai melalui SOCKS upstream. Outbound `upstream` terhubung ke proxy SOCKS5 eksternal, dan `chained` mengirimkan lalu lintasnya melaluinya (`dialerProxy`), membentuk rantai. Dalam `outbounds` :

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

Sekarang aturan routing dengan `outboundTag: "chained"` akan mengeluarkan lalu lintas ke internet melalui `upstream`.

Import outbound dari share-link

Outbound dapat diimpor dari share-link (`vless://` , `vmess://` , dll.). Saat mengimpor, pengaturan multiplexer **xmux** (XHTTP) yang diteruskan dalam blok `extra` link juga disimpan: setelah mengimpor, nilainya diisi ke dalam subformulir **XMUX** dari outbound yang dibuat.

Field Mux (multiplexing)

Maks. paralelisme, **Maks. koneksi**, **Maks. penggunaan ulang**, **Maks. permintaan**, **Maks. detik penggunaan ulang**, **Periode keep alive**. Parameter ini mengonfigurasi perilaku mux/XUDP dari outbound.

Sockopts (pengaturan soket)

Grup **Sockopts**: **Interval keep alive**, **Mark (fwmark)**, **Antarmuka**, **Hanya IPv6**, **Terima proxy protocol**, **Proxy protocol**, **TCP user timeout (md)**, **TCP keep-alive idle (d)**. Di sini juga dikonfigurasi dialer-proxy rantai.

Freedom finalRules (menimpa pemblokiran IP privat)

Untuk freedom-outbound, grup **Aturan Final** tersedia:

Field	Label	Deskripsi
<code>overrideXrayPrivateIp</code>	Timpa blok IP privat default di Xray	Menghapus larangan bawaan Xray untuk koneksi keluar ke IP privat.
<code>action</code>	Tindakan	<code>allow</code> (seperti dalam template referensi: <code>finalRules: [{action: "allow"}]</code>), <code>redirect</code> (Redirect), atau lainnya.
<code>blockDelay</code>	Penundaan blok (md)	Penundaan sebelum membuang koneksi.
<code>redirect / fragment</code>	Redirect / Fragment	Tindakan pengalihan dan fragmentasi lalu lintas.

Mask fragment: Lengths dan Delays per fragmen

Dalam mask **fragment** (tipe fragment di FinalMask, untuk TCP), field tunggal Length dan Delay diganti dengan daftar **Lengths** dan **Delays**: untuk setiap segmen, Anda dapat menentukan rentang panjang terpisah (misalnya `100-200`) dan penundaan dalam milidetik (misalnya `10-20` atau `0`). Baris daftar dapat ditambahkan dan dihapus; nilai tunggal yang disimpan sebelumnya secara otomatis dipindahkan ke array berisi satu elemen.

Field formulir lainnya

- **UDP over TCP** dan **Versi UoT** – untuk protokol mirip shadowsocks.
- **Tanpa header gRPC**, **Ukuran chunk Uplink** – parameter transport gRPC.
- Field TLS/uTLS: **Verifikasi nama peer**, **Pinned SHA256**, **Short ID**, **Vision testpre**, placeholder "nama server".

Pengujian outbounds

Tombol: **Uji**, **Uji semua**. Status: **Menguji koneksi...**, **Uji berhasil**, **Uji gagal**, **Gagal menguji koneksi keluar**. Hasil: **Hasil uji**, latensi dalam milidetik.

Dua mode (keterangan: "TCP: probe dial-only cepat. HTTP: permintaan penuh melalui xray."):

- **TCP** (`mode=tcp`) – dial sederhana ke `host:port`, dieksekusi secara paralel di semua endpoint, ~timeout 5 d. Hanya memeriksa keterjangkauan TCP, tidak memvalidasi protokol proxy. Untuk `freedom / blackhole / tag blocked` akan mengembalikan "Outbound has no testable endpoint".
- **HTTP** (`mode=http` atau kosong) – menjalankan instance Xray sementara, melakukan permintaan HTTP nyata (probe URL = `outboundTestUrl server`), mengukur latensi nyata. Mode otoritatif tetapi berat: diserialisasi dengan kunci global ("Another outbound test is already running, please wait"). Timeout satu percobaan – 10 d, jendela tunggu hasil – 15 d (ditingkatkan agar outbound yang sehat pada saluran lambat atau bertunnel tidak ditandai sebagai "Failed"). Jika gagal, penyebab sebenarnya (kesalahan DNS, connection refused, habis deadline, kesalahan TLS, dll.) ditulis ke log panel/Xray, yang ditunjukkan oleh pesan timeout umum.

Protokol UDP (`wireguard`, `hysteria`) dan transport UDP (`kcp`, `quic`, `hysteria`) **selalu** diuji dalam mode HTTP, bahkan jika TCP diminta – dial UDP murni tidak dapat membedakan endpoint "hidup" dari "mati". Untuk `wireguard` dalam konfigurasi uji, `noKernelTun: true` dipaksa.

Pemeriksaan batch dan perincian tahapan

Uji dan **Uji semua** dalam mode HTTP menjalankan satu instance Xray sementara bersama untuk sekumpulan outbounds, membuat SOCKS-inbound loopback dengan aturan untuk masing-masing, dan secara paralel mengirimkan permintaan HTTP nyata melalui; **Uji semua** memeriksa outbounds secara bertahap. **Uji semua** juga memeriksa outbounds yang diperoleh dari langganan (tabel "dari langganan", hanya baca) – baris mereka juga disorot dengan hasil uji. Dalam hal ini, outbounds `freedom` ("direct") dan `dns` tidak diuji dalam mode apa pun (ini bukan proxy): tombol uji tidak tersedia untuk mereka, **Uji semua** melewatinya, dan perlindungan server melarang pengujian HTTP mereka bahkan saat dipanggil melalui API secara langsung. Selain sukses/gagal, popup hasil menampilkan status HTTP respons dan perincian waktu per tahapan: **Proxy connect** (koneksi ke proxy), **TLS via outbound** (TLS melalui outbound), dan **First byte** (waktu hingga byte pertama) – ini membantu memahami pada langkah mana terjadi penundaan atau kegagalan.

Statistik lalu lintas outbounds

Panel menyimpan penghitung lalu lintas berdasarkan tag (`up` / `down` / `total`). Tombol reset mengatur ulang penghitung untuk tag tertentu atau untuk semua (`tag = "-alltags-"`). Field **Informasi akun** dan **Status koneksi keluar** menampilkan ringkasan.

11.5. Balancer (Balancers)

Daftar `routing.balancers`. Tombol: **Buat balancer**, **Edit balancer**.

Di tab Balancers terdapat kolom status langsung: **Live Target** menampilkan target aktif saat ini dari balancer dalam Xray yang berjalan, dan **Override** memungkinkan penggantian pilihan target secara manual (nilai **Auto (strategy)** mengembalikan pemilihan berdasarkan strategi). Status diperbarui dengan tombol terpisah. Jika balancer belum aktif dalam Xray yang berjalan, panel akan menyarankan untuk menyimpan perubahan terlebih dahulu atau menjalankan Xray.

Field	Label	Deskripsi
Tag	Tag (keterangan: " <i>Tag unik</i> ")	Identifer unik. Placeholder: " <i>tag balancer unik</i> ". Validasi: " <i>Tag wajib diisi</i> ", " <i>Tag sudah digunakan oleh balancer lain</i> ".
Selektor	Selektor	Daftar tag outbound (berdasarkan substring) yang menjadi pilihan balancer. Setidaknya satu harus dipilih: " <i>Pilih setidaknya satu outbound</i> ".
Fallback	Fallback	Tag outbound cadangan jika tidak ada selektor yang cocok.
Strategi	Strategi	Algoritma pemilihan (lihat di bawah).

Strategi dan parameter observasi

Strategi (`strategy.type`) menentukan bagaimana balancer memilih outbound dari selektor. Nilai Xray-core: `random` (acak), `roundRobin` (bergiliran), `leastPing` (latensi minimum berdasarkan hasil observatory), `leastLoad` (beban minimum). Untuk `leastLoad` / `leastPing`, parameter dari `strategy.settings` digunakan:

Field	Label	Deskripsi
expected	Diharapkan	Placeholder: <i>"jumlah node optimal"</i> – jumlah node hidup yang ditargetkan.
maxRtt	Maks. RTT	Batas atas RTT yang dapat diterima saat memilih kandidat.
tolerance	Toleransi	Toleransi saat membandingkan latensi/beban.
baselines	Baselines	Nilai ambang latensi untuk mengelompokkan node.
costs	Costs	Koefisien bobot (cost) untuk tag individual.

Contoh strategi. Blok `strategy` berada di dalam balancer (dalam JSON – berdampingan dengan `tag` dan `selector`):

```
"strategy": { "type": "random" } // pemilihan acak dari selektor
"strategy": { "type": "roundRobin" } // bergiliran, bergantian
"strategy": { "type": "leastPing" } // latensi minimum (memerlukan observer)
```

Untuk `leastLoad`, parameter ditentukan dalam `settings`:

```
"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regex": false, "match": "proxy-premium", "value": 0.1 },
      { "regex": true, "match": "^proxy-cheap-.*$", "value": 5 }
    ]
  }
}
```

Cara kerjanya (contoh). Misalkan observer mengukur latensi untuk exit: `A = 250 md`, `B = 280 md`, `C = 700 md`, `D = 1500 md`. Dengan pengaturan di atas, pemilihan berlangsung seperti ini:

1. **maxRTT: "1s"** – exit dengan latensi di atas 1 d dibuang: `D (1500 md)` gugur. Tersisa `A`, `B`, `C`.
2. **baselines + expected** – exit dikelompokkan berdasarkan ambang latensi, dan ambang **terkecil** yang berisi setidaknya `expected` exit dipilih. Ambang `500ms` sudah berisi `A` dan `B` – itulah 2 (= `expected`), sehingga grup `{ A, B }` dipilih. `C (700 md)` tidak masuk pemilihan selama exit cepat masih mencukupi (ia adalah "cadangan panas").
3. **tolerance: 0.05** – dalam grup yang dipilih, exit yang latensinya berbeda tidak lebih dari 5% dianggap setara, dan beban dibagi rata di antara mereka. `A (250)` dan `B (280)` berbeda sekitar 12% (> 5%), sehingga jika tidak ada perbedaan lain, preferensi diberikan ke `A` yang lebih cepat; jika perbedaannya dalam 5% – lalu lintas akan mengalir melalui `A` dan `B`.
4. **costs** – sebelum perbandingan, "biaya" exit individual disesuaikan: `value` yang lebih kecil membuat exit lebih menarik, yang lebih besar – sebaliknya. Dalam contoh, `proxy-premium` mendapat `0.1` (menjadi "lebih murah" dan lebih sering dipilih), dan semua `proxy-cheap-*` (berdasarkan ekspresi reguler, `regex: true`) – `5` (menjadi "lebih mahal" dan digunakan sebagai pilihan terakhir). Ini memungkinkan prioritas exit secara halus tanpa mengecualikannya sepenuhnya.

Hasilnya: lalu lintas akan mengalir terutama melalui **A** (jika latensinya serupa – sama dengan **B**), **C** tetap sebagai cadangan, **D** dikecualikan selama RTT-nya tidak turun di bawah **maxRTT**.

Observer: **observatory** dan **burstObservatory** (pengukuran untuk **leastPing** / **leastLoad**)

Strategi **leastPing** dan **leastLoad** tidak mengukur apa pun sendiri – mereka membutuhkan data tentang latensi dan ketersediaan setiap outbound. Data ini dikumpulkan oleh **observer** (observatory): ia secara berkala "melakukan ping" ke setiap outbound yang dipantau dan menyimpan waktu respons dan ketersediaan. Data yang sama ditampilkan di tab "**Observatory**" (status **Aktif / Tidak tersedia**, "**Aktivitas terakhir**", "**Percobaan terakhir**").

Mulai versi **3.4.2**, halaman «Balancer» dibagi menjadi tab «**Pengaturan balancer**» (tabel dan penambahan) dan «**Observatory**», dan observer diedit melalui **formulir terstruktur** (editor JSON mentah yang lama dihapus). Panel secara otomatis membuat dan menghapus observer sesuai kebutuhan: **observatory** biasa – untuk strategi **Least Ping**, **burstObservatory** lanjutan – untuk **Least Load**, serta untuk **Random/Round-robin dengan fallbackTag yang ditetapkan** (xray-core mensyaratkannya; saat **fallbackTag** terakhir dikosongkan, observer dihapus). Membuat atau menghapus observer memerlukan restart Xray penuh.

Dua varian tersedia:

- **observatory** – sederhana: **subjectSelector** + **probeURL** + **probeInterval**.
- **burstObservatory** – lanjutan, dengan konfigurasi ping halus melalui **pingConfig**; nyaman untuk beberapa exit.

Contoh blok **burstObservatory** :

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

Tujuan field:

Field	Keterangan
subjectSelector	Daftar prefiks tag outbound untuk dipantau. Xray mengambil semua outbound yang tagnya diawali dengan string yang ditentukan. Dalam contoh, exit WS-SE... , WS-FR... , WS-PL... dipantau. Tag ini harus cocok dengan yang dipilih di Selektor balancer.
pingConfig.destination	URL yang diminta melalui setiap outbound untuk mengukur latensi. Pilih halaman "ringan" dengan respons 204 tanpa isi – misalnya https://www.google.com/generate_204 . Waktu hingga respons adalah latensi yang diukur.

Field	Keterangan
<code>pingConfig.interval</code>	Seberapa sering melakukan ping ke setiap outbound. String durasi: <code>"1m"</code> – sekali per menit, juga <code>"30s"</code> , <code>"5m"</code> , dll. Lebih sering – data lebih segar, tetapi lebih banyak lalu lintas latar belakang.
<code>pingConfig.connectivity</code>	(opsional) URL pemeriksaan konektivitas dasar server itu sendiri. Jika tidak dapat dijangkau – berarti ada masalah pada jaringan server, dan observer tidak menandai outbound sebagai tidak tersedia (perlindungan terhadap false positive saat terjadi kegagalan lokal). Biasanya juga merupakan endpoint dengan respons <code>204</code> .
<code>pingConfig.timeout</code>	Berapa lama menunggu respons untuk satu ping sebelum menganggap percobaan gagal (misalnya <code>"5s"</code>).
<code>pingConfig.sampling</code>	Berapa banyak pengukuran terakhir yang disimpan dan dirata-ratakan per outbound. <code>2</code> – mempertimbangkan dua ping terakhir (memuluskan lonjakan acak).
<code>pingConfig.httpMethod</code>	Baru di 3.4.2. Metode HTTP untuk probe: <code>HEAD</code> (default) atau <code>GET</code> .

Di tab «**Observatory**», parameter yang sama ditetapkan melalui formulir (bukan JSON). Untuk `observatory` biasa, ini adalah **Watched Outbounds** (`subjectSelector` , hanya baca), **Probe URL** (`probeURL` , default `https://www.google.com/generate_204`), **Probe Interval** (`probeInterval` , `1m`), dan **Enable Concurrency** (`enableConcurrency` , default aktif). Untuk `burstObservatory` – field yang disebutkan di atas ditambah «**Metode HTTP**» yang baru. Jika balancer memiliki kedua observer, sakelar segmen mengalihkan antara formulir Observatory dan Burst.

Cara menghubungkan semuanya:

1. Di tab «**Observatory**», atur parameter observer (ia dibuat otomatis untuk strategi yang dipilih; `subjectSelector` mengikuti selektor balancer).
2. Buat balancer: **Strategi** = `leastPing` , di **Selektor** tentukan tag outbound yang sama (`WS-SE` , `WS-FR` , `WS-PL`).
3. Arahkan lalu lintas ke sana dengan aturan routing (field **Tag balancer**, lihat 11.3).
4. Mulai ulang Xray. Di tab "**Observatory**", status exit akan muncul, dan balancer akan mulai memilih yang tercepat di antara yang aktif.

Dalam satu aturan, `balancerTag` dan `outboundTag` tidak dapat diatur secara bersamaan – hanya `outboundTag` yang akan berfungsi.

Yang terjadi saat menghapus outbound atau balancer (mulai 3.4.2)

Saat menghapus outbound atau balancer, dalam tindakan yang sama panel **membersihkan tautan ke sana di routing** dan menampilkan **pratinjau dampak** di dialog konfirmasi (judul «Penghapusan ini juga akan memperbarui routing Anda», lalu untuk setiap aturan – «dihapus (tidak ada tujuan tersisa)» atau «dipertahankan (kini menggunakan ...)», serta «Balancer ... dihapus (tidak ada target tersisa)»). Dengan begitu dicegah tautan «menggantung» (`balancerTag` / `outboundTag` / `dialerProxy`) yang sebelumnya bisa membuat xray-core gagal start pada restart berikutnya. Penghapusan balancer: aturan mempertahankan `outboundTag` -nya, jika tidak – dihapus. Penghapusan outbound: tag-nya dihapus dari selektor dan

`fallbackTag` balancer, `dialerProxy` outbound lain dibersihkan, balancer yang menjadi kosong dihapus bersama aturan yang kini tak bertujuan, dan observer dirakit ulang.

11.6. DNS

Bagian `dns`. Aktifkan: **Aktifkan DNS** (keterangan: "Mengaktifkan server DNS bawaan").

Parameter DNS umum

Field	Label	JSON	Deskripsi / keterangan
<code>tag</code>	Nama tag DNS	<code>dns.tag</code>	"Tag ini akan tersedia sebagai tag inbound dalam aturan routing." Memungkinkan routing permintaan DNS itu sendiri melalui <code>inboundTag</code> .
<code>clientIp</code>	IP klien	<code>dns.clientIp</code>	"Digunakan untuk memberi tahu server tentang lokasi IP yang ditentukan selama permintaan DNS" (EDNS Client Subnet).
<code>strategy</code>	Strategi permintaan	<code>dns.queryStrategy</code>	"Strategi umum resolusi nama domain". Nilai: <code>UseIP</code> , <code>UseIPv4</code> , <code>UseIPv6</code> .
<code>disableCache</code>	Nonaktifkan cache	<code>dns.disableCache</code>	"Menonaktifkan caching DNS".
<code>disableFallback</code>	Nonaktifkan DNS cadangan	<code>dns.disableFallback</code>	"Menonaktifkan permintaan DNS cadangan".
<code>disableFallbackIfMatch</code>	Nonaktifkan DNS cadangan jika cocok	<code>dns.disableFallbackIfMatch</code>	"Menonaktifkan permintaan DNS cadangan saat daftar domain server DNS cocok".
<code>enableParallelQuery</code>	Aktifkan kueri paralel	—	"Aktifkan kueri DNS paralel ke beberapa server untuk resolusi lebih cepat".
<code>useSystemHosts</code>	Gunakan Hosts sistem	<code>dns.useSystemHosts</code>	"Gunakan file hosts dari sistem yang terinstal".

Contoh blok `dns`. Permintaan ke domain Google di-resolve melalui server DoH Cloudflare, sisanya melalui `1.1.1.1`; untuk permintaan Google, hanya IP non-privat yang diharapkan. Di tingkat atas konfigurasi:

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:!private"]
    },
    "1.1.1.1"
  ]
}
```

String server ("1.1.1.1") tanpa field – ini adalah server default untuk semua domain lainnya. Tag `dns-inbound` kemudian dapat digunakan sebagai `inboundTag` dalam aturan routing untuk mengarahkan permintaan DNS itu sendiri melalui outbound yang diperlukan.

Cache entri kedaluwarsa

Field	Label	Deskripsi
<code>serveStale</code>	Gunakan yang kedaluwarsa	"Mengembalikan hasil kedaluwarsa dari cache saat diperbarui di latar belakang".
<code>serveExpiredTTL</code>	TTL kedaluwarsa	"Masa berlaku (detik) entri cache kedaluwarsa; 0 = tidak terbatas".

Server DNS (daftar `dns.servers`)

Tombol: **Buat DNS**, **Edit DNS**, **Hapus semua** (konfirmasi: "Semua server DNS akan dihapus dari daftar. Tindakan ini tidak dapat dibatalkan."). Template: **Gunakan template**, jendela **Template DNS**, termasuk preset **Keluarga**.

Saat menekan **Edit DNS** pada entri server DNS (seperti pada entri Fake DNS), jendela pengeditan mengisi nilai server yang disimpan, bukan nilai default.

Field server DNS:

Field	Label	Deskripsi
<code>address</code>	–	Alamat DNS (IP, URL DoH, <code>localhost</code> , <code>fakedns</code> , dll.).
<code>domains</code>	Domain	Daftar domain yang menggunakan server ini.
<code>expectIPs</code>	IP yang diharapkan	Terima respons hanya jika IP ada dalam daftar.
<code>unexpectIPs</code>	IP yang tidak diharapkan	Buang respons dengan IP yang ditentukan.
<code>skipFallback</code>	Lewati Fallback	Jangan gunakan server ini sebagai fallback.
<code>finalQuery</code>	Kueri final	Menandai server sebagai final dalam rantai.
<code>timeoutMs</code>	Timeout (md)	Timeout permintaan ke server.

Hosts (entri statis)

Grup **Hosts** (`dns.hosts`). Tombol **Tambah Host**; status kosong **Host tidak ditentukan**. Field: domain (placeholder: *"Domain (mis. domain:example.com)"*) dan nilai (placeholder: *"IP atau domain – masukkan dan tekan Enter"*).

Log DNS

Lihat 11.10: flag **Log DNS** (`dnsLog`) di bagian logging.

11.7. Fake DNS

Bagian `fakedns`. Tombol: **Buat Fake DNS**, **Edit Fake DNS**.

Field	Label	Deskripsi
<code>ipPool</code>	Subnet pool IP	Rentang CIDR dari mana IP fiktif dikeluarkan (misalnya <code>198.18.0.0/15</code>).
<code>poolSize</code>	Ukuran pool	Berapa banyak alamat yang disimpan dalam pool sirkular.

Fake DNS digunakan bersama sniffing pada inbound: inti mengeluarkan IP fiktif kepada klien, mengingat pemetaan domain↔IP, dan memulihkan domain saat routing. Agar Fake DNS berfungsi, server DNS dengan alamat `fakedns` harus ditambahkan ke daftar server DNS.

Contoh: kombinasi Fake DNS + server DNS. Pertama, tentukan pool alamat fiktif, lalu tambahkan server DNS `fakedns` agar permintaan domain menerima IP dari pool ini:

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

Selain itu, pada inbound perlu mengaktifkan sniffing dengan `destOverride: ["fakedns"]` , jika tidak, inti tidak memiliki cara untuk mendapatkan domain nyata untuk pemulihan.

11.8. WireGuard / WARP / NordVPN

Field WireGuard (`wireguard`)

Field	Label	Deskripsi
<code>secretKey</code>	Kunci rahasia	Kunci privat antarmuka lokal.
<code>publicKey</code>	Kunci publik	Kunci publik peer.

Field	Label	Deskripsi
psk	Kunci bersama	PreShared Key (opsional).
allowedIPs	Alamat IP yang diizinkan	Rentang yang di-routing ke terowongan.
endpoint	Titik akhir	host:port peer.
domainStrategy	Strategi domain	Strategi resolve untuk WireGuard-outbound.

Cloudflare WARP (warp)

Integrasi menggunakan API `https://api.cloudflareclient.com/v0a4005` (client-version `a-6.30-3596`). Tindakan kontroler (`/xray/warp/:action`): `config`, `reg`, `license`, `data`, `del`.

Langkah demi langkah:

1. **Buat akun WARP** → `reg` : panel menghasilkan/menerima kunci privat (`privateKey`) dan publik (`publicKey`), mendaftarkan perangkat ke Cloudflare, dan menyimpan `access_token`, `device_id`, `license_key`, `private_key` (serta `client_id`) dalam pengaturan `warp`.
2. **Kunci lisensi WARP / WARP+** → `license` : mengatur kunci WARP+ 26 karakter (placeholder: "*Kunci WARP+ 26 karakter*"). Jika terjadi kesalahan: "*Gagal mengatur lisensi WARP*". Jika konfigurasi belum diperoleh: "*Dapatkan konfigurasi WARP terlebih dahulu*".
3. **Informasi akun: Nama perangkat, Model perangkat, Perangkat diaktifkan, Jenis akun, Peran, WARP+ data, Kuota, Penggunaan.**
4. **Tambah outbound** – membuat WireGuard-outbound dengan kunci dan endpoint Cloudflare yang diperoleh.
5. **Hapus akun** → `del` : menghapus data WARP yang tersimpan.

NordVPN (nord / nordvpn)

Integrasi menggunakan NordLynx (= WireGuard). Tindakan kontroler (`/xray/nord/:action`): `countries`, `servers`, `reg`, `setKey`, `data`, `del`.

Langkah demi langkah:

1. **Token akses** → `reg` : panel meminta kredensial NordLynx dari `api.nordvpn.com` dan mengekstrak `nordlynx_private_key`. Menyimpan `private_key` dan `token` dalam pengaturan `nord`. Alternatif – `setKey` : masukkan **Kunci privat** secara langsung (tidak boleh kosong).
2. **Negara** → `countries` memuat daftar negara; **Kota** (atau **Semua kota**).
3. **Server** → `servers` memuat server untuk negara yang dipilih (`countryId` divalidasi sebagai angka – perlindungan terhadap injeksi). Filter: hanya server dengan **Beban** > 7% yang ditampilkan. Jika tidak ada server: "*Tidak ada server yang ditemukan untuk negara yang dipilih*". Jika server tidak memiliki kunci publik NordLynx: "*Server yang dipilih tidak melaporkan kunci publik NordLynx*".
4. Membuat/memperbarui outbound: toast "*Outbound NordVPN ditambahkan*" / "*Outbound NordVPN diperbarui*".

Prioritas IPv4 dan userspace TUN

WireGuard-outbounds yang dihasilkan oleh wizard WARP dan NordVPN menggunakan `domainStrategy: "ForceIPv4v6"` (prioritas IPv4 dengan fallback ke IPv6 pada host hanya IPv6) alih-alih `ForceIP` — ini menghilangkan "pembekuan" handshake pada host dengan IPv6 yang dikonfigurasi sebagian, ketika record AAAA endpoint Cloudflare dipilih. Selain itu, `userspace TUN (noKernelTun: true)` diaktifkan untuk mereka alih-alih kernel TUN: yang terakhir memerlukan hak dan routing fwmark, dan diam-diam gagal pada banyak VPS, sementara pemeriksaan koneksi bawaan panel selalu diuji melalui userspace TUN — sekarang lalu lintas nyata dan pemeriksaan menggunakan jalur yang sama. Perubahan ini hanya berlaku untuk outbounds yang baru ditambahkan atau direset; template yang sudah disimpan mempertahankan pengaturannya.

11.9. Reverse-proxy dan TUN

Reverse (reverse-proxy)

Bagian `reverse` dari konfigurasi Xray. Dalam formulir outbound terdapat sakelar ke tipe **Reverse-proxy**.

Tombol: **Buat reverse-proxy, Edit reverse-proxy.**

Field	Label	Deskripsi
Tipe	Tipe	Bridge atau Portal — dua peran reverse-proxy Xray.
Domain	Domain	Domain label layanan untuk pasangan bridge ^[?] portal.
Tag / Koneksi	Tag / Koneksi	Tag untuk menghubungkan bridge dan portal.
Reverse Tag	Tag reverse-proxy	Keterangan: " <i>Tag koneksi keluar untuk reverse-proxy VLESS sederhana. Biarkan kosong untuk menonaktifkan.</i> " Placeholder: " <i>tag outbound (kosong = nonaktif)</i> ". Mengimplementasikan reverse VLESS yang disederhanakan.

Dalam formulir outbound juga terdapat field aliran balik: **Sniffing balik, Pekerja, Dicapangkan, Interval muat minimum (md), Ukuran muat maksimum (byte).**

TUN (tun)

Field	Label	Deskripsi	Default
name	—	"Nama antarmuka TUN."	xray0
mtu	—	"Satuan transmisi maksimum. Ukuran maksimum paket data."	1500
<code>userLevel</code>	Level pengguna	"Semua koneksi yang dibuat melalui aliran masuk ini akan menggunakan level pengguna ini."	0

11.10. Log dan statistik (Stats, metrics)

Log (log)

Keterangan: "Log dapat memperlambat server. Aktifkan hanya jenis log yang Anda perlukan!" Bagian `log` dari template referensi: `access: "none" , error: "" , loglevel: "warning" , dnsLog: false , maskAddress: ""`.

Field	Label	JSON	Deskripsi	Default
<code>logLevel</code>	Level log	<code>loglevel</code>	"Level jurnal untuk log kesalahan..." Nilai: <code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code> , <code>none</code> .	warning
<code>accessLog</code>	Log akses	<code>access</code>	"Jalur ke file log akses. Nilai khusus «none» menonaktifkan log akses."	none
<code>errorLog</code>	Log kesalahan	<code>error</code>	"Jalur ke file log kesalahan. Nilai khusus «none» menonaktifkan log kesalahan."	"" (default)
<code>dnsLog</code>	Log DNS	<code>dnsLog</code>	"Aktifkan log permintaan DNS"	false
<code>maskAddress</code>	Penyamaran alamat	<code>maskAddress</code>	"Saat diaktifkan, alamat IP nyata diganti dengan alamat penyamaran dalam log."	"" (nonaktif)

Statistik (stats / policy)

Grup **Statistik**. Mengaktifkan penghitung di `policy.system` dan `policy.levels` . Dalam template referensi: `statsInboundUplink: true , statsInboundDownlink: true , statsOutboundUplink: false , statsOutboundDownlink: false ;` untuk level `0` – `statsUserUplink: true , statsUserDownlink: true` .

Field	Label	Deskripsi	Default
<code>statsInboundUplink</code>	Statistik uplink inbound	"Mengaktifkan pengumpulan statistik untuk lalu lintas keluar dari semua proxy inbound."	true
<code>statsInboundDownlink</code>	Statistik downlink inbound	"Mengaktifkan pengumpulan statistik untuk lalu lintas masuk dari semua proxy inbound."	true
<code>statsOutboundUplink</code>	Statistik uplink outbound	"Mengaktifkan pengumpulan statistik untuk lalu lintas keluar dari semua proxy outbound."	false
<code>statsOutboundDownlink</code>	Statistik downlink outbound	"Mengaktifkan pengumpulan statistik untuk lalu lintas masuk dari semua proxy outbound."	false

Statistik untuk klien dan inbounds (uplink/downlink) adalah dasar tampilan lalu lintas di dashboard dan untuk klien; tidak disarankan untuk menonaktifkannya. Statistik outbound dinonaktifkan secara default dan hanya diperlukan jika Anda melacak lalu lintas berdasarkan tag outbound.

Metrics

Dalam template referensi terdapat bagian `metrics` (`listen: "127.0.0.1:11111"` , `tag: "metrics_out"`) dan API `metrics_out` yang sesuai. Panel menggunakan listener ini untuk mengumpulkan metrik dan snapshot observatory: ia mem-parse `metrics.listen` dari template, melakukan polling `/debug/vars` , dan mengagregasi riwayat latensi berdasarkan tag. Jika Anda mengubah alamat/port `metrics.listen` , panel akan mengakses alamat baru; menghapus bagian `metrics` akan menonaktifkan pengumpulan grafik observatory.

Pengujian outbound dalam mode HTTP menjalankan instance Xray sementara **terpisah** dengan listener `metrics` -nya sendiri pada port acak – ini bukan listener yang sama seperti pada konfigurasi utama.

11.11. Penyimpanan, mulai ulang, dan transformasi otomatis

Tombol

Tombol	Tindakan
Simpan	<code>POST /xray/update</code> : memvalidasi dan menyimpan template + <code>outboundTestUrl</code> .
Mulai ulang Xray	Memuat ulang layanan dengan konfigurasi yang disimpan. Konfirmasi: <i>"Mulai ulang xray?" / "Memuat ulang layanan xray dengan konfigurasi yang disimpan."</i>

Toast: sukses – *"Xray berhasil dimulai ulang"*, *"Xray berhasil dihentikan"*; kesalahan – *"Terjadi kesalahan saat memulai ulang Xray."*, *"Terjadi kesalahan saat menghentikan Xray."* Jendela **Output mulai ulang Xray** menampilkan output diagnostik inti.

Penerapan perubahan langsung (tanpa mulai ulang penuh)

Perubahan pada inbounds, outbounds, dan aturan routing diterapkan "langsung": saat menekan **Simpan**, panel menghitung perbedaan antara konfigurasi lama dan baru, dan hanya menerapkan bagian yang berubah melalui gRPC-API Xray (`HandlerService/RoutingService`), tanpa memulai ulang proses. Mulai ulang penuh dilakukan secara otomatis hanya ketika bagian tanpa API hot-reload berubah (`log` , `dns` , `policy` , `observatory` , dll.). Oleh karena itu, tombol "Mulai ulang" terpisah di halaman Xray tidak diperlukan – **Simpan** sendiri menerapkan perubahan. Mulai ulang inti saat diperlukan tetap dilakukan secara otomatis (lihat juga auto-reload saat memperbarui langganan dan rotasi WARP).

Pemulihan template default

Endpoint `GET /xray/getDefaultJsonConfig` mengembalikan template referensi (`config.json` , yang tertanam dalam biner). Ini dapat digunakan untuk mengatur ulang konfigurasi ke pabrik.

Transformasi otomatis saat penyimpanan

Saat menyimpan pengaturan Xray, panel melakukan (dalam urutan ini):

1. **Penghapusan pembungkus** – menghapus pembungkus seperti `{ "xraySetting": <config>, "inboundTags": ..., "outboundTestUrl": ... }` jika mereka tidak sengaja masuk ke nilai (jika tidak, lapisan akan menumpuk setiap kali disimpan). Hingga 8 lapisan dihapus.
2. **Pemeriksaan konfigurasi** – JSON di-parse ke dalam struktur konfigurasi Xray; jika terjadi kesalahan – ditolak dengan `"xray template config invalid"`.
3. **Jaminan aturan statistik** – aturan `inboundTag: ["api"]` → `outboundTag: "api"` dipaksa ke posisi 0 di `routing.rules` (atau ditambahkan jika tidak ada). Ini memastikan bahwa permintaan gRPC-statistik panel tidak akan disadap oleh aturan catch-all di atasnya (jika tidak, klien mungkin ditampilkan offline dengan lalu lintas nol saat proxy berjalan).

Karena butir 3, jangan mencoba menghapus atau memindahkan aturan `api` → `api` – panel akan memulihkannya ke tempatnya saat penyimpanan berikutnya. Ini adalah infrastruktur statistik layanan, bukan rute pengguna.

11.12. Outbound dari langganan (dengan pembaruan otomatis)

Mulai versi 3.3.0, panel dapat mengimpor `outbound` langsung dari URL langganan – format yang sama yang disediakan oleh penyedia VPN untuk aplikasi klien. Langganan secara berkala dibaca ulang di latar belakang, sehingga kumpulan `outbound` di server tetap terkini tanpa pengeditan manual template konfigurasi.

Bagian ini disebut **"Langganan outbound"**, deskripsi: "Impor outbounds dari URL langganan jarak jauh (vmess/vless/trojan/ss/...). Tag tetap tidak berubah untuk digunakan dalam balancer dan aturan routing. Pembaruan dilakukan secara otomatis." Bagian ini terletak di halaman Xray, di atas panel pengaturan `outbound`.

Cara kerjanya

Langganan disimpan terpisah dari template konfigurasi Xray. Template **tidak pernah ditimpa**: `outbound` yang diperoleh dari langganan ditambahkan ke konfigurasi akhir saat setiap generasi konfigurasi Xray.

Menambahkan langganan

Formulir "Tambah langganan" memiliki field berikut:

Field	Kunci	Default	Tujuan
URL langganan	<code>url</code>	– (wajib)	Alamat langganan. Placeholder: "https://..." (daftar tautan dalam base64). Hanya HTTP(S) yang diterima; alamat diperiksa keamanannya.
Catatan	<code>remark</code>	kosong	Label sembarang (placeholder "mis. node HK").
Prefiks tag	<code>tagPrefix</code>	<code>subN-</code>	Prefiks yang diawali tag <code>outbound</code> yang diimpor. Jika dibiarkan kosong, panel akan memilih nomor bebas terkecil seperti <code>sub1-</code> , <code>sub2-</code> , dll.

Field	Kunci	Default	Tujuan
Interval pembaruan	<code>updateInterval</code>	600 detik (10 menit)	Seberapa sering langganan dibaca ulang. Di UI ditentukan dalam jam/menit.
Aktifkan	<code>enabled</code>	ya (<code>true</code>)	Hanya langganan yang diaktifkan yang masuk ke konfigurasi dan diperbarui secara otomatis.
Izinkan alamat privat	<code>allowPrivate</code>	tidak (<code>false</code>)	Mengizinkan URL pada localhost, LAN, dan IP privat. Dinonaktifkan secara default untuk perlindungan terhadap SSRF – aktifkan hanya untuk sumber lokal tepercaya.
Sebelum outbounds manual	<code>prepend</code>	tidak (<code>false</code>)	Jika diaktifkan, <code>outbound</code> dari langganan ini ditempatkan sebelum <code>outbound</code> manual dari template, dan salah satunya dapat menjadi <code>outbound</code> default. Jika tidak – mereka ditambahkan setelah .

Tombol "**Pratinjau**" (`POST /outbound-subs/parse`) memungkinkan Anda mengunduh dan mem-parse URL sebelum menyimpan, dan melihat `outbound` dan tag apa yang akan dihasilkan; tidak ada yang ditulis ke database. Jika tidak ada yang dikenali dari URL, pesan "Tidak ada outbound yang ditemukan dari URL ini." ditampilkan.

Urutan beberapa langganan dalam daftar `outbound` umum diatur oleh prioritas (`priority`) dan diubah dengan panah atas/bawah (`POST /outbound-subs/:id/move`).

Format langganan yang diterima

Isi respons dari URL diproses sebagai berikut:

- Konten pertama dicoba sebagai **base64** (varian standar dan URL-safe, dengan pengisian padding otomatis dan penghapusan spasi/baris baru). Jika ini base64 – di-decode; jika tidak, digunakan apa adanya.
- Kemudian isi dipecah menjadi baris. Setiap baris yang tidak kosong dan tidak dimulai dengan `#` di-parse sebagai tautan. Baris yang tidak dikenali (komentar, protokol yang tidak didukung) diabaikan secara diam-diam.
- Skema tautan yang didukung: `vmess://` , `vless://` , `trojan://` , `ss://` (Shadowsocks), `hysteria2://` / `hy2://` , `wireguard://` / `wg://` .

Artinya, langganan biasa dalam format "daftar tautan yang dikodekan base64", seperti yang digunakan oleh sebagian besar penyedia, cocok.

Tag yang stabil

Setiap tautan dihitung "identitas" yang stabil (inti URI tanpa fragmen catatan; untuk `vmess` – JSON internal tanpa field `ps`). Pemetaan "identitas → tag" disimpan, dan pada pembaruan berikutnya, server yang sama mendapatkan tag yang sama, bahkan jika catatan atau parameter minor berubah. Ini dirancang khusus agar balancer dan aturan routing terus berfungsi setelah pembaruan:

- Tag yang tepat dalam balancer/aturan akan terus menunjuk ke server yang sama.
- Selektor berbasis prefiks/wildcard (misalnya, `hk-*`) secara otomatis akan mengambil server baru yang dikembalikan langganan nanti – ini adalah cara yang disarankan untuk "berlangganan ke pool".

- Jika server menghilang dari langganan, tagnya hanya akan hilang dari array `outbound` akhir; jika balancer memiliki `fallbackTag`, Xray akan menggunakannya.
- Jika penyedia mengubah UUID/host/kredensial server, identitasnya berubah – ini dianggap sebagai `outbound` baru dengan tag baru.

Di dalam satu unduhan, tag dideduplikasi dengan sufiks `-N`. Tag dari langganan menyimpan karakter non-ASCII (misalnya, Kiril) dan tetap mudah dibaca: huruf dan angka Unicode dipertahankan dalam slug, dan tanda baca diganti dengan tanda hubung – tag dari nama Kiril tidak lagi direduksi menjadi angka saja.

Cara kerja pembaruan otomatis

- Tugas latar belakang pembaruan langganan berjalan terjadwal **setiap 5 menit**.
- Pada setiap jalankan, ia mengulang semua langganan yang diaktifkan dan hanya memperbarui yang intervalnya telah kedaluwarsa: langganan diperbarui jika belum pernah diperbarui sama sekali atau jika setidaknya `updateInterval`-nya telah berlalu sejak pembaruan terakhir. Dengan demikian, tugas memeriksa langganan sering, tetapi setiap langganan tertentu dibaca ulang tidak lebih sering dari `updateInterval`-nya (default 10 menit). Di UI ini tercermin dalam keterangan yang sesuai.
- Pembaruan: URL diperiksa ulang keamanannya sebagai publik (alamat privat diblokir kecuali langganan memiliki `allowPrivate` yang disetel), permintaan dikirim melalui klien proxy panel dengan header `User-Agent: 3x-ui-outbound-sub/1.0`. Rantai pengalihan dibatasi hingga 10 hop, dan setiap hop juga diperiksa privasi (perlindungan SSRF). HTTP 200 diharapkan; jika tidak, kesalahan dicatat.
- Setelah parsing yang berhasil, hasilnya disimpan, waktu pembaruan terakhir disetel, kesalahan dihapus. Jika terjadi kesalahan, teksnya terlihat di UI sebagai "Kesalahan terakhir", dan `outbound` yang sebelumnya diperoleh tetap berlaku.
- Jika setidaknya satu langganan benar-benar diperbarui, tugas menandai Xray untuk dimulai ulang dan mengirimkan invalidasi UI agar antarmuka mengambil `outbound` baru. Muat ulang Xray yang sebenarnya terjadi pada siklus 30 detik terdekat dari manajer.

Pembaruan manual satu langganan – tombol **"Perbarui sekarang"** (`POST /outbound-sub/:id/refresh`); ini juga menandai Xray untuk dimulai ulang. Menambahkan, mengubah, menghapus langganan juga menetapkan flag mulai ulang Xray (saat penghapusan, `outbound`-nya gugur dari konfigurasi pada pemuatan ulang berikutnya). UI memberi tahu: "Setelah menambahkan atau memperbarui, mulai ulang Xray (atau tunggu auto-reload berikutnya) agar outbounds menjadi aktif."

Cara masuk ke konfigurasi Xray

Pada setiap generasi konfigurasi Xray, `outbound` langganan aktif dibagi menjadi dua grup – `prepend` (flag "Sebelum outbounds manual") dan lainnya – dan digabungkan dengan template: `[prepend langganan] + [outbound template] + [langganan lainnya]`. Di dalam setiap grup, langganan diurutkan berdasarkan prioritas. `outbound` manual dari template tidak terpengaruh; jika array `outbound` template karena alasan tertentu tidak dapat di-parse, `outbound` langganan tidak dicampur ke dalamnya (agar tidak kehilangan yang manual).

`outbound` yang diimpor juga ditampilkan di panel `outbound` itu sendiri dalam blok terpisah **"Dari langganan outbound (hanya baca)"** – mereka tidak dapat diedit di sana, pengelolaan hanya melalui bagian "Langganan outbound".

11.13. Rotasi IP di WARP

Di 3X-UI, Anda dapat mengatur WARP-outbound — koneksi WireGuard keluar ke Cloudflare WARP (tag `warp` dalam konfigurasi Xray). Panel sendiri mendaftarkan akun perangkat di server Cloudflare, mendapatkan kunci WireGuard dan alamat, dan memasukkannya ke dalam outbound dengan tag `warp`. Melalui outbound ini, lalu lintas keluar ke internet dengan alamat IP Cloudflare WARP. Fitur baru di versi 3.3.0 — kemampuan untuk mengubah IP keluar ini secara manual atau terjadwal, tanpa membuat ulang akun WARP secara manual.

Pengelolaan terletak di bagian **Xray** dalam kartu WARP (setelah menekan "Buat akun WARP" dan mendapatkan konfigurasi; sebelum itu, tindakan tidak tersedia — panel akan memberi tahu "Dapatkan konfigurasi WARP terlebih dahulu").

Apa yang terjadi saat mengganti IP

Tombol **"Ganti IP"** memulai penggantian IP. Logika:

1. Pasangan kunci WireGuard baru dihasilkan.
2. Dengan kunci baru, perangkat WARP didaftarkan ulang di server Cloudflare (baru `device_id`, `access_token`, alamat, dan data peer).
3. Data baru ditulis ke WARP-outbound konfigurasi Xray: `secretKey`, `address` (v4 /32 dan v6 /128), `reserved` (dari `client_id`), serta `publicKey` dan `endpoint` pada peer diperbarui.
4. Jika kunci lisensi WARP+ sebelumnya telah ditetapkan (panjang setidaknya 26 karakter), kunci tersebut secara otomatis diinstal ulang ke akun baru. Jika gagal, ini hanya peringatan di log — penggantian IP tidak dibatalkan.
5. Setelah penggantian berhasil, Xray ditandai sebagai memerlukan mulai ulang agar outbound baru berlaku.

Jika berhasil, antarmuka menampilkan "Alamat IP WARP berhasil diubah!".

Rotasi otomatis terjadwal

Dalam kartu WARP terdapat sakelar **"Pembaruan alamat IP otomatis"** dan field **"Interval (hari)"**. Keterangan: "0 — nonaktifkan. Secara otomatis mengubah alamat IP"

Parameter	Nilai
Pengaturan di database	<code>warpUpdateInterval</code> (bilangan bulat, ≥ 0)
Nilai default	0 (rotasi otomatis dinonaktifkan)
Satuan	hari
0	menonaktifkan penggantian otomatis
> 0	ganti IP setiap N hari

Menyimpan interval menyimpan `warpUpdateInterval`, dan untuk nilai lebih dari 0, mengatur ulang "waktu pembaruan terakhir" ke saat ini — jika tidak, penjadwal akan mengganti IP pada tick terdekat.

Jadwal dieksekusi oleh tugas latar belakang yang berjalan sekali per jam – yaitu, panel memeriksa sekali per jam apakah sudah waktunya untuk merotasi. Algoritma pemeriksaan:

- jika $\text{interval} \leq 0$ – tidak melakukan apa-apa;
- jika "waktu pembaruan terakhir" sama dengan 0 (misalnya, interval disetel dengan mengedit database secara langsung) – ini adalah jalankan pertama: tugas hanya mencatat tanda waktu dasar dan TIDAK langsung mengganti IP;
- jika setidaknya $\text{interval} \times 24 \times 3600$ detik telah berlalu sejak pembaruan terakhir – penggantian IP yang sama dilakukan, tanda waktu diperbarui, dan mulai ulang Xray dijadwalkan.

Detail penting: penggantian manual dengan tombol "Ganti IP" juga mengatur ulang tanda waktu pembaruan terakhir. Oleh karena itu, setelah rotasi manual, hitungan mundur interval otomatis dimulai kembali dan penggantian terjadwal tidak akan segera menyusul.

"Melalui proxy panel"

Diubah di 3.3.1. Pengaturan terpisah "Proxy jaringan panel" (`panelProxy`) dihapus. Lalu lintas keluar dari panel itu sendiri (termasuk permintaan ke WARP API) sekarang diarahkan melalui **outbound untuk lalu lintas panel** yang dipilih – Xray-outbound atau balancer (lihat bagian 13). Deskripsi di bawah berlaku untuk versi sebelum 3.3.1.

Semua permintaan ke Cloudflare WARP API (pendaftaran, mendapatkan konfigurasi, mengatur lisensi, mengganti IP) tidak dikirim secara langsung, melainkan melalui klien HTTP panel dengan timeout 15 detik. Klien ini menghormati pengaturan "**Proxy jaringan panel**" (`panelProxy`) dari pengaturan panel.

Dari deskripsi pengaturan: proxy me-routing permintaan keluar panel itu sendiri (pembaruan basis geo, pemeriksaan versi Xray/panel, Telegram, dan sekarang juga akses ke WARP) – untuk melewati pemfilteran server. Alamat seperti `socks5://` atau `http(s)://` diterima, misalnya SOCKS-inbound lokal dari Xray itu sendiri. Jika field kosong atau proxy dikonfigurasi secara tidak benar – koneksi langsung digunakan (perilaku tidak rusak).

Kegunaan untuk WARP: jika server tidak dapat langsung menjangkau `api.cloudflareclient.com`, pendaftaran dan rotasi sebelumnya gagal. Sekarang, dengan menentukan proxy yang berfungsi di `panelProxy` (termasuk inbound Xray sendiri), Anda dapat memastikan ketersediaan WARP API dan fungsionalitas baik tombol manual maupun rotasi terjadwal.

Kapan ini berguna

- Penggantian IP keluar secara berkala untuk outbound yang melalui WARP – mengurangi risiko pemblokiran dan pelacakan berdasarkan satu alamat.
- "Menyegarkan" IP secara manual jika alamat Cloudflare saat ini masuk daftar hitam atau bekerja lambat.
- Server yang tidak memiliki akses langsung ke Cloudflare WARP API: me-routing permintaan melalui `panelProxy` membuat pendaftaran dan rotasi dapat berfungsi.

12. Node (multipanel, master/slave)

Bagian **Node** mengubah instalasi 3X-UI biasa menjadi **panel pusat (master)** yang memantau dan mengelola panel 3X-UI lain (panel anak) secara jarak jauh. Setiap node adalah instalasi 3X-UI terpisah di servernya sendiri; master menghubungkannya melalui HTTP API miliknya, mengumpulkan statusnya, serta menyinkronkan inbound dan klien yang ditugaskan kepadanya. Inilah fitur **multipanel**: alih-alih masuk ke setiap panel satu per satu, Anda melihat semua server dalam satu daftar dan mengelolanya secara terpusat.

Prinsip penting: **node bukanlah agen, melainkan panel 3X-UI yang lengkap**. Master tidak "menginstal" apa pun padanya – ia hanya terhubung ke API-nya menggunakan token. Menghapus node dari daftar hanya menghentikan pemantauan; panel jarak jauh itu sendiri tidak terpengaruh (petunjuk: "Ini akan menghentikan pemantauan node. Panel jarak jauh itu sendiri tidak akan terpengaruh").

12.1. Ringkasan di bagian atas daftar

Di atas tabel node ditampilkan penghitung agregat:

Kolom	Deskripsi
Total node	Jumlah total node dalam daftar.
Online	Berapa banyak node yang berstatus <code>online</code> .
Offline	Berapa banyak node yang berstatus <code>offline</code> .
Latensi rata-rata	Rata-rata latensi (ping) ke node, dalam milidetik.

12.2. Menambah dan mengedit node

Tombol **Tambah node** dan **Edit node** membuka formulir dengan kolom-kolom node.

Kolom yang wajib diisi (petunjuk: "Nama, alamat, port, dan token API wajib diisi") adalah **Nama**, **Alamat**, **Port**, dan **Token API**.

Saat menekan "Simpan" (baik saat menambah maupun mengedit), panel **terlebih dahulu memeriksa keterjangkauan node** dengan batas waktu 6 detik. Jika node tidak merespons, data tidak tersimpan dan akan muncul pesan kesalahan. Artinya, Anda tidak bisa menambahkan node yang jelas-jelas tidak dapat dijangkau.

Kolom formulir

Kolom (RU)	Default	Nilai yang diizinkan	Deskripsi
Nama	— (wajib)	string tidak kosong, unik	Nama internal node. Kolom nama memiliki batasan keunikan – dua node dengan nama yang sama tidak dapat dibuat. Teks placeholder: <code>mis. de-frankfurt-1</code> . Spasi di awal/akhir dihapus saat menyimpan.
Catatan	kosong	sembarang string	Catatan/deskripsi opsional untuk node. Tidak memengaruhi fungsi.

Kolom (RU)	Default	Nilai yang diizinkan	Deskripsi
Skema	<code>https</code>	<code>http / https</code>	Protokol koneksi ke panel jarak jauh. Jika dikosongkan atau nilai tidak valid, normalisasi akan menetapkan <code>https</code> . Jika node merespons melalui HTTP biasa tetapi skema diatur ke <code>https</code> , panel akan menampilkan petunjuk yang jelas: "the server speaks HTTP, not HTTPS; set the node scheme to http".
Alamat	– (wajib)	host atau IP	Alamat panel jarak jauh. Placeholder: <code>panel.example.com</code> atau <code>1.2.3.4</code> . Alamat dinormalisasi; secara default alamat privat/lokal dilarang untuk perlindungan dari SSRF – lihat "Izinkan alamat privat".
Port	– (wajib)	bilangan bulat 1–65535	Port panel web node jarak jauh. Nilai di luar rentang akan ditolak ("node port must be 1-65535").
Jalur dasar	<code>/</code>	string path	Jalur dasar (web base path) panel jarak jauh, jika dikonfigurasi. Dinormalisasi: dijamin diawali dan diakhiri dengan <code>/</code> (nilai kosong → <code>/</code>). Panel menambahkan <code>panel/api/server/status</code> di belakangnya saat melakukan polling.
Token API	– (wajib)	token panel jarak jauh	Token Bearer untuk mengakses API node. Dikirim dalam header <code>Authorization: Bearer <token></code> . Placeholder: "Token dari halaman Pengaturan panel jarak jauh". Petunjuk: "Panel jarak jauh menampilkan token API-nya di bagian Pengaturan → Token API". Artinya, token harus dibuat di node itu sendiri (Pengaturan → Token API), lalu ditempelkan di sini.
Aktif	<code>true</code>	ya/tidak	Mengaktifkan pemantauan dan sinkronisasi node. Node yang dinonaktifkan tidak di-polling oleh tugas latar belakang (heartbeat dan traffic-sync melewatinya) dan tidak ikut dalam pembaruan panel massal.
Izinkan alamat privat	<code>false</code>	ya/tidak	Menonaktifkan perlindungan SSRF dan mengizinkan koneksi ke node melalui alamat privat/lokal. Petunjuk: "Aktifkan hanya untuk node di jaringan privat atau VPN". Aktifkan hanya jika node benar-benar berada di jaringan privat atau dapat diakses melalui VPN.

Mendapatkan dan meregenerasi token di sisi node

Token diambil dari panel jarak jauh di bagian **Pengaturan** → **Token API**. Di sana pula token dapat diperbarui: tombol **Buat ulang token** dengan peringatan: "Pembuatan ulang akan membatalkan token saat ini. Panel pusat mana pun yang menggunakannya akan kehilangan akses hingga diperbarui. Lanjutkan?". Setelah diperbarui, token lama di panel master tidak akan berfungsi – token tersebut perlu diperbarui di formulir node.

Koneksi keluar (Connection outbound)

Kolom **Connection outbound** (Koneksi keluar, `outboundTag`) menentukan bagaimana lalu lintas permintaan master ke API node ini meninggalkan server. Jika tag Xray-outbound dipilih, permintaan panel ke node tidak akan langsung, melainkan melalui outbound yang ditentukan; panel secara otomatis menambahkan bridge-inbound pada loopback ke konfigurasi aktif dan menerapkan perubahan secara langsung, tanpa perlu restart. Petunjuk: "Route this node's panel API traffic through the selected Xray outbound. A loopback bridge inbound is added to the running config automatically and applied live. Leave empty for a direct connection".

Selektor dirancang seperti pemilihan outbound di panel: tag dikelompokkan menjadi **Outbounds** (outbound biasa) dan **Balancers** (penyeimbang beban), outbound blackhole disembunyikan dari daftar. Nilai kosong (placeholder "Direct connection") = koneksi langsung ke node.

Impor inbound (pemilihan inbound yang disinkronkan)

Formulir node memiliki pengaturan **Impor inbound** (`inboundSyncMode`) dengan dua mode: **Semua inbound** (`all` , default) dan **Dipilih** (`selected`). Secara default, master menyinkronkan semua inbound yang memilih node ini ke node tersebut; node yang sudah ada tetap bekerja dalam mode "Semua inbound".

Dalam mode **Dipilih**, di bawah kolom muncul multi-pilih tag inbound. Klik **Muat inbound** – master akan meminta daftar inbound kepada node menggunakan parameter koneksi yang sudah dimasukkan (belum tersimpan) (endpoint `POST /panel/api/nodes/inbounds`) dan menampilkan tagnya; pilih yang diperlukan. Panel hanya akan menyinkronkan dan men-deploy tag yang dipilih ke node, sementara inbound lain yang ada langsung di node akan dibiarkan – master tidak menghapus atau mengelolanya.

Contoh: meminta daftar inbound node untuk impor selektif. Isi permintaan berisi parameter koneksi yang belum tersimpan; responsnya berisi tag inbound yang tersedia di node:

```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. Verifikasi TLS (untuk node https)

Sekelompok kolom menentukan cara master memverifikasi sertifikat HTTPS node. Pengaturan ini **hanya relevan untuk skema https** ; untuk node `http` , pengaturan ini diabaikan.

Verifikasi TLS – daftar dropdown, petunjuk: "Cara panel memverifikasi sertifikat HTTPS node. Pinning atau Lewati – untuk sertifikat yang ditandatangani sendiri (hanya node https)".

Mode (RU)	Nilai	Default	Deskripsi
Verifikasi (CA standar)	<code>verify</code>	ya (default)	Verifikasi rantai sertifikat biasa oleh CA tepercaya. Cocok untuk node dengan sertifikat publik/Let's Encrypt. Juga digunakan untuk semua node <code>http</code> .
Sematkan sertifikat (SHA-256)	<code>pin</code>	–	Rantai CA tidak diverifikasi, tetapi SHA-256 sertifikat leaf node dibandingkan dengan sidik jari yang tersimpan (perbandingan constant-time). Mempertahankan perlindungan dari MITM untuk sertifikat yang ditandatangani sendiri . Memerlukan kolom sidik jari diisi.
Lewati verifikasi	<code>skip</code>	–	Verifikasi sertifikat dinonaktifkan sepenuhnya. Peringatan: "Melewati verifikasi menghilangkan perlindungan dari serangan man-in-the-middle – token API dapat dicegat. Lebih baik sematkan sertifikat".

Selain tiga mode di atas, di 3.4.0 ditambahkan mode keempat – **Mutual TLS (client certificate)** (`mtls`), tersedia seperti yang lainnya, hanya untuk skema `https` .

Mode (RU)	Nilai	Default	Deskripsi
Mutual TLS (sertifikat klien)	mtls	—	Selain memverifikasi sertifikat node, master juga mengautentikasi dirinya ke node menggunakan sertifikat klien yang diterbitkan oleh CA-nya sendiri. Dalam mode ini token API menjadi opsional – node mengenali master berdasarkan sertifikat. Saat mode dipilih, ditampilkan petunjuk: "This node authenticates the panel with a client certificate. Copy this panel's CA from the Node mTLS section onto the node, set its Trusted parent CA, then restart it".

Untuk mengaktifkan mutual TLS untuk node: di sisi node, atur mode **Mutual TLS**, salin CA panel pengendali dari bagian **Node mTLS** (lihat di bawah), daftarkan sebagai **CA induk tepercaya** di node, lalu restart node.

Jika nilai selain `skip`, `pin`, atau `mtls` dipilih, normalisasi akan memaksa menetapkan `verify`.

Penyematan sertifikat

Saat memilih **Sematkan sertifikat**, akan muncul:

- **SHA-256 sertifikat yang disematkan** – kolom input. Menerima sidik jari dalam format **base64** (format `pinnedPeerCertSha256` dari Xray) atau dalam format **hex** dengan titik dua atau tanpa (gaya `openssl -fingerprint`). Petunjuk: "SHA-256 sertifikat node dalam base64 atau hex. Klik 'Ambil' untuk membacanya dari node sekarang". Placeholder: "SHA-256 dalam base64 atau hex". Saat memilih `pin`, sidik jari yang kosong atau tidak valid akan menyebabkan kesalahan validasi saat menyimpan.

Contoh: sidik jari yang sama dalam dua format. Kolom menerima salah satu dari keduanya – keduanya merujuk ke sertifikat yang sama:

```
# base64 (format pinnedPeerCertSha256 dari Xray)
607TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# hex dengan titik dua (gaya openssl x509 -fingerprint -sha256)
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```

Jika sidik jari belum diketahui, klik **Ambil** – master akan membacanya sendiri dari node melalui HTTPS dan mengisinya ke kolom.

- Tombol **Ambil** – terhubung ke node melalui HTTPS tanpa verifikasi sertifikat dan membaca SHA-256 sertifikat leaf saat ini (endpoint `POST /certFingerprint`), lalu mengisinya ke kolom. Setelah berhasil – "Sertifikat node saat ini berhasil diambil"; jika gagal – "Gagal mengambil sertifikat". Hanya tersedia untuk node https.

Node mTLS (autentikasi TLS mutual antar panel)

Di halaman **Node** terdapat bagian terpisah **Node mTLS** – pengaturan autentikasi TLS mutual yang menambahkan faktor kedua (sertifikat klien) selain token API untuk panggilan "panel → node". Mutual TLS bersifat opsional; jika kolom bagian ini kosong, node bekerja dengan skema sebelumnya – **hanya dengan**

token API (petunjuk: "Mutual TLS adds a client-certificate factor on top of the API token for node-to-node calls. It is opt-in: leave it empty to keep token-only auth"). Bagian ini memiliki dua operasi:

- **Salin CA panel ini** (`POST /panel/api/nodes/mtls/ca`) – menyalin sertifikat root (CA) panel ini ke clipboard. CA ini perlu diserahkan ke node yang dikelola agar mereka mempercayai sertifikat klien panel; pada node-node tersebut kemudian mode verifikasi TLS diatur ke **Mutual TLS** (petunjuk: "Hand this CA to the nodes this panel manages, then set their TLS verification to Mutual TLS"). Setelah disalin – "CA certificate copied to clipboard".
- **CA induk terpercaya** (`Trusted parent CA` , `POST /panel/api/nodes/mtls/trustCA`) – kolom yang digunakan ketika panel ini sendiri berperan sebagai node bagi panel pengendali (yang lebih tinggi). Tempelkan CA panel pengendali di sini untuk meminta sertifikat kliennya, lalu klik **Save trust CA**. Perubahan ini memerlukan **restart panel** (petunjuk: "When this panel is itself a node, paste the managing panel's CA here to require its client certificate. Restart the panel to apply").

12.4. Informasi yang ditampilkan untuk setiap node

Kolom tabel dan kolom kartu node (status yang diamati, diisi pada setiap polling heartbeat):

Kolom (RU)	Deskripsi
Status	<code>online</code> / <code>offline</code> / <code>unknown</code> – lihat di bawah.
CPU	Beban prosesor server jarak jauh dalam persen.
Memori	Penggunaan RAM dalam persen (dihitung sebagai $\text{current}/\text{total} \times 100$).
Uptime	Waktu server berjalan terus-menerus (dalam detik).
Latensi	Waktu respons node pada polling terakhir (ms).
Ping terakhir	Waktu heartbeat terakhir yang berhasil (detik unix; <code>0</code> = "belum pernah"; nilai terkini ditampilkan sebagai "baru saja").
Versi Xray	Versi Xray-core yang berjalan di node.
Versi panel	Versi 3X-UI di node – dibandingkan dengan versi terkini untuk indikator pembaruan.
(inbound)	Berapa banyak inbound yang secara fisik ditempatkan di node ini.
(klien)	Jumlah klien pada inbound node.
(online)	Berapa banyak klien node yang saat ini terhubung.
(habis)	Berapa banyak klien node yang kedaluwarsa atau telah melampaui batas lalu lintas . Klien yang dinonaktifkan secara manual tidak masuk dalam penghitungan ini.
(kecepatan)	Kecepatan transfer saat ini (langsung) pada inbound yang ditempatkan di node.

Penghitung inbound/klien/online dikaitkan ke node berdasarkan GUID stabil-nya (`panelGuid`), bukan berdasarkan id lokal – agar klien pada sub-node dihitung tepat di bawah sub-node tersebut, bukan di bawah node perantara yang menjadi jalur sinkronisasinya.

Untuk inbound yang ditempatkan di node, halaman menampilkan klien online, penghitung, dan **kecepatan transfer saat ini**. Pengikatan berdasarkan GUID stabil dengan benar memisahkan node yang "dikloning" dengan `panelGuid` yang sama.

Status node

Status	Arti	Kapan ditetapkan
online	Online	Node merespons <code>success=true</code> pada polling <code>panel/api/server/status</code> .
offline	Offline	Node tidak merespons, mengembalikan kesalahan HTTP, <code>success=false</code> , atau respons yang tidak dapat dikenali.
unknown	Tidak diketahui	Nilai awal, sebelum node pernah di-poll.

Saat polling gagal, teks kesalahan disimpan dan ditampilkan dalam formulasi yang jelas, membantu mendiagnosis penyebab "offline".

12.5. Tindakan pada node

- **Uji koneksi** (`POST /test`) – di formulir node, menguji koneksi menggunakan parameter yang dimasukkan (belum tersimpan) dengan batas waktu 6 detik. Hasilnya: "Koneksi berhasil ({ms} ms)" atau "Gagal terhubung". Berguna untuk men-debug alamat/port/token/TLS sebelum menyimpan.
- **Periksa sekarang** (tombol "Periksa sekarang", `POST /probe/:id`) – polling tidak terjadwal pada node yang sudah tersimpan; segera memperbarui status dan metrik (CPU/memori/uptime/latensi/versi) serta mencatat heartbeat. Jika gagal – "Pemeriksaan gagal".

Contoh: menguji dan mem-poll node melalui API master. "Uji koneksi" menguji parameter yang belum tersimpan dari formulir:

```
POST /panel/api/nodes/test
Content-Type: application/json

{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGciOi...", "tlsMode": "verify" }
```

Polling tidak terjadwal pada node yang sudah tersimpan dengan id 7:

```
POST /panel/api/nodes/probe/7
```

- **Perbarui panel** (`POST /updatePanel` dengan isi `{ids:[...]}`) – menjalankan pembaruan diri otomatis standar pada node: node mengunduh rilis terbaru 3X-UI dan melakukan restart. Tombol **Perbarui yang dipilih ({count})** menjalankan ini untuk beberapa node yang ditandai sekaligus. Di samping node ditampilkan indikator: **Pembaruan tersedia** atau **Sudah terbaru**, berdasarkan perbandingan versi panel node dengan versi terbaru.

Contoh: memperbarui beberapa node dengan satu permintaan. Isi permintaan berisi id node yang ditandai; hanya node yang aktif dan `online` yang akan diperbarui, sisanya dikembalikan sebagai dilewati.

```
POST /panel/api/nodes/updatePanel
Content-Type: application/json

{ "ids": [3, 7, 12] }
```

Respons seperti "Pembaruan dimulai pada 2 node, 1 gagal": node 12, misalnya, mungkin sedang offline sehingga dilewati.

- Konfirmasi: "Perbarui {count} node ke versi terbaru? Setiap node yang dipilih akan mengunduh rilis terbaru dan melakukan restart. Hanya node aktif yang online yang akan diperbarui".
- **Hanya node aktif dengan status online yang diperbarui.** Node yang dinonaktifkan ditandai "node is disabled" dalam hasil, yang offline ditandai "node is offline". Hasilnya: "Pembaruan dimulai pada {ok} node, {failed} gagal". Jika tidak ada node yang memenuhi syarat yang dipilih – "Pilih setidaknya satu node aktif yang online".

Dalam dialog konfirmasi pembaruan (baik untuk satu node maupun massal) terdapat kotak centang **Perbarui ke kanal pengembangan (commit terbaru)**. Jika dicentang, node yang dipilih akan menginstal build rolling dev-latest (commit terbaru dari branch main) alih-alih rilis stabil; jika tidak dicentang, node diperbarui melalui kanal biasanya. Saat kotak dicentang, peringatan ditampilkan di bawahnya: "Build pengembangan mengikuti setiap commit di main dan bukan rilis stabil – tidak ada rollback otomatis". Flag dev diteruskan melalui `POST /panel/api/nodes/updatePanel` ke node, dan node memulai pembaruan melalui kanal dev.

- **Set Cert from Panel** (opsional, `GET /webCert/:id`) – saat membuat inbound di node, memungkinkan pengisian path ke sertifikat web-TLS **milik** node itu sendiri (bukan panel pusat), agar file-file tersebut ada tepat di node. Memerlukan node dalam keadaan aktif dan dapat dijangkau.
- **Hapus node** (`POST /del/:id`) – konfirmasi: "Hapus node "{name}"? Ini akan menghentikan pemantauan node. Panel jarak jauh itu sendiri tidak akan terpengaruh". Menghapus data node dan statistik lalu lintas yang telah terkumpul; panel jarak jauh terus bekerja seperti biasa. **Node hanya dapat dihapus setelah semua inbound dilepas darinya.** Jika setidaknya satu inbound masih terhubung ke node (melalui `node_id`), panel akan menolak penghapusan dengan pesan kesalahan seperti "cannot delete node: N inbound(s) still attached to it; detach or delete them first" – lepas atau hapus inbound tersebut terlebih dahulu, baru hapus node. Ini mencegah inbound "yatim piatu" dengan referensi menggantung ke node yang sudah dihapus.

12.6. Riwayat metrik

Tombol/grafik riwayat mengakses `GET /history/:id/:metric/:bucket`. Metrik yang tersedia: **cpu** dan **mem** – keduanya diakumulasikan pada setiap heartbeat yang berhasil. Ukuran interval agregasi (`bucket`, dalam detik) dibatasi oleh daftar yang diizinkan:

Contoh: permintaan riwayat. Grafik beban CPU node 7 dengan agregasi per interval 60 detik (dikembalikan hingga 60 titik data):

```
GET /panel/api/nodes/history/7/cpu/60
```

Untuk memori dan mode "real-time" (2 detik) – masing-masing `.../7/mem/60` dan `.../7/cpu/2`. Nilai di luar daftar yang diizinkan akan ditolak ("invalid metric" / "invalid bucket").

Bucket (detik)	Tujuan
2	Mode real-time
30	Interval 30 detik
60	Interval 1 menit
120	Interval 2 menit
180	Interval 3 menit
300	Interval 5 menit

Dikembalikan hingga 60 titik data. Metrik atau bucket yang tidak valid akan ditolak ("invalid metric" / "invalid bucket").

12.7. Cara inbound dan klien disinkronkan

Inbound "dimiliki" oleh node melalui kolom `node_id` (di editor inbound, node dipilih):

Contoh: token di formulir node. Token diambil dari panel anak (Pengaturan → Token API) dan ditempelkan ke kolom **Token API** master. Pada setiap polling, master mengirimkannya dalam header:

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

Jika panel anak memiliki **jalur dasar** (web base path), misalnya `/secret/`, master akan secara otomatis menambahkannya sebelum `panel/api/server/status` → `https://panel.example.com:2053/secret/panel/api/server/status`.

1. **Deployment konfigurasi (reconcile).** Setiap kali ada perubahan pada inbound/klien yang terhubung ke node, node ditandai "kotor". Tugas latar belakang untuk setiap node aktif **berstatus online** yang memiliki perubahan akan men-deploy inbound-nya ke node (berdasarkan `node_id`) lalu mereset tanda "kotor". Node yang dinonaktifkan, offline, atau "kotor" dianggap "menunggu" – deployment ditunda hingga koneksi pulih.
2. **Pengumpulan lalu lintas.** Tugas yang sama meminta snapshot lalu lintas dari node dan menggabungkannya ke statistik lokal. Berdasarkan lalu lintas yang digabungkan, dilakukan pemeriksaan apakah batas/masa berlaku telah habis dan jika perlu klien dinonaktifkan; penghitung "habis" per node mencerminkan hal ini. Jika node tidak dapat dijangkau, klien online-nya dihapus.

Untuk klien yang terhubung ke beberapa panel sekaligus, master dalam tugas yang sama juga mengirimkan ke node-node **total penggunaan lalu lintas klien tersebut di semua panel** (dalam tabel terpisah di node, dengan kunci GUID master; ditimpa setiap kali pengiriman, sehingga reset di sisi master juga disebarkan). Di node, penggunaan lalu lintas klien menampilkan nilai yang lebih besar antara nilai lokal atau nilai yang dikirimkan, dan saat kuota total terlampaui, klien dinonaktifkan **secara lokal di node itu sendiri** (melalui mekanisme restart Xray yang sama saat auto-nonaktifkan, yang memutuskan koneksi yang sudah terbentuk). Ini menghilangkan situasi di mana node hanya melihat porsi lalu lintasnya sendiri, kekurangan dalam penghitungan, dan terus melayani klien yang sudah melampaui batas total. Saat lalu lintas direset, auto-perpanjang, atau klien dihapus, penghitung yang dikirimkan dibersihkan.

Saat **pertama kali** sinkronisasi inbound yang ditempatkan di node (menambahkan node baru atau mengimpor ulang inbound), master menginisialisasi penghitung lalu lintas klien dengan nilai aktual dari node. Sebelumnya dalam situasi ini, penghitung inbound keseluruhan dipindahkan dengan benar, tetapi penghitung klien individual direset ke nol, dan master meremehkan penggunaan klien untuk seluruh riwayat yang terkumpul sebelum node terhubung. Kini, jika inbound dibuat dalam sinkronisasi yang sama, baris `client_traffics` baru mewarisi nilai penghitung dari node (baseline ditetapkan sama dengan nilai tersebut, sehingga delta berikutnya adalah nol dan lalu lintas tidak dihitung dua kali). Pengisian penghitung hanya diterapkan untuk inbound yang dibuat dalam pass yang sama: klien yang muncul kembali di bawah inbound yang sudah ada tetap dimulai dari nol (perlindungan dari lalu lintas "phantom"), dan klien yang baru saja dihapus tidak "hidup kembali" saat inbound-nya dibuat ulang.

3. **Heartbeat.** Tugas latar belakang terpisah secara berkala mem-poll semua node **aktif** (dengan batasan konkurensi) melalui `panel/api/server/status`, memperbarui status/metrik/versi, dan jika ada klien web, menyebarkan pohon node yang diperbarui melalui WebSocket.

12.8. Rantai node (sub-node / node transitif)

Topologi tidak harus datar: sebuah node sendiri bisa menjadi master bagi node-nodenya sendiri. Panel-panel di bawahnya yang demikian ditampilkan kepada Anda sebagai **Sub-node** – ini adalah **proyeksi "hanya baca"** yang diterima dari node langsung.

- Petunjuk: "Hanya baca: node bawahan yang dapat diakses melalui {induk}. Kelola dari panel {induk} sendiri". Artinya, sub-node tidak dapat diedit, dihapus, atau diperbarui dari sini – semua operasi padanya dilakukan dari panel induk langsungnya.
- Identitas sub-node ditentukan oleh GUID-nya; berkat ini klien online dan inbound dihitung tepat di bawah node fisik yang menghosting mereka, bahkan dalam rantai `Node1 → Node2 → Node3` (master "menembus" satu level lebih dalam melalui setiap node langsung).
- Jika node langsung menjadi tidak dapat dijangkau, cache sub-nodenya dihapus, dan sub-node menghilang dari pohon hingga koneksi pulih.

12.9. Node: hal baru di 3.3.0

Pada versi 3.3.0, bagian **Node** mendapatkan tiga peningkatan signifikan: atribusi lalu lintas dan klien online yang benar dalam topologi multi-level (multi-hop), sinkronisasi client-IP antar node, dan indikator status terpisah untuk kasus ketika panel node aktif tetapi inti Xray di dalamnya crash.

1. Multi-hop: atribusi lalu lintas yang benar sepanjang rantai sub-node

Sebelumnya, penghitung (jumlah inbound, klien online, yang habis) dihitung di tingkat node "langsung". Jika Anda memiliki rantai seperti `Master → Node1 → Node2 → Node3`, semua yang secara fisik berada di `Node2 / Node3` secara keliru dikaitkan ke `Node1`, yang menjadi jalurnya ke master. Di 3.3.0, atribusi dilakukan berdasarkan sumber sebenarnya.

Cara kerjanya:

- **Sub-node terlihat sebagai baris terpisah.** Setiap panel mempublikasikan daftar node langsungnya; hanya node dengan `Guid` yang diketahui yang disertakan – identitas stabil diperlukan untuk mengaitkan node

satu "hop" ke atas. Master secara berkala (dari tugas heartbeat) mengambil daftar-daftar ini dan meng-cache-nya, lalu menambahkan sub-node "transitif" ke node-node langsung.

- **Node transitif bersifat hanya baca.** Di UI mereka ditandai sebagai **"Sub-node"** dengan petunjuk: *"Hanya baca: node bawahan yang dapat diakses melalui {induk}. Kelola dari panel {induk} sendiri."* Baris tersebut tidak memiliki tombol kontrol – node dikelola dari panel induk langsungnya.
- **Hierarki melalui GUID.** Node langsung memiliki `ParentGuid` berupa GUID master itu sendiri; node transitif memiliki GUID node induknya. Dengan demikian, pohon dibangun.
- **Sumber kebenaran untuk penghitung adalah `origin_node_guid` pada inbound.** Ini adalah `panelGuid` node yang secara fisik menyimpan inbound tersebut. Nilai ini ditetapkan saat sinkronisasi inbound dari node dan **disimpan apa adanya saat hop selanjutnya**, sehingga inbound yang sangat dalam dalam hierarki dikaitkan ke node sebenarnya, bukan ke perantara. Berdasarkan GUID ini, penghitung jumlah inbound, klien online, dan klien yang habis dihitung ulang. Logika pemilihan kunci:

Status inbound	Dikaitkan ke
<code>origin_node_guid</code> diisi	GUID ini (node sumber sebenarnya)
kosong, tetapi <code>node_id</code> diisi	GUID sintetis node (build lama, belum melaporkan <code>panelGuid</code> -nya)
kosong dan <code>node_id</code> kosong	GUID master sendiri (inbound pada Xray lokal)

Klien online juga dikelompokkan berdasarkan GUID, sehingga setiap baris node hanya menampilkan mereka yang benar-benar terhubung ke node tersebut.

Yang dilihat pengguna: dalam topologi datar (node langsung di bawah master) tidak ada yang berubah – penghitung berdasarkan GUID dan berdasarkan `id` cocok. Namun begitu ada rantai node, baris "Sub-node" muncul dalam daftar, dan angka inbound/online/habis untuk setiap node kini mencerminkan beban node itu sendiri, bukan jumlah dari semua yang melewatinya secara transit.

2. Sinkronisasi client-IP dari access.log antar node

Batas IP (`limitIp` pada klien) bergantung pada alamat yang ditulis Xray ke access.log-nya. Sebelumnya, setiap node hanya melihat koneksi ke dirinya sendiri, sehingga pembatasan "tidak lebih dari N IP per klien" tidak berfungsi dalam kluster: klien dapat terhubung ke node berbeda dan melewati batas. Di 3.3.0, IP yang diamati disinkronkan ke seluruh kluster.

Cara kerjanya:

- Di setiap node, tugas latar belakang mengurai access.log, mengekstrak IP, email klien, dan stempel waktu dari setiap baris, lalu menyimpannya ke tabel lokal (satu catatan per email, IP disimpan sebagai array JSON `{ip, timestamp}`). Alamat lokal `127.0.0.1` dan `::1` diabaikan.
- Sinkronisasi **setiap 10 detik** melakukan pertukaran dua arah per setiap node aktif yang online: mengambil IP dari node dan menggabungkannya ke tabel lokal, lalu mengirimkan gambaran keseluruhan master ke node.
- Penggabungan menyatukan pengamatan lama dan baru **tanpa penghitungan ganda** untuk satu IP yang terlihat di beberapa node, dan **tanpa menghidupkan kembali** catatan yang sudah usang: ambang batas usia yang sama digunakan seperti pada tugas lokal – **30 menit**. Untuk setiap IP, stempel waktu terbaru

disimpan. Catatan dari node lain mendapatkan id lokal baru (ruang id node bersifat independen); penyisipan konkuren dari email yang sama dilindungi dari duplikat.

- Saat menghitung batas, IP dianggap "hidup" jika terlihat dalam pemindaian lokal saat ini, atau memiliki stempel waktu yang sangat baru dari database yang disinkronkan (**dalam 2 menit**). Inilah yang membuat batas berfungsi di seluruh kluster, bahkan jika alamat terlihat di node lain. Saat batas terlampaui, IP "hidup" yang paling lama dikirim ke log fail2ban, dan koneksi diputus secara paksa (remove/re-add klien melalui API Xray).

Yang dilihat pengguna: pembatasan jumlah IP kini berlaku untuk seluruh kluster, bukan untuk setiap node secara terpisah; di panel untuk satu klien terlihat IP yang diamati pada node mana pun (dalam jendela 30 menit). Tidak ada tombol/pengaturan terpisah untuk ini – sinkronisasi berjalan otomatis di latar belakang, asalkan node memiliki access.log yang aktif dan dapat diakses (untuk batas itu sendiri, Fail2Ban juga diperlukan di node).

3. Indikator status terpisah: panel node online, tetapi Xray crash

Sebelumnya, status node pada dasarnya adalah "online / offline". Jika panel node merespons, node dianggap online – bahkan ketika inti Xray di dalamnya tidak berjalan, dan klien sebenarnya tidak dapat terhubung. Di 3.3.0, kesehatan panel dan kesehatan inti Xray dipisahkan.

Cara kerjanya:

- Saat mem-poll node, master mengambil kolom `xray.state` dan `xray.errorMessage` dari respons `/panel/api/server/status` jarak jauh dan menyimpannya di node. Kolom-kolom ini diisi bahkan saat ping panel berhasil tetapi inti tidak sehat – justru untuk membedakan keterjangkauan panel dari status Xray.
- Nilai `xray.state` : "running" (berjalan), "stop" (berhenti), "error" (error).
- Nilai-nilai ini ditranslasikan ke status node. Selain status yang sudah dikenal, ditambahkan yang baru:

Kunci status	Keterangan	Kapan ditampilkan
online	"Online"	panel merespons, Xray berjalan (running)
offline	"Offline"	panel tidak dapat dijangkau / ping gagal
unknown	"Tidak diketahui"	status belum ditentukan
xrayError	"Error Xray"	panel online, tetapi inti Xray dalam status error (ada errorMessage)
xrayStopped	"Berhenti"	panel online, tetapi Xray berhenti (stop)

- Untuk status seperti ini, UI menggunakan **indikator ungu terpisah** (warna berbeda dari hijau "online" dan merah "offline"). Ungu langsung memberi sinyal: node dapat dihubungi, masalahnya ada pada inti Xray itu sendiri, bukan pada jaringan atau panel itu sendiri.

Yang dilihat pengguna: alih-alih "hijau" yang menyesatkan saat inti crash, node disorot dengan **ungu** dengan status "**Error Xray**" atau "**Berhenti**". Ini langsung menunjukkan bahwa yang perlu diperbaiki adalah Xray di node (restart inti, lihat `errorMessage`), bukan menyelidiki keterjangkauan node itu sendiri. `xrayState` / `xrayError` yang sama juga diteruskan ke sub-node transitif (lihat poin 1), sehingga status inti yang tidak normal terlihat di seluruh rantai.

13. Pengaturan Panel

Bagian "Pengaturan" (judul halaman — **Pengaturan**, Ingg. *Panel Settings*) mengontrol perilaku panel web 3X-UI itu sendiri: di alamat dan port mana ia mendengarkan, bagaimana ia diamankan, bagaimana ia berinteraksi dengan bot Telegram dan layanan eksternal, serta di zona waktu mana ia menjalankan tugas terjadwal. Setiap parameter disimpan dalam tabel `settings` database sebagai pasangan "kunci — nilai"; jika nilai tidak ada di database, nilai default akan digunakan.

Penting — penerapan perubahan. Setiap perubahan di halaman ini harus disimpan dengan tombol **Simpan** (*Save*), lalu panel harus di-restart agar perubahan berlaku. Petunjuk resminya: "Simpan perubahan dan restart panel untuk menerapkannya." Saat menyimpan, muncul notifikasi "Pengaturan telah diubah".

13.1. Menyimpan dan Me-restart Panel

Elemen	Fungsi
Simpan (<i>Save</i>)	Menyimpan semua kolom formulir ke database (<code>POST /panel/setting/update</code>). Sebelum disimpan, nilai-nilai divalidasi — alamat, port, atau jalur yang tidak valid akan ditolak, dan panel akan mengembalikan error.
Restart Panel (<i>Restart Panel</i>)	Me-restart server web panel (<code>POST /panel/setting/restartPanel</code>). Restart dilakukan dengan jeda 3 detik. Petunjuknya: "Apakah Anda yakin ingin me-restart panel? Konfirmasi, dan restart akan terjadi dalam 3 detik. Jika panel tidak dapat diakses, periksa log server." Jika berhasil — "Panel berhasil di-restart."
Reset ke Default (<i>Reset to Default</i>)	Menghapus semua pengaturan yang tersimpan di database, setelah itu panel menggunakan nilai default. Kredensial administrator tidak direset oleh operasi ini.

Restart dilakukan dengan mengirimkan sinyal `SIGHUP` ke proses panel (atau melalui hook restart yang terdaftar). Di Windows, restart otomatis melalui sinyal tidak didukung. **Perubahan pada parameter listening (IP, port, jalur, domain, sertifikat, zona waktu) hanya diterapkan setelah panel di-restart.**

13.2. Pengaturan Umum (tab "Panel" / *General*)

Bahasa Antarmuka (*Language*)

Bahasa antarmuka web panel. Bahasa yang tersedia: `en-US` (Inggris), `ru-RU` (Rusia), `zh-CN`, `zh-TW`, `fa-IR`, `ar-EG`, `es-ES`, `id-ID`, `ja-JP`, `pt-BR`, `tr-TR`, `uk-UA`, `vi-VN`. Ini adalah pengaturan tampilan dan tidak memengaruhi cara kerja Xray.

Jenis Kalender (*Calendar Type*)

- **Kunci:** `datepicker`
- **Nilai default:** `gregorian` (Gregorian).
- **Fungsi:** jenis kalender yang digunakan dalam pemilihan tanggal (misalnya, saat menetapkan masa berlaku klien). Petunjuk: "Tugas terjadwal akan dijalankan sesuai dengan kalender ini." Nilai alternatif — kalender Persia (Jalali), yang populer di kalangan pengguna panel dari Iran.

Ukuran Paginasi (*Pagination Size*)

- **Kunci:** `pageSize`
- **Nilai default:** `25`
- **Nilai yang diizinkan:** bilangan bulat dari `0` hingga `1000`.
- **Fungsi:** jumlah baris per halaman dalam tabel (daftar koneksi/inbound). Petunjuk: "Tentukan ukuran halaman untuk tabel koneksi. Setel ke 0 untuk menonaktifkan" – jika `0`, tampilan halaman dinonaktifkan dan semua entri ditampilkan dalam satu daftar.
- **Restart panel tidak diperlukan** (pengaturan tampilan).

Restart Xray Setelah Penonaktifan Otomatis (*Restart Xray After Auto Disable*)

- **Kunci:** `restartXrayOnClientDisable`
- **Nilai default:** `true`
- **Fungsi:** saat klien dinonaktifkan secara otomatis (karena masa berlaku habis atau batas trafik tercapai), Xray di-restart untuk memutus koneksi yang sudah dibuat oleh klien tersebut. Petunjuk: "Saat klien dinonaktifkan secara otomatis karena masa berlaku habis atau batas trafik, restart Xray." Fungsionalitasnya tidak berubah – tombol toggle hanya berada di tab "Panel" (*General*) bersama pengaturan umum lainnya.

Model Keterangan dan Karakter Pemisah (*Remark Model & Separation Character*)

- **Kunci:** `remarkModel`
- **Nilai default:** `-ieo`
- **Fungsi:** menentukan bagaimana nama (remark) konfigurasi dalam subscription dibentuk. String terdiri dari **karakter pertama** – pemisah, diikuti **urutan huruf**:
 - `i` – keterangan inbound (*inbound remark*);
 - `e` – email klien;
 - `o` – label tambahan (*extra*).

Dengan nilai default `-ieo`, pemisahannya adalah `-`, dan urutan bagiannya: inbound → email → extra (misalnya, `MyInbound-user@mail-extra`). Bagian yang kosong akan dilewati. Kolom "Contoh Keterangan" (*Sample Remark*) di antarmuka menampilkan pratinjau nama yang dibentuk. Penyertaan email dalam nama juga bergantung pada parameter "Sertakan Email dalam nama" di pengaturan subscription (bagian tentang subscription).

Contoh: bagaimana nilai `remarkModel` memengaruhi nama konfigurasi. Misalkan inbound bernama `VLESS-Reality`, email klien – `alex@vpn`, dan label tambahan – `RU`. Maka:

Nilai kolom	Nama akhir (remark)
<code>-ieo</code> (default)	<code>VLESS-Reality-alex@vpn-RU</code>
<code>_ie</code>	<code>VLESS-Reality_alex@vpn</code>
<code>-ei</code>	<code>alex@vpn-VLESS-Reality</code>
<code>o</code> (spasi sebagai pemisah, hanya label)	<code>RU</code>

Karakter pertama string selalu menjadi pemisah; huruf-huruf selanjutnya menentukan bagian mana dan dalam urutan apa yang akan masuk ke nama.

13.3. Akses ke Panel: IP, Port, Jalur, Domain, Sertifikat

Grup ini menentukan titik masuk jaringan panel. **Semua perubahan di sini hanya diterapkan setelah panel di-restart.**

Kolom	Kunci	Nilai default	Deskripsi
Alamat IP untuk manajemen panel (<i>Listen IP</i>)	<code>webListen</code>	<code>""</code> (kosong)	IP tempat panel web mendengarkan. Kosong = mendengarkan di semua IP. Petunjuk: "Biarkan kosong untuk koneksi dari IP mana pun". Jika diisi, harus berupa alamat IP yang valid (jika tidak, penyimpanan akan ditolak).
Domain panel (<i>Listen Domain</i>)	<code>webDomain</code>	<code>""</code> (kosong)	Nama domain panel untuk memverifikasi permintaan berdasarkan domain. Kosong = menerima koneksi dari domain dan IP mana pun. Petunjuk: "Biarkan kosong untuk koneksi dari domain dan IP mana pun."
Port panel (<i>Listen Port</i>)	<code>webPort</code>	<code>2053</code>	Port tempat panel beroperasi. Petunjuk: "Port tempat panel beroperasi". Diizinkan <code>1–65535</code> . Port harus bebas; panel dan layanan subscription tidak dapat menggunakan pasangan <code>IP:port</code> yang sama secara bersamaan.
Jalur URI (<i>URI Path</i>)	<code>webBasePath</code>	<code>/</code>	Jalur URL dasar panel (<code>basePath</code>). Petunjuk: "Harus dimulai dengan <code>'</code> ' dan diakhiri dengan <code>'/'</code> ". Saat menyimpan, panel secara otomatis menambahkan <code>/</code> di awal dan akhir jika tidak ada. Karakter yang dilarang dalam jalur akan ditolak.

Sertifikat Panel (TLS / HTTPS)

Kolom	Kunci	Nilai default	Deskripsi
Jalur file kunci publik sertifikat panel (<i>Public Key Path</i>)	<code>webCertFile</code>	<code>""</code>	Jalur lengkap ke file sertifikat (rantai). Petunjuk: "Masukkan jalur lengkap yang dimulai dengan <code>'/'</code> ".
Jalur file kunci privat sertifikat panel (<i>Private Key Path</i>)	<code>webKeyFile</code>	<code>""</code>	Jalur lengkap ke file kunci privat. Petunjuk: "Masukkan jalur lengkap yang dimulai dengan <code>'/'</code> ".

Jika **setidaknya satu** dari jalur sertifikat/kunci diisi, panel saat menyimpan akan mencoba memuat pasangan "sertifikat + kunci"; jika terjadi error (file tidak ada, kunci dan sertifikat tidak cocok), penyimpanan akan ditolak. Jika kedua jalur yang valid diisi, panel beralih ke HTTPS. Keduanya kosong = panel beroperasi dengan HTTP biasa.

Peringatan keamanan (*Security warnings*). Panel menampilkan blok "Panel Anda mungkin terbuka:" dengan peringatan jika konfigurasi yang tidak aman terdeteksi:

- beroperasi dengan HTTP biasa – "konfigurasi TLS untuk produksi";
- port standar 2053 – "ubah ke port acak";

- jalur dasar default / – "ubah ke jalur acak";
- jalur subscription standar /sub/ dan JSON-subscription /json/ – "ubah ini". Ini adalah rekomendasi, bukan pemblokiran.

13.4. Sesi, Proxy Panel, dan Proxy Tepercaya (tab "Proxy dan Server" / *Proxy and Server*)

Durasi Sesi (*Session Duration*)

- **Kunci:** `sessionMaxAge`
- **Nilai default:** 360 (menit, yaitu 6 jam).
- **Nilai yang diizinkan:** dari 1 hingga 525600 menit (1 tahun).
- **Fungsi:** berapa lama administrator tetap terautentikasi tanpa login ulang. Satuannya – **menit**. Petunjuk: "Durasi sesi dalam sistem (nilai: menit)".

Outbound untuk Trafik Panel (*Panel Traffic Outbound*)

- **Kunci:** `panelOutbound`
- **Nilai default:** "" (kosong = koneksi langsung).
- **Fungsi:** menentukan **outbound** Xray yang digunakan panel untuk mengirimkan **permintaannya sendiri** – pemeriksaan versi dan pengunduhan panel/Xray, permintaan ke Telegram, pembaruan biasa file geo – untuk melewati pemfilteran server GitHub/Telegram. Kolom ini berupa **daftar dropdown**: berisi outbound dari template konfigurasi Xray, outbound dari subscription ke outbound, serta **balancer** rute (dalam grup terpisah). Outbound bertipe `blackhole` dikecualikan dari daftar – mengarahkan unduhan ke "lubang hitam" tidak masuk akal. Petunjuk resminya: "Merutekan permintaan internal panel sendiri – pemeriksaan versi dan unduhan panel/Xray, Telegram, dan pembaruan biasa file geo – melalui outbound Xray ini untuk melewati pemfilteran server GitHub/Telegram. Inbound loopback bridge ditambahkan ke konfigurasi aktif secara otomatis dan diterapkan secara langsung. Pembaruan otomatis Geodata bawaan Xray tidak terpengaruh; ia memiliki outbound sendiri untuk pengunduhan. Biarkan kosong untuk koneksi langsung."

Cara kerjanya. Saat outbound dipilih, panel sendiri menambahkan ke konfigurasi aktif sebuah inbound loopback layanan (SOCKS bridge dengan tag `panel-egress`) dan aturan routing yang mengarahkan trafik HTTP internal panel ke outbound yang dipilih. Jika balancer dipilih, `balancerTag` dimasukkan ke aturan, dan trafik panel didistribusikan di antara anggota-anggotanya. Bridge dan aturan diterapkan **secara langsung**, tanpa restart penuh panel. Biarkan kolom kosong untuk koneksi langsung. Pembaruan otomatis geodata bawaan Xray **tidak terpengaruh** oleh pengaturan ini – ia memiliki outbound sendiri di dalam routing Xray.

- **Format:** `socks5://` (atau `socks5h://`) atau `http(s)://`, dengan otorisasi jika diperlukan dalam bentuk `socks5://user:pass@host:port`. Skema yang didukung secara ketat: `socks5`, `socks5h`, `http`, `https` – skema lain dianggap tidak valid, dan panel akan kembali ke koneksi langsung. Contoh umum – inbound SOCKS lokal Xray itu sendiri.
- Petunjuk resminya: "Merutekan permintaan keluar panel sendiri (pembaruan geo, pemeriksaan versi Xray/panel, Telegram) melalui proxy ini untuk melewati pemfilteran server GitHub/Telegram. Menerima `socks5://` atau `http(s)://`, misalnya inbound SOCKS lokal Xray. Biarkan kosong untuk koneksi langsung."

- Proxy yang tidak valid tidak menyebabkan error saat menyimpan – panel hanya menggunakan koneksi langsung dan mencatat peringatan di log.

Contoh nilai kolom. Jika server sudah memiliki inbound SOCKS lokal Xray di port `10808`, arahkan permintaan internal panel melaluinya:

```
socks5://127.0.0.1:10808
```

Untuk proxy HTTP eksternal dengan otorisasi:

```
http://user:pass@proxy.example.com:8080
```

Setelah menyimpan dan me-restart, panel akan mengunduh pembaruan database geo, memeriksa versi, dan menghubungi Telegram melalui proxy yang ditentukan.

CIDR Proxy Tepercaya (*Trusted proxy CIDRs*)

- **Kunci:** `trustedProxyCIDRs`
- **Nilai default:** `127.0.0.1/32,::1/128` (hanya localhost).
- **Format:** daftar alamat IP atau subnet CIDR yang dipisahkan koma (misalnya `10.0.0.0/8, 192.168.1.5`). Setiap elemen diverifikasi sebagai IP atau CIDR – nilai yang tidak valid ditolak saat menyimpan.
- **Fungsi:** mencantumkan sumber yang diizinkan untuk menetapkan header `X-Forwarded-Host`, `X-Forwarded-Proto`, dan header IP klien nyata. Petunjuk: "IP/CIDR yang dipisahkan koma, yang diizinkan untuk menetapkan header forwarded host, proto, dan client IP." Perlu dikonfigurasi jika panel beroperasi di belakang reverse proxy (nginx, Caddy, dll.) agar IP klien dan skema teridentifikasi dengan benar.

Contoh: panel di belakang reverse proxy. Jika nginx berada di host yang sama dan mem-proxy permintaan ke panel, biarkan kepercayaan hanya ke localhost (nilai default):

```
127.0.0.1/32,::1/128
```

Jika proxy berada di server terpisah di jaringan internal `10.0.0.0/8`, tambahkan subnetnya, jika tidak panel akan mengabaikan header yang dikirim olehnya dan akan melihat IP proxy alih-alih klien nyata:

```
127.0.0.1/32,::1/128,10.0.0.0/8
```

Contoh blok nginx yang sesuai yang meneruskan IP nyata dan skema:

```
proxy_set_header X-Real-IP      $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

13.5. Bot Telegram (tab "Bot Telegram" / *Telegram Bot*)

Aktifkan Bot Telegram (*Enable Telegram Bot*)

- **Kunci:** `tgBotEnable`
- **Tipe/default:** boolean, `false`.
- **Fungsi:** mengaktifkan bot Telegram. Petunjuk: "Akses fitur panel melalui bot Telegram".

Token Telegram (*Telegram Token*)

- **Kunci:** `tgBotToken`
- **Default:** `""`.
- **Fungsi:** token bot. Petunjuk: "Anda perlu mendapatkan token dari manajer bot Telegram @botfather".
- **Fitur keamanan:** token termasuk nilai rahasia. Dalam respons panel saat membaca pengaturan, token tidak dikembalikan (kolom dikosongkan, hanya flag "dikonfigurasi/tidak dikonfigurasi" yang dikembalikan). Jika kolom dibiarkan kosong saat menyimpan, token yang tersimpan sebelumnya **tetap disimpan** (tidak dihapus).

Bahasa Bot Telegram (*Telegram Bot Language*)

- **Kunci:** `tgLang`
- **Default:** `en-US`.
- **Fungsi:** bahasa pesan bot (terlepas dari bahasa antarmuka web). Daftar bahasa yang tersedia sama dengan bahasa panel.

User ID Administrator Bot (*Admin Chat ID*)

- **Kunci:** `tgBotChatId`
- **Default:** `""`.
- **Format:** satu atau beberapa Telegram User ID numerik **dipisahkan koma**.
- **Fungsi:** penerima notifikasi dan administrator yang diizinkan mengelola panel melalui bot. Petunjuk: "Satu atau beberapa User ID administrator bot Telegram. Untuk mendapatkan User ID, gunakan @userinfobot atau perintah '/id' di bot."

Frekuensi Notifikasi (*Notification Time*)

- **Kunci:** `tgRunTime`
- **Default:** `@daily` (sekali sehari).
- **Format:** string dalam format **Crontab** (mendukung ekspresi cron standar maupun singkatan seperti `@daily`, `@hourly`, `@every 1h`). Petunjuk: "Tentukan interval notifikasi dalam format Crontab". Mengontrol laporan berkala bot.

Contoh nilai kolom.

Nilai	Kapan bot mengirim laporan
@daily	sekali sehari pada tengah malam (default)
@hourly	setiap jam
@every 6h	setiap 6 jam
0 9 * * *	setiap hari pukul 09:00
30 8 * * 1	setiap Senin pukul 08:30

Waktu dihitung dalam zona waktu dari pengaturan "Zona Waktu" (p. 13.6).

SOCKS Proxy (*SOCKS Proxy*)

- **Kunci:** `tgBotProxy`
- **Default:** `""`.
- **Fungsi:** SOCKS5 proxy khusus untuk koneksi bot ke Telegram. Petunjuk: "Jika Anda membutuhkan proxy Socks5 untuk terhubung ke Telegram, konfigurasi parameter sesuai panduan." Berlaku khusus untuk trafik bot (berbeda dari "Proxy Jaringan Panel" umum dari p. 13.4).

Telegram API Server (*Telegram API Server*)

- **Kunci:** `tgBotAPIServer`
- **Default:** `""` (gunakan server standar `api.telegram.org`).
- **Format:** URL `http(s)://...`; saat menyimpan, validitas URL diperiksa — alamat yang tidak valid ditolak. Petunjuk: "Server API Telegram yang digunakan. Biarkan kosong untuk menggunakan server default." Diperlukan untuk Telegram Bot API server yang di-deploy sendiri.

Notifikasi Bot (grup "Notifikasi" / *Notifications*)

Kolom	Kunci	Default	Deskripsi
Pencadangan Database (<i>Database Backup</i>)	<code>tgBotBackup</code>	<code>false</code>	Kirim file cadangan database ke Telegram bersama laporan. Petunjuk: "Kirim notifikasi dengan file cadangan database".
Notifikasi Login (<i>Login Notification</i>)	<code>tgBotLoginNotify</code>	<code>true</code>	Beri notifikasi saat ada upaya login ke panel. Petunjuk: "Menampilkan nama pengguna, alamat IP, dan waktu saat seseorang mencoba login ke panel Anda."
Jeda Notifikasi Kedaluwarsa Sesi (<i>Expiration Date Notification</i>)	<code>expireDiff</code>	<code>0</code>	Berapa hari sebelum masa berlaku klien habis untuk mengirim notifikasi. <code>0</code> — dinonaktifkan. Diizinkan <code>>= 0</code> . Petunjuk: "Terima notifikasi tentang kedaluwarsa sesi sebelum nilai ambang batas tercapai (nilai: hari)".
Ambang Batas Trafik untuk Notifikasi (<i>Traffic Cap Notification</i>)	<code>trafficDiff</code>	<code>0</code>	Ambang batas sisa trafik untuk notifikasi. Petunjuk: "Terima notifikasi tentang trafik yang habis sebelum ambang batas tercapai (nilai: GB)". Diizinkan <code>0-100</code> .

Kolom	Kunci	Default	Deskripsi
Ambang Batas Beban CPU (<i>CPU Load Notification</i>)	<code>tgCpu</code>	<code>80</code>	Beri notifikasi kepada administrator jika beban CPU melebihi ambang batas (dalam %). Diizinkan <code>0–100</code> . Petunjuk: "Beri notifikasi kepada administrator di Telegram jika beban CPU melebihi ambang batas ini (nilai: %)".

13.6. Tanggal dan Waktu (tab "Tanggal dan Waktu" / *Date and Time*)

Zona Waktu (*Time Zone*)

- **Kunci:** `timeLocation`
- **Nilai default:** `Local` (zona waktu sistem server).
- **Format:** nama zona dari database IANA tz (misalnya, `Europe/Moscow` , `UTC` , `Asia/Tehran`).
- **Fungsi:** zona waktu tempat panel menjalankan tugas terjadwal (laporan bot, reset/pemeriksaan trafik, kedaluwarsa masa berlaku). Petunjuk: "Tugas terjadwal dijalankan sesuai waktu di zona waktu ini".
- **Validasi:** zona waktu diperiksa saat menyimpan – zona yang tidak ada ditolak. Jika nilai yang tidak valid ditemukan di database di kemudian hari, panel akan kembali ke `Local` saat runtime, dan jika itu pun tidak tersedia – ke `UTC` .

13.7. Trafik Eksternal dan Perilaku Xray (tab "Trafik Eksternal" / *External Traffic*)

Kolom	Kunci	Default	Deskripsi
Informasi Trafik Eksternal (<i>External Traffic Inform</i>)	<code>externalTrafficInformEnable</code>	<code>false</code>	Beri notifikasi ke API eksternal setiap kali trafik diperbarui. Petunjuk: "Beri notifikasi ke API eksternal setiap kali trafik diperbarui."
URI Informasi Trafik Eksternal (<i>External Traffic Inform URI</i>)	<code>externalTrafficInformURI</code>	<code>""</code>	URL tempat panel mengirimkan pembaruan trafik. Melewati pemeriksaan validitas URL saat menyimpan. Petunjuk: "Pembaruan trafik dikirimkan ke URI ini".
Restart Xray Setelah Penonaktifan Otomatis (<i>Restart Xray After Auto Disable</i>)	<code>restartXrayOnClientDisable</code>	<code>true</code>	Restart Xray saat klien dinonaktifkan secara otomatis karena masa berlaku habis atau batas trafik terlampaui. Petunjuk: "Saat klien dinonaktifkan secara otomatis karena masa berlaku habis atau batas trafik, restart Xray." Tombol toggle berada di tab "Panel" (<i>General</i>) – lihat p. 13.2; dicantumkan di sini untuk kelengkapan.

13.8. Lainnya: Template Konfigurasi Xray dan URL Pengujian

Template Konfigurasi Xray (*xrayTemplateConfig*)

- **Kunci:** `xrayTemplateConfig`
- **Default:** template JSON bawaan (embedded) yang disertakan dengan build.

- **Fungsi:** template JSON dasar konfigurasi Xray-core, di atas mana panel membangun inbound/outbound. Nilai ini **tidak dikembalikan** dalam output pengaturan biasa dan diedit di halaman konfigurasi Xray terpisah, bukan dalam daftar kolom pengaturan panel umum. Template standar default tersedia melalui `GET /panel/setting/getDefaultJsonConfig`.

URL Pengujian Outbound (`xrayOutboundTestUrl`)

- **Kunci:** `xrayOutboundTestUrl`
- **Default:** `https://www.google.com/generate_204`
- **Fungsi:** URL yang digunakan saat memeriksa ketersediaan koneksi outbound. Saat ditetapkan, melewati sanitasi sebagai URL HTTP(S).

13.9. Akun Administrator dan Token API

Parameter ini berada di tab terkait ("Akun" / *Authentication*) dan dibahas secara rinci di bagian keamanan; berikut adalah ringkasan singkat kunci-kuncinya.

- **Perubahan kredensial** (kolom "Login Saat Ini", "Kata Sandi Saat Ini", "Login Baru", "Kata Sandi Baru") disimpan dengan permintaan terpisah `POST /panel/setting/updateUser`. Memerlukan login dan kata sandi saat ini yang benar; login dan kata sandi baru tidak boleh kosong. Pesan: "Anda berhasil mengubah kredensial administrator." / "Nama pengguna atau kata sandi salah".
- **Autentikasi dua faktor (2FA)** – kunci `twoFactorEnable` (default `false`) dan rahasia `twoFactorToken`. Token adalah rahasia: saat 2FA diaktifkan, kolom yang dikosongkan saat menyimpan tidak menghapus token yang ada. Saat 2FA **pertama kali** diaktifkan, panel menginvalidasi sesi saat ini (meningkatkan "era login").
- **Token API** dikelola oleh endpoint terpisah (`/panel/setting/apiTokens...`): daftar, pembuatan (`apiTokens/create`), penghapusan, pengaktifan/penonaktifan. Token itu sendiri **hanya ditampilkan sekali saat pembuatan** dan tidak disimpan dalam format yang dapat dibaca: "Salin token ini sekarang. Demi keamanan, token tidak disimpan dalam format yang dapat dibaca dan tidak akan ditampilkan lagi."

Detail mengenai 2FA, kata sandi, sinkronisasi LDAP, dan format subscription (JSON/Clash, fragmentation, noises, mux) dibahas di bagian panduan terpisah yang sesuai.

13.10. Perubahan API di 3.3.0 (penting untuk integrasi)

Pada versi 3.3.0, struktur jalur API server berubah. Jika Anda memiliki integrasi eksternal (skrip, bot, panel pusat, tugas CI) yang mengakses panel melalui HTTP, integrasi tersebut **perlu diperbarui**, jika tidak akan berhenti berfungsi.

 **BREAKING CHANGE:** endpoint `/panel/setting/*` dan `/panel/xray/*` pindah ke bawah `/panel/api`

Sebelumnya, manajemen pengaturan panel dan konfigurasi Xray berada secara terpisah, di bawah jalur `/panel/setting/*` dan `/panel/xray/*`. Sekarang keduanya terdaftar di dalam grup API umum `/panel/api`. Jalur lama **dihapus sepenuhnya** – permintaan ke jalur tersebut akan mengembalikan 404.

Alasan perubahan ini: seluruh grup `/panel/api` melewati pemeriksaan akses terpadu, artinya endpoint ini sekarang menerima header `Authorization: Bearer <token>` yang sama dengan API lainnya. Token API adalah akses administratif penuh, sehingga seluruh permukaan API menjadi seragam.

Yang TIDAK berubah: halaman antarmuka web (rute SPA) `/panel/settings` dan `/panel/xray` tetap di tempatnya – ini hanya tentang endpoint API server.

Tabel Korespondensi Jalur (lama → baru)

Prefiks untuk semua jalur di bawah – hanya ditambahkan `api/` setelah `/panel/`.

Sebelumnya (≤ 3.2.x)	Sekarang (3.3.0)	Metode
<code>/panel/setting/all</code>	<code>/panel/api/setting/all</code>	POST
<code>/panel/setting/defaultSettings</code>	<code>/panel/api/setting/defaultSettings</code>	POST
<code>/panel/setting/update</code>	<code>/panel/api/setting/update</code>	POST
<code>/panel/setting/updateUser</code>	<code>/panel/api/setting/updateUser</code>	POST
<code>/panel/setting/restartPanel</code>	<code>/panel/api/setting/restartPanel</code>	POST
<code>/panel/setting/getDefaultJsonConfig</code>	<code>/panel/api/setting/getDefaultJsonConfig</code>	GET
<code>/panel/setting/apiTokens</code>	<code>/panel/api/setting/apiTokens</code>	GET
<code>/panel/setting/apiTokens/create</code>	<code>/panel/api/setting/apiTokens/create</code>	POST
<code>/panel/setting/apiTokens/delete/:id</code>	<code>/panel/api/setting/apiTokens/delete/:id</code>	POST
<code>/panel/setting/apiTokens/setEnabled/:id</code>	<code>/panel/api/setting/apiTokens/setEnabled/:id</code>	POST
<code>/panel/xray/</code>	<code>/panel/api/xray/</code>	POST
<code>/panel/xray/update</code>	<code>/panel/api/xray/update</code>	POST
<code>/panel/xray/getDefaultJsonConfig</code>	<code>/panel/api/xray/getDefaultJsonConfig</code>	GET
<code>/panel/xray/getXrayResult</code>	<code>/panel/api/xray/getXrayResult</code>	GET
<code>/panel/xray/getOutboundsTraffic</code>	<code>/panel/api/xray/getOutboundsTraffic</code>	GET
<code>/panel/xray/resetOutboundsTraffic</code>	<code>/panel/api/xray/resetOutboundsTraffic</code>	POST
<code>/panel/xray/testOutbound</code>	<code>/panel/api/xray/testOutbound</code>	POST
<code>/panel/xray/warp/:action</code>	<code>/panel/api/xray/warp/:action</code>	POST
<code>/panel/xray/nord/:action</code>	<code>/panel/api/xray/nord/:action</code>	POST
<code>/panel/xray/outbound-subs</code> (dan <code>/outbound-subs/*</code>)	<code>/panel/api/xray/outbound-subs</code> (dan <code>/outbound-subs/*</code>)	GET/POST/DELETE

Sub-jalur, isi permintaan, dan format respons tidak berubah – hanya **prefiks** yang berubah.

Cara Memperbaiki Integrasi yang Ada

1. Temukan semua kemunculan `/panel/setting/` dan `/panel/xray/` dalam skrip/konfigurasi Anda.
2. Ganti prefiks: tambahkan `api/` tepat setelah `/panel/` (misalnya, `/panel/setting/all` → `/panel/api/setting/all`).
3. Isi permintaan, parameter, dan format respons tidak perlu diubah – hanya URL yang berubah.
4. Karena pengaturan dan konfigurasi Xray sekarang berada di bawah `/panel/api`, keduanya dapat (dan harus) diakses menggunakan token API `Authorization: Bearer <token>` yang sama dengan `/panel/api/inbounds/*` dan endpoint lainnya. Ingat middleware CSRF yang diaktifkan untuk seluruh grup `/panel/api`.

Contoh: membaca semua pengaturan melalui API. Sebelumnya ($\leq 3.2.x$):

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
-H "Authorization: Bearer <token>"
```

Sekarang (3.3.0) – tambahkan `api/` setelah `/panel/`:

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
-H "Authorization: Bearer <token>"
```

Demikian pula restart panel: `POST /panel/api/setting/restartPanel`. Jalur lama `/panel/setting/restartPanel` sekarang akan mengembalikan 404.

API Bertipe: Skema dan Dokumentasi (Swagger / OpenAPI)

Di 3.3.0 spesifikasi OpenAPI menjadi bertipe penuh. Sebelumnya respons bertipe dijelaskan dengan objek kosong `{}`; kini komponen dan skema (`components.schemas`) dibuat langsung dari model data. Berkat ini:

- Swagger UI menampilkan model data nyata, bukan placeholder kosong.
- Generator eksternal (`openapi-generator`, dll.) dapat membangun klien siap pakai dalam bahasa yang diperlukan berdasarkan spesifikasi.
- Setiap respons bertipe dilampiri `$ref` ke model tertentu dan disertai contoh respons.

Tempat melihat dokumentasi API:

- **Halaman Swagger bawaan.** Di menu panel – item **"Dokumentasi API"** (rute SPA `/panel/api-docs`). Di sini semua endpoint tercantum secara interaktif dengan deskripsi, isi permintaan, dan contoh respons.
- **Spesifikasi OpenAPI 3.0 mentah** tersedia di `/panel/api/openapi.json`. URL ini dapat langsung dimasukkan ke Postman, Insomnia, atau `openapi-generator`. Spesifikasi tertanam dalam biner pada saat build; saat panel beroperasi dengan `webBasePath` non-standar, kolom `servers` dalam spesifikasi secara otomatis disesuaikan dengan jalur dasar saat ini, agar tombol "Try it out" dan generator eksternal mengarah ke prefiks yang benar.

14. Bot Telegram

Panel 3X-UI memiliki bot Telegram bawaan yang dapat digunakan untuk menerima notifikasi tentang status server dan klien, serta mengelola klien tertentu langsung dari messenger. Bot bekerja menggunakan teknologi long polling (polling berkelanjutan ke Telegram), sehingga tidak memerlukan domain eksternal atau port terbuka – cukup akses keluar ke server Telegram.

Bot membedakan dua jenis pengguna:

- **Administrator** – pengguna yang Telegram User ID-nya dicantumkan dalam pengaturan bot (kolom «User ID administrator bot»). Memiliki akses ke semua fitur: statistik server, backup, manajemen klien, restart Xray.
- **Klien** – pengguna lain yang Telegram User ID-nya terhubung ke klien inbound tertentu (kolom `tgId` klien). Hanya dapat melihat informasi tentang langganannya sendiri.

Contoh: menghubungkan klien ke Telegram. Agar pengguna dapat menerima statistik langganannya, Telegram User ID numerik mereka dicatat di kolom `tgId` klien. Dalam pengaturan JSON klien, tampilannya seperti ini:

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

Setelah itu, pengguna dengan User ID `123456789` dapat meminta bot `/usage ivan` dan melihat statistiknya. ID yang sama dapat diatur oleh administrator melalui tombol « Установить пользователя Telegram» di kartu klien – tidak perlu mengedit JSON secara manual.

14.1. Mengaktifkan dan mengonfigurasi bot

Semua parameter bot diatur di panel pada bagian **Настройки** → **Telegram-бот**. Setelah mengubah pengaturan, cukup simpan – panel menerapkannya segera, restart panel tidak diperlukan. Jika flag aktivasi (`tgBotEnable`), token, User ID administrator, atau alamat server API diubah, panel secara otomatis menghentikan dan memulai ulang bot dengan parameter baru. Aturan lama yang mengharuskan restart panel setelah mengganti token tidak lagi berlaku.

Kolom (UI)	Kunci pengaturan	Nilai default	Deskripsi
Включить Telegram бота	<code>tgBotEnable</code>	<code>false</code>	Sakelar utama. Keterangan: «Доступ к функциям панели через Telegram-бота». Selama dinonaktifkan, bot tidak berjalan dan tugas notifikasi tidak dijadwalkan.
Telegram-токен	<code>tgBotToken</code>	(kosong)	Token bot. Keterangan: «Необходимо получить токен у менеджера ботов Telegram @botfather». Tanpa token yang terisi, bot tidak akan berjalan.

Kolom (UI)	Kunci pengaturan	Nilai default	Deskripsi
SOCKS-прокси	tgBotProxy	(kosong)	Proxy untuk koneksi ke Telegram. Keterangan: «Если для подключения к Telegram вам нужен прокси Socks5, настройте его параметры согласно руководству».
Telegram API Server	tgBotAPIServer	(kosong)	Server API Telegram alternatif. Keterangan: «Используемый API-сервер Telegram. Оставьте пустым, чтобы использовать сервер по умолчанию».
User ID администратора бота	tgBotChatId	(kosong)	Satu atau beberapa Telegram User ID administrator, dipisahkan koma. Keterangan: «Для получения User ID используйте @userinfobot или команду /id в боте».
Частота уведомлений для администраторов от бота	tgRunTime	@daily	Jadwal laporan berkala dalam format crontab. Keterangan: «Укажите интервал уведомлений в формате Crontab».
Резервное копирование базы данных	tgBotBackup	false	Keterangan: «Отправлять уведомление с файлом резервной копии базы данных». Melampirkan backup ke laporan berkala.
Уведомление о входе	tgBotLoginNotify	true	Keterangan: «Отображает имя пользователя, IP-адрес и время, когда кто-то пытается войти в вашу панель».
Порог нагрузки на ЦП для уведомления	tgCpu	80	Ambang batas penggunaan CPU dalam persen (validasi 0–100). Keterangan: «Уведомление администраторов в Telegram, если нагрузка на ЦП превышает этот порог (значение: %)». Jika nilainya 0, pemeriksaan CPU dinonaktifkan.
Язык Telegram-бота	—	—	Bahasa yang digunakan bot untuk membuat semua pesan.

Mendapatkan token melalui @BotFather

1. Buka percakapan dengan **@BotFather** di Telegram.
2. Kirimkan perintah `/newbot` dan ikuti instruksinya (nama bot dan `username` unik yang berakhiran `bot`).
3. BotFather akan memberikan token dengan format `123456789:AA...`. Salin ke kolom **Telegram-токен**.

Mendapatkan User ID administrator

User ID adalah pengenal numerik akun (bukan username). Ada dua cara untuk mengetahuinya:

- Kirim pesan ke bot **@userinfobot**.
- Jalankan bot yang sudah dikonfigurasi dan kirimkan perintah `/id` — bot akan mengembalikan ID Anda.

Masukkan angka yang diperoleh ke kolom **User ID администратора бота**. Untuk menunjuk beberapa administrator, cantumkan ID mereka dengan koma (misalnya, `1111111,2222222`). Setiap ID divalidasi sebagai bilangan bulat; nilai yang tidak valid akan menyebabkan error saat bot dijalankan.

Contoh: nilai kolom «User ID администратора бота». Satu administrator — cukup satu angka:

123456789

Dua administrator dengan koma (spasi tidak diperlukan):

123456789,987654321

Setiap nilai harus berupa bilangan bulat. Format seperti `@username` atau `123 456` (dengan spasi di dalam angka) tidak valid – bot tidak akan berjalan.

Proxy

Skema yang didukung: `socks5://`, `http://`, dan `https://`. Jika kolom proxy dikosongkan, bot mencoba menggunakan proxy umum panel (jika diatur dan skemanya didukung). URL dengan skema yang tidak didukung atau sintaks yang tidak valid akan diabaikan – bot terhubung secara langsung. Proxy berguna ketika akses langsung ke API Telegram dari server diblokir.

Notifikasi email (SMTP)

Selain Telegram, event yang sama dapat diterima melalui email. Saluran ini dikonfigurasi di bagian **Настройки** → **Email** pada tab **SMTP Settings**:

Kolom (UI)	Kunci pengaturan	Nilai default	Deskripsi
Enable Email Notifications	<code>smtpEnable</code>	<code>false</code>	Sakelar utama notifikasi email melalui SMTP.
SMTP Host	<code>smtpHost</code>	(kosong)	Host server SMTP (misalnya, <code>smtp.gmail.com</code>).
SMTP Port	<code>smtpPort</code>	<code>587</code>	Port server SMTP.
SMTP Username	<code>smtpUsername</code>	(kosong)	Nama pengguna untuk autentikasi SMTP. Digunakan juga sebagai alamat pengirim (From).
SMTP Password	<code>smtpPassword</code>	(kosong)	Kata sandi untuk autentikasi SMTP. Disimpan tersembunyi; jika kata sandi sudah diatur, kolom menampilkan indikator «dikonfigurasi» dan dapat dikosongkan untuk mempertahankan nilai saat ini.
Recipients	<code>smtpTo</code>	(kosong)	Daftar penerima dipisahkan koma (misalnya, <code>admin@example.com, ops@example.com</code>).
Encryption	<code>smtpEncryptionType</code>	<code>starttls</code>	Jenis enkripsi koneksi: <code>none</code> (tanpa enkripsi), <code>starttls</code> (STARTTLS), atau <code>tls</code> (TLS implisit).

Tombol **Send Test Email** mengirimkan email percobaan dan menampilkan hasilnya per tahap: **Connection** (koneksi), **Authentication** (autentikasi), dan **Send** (pengiriman). Jika ada yang salah, diagnostik menunjukkan di tahap mana error terjadi (misalnya, «Authentication failed – check username and password» atau «Server requires STARTTLS – change encryption type»), sehingga memudahkan penyesuaian parameter.

Pada tab kedua (**Notifications**) dipilih event mana yang akan dikirimkan melalui email – menggunakan grup kartu yang sama seperti untuk Telegram (lihat «Bus event dan pemilihan notifikasi» di bagian 14.5).

Server API Telegram

Secara default, bot menggunakan API Telegram resmi. Di kolom **Telegram API Server** dapat dicantumkan alamat server Bot API sendiri (telegram-bot-api). URL diperiksa keamanannya; alamat yang diblokir atau tidak valid akan diabaikan, dan server default akan digunakan.

14.2. Menu utama dan tombol

Menu dipanggil dengan perintah `/start`. Tombol-tombol adalah inline keyboard yang dilampirkan pada pesan; rangkaian tombol bergantung pada apakah Anda administrator atau klien.

Menu administrator

Tombol	Tindakan
 Отсортированный отчёт об использовании трафика	Mencantumkan semua klien yang diurutkan berdasarkan traffic, dengan penggunaan masing-masing; email «ekstra» tanpa data ditandai « Нет результатов».
 Состояние сервера	Ringkasan server (lihat bagian 14.5). Tombol « Обновить» memperbarui data.
Сбросить весь трафик	Mereset penghitung traffic semua klien. Meminta konfirmasi («Вы уверены? »), kemudian untuk setiap klien menampilkan « Успешно» atau « Неудача», diakhiri dengan « Сброс трафика завершён для всех клиентов».
 Бэкап БД	Mengirimkan file database dan <code>config.json</code> (lihat bagian 14.6).
 Лог банов	Mengirimkan file log alamat IP yang diblokir karena melebihi batas IP.
 Входящие подключения	Ringkasan semua inbound: Remark, port, traffic, jumlah klien, tanggal kedaluwarsa.
 Скоро конец	Daftar inbound dan klien yang traffic atau masa aktifnya akan segera habis (lihat bagian 14.5).
 Команды	Menampilkan bantuan perintah administrator.
 Онлайн	Jumlah dan daftar klien yang sedang online; mengetuk email membuka kartu klien. Tombol « Обновить».
 Все клиенты	Membuka pilihan inbound, kemudian daftar kliennya – untuk melihat/mengelola.
 Новый клиент	Menjalankan wizard penambahan klien (pilih inbound → draf → konfirmasi).
Настройки подписки / индивидуальные ссылки / QR-код	Pilih inbound dan klien untuk mendapatkan tautan langganan, tautan individual, atau kode QR.

Menu klien

Klien memiliki rangkaian tombol yang terbatas:

Tombol	Tindakan
Статистика клиента	Menampilkan data semua langganan yang terhubung dengan Telegram User ID klien.

Tombol	Tindakan
Команды	Menampilkan bantuan perintah klien.
Настройки подписки	Pilih klien Anda sendiri → tautan langganan.
Индивидуальные ссылки	Pilih klien Anda sendiri → tautan individual.
QR-код	Pilih klien Anda sendiri → kode QR.

Jika pengguna tidak memiliki klien yang terhubung dengan Telegram User ID-nya, bot akan menjawab: «
Ваша конфигурация не найдена! Пожалуйста, попросите администратора использовать ваш Telegram
User ID в конфигурации. Ваш User ID: ...». ID ini perlu diberikan ke administrator agar dimasukkan ke kolom
klien.

14.3. Perintah bot

Bot memiliki empat perintah yang terdaftar dan terlihat di menu «/» Telegram:

Perintah	Deskripsi (dari menu)	Akses	Yang dilakukan
/start	Показать главное меню	semua	Sambutan; untuk administrator juga menampilkan « Добро пожаловать в бота управления !» dan menu utama.
/help	Справка по боту	semua	Menampilkan sambutan umum dan ajakan memilih item menu.
/status	Проверить статус бота	semua	Menjawab « Бот функционирует нормально».
/id	Показать ваш Telegram ID	semua	Mengembalikan « Ваш User ID: ... ». Berguna untuk mengetahui User ID Anda sendiri.

Selain perintah yang terdaftar, ada tiga perintah argumen lagi yang diproses (tidak ditampilkan di menu «/», tetapi berfungsi):

- **/usage [Email]** — mencari klien berdasarkan email.
 - Untuk **administrator** menampilkan kartu klien lengkap (dengan tombol manajemen).
 - Untuk **klien** menampilkan hanya langganannya sendiri dengan email yang ditentukan (berdasarkan tautan Telegram User ID). Tanpa argumen, bot meminta email: « Пожалуйста, укажите email для поиска».
- **/inbound [nama koneksi]** — hanya untuk administrator. Mencari inbound berdasarkan Remark dan menampilkan parameter-nya dengan statistik semua klien. Tanpa argumen (atau untuk klien) — « Неизвестная команда».
- **/restart** — hanya untuk administrator. Memulai ulang Xray Core. Kemungkinan jawaban: « Ядро Xray успешно перезапущено», « Xray Core не запущен» (jika inti tidak berjalan), « Ошибка при перезапуске Xray-core. <Ошибка>». Argumen apa pun setelah **/restart** akan menghasilkan pesan perintah tidak dikenal dengan petunjuk **/restart**.

Di obrolan grup, perintah dengan format **/perintah@botusername** hanya diproses jika username cocok dengan nama bot saat ini.

Bantuan administrator (tombol «Команды»):

-  Для перезапуска Xray Core: /restart
-  Для поиска клиента по email: /usage [Email]
-  Для поиска входящих подключений (со статистикой клиентов): /inbound [имя подключения]
-  Ваш Telegram User ID: /id

Bantuan klien:

-  Для просмотра информации о вашей подписке: /usage [Email]
-  Ваш Telegram User ID: /id

14.4. Manajemen klien (hanya administrator)

Setelah membuka kartu klien (melalui «Все клиенты», «Онлайн», «Скоро конец», atau `/usage`), administrator melihat informasi klien (email, inbound yang terhubung, status «Aktif», status koneksi, tanggal kedaluwarsa, penggunaan traffic) dan tombol inline untuk manajemen:

Tombol	Fungsi
 Обновить	Memuat ulang kartu klien.
 Сбросить трафик	Mereset penghitung traffic klien. Memerlukan konfirmasi «  Подтвердить сброс трафика?».
 Лимит трафика	Menetapkan batas traffic. Nilai siap pakai:  Безлимит (0), 1/5/10/20/30/40/50/60/80/100/150/200 GB atau «  Своё» – input angka via keyboard numerik bawaan (tombol 0–9, «  » – reset ke 0, «  » – hapus digit terakhir, «  Подтвердить: N»). Nilai diatur dalam gigabyte.
 Изменить дату окончания	Pilihan siap pakai:  Безлимит, «  Своё», tambah 7/10/14/20 hari, 1/3/6/12 bulan. Angka positif memperpanjang masa aktif (menambah hari ke tanggal kedaluwarsa saat ini atau ke «sekarang» jika sudah kedaluwarsa); 0 menghapus batas waktu.
 Лог IP	Menampilkan alamat IP klien yang tercatat (dengan cap waktu jika ada). Dari log tersedia «  Обновить» dan «  Очистить IP» (dengan konfirmasi «  Подтвердить очистку IP?»).
 Лимит IP	Batas koneksi IP bersamaan. Pilihan:  Безлимит (0), 1–10 atau «  Своё» (keyboard numerik).
 Установить пользователя Telegram	 Menampilkan Telegram User ID klien yang saat ini terhubung; memungkinkan penghapusan tautan («  Удалить пользователя Telegram» dengan konfirmasi). Tautan pengguna baru dilakukan melalui pemilihan kontak Telegram sistem.
 Вкл./Выкл.	 Mengaktifkan atau menonaktifkan klien. Memerlukan konfirmasi «  Подтвердить вкл/выкл пользователя?».

Semua operasi yang mengubah konfigurasi (batas traffic/IP, tanggal kedaluwarsa, tautan/pemutusan pengguna Telegram, aktif/nonaktif), jika perlu, menandai Xray untuk restart agar perubahan berlaku. Setelah operasi berhasil, bot menampilkan konfirmasi berupa « : ...» dan menampilkan ulang kartu klien.

Input angka apa pun dalam wizard dibatasi pada nilai < 999999.

14.5. Notifikasi dan laporan

Notifikasi dikirimkan ke semua administrator (semua User ID dari `tgBotChatId`).

Bus event dan pemilihan notifikasi

Notifikasi dibangun di atas bus event tunggal, dengan dua saluran pengiriman — **Telegram** dan **email (SMTP)**. Untuk setiap saluran, dipilih secara terpisah event mana yang akan dinotifikasikan. Di **Настройки** → **Telegram** ini dilakukan pada tab **Notifications**, di **Настройк** → **Email** — pada tab dengan nama yang sama.

Event dikelompokkan dalam kartu; setiap grup memiliki sakelar utama dengan penghitung event yang aktif (n/total) dan status peralihan ketika hanya sebagian yang dipilih. Grup yang tersedia:

- **Outbound** — «Down» (`outbound.down`) dan «Up» (`outbound.up`): outbound turun dan pulih kembali.
- **Xray Core** — «Crash» (`xray.crash`): inti Xray berhenti secara tidak terduga.
- **Nodes** — «Down» (`node.down`) dan «Up» (`node.up`): node menjadi tidak tersedia atau pulih kembali.
- **System** — «CPU high (%)» (`cpu.high`) dan «Memory high (%)» (`memory.high`): penggunaan CPU dan RAM yang tinggi. Kedua event ini memiliki kolom inline ambang batas dalam persen di sebelahnya.
- **Security** — «Login attempt» (`login.attempt`): percobaan masuk ke panel.

Rangkaian event yang diaktifkan disimpan secara terpisah: untuk Telegram — di `tgEnabledEvents`, untuk Email — di `smtpEnabledEvents`. Secara default di kedua saluran, «Login attempt» dan «CPU high» diaktifkan (nilai `login.attempt,cpu.high`).

Notifikasi login ke panel

Dikendalikan oleh kotak centang **Уведомление о входе** (`tgBotLoginNotify`, diaktifkan secara default). Setiap percobaan masuk ke panel web akan mengirimkan pesan ke administrator:

- Jika berhasil: « Успешный вход в панель.» + host, nama pengguna, IP, waktu.
- Jika gagal: « Ошибка входа в панель.» + host, **alasan** (misalnya, «Ошибка 2FA» jika faktor kedua salah), nama pengguna, IP, waktu.

Melebihi batas beban CPU dan memori

Setiap menit, panel memeriksa penggunaan CPU dan RAM. Jika ambang batas `tgCpu` > 0 dan rata-rata beban CPU per menit melampaui nilai tersebut, administrator akan menerima pesan: « Загрузка процессора составляет N%, что превышает пороговое значение M%». Penggunaan RAM juga diperiksa terhadap ambang `tgMemory` (default 80%) — event «Memory high (%)».

Kedua ambang batas diatur melalui kolom inline di sebelah event «CPU high (%)» dan «Memory high (%)» dalam grup **System** pada tab Notifications (lihat «Bus event dan pemilihan notifikasi» di atas). Untuk saluran Email, kunci terpisah `smtpCpu` dan `smtpMemory` berlaku. Jika nilai ambang batas adalah 0, pemeriksaan yang bersangkutan dinonaktifkan.

Laporan berkala (terjadwal)

Dijadwalkan berdasarkan ekspresi cron dari kolom **Частота уведомлений** (`tgRunTime` , default `@daily`). Jika nilainya kosong atau tidak valid, `@daily` digunakan. Laporan mencakup:

Pembuat jadwal

Kolom **Частота уведомлений для администраторов от бота** tidak diisi dengan mengetik string secara manual, melainkan melalui pembuat jadwal. Pertama, pilih mode dari daftar dropdown:

- **@every** – ulangi dengan interval – muncul kolom angka dan pilihan satuan (**Секунды / Минуты / Часы**); hasilnya disusun menjadi ekspresi seperti `@every 6h` .
- **@hourly** – setiap jam, **@daily** – setiap hari pukul 00:00, **@weekly** – setiap minggu, **@monthly** – setiap bulan – preset siap pakai yang disimpan sebagai makro yang sesuai (`@hourly` , `@daily` , `@weekly` , `@monthly`).
- **Произвольный (crontab)** – kolom untuk ekspresi crontab khusus. Penjadwal panel bekerja dengan detik yang diaktifkan, sehingga ekspresi khusus terdiri dari **6 kolom**: detik, menit, jam, hari dalam bulan, bulan, hari dalam minggu (misalnya, `0 30 8 * * *` – setiap hari pukul 08:30:00). Saat beralih ke mode ini, kolom diisi dengan ekspresi crontab setara dari pilihan saat ini sebagai titik awal.

Contoh: nilai kolom «Частота уведомлений» (`tgRunTime`). Didukung singkatan siap pakai dan format crontab lengkap:

Nilai	Kapan dipicu
<code>@daily</code>	sekali sehari pada tengah malam (nilai default)
<code>@hourly</code>	setiap jam
<code>@every 6h</code>	setiap 6 jam
<code>0 9 * * *</code>	setiap hari pukul 09:00
<code>0 9 * * 1</code>	setiap Senin pukul 09:00
<code>0 */12 * * *</code>	setiap 12 jam (pukul 00:00 dan 12:00)

Urutan kolom dalam crontab: menit, jam, hari dalam bulan, bulan, hari dalam minggu.

1. Baris «**[?] [?] Запланированные отчёты: »** dan tanggal/waktu saat ini.
2. **Status server** (lihat di bawah).
3. Blok «Скоро конец» berdasarkan inbound dan klien.
4. Notifikasi personal ke klien yang memiliki Telegram User ID terhubung – setiap klien non-admin menerima daftar langganannya yang traffic atau masa aktifnya akan segera habis (dengan mempertimbangkan yang dinonaktifkan).
5. Jika **Резервное копирование базы данных** (`tgBotBackup`) diaktifkan – backup database dikirimkan ke administrator.

Status server mencakup: host, versi 3X-UI dan Xray, IPv4/IPv6, uptime (dalam hari), rata-rata beban (Load1/2/3), RAM (saat ini/total), jumlah klien online, penghitung koneksi TCP/UDP, total traffic jaringan (↑/↓) dan status Xray.

«Скоро конец» menampilkan:

- per inbound: jumlah yang dinonaktifkan dan jumlah yang «akan segera habis», kemudian daftar inbound tersebut (Remark, port, traffic, tanggal kedaluwarsa);
- per klien: hal yang sama, ditambah kartu klien dan tombol dengan email mereka (mengetuk membuka kartu klien).

Ambang batas «akan segera habis» diambil dari pengaturan umum panel: margin traffic (dalam GB) dan margin waktu (dalam hari). Inbound/klien dianggap «akan habis» jika sisa traffic hingga batas lebih kecil dari ambang ATAU sisa hari hingga tanggal kedaluwarsa lebih kecil dari ambang.

14.6. Backup dan log

- **Backup database** (tombol  «Бэкап БД» atau kotak centang di laporan berkala): bot mengirimkan waktu backup, file database (`x-ui.db`, atau `x-ui.dump` untuk PostgreSQL) dan file konfigurasi Xray `config.json`.

Nama file backup yang dikirimkan bot dibentuk berdasarkan alamat server: menggunakan nilai **webDomain**, dan jika tidak diatur – IP publik server. Ini membantu mengidentifikasi dari server mana file tersebut berasal ketika backup dikumpulkan dari beberapa panel. Jika alamat tidak dapat ditentukan, nama generik akan digunakan.

- **Log ban** (tombol  «Лог банов»): mengirimkan file log saat ini dan sebelumnya dari alamat IP yang diblokir karena melebihi batas IP. File kosong tidak dikirimkan.

14.7. Fitur operasional

- **Pesan panjang** dipecah menjadi beberapa bagian (ambang ~2000 karakter), inline keyboard dilampirkan pada bagian terakhir.
- **Paralelisme**: perintah dan penekanan tombol diproses secara bersamaan (pool hingga 10 handler simultan).
- **Keandalan pengiriman**: saat terjadi error koneksi, pesan dikirim ulang dengan jeda eksponensial (1d/2d/4d, hingga 3 percobaan).
- **Caching**: data «Status server» di-cache agar penekanan «Обновить» yang sering tidak membebani sistem.
- **Restart bot**: saat menyimpan pengaturan yang mempengaruhi bot (flag aktivasi, token, User ID administrator, atau alamat server API), panel secara otomatis menghentikan siklus polling sebelumnya dan memulai yang baru dengan parameter terbaru – restart panel tidak diperlukan. Hanya satu instance penerima pembaruan yang berjalan sekaligus.

15. Basis Geo (geoip / geosite dan kustom)

Basis geo adalah file biner `.dat` yang digunakan Xray-core untuk merutekan dan memfilter lalu lintas berdasarkan kepemilikan negara (rentang IP) atau kategori domain. Panel dapat mengunduh dan memperbarui baik set file geo standar maupun sumber kustom yang ditentukan oleh pengguna melalui URL. Semua file disimpan di direktori `bin` yang berdekatan dengan biner Xray (jalur default `bin`, dapat diganti dengan variabel lingkungan `XUI_BIN_FOLDER`).

15.1. Apa itu geoip.dat dan geosite.dat

- **geoip.dat** – basis pemetaan "alamat IP → kode negara/wilayah". Digunakan dalam aturan perutean dalam bentuk `geoip:<kode>`, misalnya `geoip:ru`, `geoip:cn`, serta untuk label khusus `geoip:private` (jaringan privat/lokal). Secara semantis ini menjawab pertanyaan "di negara mana IP ini berada".
- **geosite.dat** – basis pemetaan "domain → kategori/daftar". Digunakan dalam bentuk `geosite:<kategori>`, misalnya `geosite:category-ads-all` (domain iklan), `geosite:google`, `geosite:ru`. Secara semantis ini adalah daftar domain yang dikelompokkan.

File-file ini diperlukan untuk membangun aturan seperti "semua lalu lintas ke IP/domain Rusia – langsung, sisanya – melalui outbound" dan sejenisnya. Aturan itu sendiri dikonfigurasi di bagian perutean Xray; basis geo hanya menyediakan data untuk aturan tersebut. Tanpa file geo yang mutakhir, aturan yang merujuk `geoip: / geosite:` tidak akan berfungsi atau akan mengandalkan daftar yang sudah usang.

Contoh: aturan "domain dan IP Rusia – langsung". Aturan seperti ini di bagian perutean mengarahkan semua lalu lintas ke sumber daya Rusia ke outbound dengan tag `direct`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. File geo standar dan pembaruannya

Panel berisi "daftar putih" (allowlist) tetap dari enam file standar dengan sumber unduhan yang telah dikodekan secara permanen. Pembaruan dilakukan melalui `POST /panel/api/server/updateGeofile/:fileName` (atau tanpa nama file – untuk memperbarui semua sekaligus).

Contoh: memperbarui satu file dan semua sekaligus melalui API. Memperbarui hanya `geoip_RU.dat`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/
geoip_RU.dat' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Memperbarui semua enam file standar dalam satu permintaan (nama file tidak ditentukan):

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Respons sukses:

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

Nama file	Sumber (repositories rilis)
geoip.dat	github.com/Loyalsoldier/v2ray-rules-dat (geoip.dat)
geosite.dat	github.com/Loyalsoldier/v2ray-rules-dat (geosite.dat)
geoip_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geoip.dat)
geosite_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geosite.dat)
geoip_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geoip.dat)
geosite_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geosite.dat)

Kekhususan pembaruan file standar:

- **Tombol pembaruan satu file.** Sebelum mengunduh, ditampilkan konfirmasi: "Do you really want to update the geofile? This will update the #filename# file." Jika berhasil, muncul notifikasi "Geofile updated successfully".
- **Tombol "Update all"** mengunduh semua enam file. Konfirmasi: "This will update all geofiles."
- **Unduhan bersyarat.** Jika file lokal sudah ada, header `If-Modified-Since` dengan waktu modifikasi file disertakan dalam permintaan. Respons server `304 Not Modified` berarti file tidak berubah – file tidak diunduh ulang, hanya cap waktu file yang diperbarui.
- **Keamanan nama file.** Hanya nama-nama dari allowlist yang diterima; nama diperiksa untuk memastikan tidak mengandung `..`, pemisah jalur `/` dan `\`, jalur absolut, dan harus sesuai pola `^[a-zA-Z0-9._-]+\ .dat$`. Nama apa pun di luar daftar ditolak dengan kesalahan "Invalid geofile name".
- **Restart Xray.** Setelah mengunduh file geo, Xray-core di-restart agar membaca ulang basis yang diperbarui. Jika restart gagal, pesan kesalahan yang sesuai ditambahkan.

Memperbarui basis geo dari baris perintah (x-ui)

Basis geo juga dapat diperbarui tanpa panel – melalui menu interaktif `x-ui` (item pembaruan file geo) atau dengan perintah non-interaktif `x-ui update-all-geofiles`. Untuk setiap file dalam set (geoip/geosite, termasuk set IR dan RU) ditampilkan status terpisah: "diperbarui", "sudah terkini", atau "gagal mengunduh". Jika pengunduhan gagal, pesan sukses palsu tidak dicetak. Restart Xray (dan pemutusan koneksi aktif) hanya terjadi jika setidaknya satu file benar-benar telah diperbarui; jika tidak ada file yang berubah (semua mengembalikan `304 Not Modified`), panel dan Xray tidak di-restart.

15.3. Pembaruan otomatis data geo melalui Xray (Geodata Auto-Update)

Sumber `.dat` tambahan dari URL sembarang tidak ditambahkan melalui fasilitas panel, melainkan melalui seksi native `geodata` Xray-core. Seksi yang bersangkutan ditempatkan di jendela modal pembaruan Xray

(Dashboard → pembaruan Xray, `xrayUpdates`) – ini adalah tab "Geodata Auto-Update". Panel di sini hanya mengedit kunci `geodata` dalam template konfigurasi Xray; pengunduhan, verifikasi, dan pemuatan ulang file secara langsung ditangani oleh inti Xray itu sendiri.

Di bagian atas seksi ditampilkan petunjuk: "Xray downloads these files on schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it."

Kolom dan field seksi

- **Schedule (cron)** – string cron 5 field; nilai default `0 4 * * *` (setiap hari pukul 04:00). Saat menyimpan, diperiksa bahwa string berisi tepat 5 field, jika tidak ditampilkan kesalahan "Cron must contain 5 fields, e.g. `0 4 * * *`".
- **Download through outbound (optional)** – daftar dropdown dengan tag outbound yang tersedia (plus outbound langganan), yang melaluinya Xray akan mengunduh file; outbound dengan protokol `blackhole` tidak masuk ke daftar. Field ini boleh dikosongkan – dalam hal ini koneksi langsung digunakan. Pilihan ini tidak bergantung pada outbound untuk permintaan panel sendiri (lihat §11): pembaruan otomatis geodata memiliki outbound tersendiri untuk pengunduhan.
- **Daftar file** – setiap baris menentukan pasangan "URL + File name". URL harus dimulai dengan `https://` (jika tidak: "Each file needs an HTTPS URL."). Nama file ditentukan secara sederhana, tanpa jalur dan pemisah – hanya karakter `^[A-Za-z0-9._-]+$` (jika tidak: "File name must be simple, e.g. `geosite_custom.dat` (no paths)."). Saat memasukkan URL, panel mencoba mengisi nama file secara otomatis dari segmen jalur terakhir. Tombol "Add file" menambah baris, tombol keranjang menghapusnya.

Jika daftar kosong, ditampilkan petunjuk: "No files configured. Reference files in routing rules as `ext:geosite_custom.dat:category`."

Menyimpan

Tombol "Save & Restart Xray" menampilkan konfirmasi "Save geodata settings?" dengan penjelasan "This updates the Xray config template and restarts Xray." Setelah menyimpan, kunci `geodata` ditulis ke template konfigurasi (`POST /panel/api/xray/update`) dan Xray di-restart (`POST /panel/api/server/restartXrayService`). Jika daftar file kosong, kunci `geodata` dihapus dari template.

Hal-hal penting:

- **File harus sudah ada di `bin`**. Xray hanya memperbarui file `.dat` yang sudah ada di folder `bin` saat startup. Oleh karena itu, file kustom baru pertama-tama ditempatkan ke `bin` secara manual (atau setidaknya dibuat di sana versi kosong/usang dengan nama yang diinginkan), dan baru setelah itu Xray mulai memeliharanya agar tetap terkini sesuai jadwal.
- **Pemuatan ulang langsung (hot-reload)**. Setelah pengunduhan terjadwal, Xray membaca ulang basis yang diperbarui tanpa me-restart proses secara penuh.
- **Kompatibilitas**. File geo yang sebelumnya diunduh (baik standar maupun kustom) terus bekerja dalam aturan perutean dengan sintaks `ext:` tanpa perubahan.

Jika daftar kosong, ditampilkan petunjuk: "No custom geo sources yet – click Add to create one".

Kolom tabel dan field sumber

Field (UI)	JSON	Nilai default	Keterangan
Type	<code>type</code>	– (wajib)	Jenis sumber daya: hanya <code>geosite</code> atau <code>geoip</code> . Menentukan nama file hasil akhir.
Alias	<code>alias</code>	– (wajib)	Pengenal singkat sumber. Nama file dibentuk dari alias dan type.
URL	<code>url</code>	– (wajib)	Tautan langsung ke file <code>.dat</code> (http/https).
Enabled	–	–	Tanda aktif sumber dalam daftar.
Last updated	<code>lastUpdatedAt</code>	<code>0</code>	Waktu pembaruan terakhir yang berhasil (waktu Unix; <code>0</code> – belum pernah diperbarui).
Routing (ext:...)	–	–	String siap pakai untuk aturan perutean: <code>ext:<file.dat>:tag</code> .
Actions	–	–	Tombol "Edit", "Delete", "Update now".

Selain itu, field layanan berikut disimpan dalam basis data: `localPath` (jalur aktual ke file di direktori `bin`), `lastModified` (nilai header `Last-Modified` dari server, digunakan untuk unduhan bersyarat), `createdAt` dan `updatedAt`.

Penamaan file

Nama file hasil akhir dibentuk secara otomatis dari type dan alias:

- type `geoip` → `geoip_<alias>.dat` ;
- type `geosite` → `geosite_<alias>.dat` .

Misalnya, sumber dengan type `geosite` dan alias `myads` akan membuat file `geosite_myads.dat` .

Contoh: menambahkan sumber melalui API. Menambahkan daftar domain iklan kustom sebagai sumber daya `geosite` dengan alias `myads` :

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
-H 'Cookie: 3x-ui=<session-cookie>' \
-H 'Content-Type: application/json' \
-d '{
  "type": "geosite",
  "alias": "myads",
  "url": "https://example.com/lists/myads.dat"
}'
```

Panel akan mengunduh file ke direktori `bin` sebagai `geosite_myads.dat` , menyimpan catatan, dan me-restart Xray.

Tombol dan tindakan

- **Add** – membuka formulir "Add custom geo". Tombol simpan – "Save". API: `POST /add` .

- **Edit** – formulir "Edit custom geo". API: `POST /update/:id`. Saat type atau alias berubah, file lama dihapus dan yang baru diunduh ulang.
- **Delete** – konfirmasi "Delete this custom geo source?". Menghapus catatan dari basis data dan file `.dat` itu sendiri. API: `POST /delete/:id`. Jika berhasil: "Рользовательский гео-файл «» удалён" – file geo kustom "" telah dihapus.
- **Update now** – mengunduh ulang sumber tertentu dan memperbarui cap waktu. API: `POST /download/:id`. Jika berhasil: "Geofile «» diperbarui".
- **Update all** – memperbarui semua sumber kustom sekaligus. API: `POST /update-all`. Jika semua berhasil: "All custom geo sources updated". Jika setidaknya satu sumber gagal diperbarui, operasi dianggap sebagian gagal dengan pesan "One or more custom geo sources failed to update", dan dalam respons dicantumkan sumber yang berhasil dan yang gagal.

Setelah tindakan apa pun (penambahan, pengeditan, penghapusan, pembaruan, pembaruan semua jika ada yang berhasil) Xray-core di-restart.

Langkah demi langkah: menambahkan sumber

1. Klik "Add".
2. Di field "Type", pilih `geosite` atau `geoip`.
3. Di field "Alias", masukkan pengenalan (hanya huruf latin kecil, angka, `-` dan `_`; placeholder petunjuk: `a-z 0-9 _ -`).
4. Di field "URL", tentukan tautan langsung ke file `.dat` (harus dimulai dengan `http://` atau `https://`).
5. Klik "Save". Panel akan segera mengunduh file ke direktori `bin`, menyimpan catatan, dan me-restart Xray.

15.4. Validasi dan batasan

Saat membuat dan mengubah sumber, pemeriksaan ketat dilakukan. Pesan kesalahan:

Kondisi	Pesan (ID)	Pesan (EN)
Type bukan <code>geosite / geoip</code>	Type harus geosite atau geoip	<i>Type must be geosite or geoip</i>
Alias kosong	Tentukan alias	<i>Alias is required</i>
Karakter tidak diizinkan dalam alias (bukan <code>^[a-z0-9_-]+\$</code>)	Alias mengandung karakter yang tidak diizinkan	<i>Alias must match allowed characters</i>
Alias dicadangkan	Alias ini dicadangkan	<i>This alias is reserved</i>
URL kosong	Tentukan URL	<i>URL is required</i>
URL tidak dapat diurai	URL tidak valid	<i>URL is invalid</i>
Skema bukan http/https	URL harus menggunakan http atau https	<i>URL must use http or https</i>
Host kosong/tidak valid, atau diblokir proteksi SSRF	Host URL tidak valid	<i>URL host is invalid</i>
Duplikat "type + alias"	Alias ini sudah digunakan untuk type ini	<i>This alias is already used for this type</i>
Sumber tidak ditemukan	Sumber tidak ditemukan	<i>Custom geo source not found</i>

Kondisi	Pesan (ID)	Pesan (EN)
Kesalahan pengunduhan	Gagal mengunduh	<i>Download failed</i>

Petunjuk dalam formulir (validasi di sisi klien): "Alias may only contain lowercase letters, digits, - and _" dan "URL must start with http:// or https://".

Batasan teknis tambahan:

- **Alias yang dicadangkan.** Alias yang berkonflik dengan file standar tidak dapat digunakan. Yang dicadangkan (perbandingan tidak peka huruf besar-kecil, tanda hubung disamakan dengan garis bawah): `geoip`, `geosite`, `geoip_ir`, `geosite_ir`, `geoip_ru`, `geosite_ru`. Misalnya, `geosite-ru` akan ditolak sebagai `geosite_ru`.
- **Proteksi SSRF.** Host URL di-resolve ke IP, dan jika mengarah ke alamat privat/internal, pengunduhan diblokir (pengguna melihat "Host URL tidak valid"). Ini mencegah panel digunakan untuk mengakses layanan internal.
- **Proteksi path traversal.** Jalur akhir file harus berada di dalam direktori `bin` (dengan resolusi symlink); upaya keluar dari batasnya ditolak.
- **Ukuran file minimum.** File yang diunduh dianggap valid hanya jika tidak kurang dari 64 byte; file yang terlalu kecil ditolak dengan kesalahan unduhan.
- **Proxy dan unduhan bersyarat.** Jika proxy dikonfigurasi dalam pengaturan panel, pengunduhan dilakukan melaluinya; dalam kasus lain, koneksi langsung dengan transport aman SSRF digunakan. Seperti untuk file standar, `If-Modified-Since / 304 Not Modified` diterapkan (file yang tidak berubah tidak diunduh ulang). Batas waktu pengunduhan – 10 menit, pemeriksaan ketersediaan URL (HEAD, jika perlu – GET parsial) – 12 detik.

15.5. Pemeriksaan otomatis saat panel dijalankan

Saat startup, panel memeriksa semua sumber kustom dan untuk masing-masing memeriksa keberadaan dan integritas file lokal (file tidak ada, merupakan direktori, atau berukuran kurang dari 64 byte). Jika file tidak ada atau rusak, pemeriksaan sumber dilakukan dan pengunduhan ulang dicoba. Ini memastikan bahwa setelah instalasi ulang atau kehilangan direktori `bin`, file geo kustom akan dipulihkan secara otomatis.

15.6. Penggunaan basis geo dalam aturan perutean

Dalam aturan perutean Xray, basis geo digunakan di field seperti `domain / ip` melalui prefix:

- **geoip:** untuk basis IP – `geoip:<kode>`. Contoh: `geoip:ru`, `geoip:cn`, `geoip:private`. Diambil dari `geoip.dat` (atau `geoip_RU.dat` dan sejenisnya, jika aturan menunjuk ke file tertentu).
- **geosite:** untuk basis domain – `geosite:<kategori>`. Contoh: `geosite:category-ads-all`, `geosite:google`, `geosite:ru`. Diambil dari `geosite.dat`.

Contoh: pemblokiran iklan melalui geosite. Aturan yang mengirimkan semua domain iklan ke "lubang hitam" (diasumsikan outbound dengan tag `blocked` dan protokol `blackhole`):

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

Untuk file **kustom**, digunakan sintaks file eksternal `ext: .` Petunjuk di UI: "In routing rules use the value column as `ext:file.dat:tag` (replace tag)." Formatnya:

```
ext:<nama_file.dat>:<tag>
```

di mana `<nama_file.dat>` adalah `geoip-<alias>.dat` atau `geosite-<alias>.dat`, dan `<tag>` adalah daftar/kategori tertentu di dalam file. Panel di kolom "Routing (ext:...)" menampilkan template siap pakai seperti `ext:geosite_myads.dat:tag` – tinggal ganti `tag` dengan tag yang diinginkan. Nama file tersebut ditentukan di bagian "Geodata Auto-Update" (lihat §15.3) di field "File name" – misalnya `geosite_custom.dat`; dirujuk dalam aturan sebagai `ext:geosite_custom.dat:category`.

Contoh: aturan berbasis file kustom. Jika sumber dengan type `geosite` dan alias `myads` ditambahkan, dan di dalam file `.dat` daftarnya diberi tag `ads`, aturan peruteannya terlihat seperti ini:

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

Untuk sumber IP (type `geoip`, alias `mycorp`, tag `office`) fieldnya adalah `"ip": ["ext:geoip_mycorp.dat:office"]`.

16. Operasional: backup, log, pembaruan, CLI

Bagian ini mencakup pemeliharaan panel sehari-hari: pembuatan dan pemulihan cadangan database, melihat log panel dan Xray, me-restart dan menghentikan layanan, memperbarui Xray dan panel itu sendiri, tugas berkala (cron), serta menghapus panel. Sebagian operasi dilakukan dari antarmuka web (tab di halaman «Dashboard» dan «Pengaturan Panel»), sebagian lagi dari menu konsol `x-ui` di server.

16.1. Pencadangan dan pemulihan database

Semua data panel (inbound, klien, grup, node, pengaturan) disimpan dalam satu database. Manajemen backup tersedia di halaman «Dashboard» pada tab «Cadangan», dengan judul blok «Backup dan Pemulihan».

Panel mendukung dua mesin DB, dan perilaku backup bergantung pada pilihan tersebut:

- **SQLite** (pilihan default) – data disimpan dalam file `x-ui.db`.
- **PostgreSQL** – jika panel dikonfigurasi untuk PostgreSQL, blok menampilkan petunjuk:

«Panel ini berjalan di PostgreSQL. «Cadangan» mengunduh arsip `pg_dump` (.dump), sedangkan «Pemulihan» mengunggahnya kembali melalui `pg_restore`. Alat klien PostgreSQL (`pg_dump` dan `pg_restore`) harus terinstal di server.»

Ekspor (membuat cadangan)

Tombol «Ekspor database» (Ingg. Back Up) mengunduh file cadangan ke perangkat Anda.

Mesin DB	Nama file	Yang terjadi di server
SQLite	<code>{host}_YYYY-MM-DD_HHMMSS.db</code>	Checkpoint WAL dijalankan terlebih dahulu agar file berisi catatan terbaru, kemudian file dibaca seluruhnya dan dikirim untuk diunduh
PostgreSQL	<code>{host}_YYYY-MM-DD_HHMMSS.dump</code>	<code>pg_dump</code> dijalankan, arsip dikirim untuk diunduh

Mulai versi 3.4.2, nama file cadangan yang diunduh menyertakan alamat tempat Anda membuka panel dan **tanggal-waktu**: `{host}_YYYY-MM-DD_HHMMSS` dengan ekstensi `.db` (SQLite) atau `.dump` (PostgreSQL) – misalnya `panel.example.com_2026-06-27_000000.db`. Dengan begitu file dikelompokkan berdasarkan server dan diurutkan berdasarkan waktu, dan beberapa salinan dalam satu hari tidak saling menimpa. Nama yang sama digunakan pada cadangan yang dikirim bot Telegram (`host` diambil dari domain/IP panel, fallback-nya – `x-ui`).

Petunjuk di antarmuka:

- **SQLite**: «Klik untuk mengunduh file .db yang berisi cadangan database Anda saat ini ke perangkat Anda.»
- **PostgreSQL**: «Klik untuk mengunduh dump PostgreSQL (.dump) dari database Anda saat ini ke perangkat Anda.»

Secara teknis, ekspor adalah permintaan `GET /panel/api/server/getDb`. Nama lampiran dibentuk oleh server (`Content-Disposition`) tergantung mesin yang digunakan.

Nama file cadangan dibentuk berdasarkan alamat server, bukan nama tetap `x-ui.db` / `x-ui.dump`. Saat diunduh dari browser, nama diambil dari alamat panel di bilah alamat (host permintaan), atau dari domain web yang dikonfigurasi, atau dari IP publik server (IPv4 dahulu, lalu IPv6), dengan fallback ke `x-ui`. Dengan begitu, backup dari server yang berbeda mudah dibedakan. Ekstensi tetap `.db` untuk SQLite dan `.dump` untuk PostgreSQL; backup melalui Telegram juga diberi nama berdasarkan domain/IP yang sama.

Contoh: mengunduh backup melalui API. Ekspor yang sama bisa diperoleh dengan permintaan dari konsol – misalnya untuk skrip pencadangan otomatis. Diperlukan sesi terautentikasi (cookie login):

```
# 1) Login dan simpan cookie sesi
curl -s -c cookies.txt \
  -d 'username=admin&password=admin' \
  https://panel.example.com:2053/panel/login

# 2) Unduh file database (nama ditentukan server: x-ui.db atau x-ui.dump)
curl -s -b cookies.txt -OJ \
  https://panel.example.com:2053/panel/api/server/getDb
```

Jika panel dibuka melalui base path (Web Base Path), tambahkan ke URL: `...:2053/<base_path>/panel/api/server/getDb`.

Impor (pemulihan)

Tombol **«Impor database»** (Ingg. `Restore`) membuka pemilih file dan mengunggahnya ke server untuk pemulihan (POST `/panel/api/server/importDB`, field formulir `db`).

Petunjuk di antarmuka:

- SQLite: «Klik untuk memilih dan mengunggah file `.db` dari perangkat Anda guna memulihkan database dari cadangan.»
- PostgreSQL: «Klik untuk memilih dan mengunggah file `.dump` guna memulihkan database PostgreSQL. Ini akan menggantikan semua data saat ini.»

Proses impor untuk SQLite (penting dipahami: bersifat atomik dengan rollback):

1. File yang diunggah diperiksa formatnya – harus merupakan database SQLite yang valid; jika tidak, dikembalikan error «Invalid db file format».
2. File disimpan ke `x-ui.db.temp` sementara dan melewati pemeriksaan integritas.
3. **Xray dihentikan** sebelum penggantian DB.
4. Database saat ini diubah namanya menjadi cadangan `x-ui.db.backup` (fallback).
5. File sementara dipindahkan ke posisi DB aktif, inisialisasi dan migrasi skema dijalankan, kemudian migrasi inbound.
6. **Jika langkah apa pun gagal** – rollback dilakukan: database lama dipulihkan dari `x-ui.db.backup`, dan Xray di-restart dengan data lama.
7. Jika berhasil, file fallback dihapus, dan **Xray di-restart otomatis** dengan data yang telah dipulihkan.

Pesan antarmuka berdasarkan hasil:

Hasil	Teks
Sukses	«Database berhasil diimpor»
Error impor	«Terjadi kesalahan saat mengimpor database»
Error membaca file	«Terjadi kesalahan saat membaca database»

Pemulihan sepenuhnya menggantikan data saat ini. Karena Xray berhenti sejenak selama proses berlangsung, koneksi klien yang ada akan terputus sementara selama impor.

File migrasi antar mesin (SQLite ⇔ PostgreSQL)

Terpisah dari backup biasa, terdapat fitur «**Unduh file migrasi**» (`Download Migration`, permintaan `GET /panel/api/server/getMigration`). Fitur ini menghasilkan file portabel untuk berpindah ke mesin DB lain:

Mesin saat ini	Yang diunduh	Nama file	Tujuan
SQLite	Dump SQL portabel (teks)	<code>x-ui.dump</code>	Mengisi PostgreSQL dengan data Anda
PostgreSQL	Database SQLite yang dibangun dari data PostgreSQL	<code>x-ui.db</code>	Memindahkan panel kembali ke SQLite

Petunjuk:

- Di SQLite: «Klik untuk mengunduh ekspor `.dump` portabel (teks SQL) dari database SQLite Anda.»
- Di PostgreSQL: «Klik untuk mengunduh database SQLite (`.db`) yang dibangun dari data PostgreSQL Anda dan siap untuk menjalankan panel di SQLite.»

Konversi `.db` ⇔ `.dump` untuk SQLite juga dapat dilakukan dari CLI dengan perintah `x-ui migrateDB [file]` (lihat bagian 16.7).

Backup melalui bot Telegram

Jika bot Telegram dikonfigurasi (lihat bagian tentang notifikasi), bot dapat mengirimkan cadangan langsung ke obrolan administrator. Backup melalui Telegram mencakup **dua file**: database itu sendiri (`x-ui.db`, atau `x-ui.dump` untuk PostgreSQL) dan konfigurasi Xray `config.json`. Pesan didahului baris « Waktu pencadangan: ...».

Ada dua cara mendapatkan backup di Telegram:

1. **Sesuai permintaan.** Tombol « **Backup DB**» di menu bot – bot segera mengirimkan file ke obrolan saat ini.
2. **Otomatis bersama laporan.** Di pengaturan bot terdapat sakelar «**Pencadangan database**» (Database Backup) dengan deskripsi «Kirim notifikasi dengan file cadangan database». Jika diaktifkan, pada setiap pengiriman laporan berkala, bot akan mengirimkan cadangan kepada semua administrator setelah laporan.

Periode pengiriman laporan diatur oleh jadwal cron bot (lihat bagian 16.6). Bot memberikan jeda antar file dan antar administrator agar tidak melampaui batas Telegram.

Backup melalui bot hanya dikirimkan jika bot berjalan; di PostgreSQL juga memerlukan ketersediaan `pg_dump` di server.

16.2. Melihat log

Panel memiliki dua penampil log yang independen, keduanya dapat dibuka dari tab «Log» di «Dashboard». Setiap jendela dapat diperbarui (ikon «refresh» di judul) dan mengunduh yang ditampilkan ke file `x-ui.log` (tombol dengan ikon unduhan).

Log panel (aplikasi / syslog)

Jendela log panel (`POST /panel/api/server/logs/{count}`). Kontrol:

Elemen	Nilai default	Deskripsi
Jumlah baris	20	Daftar dropdown: 20 / 50 / 100 / 500 / 1000
Level	Info	Level minimum: Debug / Info / Notice / Warning / Error
SysLog (kotak centang)	dinonaktifkan	Sumber log: dari buffer aplikasi atau jurnal sistem
Perbarui otomatis (kotak centang)	dinonaktifkan	Membaca ulang log setiap 5 detik (lihat di bawah)

Perilaku bergantung pada kotak centang **SysLog**:

- **Dinonaktifkan (default):** log diambil dari buffer ring internal panel, difilter berdasarkan level yang dipilih. Entri ditampilkan dengan level (DEBUG / INFO / NOTICE / WARNING / ERROR) dan sumber: `X-UI` – pesan panel itu sendiri, `XRAY` – pesan Xray yang diteruskan.

Notifikasi sederhana tanpa cap waktu dan level (misalnya pesan sistem «Syslog is not supported» di Windows) kini ditampilkan sepenuhnya apa adanya. Format `YYYY/MM/DD LEVEL – isi` dikenali secara ketat; semua yang lain ditampilkan tanpa penguraian, sehingga baris tersebut tidak lagi terpotong (sebelumnya tiga kata pertama keliru dianggap sebagai tanggal/waktu/level).

- **Diaktifkan:** panel menjalankan `journalctl -u x-ui --no-pager -n <count> -p <level>` di server, yaitu menampilkan jurnal sistem layanan `x-ui`. Jumlah baris yang diizinkan adalah 1 hingga 10000; level menerima nilai syslog (`emerg/0` , `alert/1` , `crit/2` , `err/3` , `warning/4` , `notice/5` , `info/6` , `debug/7`). Di Windows mode SysLog tidak didukung – akan muncul peringatan untuk menghapus centang dan menggunakan log aplikasi. Jika `systemd` /layanan tidak tersedia, akan muncul pesan error saat menjalankan `journalctl`.

Contoh: jurnal yang sama dari konsol server. Ketika panel tidak dapat diakses (misalnya tidak dapat dimulai), jurnal sistem dapat dibaca langsung – ini persis perintah yang dijalankan panel dalam mode SysLog:

```
# 100 baris terakhir level warning ke atas
journalctl -u x-ui --no-pager -n 100 -p warning

# memantau jurnal secara real-time
journalctl -u x-ui -f
```

Level di jendela ini memfilter **keluaran**. Level minimum yang sebenarnya ditulis ke konsol/syslog ditentukan oleh level logging panel (variabel lingkungan, default Info ; ke file, panel selalu menulis di level DEBUG).

Log akses Xray (jurnal akses)

Jendela terpisah untuk access-log Xray (POST /panel/api/server/xraylogs/{count}). Jendela ini mengurai baris jurnal akses Xray dan menampilkannya dalam tabel: **Date, From, To, Inbound, Outbound, Email**.

Mulai dari 3.4.1, jendela ini dan tombol pemanggilnya di kartu status Xray diberi label «**Log Akses**» (Access Logs) – sebelumnya cukup disebut «Log». Penamaan ulang dilakukan agar penampil access-log Xray tidak membingungkan dengan penampil log panel itu sendiri, yang sebelumnya memiliki nama yang sama.

Elemen	Nilai default	Deskripsi
Jumlah baris	20	20 / 50 / 100 / 500 / 1000
Filter	kosong	Pencarian teks berdasarkan substring (diterapkan dengan menekan Enter)
Perbarui otomatis (kotak centang)	dinonaktifkan	Membaca ulang log setiap 5 detik (lihat di bawah)
Direct (kotak centang)	diaktifkan	Tampilkan koneksi langsung (traffic melalui freedom-outbound)
Blocked (kotak centang)	diaktifkan	Tampilkan koneksi yang diblokir (traffic ke blackhole-outbound)
Proxy (kotak centang)	diaktifkan	Tampilkan traffic yang diprosikan

Jenis kejadian ditentukan secara otomatis berdasarkan tag koneksi keluar di baris log: kecocokan dengan tag freedom → «DIRECT» (hijau), blackhole → «BLOCKED» (merah), yang lainnya → «PROXY» (biru). Baris api → api dan baris kosong dilewati.

Perbarui otomatis. Di kedua jendela log (baik «Log» maupun «Log Akses») terdapat kotak centang «**Perbarui otomatis**» (Auto Update). Jika diaktifkan, konten log dibaca ulang secara otomatis setiap 5 detik dengan mempertahankan semua pengaturan jendela saat ini – jumlah baris yang dipilih, level/filter, dan kotak centang Direct / Blocked / Proxy. Polling berhenti saat jendela ditutup atau kotak centang dinonaktifkan.

Agar jendela ini menampilkan entri, Xray harus mengaktifkan **jurnal akses** dengan jalur ke file (bukan none) – lihat di bawah. Jika access-log dinonaktifkan atau file tidak dapat diakses, jendela akan kosong («No Record...»).

16.3. Level dan pengaturan logging Xray

Parameter logging Xray itu sendiri diatur di halaman «**Konfigurasi Xray**» di blok «**Log**» (Log) dengan peringatan:

«Log dapat memperlambat server. Aktifkan hanya jenis log yang Anda butuhkan bila diperlukan!»

Field	Terjemahan	Nilai default	Deskripsi
Level log (<code>logLevel</code>)	Log Level	<code>warning</code>	Level detail log error Xray. Nilai yang diizinkan: <code>debug</code> , <code>info</code> , <code>notice</code> , <code>warning</code> , <code>error</code> . Petunjuk: «Level log untuk log error, menunjukkan informasi yang perlu dicatat.»
Log akses (<code>accessLog</code>)	Access Log	<code>none</code>	Jalur ke file jurnal akses. Nilai khusus <code>none</code> menonaktifkan log akses. Petunjuk: «Jalur ke file log akses. Nilai khusus „none” menonaktifkan log akses.»
Log error (<code>errorLog</code>)	Error Log	kosong (jalur default)	Jalur ke file log error; <code>none</code> menonaktifkannya. Petunjuk: «Jalur ke file log error. Nilai khusus „none” menonaktifkan log error.»
Log DNS (<code>dnsLog</code>)	DNS Log	<code>false</code> (nonaktif)	Aktifkan logging permintaan DNS. Petunjuk: «Aktifkan log permintaan DNS.»
Masking alamat (<code>maskAddress</code>)	Mask Address	kosong (nonaktif)	Saat diaktifkan, alamat IP nyata secara otomatis diganti dengan alamat masking di log. Petunjuk: «Saat diaktifkan, alamat IP nyata diganti dengan alamat masking di log.»

Karena secara default «**Log akses**» = `none` , jendela «Log Xray» (bagian 16.2) awalnya kosong. Agar berfungsi, tentukan jalur ke access-log di sini dan restart Xray.

Perlu diperhatikan: access-log kosong hanya memengaruhi jendela ini. Daftar klien online di «Dashboard» dan batas jumlah IP di formulir klien **tidak bergantung** pada access-log – panel menentukan klien online dan menghitung alamat IP mereka melalui online-stats API inti Xray (statistik koneksi). Pada versi inti yang lebih lama yang tidak memiliki API ini, panel secara otomatis kembali ke metode lama (membaca access-log), dan dalam hal itu jalur ke access-log di sini tetap diperlukan untuk batas IP.

Batas jumlah IP dan fail2ban. Pembatasan jumlah IP klien (field «IP Limit» di formulir klien dan saat penambahan massal) diterapkan di server hanya jika **fail2ban** terinstal – fail2ban-lah yang memblokir alamat yang melampaui batas. Panel memeriksa keberadaan fail2ban (`GET /panel/api/server/fail2banStatus`); jika tidak ada, field «IP Limit» menjadi tidak aktif dengan petunjuk penjelasan (di Windows – dengan pesan terpisah), dan batas yang telah ditetapkan sebelumnya di server tersebut secara otomatis direset karena memang tidak berlaku. Pemblokiran fail2ban berlaku untuk TCP maupun UDP. Di server biasa, fail2ban kini diinstal otomatis saat instalasi dan pembaruan panel (lihat bagian 16.5).

Contoh: blok `log` yang membuat jendela «Log Xray» mulai menampilkan entri. Dalam konfigurasi JSON Xray, tampilannya seperti ini:

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

Yang utama adalah mengganti `"access": "none"` dengan jalur ke file (misalnya `"./access.log"`). Setelah menyimpan, restart Xray, dan tabel di jendela «Log Xray» akan terisi baris.

16.4. Mengelola Xray: menghentikan dan me-restart

Status Xray dikelola dari kartu Xray di «Dashboard». Status saat ini ditampilkan sebagai salah satu nilai:

Berjalan (`Running`), **Berhenti** (`Stopped`), **Tidak Diketahui** (`Unknown`), **Error** (`Error`). Saat error, tersedia tooltip «Kesalahan saat menjalankan Xray».

Tombol	Terjemahan	Endpoint	Tindakan
Stop	Stop	POST /panel/api/server/stopXrayService	Menghentikan proses Xray. Jika berhasil – notifikasi peringatan «Xray service has been stopped».
Restart	Restart	POST /panel/api/server/restartXrayService	Me-restart (atau menjalankan) Xray dengan konfigurasi saat ini. Jika berhasil – notifikasi «Xray service has been restarted successfully».

Setelah operasi apa pun, panel menyiarkan status baru melalui WebSocket, sehingga status di «Dashboard» diperbarui tanpa memuat ulang halaman. Jika operasi gagal, status Xray menjadi «Error», dan teks error masuk ke notifikasi.

Selain restart manual, panel sendiri memeriksa apakah Xray perlu di-restart (tugas latar belakang setiap 30 detik) dan apakah proses jatuh (pemeriksaan setiap detik) – lihat bagian 16.6.

Monitor kesehatan tunnel (auto-restart Xray)

Di versi 3.4.1 diperkenalkan **monitor kesehatan tunnel** yang bersifat opsional. Jika diaktifkan, panel secara berkala memeriksa ketersediaan URL yang ditentukan dan setelah beberapa kali pemeriksaan gagal berturut-turut secara otomatis me-restart inti Xray – ini membantu memulihkan tunnel yang berhenti mengalirkan traffic. Secara default, monitor **dinonaktifkan** dan dikonfigurasi **hanya melalui variabel lingkungan layanan** (tidak ada pengaturannya di antarmuka web – ini memang disengaja oleh para pengembang).

Monitor diaktifkan oleh variabel `XUI_TUNNEL_HEALTH_MONITOR=true`. Variabel `XUI_TUNNEL_HEALTH_PROXY` harus diarahkan ke xray-inbound lokal (misalnya `socks5://127.0.0.1:1080`) – maka probe melewati Xray

itu sendiri dan benar-benar memeriksa tunnel; tanpanya, hanya konektivitas host yang diperiksa, dan restart tidak akan memperbaiki masalah koneksi jaringan server. Variabel lainnya menentukan parameter pemeriksaan:

Variabel	Fungsi	Default
<code>XUI_TUNNEL_HEALTH_MONITOR</code>	Aktifkan monitor (aktif/nonaktif)	<code>false</code>
<code>XUI_TUNNEL_HEALTH_PROXY</code>	Proxy yang digunakan untuk probe (tentukan xray-inbound lokal)	kosong
<code>XUI_TUNNEL_HEALTH_URL</code>	URL yang diperiksa	<code>https://www.cloudflare.com/cdn-cgi/trace</code>
<code>XUI_TUNNEL_HEALTH_INTERVAL</code>	Interval antar pemeriksaan	<code>30s</code>
<code>XUI_TUNNEL_HEALTH_TIMEOUT</code>	Timeout satu pemeriksaan	<code>10s</code>
<code>XUI_TUNNEL_HEALTH_FAILURES</code>	Jumlah kegagalan berturut-turut sebelum restart	<code>3</code>
<code>XUI_TUNNEL_HEALTH_COOLDOWN</code>	Jeda minimum antar restart	<code>5m</code>

Restart Xray memutuskan koneksi semua klien yang terhubung, sehingga sebaiknya biarkan interval dan ambang jumlah kegagalan cukup besar agar kegagalan probe acak tidak menyebabkan restart yang tidak perlu.

16.5. Me-restart dan memperbarui panel

Me-restart panel

Di halaman «**Pengaturan Panel**» terdapat tindakan «**Restart Panel**» (`Restart Panel` , `POST /panel/api/setting/restartPanel`). Setelah konfirmasi, panel di-restart **dalam 3 detik**.

Pesan:

- Konfirmasi: «Apakah Anda yakin ingin me-restart panel? Konfirmasikan, dan restart akan terjadi dalam 3 detik. Jika panel tidak dapat diakses, periksa log server.»
- Sukses: «Panel berhasil di-restart».

Secara teknis di Linux, restart dilakukan dengan mengirimkan sinyal `SIGHUP` ke proses panel (atau melalui hook yang terdaftar). Di Windows, pengiriman `SIGHUP` tidak didukung.

Pembaruan mandiri panel (Update Panel)

Di «**Dashboard**» tersedia fungsi «**Perbarui Panel**» (`Update Panel`) – memperbarui 3X-UI ke rilis terbaru langsung dari antarmuka web.

Sebelum pembaruan, panel membandingkan versi (`GET /panel/api/server/getPanelUpdateInfo`), meminta rilis terbaru 3x-ui dari GitHub:

Field	Terjemahan
Versi panel saat ini	Current panel version
Versi panel terbaru	Latest panel version
Panel sudah terbaru / «Terbaru»	Panel is up to date / Up to date – ditampilkan jika tidak ada versi baru

Memulai pembaruan – `POST /panel/api/server/updatePanel` . Dialog konfirmasi:

- «Apakah Anda benar-benar ingin memperbarui panel?»
- «Ini akan memperbarui 3X-UI ke versi #version# dan me-restart layanan panel.»

Setelah dimulai – pesan pop-up «Pembaruan panel dimulai» (`Panel update started`); jika pemeriksaan versi gagal – «Pemeriksaan pembaruan panel gagal» (`Panel update check failed`).

Yang terjadi di server: pembaruan mandiri hanya didukung **di Linux** (di OS lain akan dikembalikan error «panel web update is supported only on Linux installations»). Panel mengunduh skrip resmi `update.sh` dari GitHub (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`) dan menjalankannya dalam proses terpisah: preferensikan melalui `systemd-run` dalam unit terpisah (`x-ui-web-update-<timestamp>`), dan jika `systemd` tidak tersedia – sebagai proses terpisah yang dilepas. Setelah selesai, skrip memperbarui komponen dan me-restart layanan panel. Memerlukan `bash` untuk dijalankan.

Jika selama pembaruan skrip menghasilkan base path web panel baru yang acak (Web Base Path), layanan `x-ui` di-restart otomatis agar path baru langsung berfungsi. (Tanpa restart, server akan terus menyajikan path lama, sedangkan antarmuka menampilkan yang baru, dan alamat baru tidak dapat diakses sampai restart manual.)

Saluran Dev (rolling), mulai 3.4.2. Tombol versi panel di «Dashboard» kini selalu membuka jendela pembaruan (sebelumnya, pada build stabil terkini, tombol ini mengarah ke halaman rilis GitHub), dan sakelar «**Saluran Dev**» (`devChannelEnable`) selalu ditampilkan – bahkan pada build stabil. Dengan mengaktifkannya, pada pembaruan berikutnya Anda dapat beralih ke saluran dev «yang menggeling» (build per setiap commit ke `main`). Saat diaktifkan, ditampilkan peringatan: «Build dev mengikuti setiap commit ke main dan bukan merupakan rilis stabil – tidak ada rollback otomatis». Tanpa tindakan eksplisit dari pengguna, tidak ada yang diperbarui.

Saluran pembaruan Dev (rolling build per commit)

Selain pembaruan biasa ke rilis stabil, terdapat «**Saluran Pengembang**» (`Dev`) yang bersifat opsional. Sakelar muncul di jendela pembaruan panel **hanya pada build dev** (build CI yang dikompilasi dari commit terpisah); pada rilis stabil tidak terlihat. Jika diaktifkan, panel akan diperbarui ke rolling build `dev-latest` yang mengikuti setiap commit cabang `main` dan bukan merupakan rilis stabil – ditampilkan peringatan bahwa build dev tidak stabil dan tidak ada rollback otomatis. Dalam mode dev, jendela menampilkan «Commit saat ini» / «Commit terbaru» alih-alih nomor versi. Fitur ini hanya tersedia di Linux dengan `systemd`.

Pada build dev, panel menampilkan versinya sebagai `dev+<commit-pendek>` alih-alih nomor rilis stabil yang menyesatkan – di badge panel samping, kartu «Dashboard», jendela pembaruan, laporan status bot Telegram, dan keluaran perintah `x-ui -v` . Pada rilis stabil, tampilan versi tidak berubah.

Pada node (node), panel 3x-ui yang sama diperbarui secara terpusat melalui `POST /panel/api/nodes/updatePanel` — lihat bagian tentang node.

Instalasi fail2ban otomatis

Agar batas jumlah IP klien (bagian 16.3) berfungsi langsung, saat instalasi dan pembaruan panel di server biasa, `fail2ban` kini diinstal dan dikonfigurasi secara otomatis (sebelumnya ini hanya terjadi di image Docker). Perilaku dikontrol oleh variabel lingkungan `XUI_ENABLE_FAIL2BAN` : pengaturan dilakukan jika variabel tidak ditentukan atau bernilai `true` . Pemanggilan manual tersedia dengan perintah `x-ui setup-fail2ban` . Kegagalan konfigurasi fail2ban tidak menghentikan instalasi atau pembaruan panel.

Instalasi dan pembaruan di host IPv6-only

Skrip `install.sh` dan `update.sh` kini berfungsi dengan benar di server yang hanya memiliki IPv6: pengunduhan rilis, skrip `x-ui.sh` , dan file layanan tidak lagi memaksa IPv4 (`curl -4`), melainkan menggunakan protokol yang tersedia. Oleh karena itu, panel dapat diinstal dan diperbarui di host tanpa alamat IPv4.

Mengganti port panel dengan variabel `XUI_PORT`

Port listen panel web dapat diganti dengan variabel lingkungan `XUI_PORT` — variabel ini berlaku hanya selama proses saat ini berjalan dan **tidak mengubah** nilai `webPort` yang tersimpan di database. Nilai yang diizinkan adalah 1 hingga 65535 ; nilai kosong, salah, atau di luar rentang diabaikan (digunakan `webPort`) dengan peringatan di log. Ini berguna saat deployment, terutama di Docker: saat menggunakan bridge network, port container yang dipublikasikan harus sesuai dengan `XUI_PORT` — misalnya, `XUI_PORT=8080` dan `ports: "8080:8080"` .

Memperbarui dan mengganti versi Xray-core

Di «Dashboard» yang sama, Anda dapat mengelola versi Xray-core secara terpisah dari panel.

- **Pembaruan Xray** (`Xray Updates`) / **Pilih versi** (`Version`) — daftar dropdown versi yang tersedia. Petunjuk: «Pilih versi yang diperlukan» dan peringatan «Penting: versi lama mungkin tidak mendukung pengaturan saat ini».
- Instalasi/penggantian versi — `POST /panel/api/server/installXray/{version}` . Dialog: «Ganti versi Xray» / «Apakah Anda benar-benar ingin mengganti versi Xray?». Jika berhasil — «Xray berhasil diperbarui».

Contoh: mengganti versi Xray-core melalui permintaan API. Versi ditentukan dengan tag rilis dari XTLS/Xray-core (dengan awalan `v`). Misalnya, beralih ke `v1.8.24` :

```
curl -s -b cookies.txt -X POST \
  https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

(`cookies.txt` — file cookie dari contoh di bagian 16.1.) Setelah instalasi, Xray akan di-restart otomatis dengan versi yang dipilih.

Di server saat mengganti versi, Xray pertama-tama dihentikan, arsip versi yang diperlukan diunduh dari GitHub (XTLS/Xray-core), biner diekstrak dan diganti, kemudian Xray di-restart dengan verifikasi ukuran checksum arsip/biner.

16.6. Tugas berkala (cron)

Panel mendaftarkan sejumlah tugas latar belakang saat startup. Jadwalnya tetap (tidak dapat dikonfigurasi di UI, kecuali jadwal laporan Telegram dan sinkronisasi LDAP). Berikut tugas-tugas yang berkaitan dengan operasional.

Tugas	Jadwal	Tujuan
Pemeriksaan operasi Xray	setiap 1 detik	Memantau bahwa proses Xray berjalan
Pemeriksaan kebutuhan restart Xray	setiap 30 detik	Restart jika konfigurasi ditandai sebagai telah diubah
Pengumpulan traffic Xray	setiap 5 detik (mulai 5 detik setelah startup)	Pencatatan traffic inbound/klien
Pemeriksaan IP klien	setiap 10 detik	Kontrol batas IP berdasarkan log
Heartbeat dan sinkronisasi traffic node	setiap 5 detik	Komunikasi dengan node
Pembersihan log	harian (@daily)	Membersihkan log batas IP dan persistent access-log, merotasi log saat ini ke *.prev.log
Reset traffic berdasarkan periode	@hourly , @daily , @weekly , @monthly	Mereset penghitung traffic inbound (dan kliennya) yang memiliki periode reset otomatis yang sesuai
Laporan Telegram	diatur di pengaturan bot (default @daily)	Mengirim laporan ke administrator; jika opsi diaktifkan – dengan cadangan DB terlampir (bagian 16.1)
Reset penyimpanan hash Telegram	setiap 2 menit	Hanya jika bot diaktifkan
Kontrol beban CPU untuk Telegram	setiap 10 detik	Hanya jika ambang CPU > 0 ditentukan

Tambahan:

- **Reset traffic berkala** hanya berlaku untuk inbound yang memiliki mode reset otomatis yang dipilih (per jam/harian/mingguan/bulanan). Tugas mereset traffic inbound itu sendiri dan semua kliennya.
- **Pemeriksaan kedaluwarsa dan habis batas.** Penonaktifan klien saat kedaluwarsa dan saat batas traffic habis dilakukan dalam kerangka pencatatan traffic: klien dengan expiry_time yang telah lewat atau volume yang habis ditandai dan dinonaktifkan, jika diperlukan tenggat berikutnya dihitung (untuk batas siklus dan mode «hitung mundur dari penggunaan pertama»). Di «Dashboard» dan daftar, ini tercermin dalam status «Kedaluwarsa»/«Habis»/«Segera Berakhir».
- **Backup otomatis ke Telegram** adalah efek samping dari tugas laporan, tidak ada jadwal cron terpisah hanya untuk backup. Oleh karena itu, frekuensi backup otomatis sama dengan frekuensi laporan bot.

16.7. Menu konsol dan CLI (`x-ui`)

Di server, panel dikelola dengan perintah `x-ui` . Tanpa argumen, menu interaktif «3X-UI Panel Management Script» dibuka; dengan argumen, subperintah tertentu dijalankan. Poin menu yang berkaitan dengan operasional:

No. di menu	Poin	Tindakan
1	Install	Instalasi panel (mengunduh dan menjalankan <code>install.sh</code>)
2	Update	Memperbarui semua komponen x-ui ke versi terbaru tanpa kehilangan data; setelah itu – restart otomatis
3	Update to Dev Channel (latest commit)	Memperbarui ke rolling build <code>dev-latest</code> (commit terbaru cabang <code>main</code>) dengan konfirmasi (lihat 16.5)
4	Update Menu	Memperbarui hanya skrip menu <code>x-ui</code> itu sendiri
5	Legacy Version	Menginstal versi panel tertentu (lama) berdasarkan nomor yang dimasukkan (misalnya <code>2.4.0</code>)
6	Uninstall	Penghapusan lengkap panel dan Xray (lihat 16.8)
7	Reset Username & Password	Reset login/password administrator
8	Reset Web Base Path	Reset base path panel web
9	Reset Settings	Reset pengaturan ke nilai default
10	Change Port	Mengubah port panel
11	View Current Settings	Melihat pengaturan saat ini
12–14	Start / Stop / Restart	Memulai, menghentikan, me-restart layanan panel
15	Restart Xray	Me-restart hanya Xray
16	Check Status	Status layanan saat ini
17	Logs Management	Melihat dan membersihkan log (lihat di bawah)
18–19	Enable / Disable Autostart	Mengaktifkan/menonaktifkan autostart layanan saat OS dimulai
27	Update Geo Files	Memperbarui file geo (GeoIP/GeoSite)
25	PostgreSQL Management	Manajemen PostgreSQL

Penomoran poin menu berubah di versi 3.4.1: karena penambahan poin 3 «Update to Dev Channel», semua poin berikutnya bergeser satu angka. Total poin menjadi 28, pilihan dimasukkan dalam rentang `[0–28]` .

Manajemen log di CLI (poin 16)

Submenu «Logs Management» kini dibuka di poin **17** (sebelumnya – 16):

- **Debug Log** – tampilan streaming jurnal layanan: `journalctl -u x-ui -e --no-pager -f -p debug` (di Alpine – `grep` terhadap `/var/log/messages`).
- **Clear All logs** – pembersihan jurnal sistem: `journalctl --rotate + journalctl --vacuum-time=1s` , setelah itu layanan di-restart. (Tidak tersedia di Alpine.)

Subperintah langsung `x-ui`

Semua subperintah yang tersedia:

Perintah	Deskripsi
<code>x-ui</code>	Membuka menu administrasi
<code>x-ui start</code>	Menjalankan panel
<code>x-ui stop</code>	Menghentikan panel
<code>x-ui restart</code>	Me-restart panel
<code>x-ui restart-xray</code>	Me-restart Xray
<code>x-ui status</code>	Status saat ini
<code>x-ui settings</code>	Menampilkan pengaturan saat ini
<code>x-ui enable</code>	Mengaktifkan autostart saat OS dimulai
<code>x-ui disable</code>	Menonaktifkan autostart
<code>x-ui log</code>	Melihat log
<code>x-ui banlog</code>	Melihat log blokir Fail2ban
<code>x-ui setup-fail2ban</code>	Menginstal dan mengonfigurasi fail2ban untuk batas IP (lihat 16.5)
<code>x-ui update</code>	Memperbarui panel

| `x-ui update-dev` | Memperbarui panel ke saluran pengembang (rolling build `dev-latest`) || `x-ui update-all-geofiles` | Memperbarui semua file geo (dengan restart berikutnya) || `x-ui migrateDB [file]` | Konversi database `.db` ➔ `.dump` (SQLite) || `x-ui legacy` | Menginstal versi usang || `x-ui install` | Menginstal panel || `x-ui uninstall` | Menghapus panel |

Perintah `x-ui update` mengunduh dan menjalankan `update.sh` resmi (sama seperti pembaruan web dari bagian 16.5), dengan meminta konfirmasi: «This function will update all x-ui components to the latest version, and the data will not be lost.» Setelah selesai, panel di-restart otomatis.

Flag `-webCert` / `-webCertKey` dalam subperintah `setting`. Jalur ke sertifikat dan kunci privat panel web dapat ditentukan langsung dalam subperintah `x-ui setting -webCert <jalur> -webCertKey`

`<jalur>` – menentukan salah satu dari flag ini akan menyimpan jalur yang sesuai (seperti halnya subperintah `cert` terpisah), dan panel langsung beralih ke HTTPS.

Mendapatkan token API melalui CLI

Perintah mendapatkan token API melalui CLI (poin menu/perintah `x-ui`) tidak menampilkan token yang sebelumnya dikeluarkan. Token API hanya disimpan dalam bentuk hash, sehingga token yang ada tidak dapat diperoleh dalam bentuk terbuka. Jika token sudah dikonfigurasi, perintah menginformasikan jumlahnya, menyarankan untuk mengelola token di panel (**Settings** → **API Tokens**, lihat bagian tentang token API), dan segera menghasilkan **token cadangan baru** dengan nama seperti `cli-fallback-<timestamp>` dan menampilkannya agar CLI tetap berguna tanpa harus masuk ke antarmuka.

16.8. Menghapus panel

Penghapusan dilakukan dari CLI – poin menu **5 (Uninstall)** atau perintah `x-ui uninstall`. Sebelum penghapusan, konfirmasi diminta (default «tidak»): «Are you sure you want to uninstall the panel? xray will also be uninstalled!».

Setelah dikonfirmasi, skrip:

1. Menghentikan layanan dan menonaktifkan autostartnya (`systemctl stop/disable x-ui`, atau di Alpine – `rc-service / rc-update`), menghapus file unit layanan dan memuat ulang konfigurasi systemd.
2. Menghapus direktori data dan aplikasi (`/etc/x-ui/`, direktori instalasi) dan file env layanan (`/etc/default/x-ui`, `/etc/conf.d/x-ui` atau `/etc/sysconfig/x-ui` – tergantung distribusi).
3. Menghapus skrip `x-ui` itu sendiri dan menampilkan pesan «Uninstalled Successfully.», serta perintah untuk instalasi ulang.

Jika panel menggunakan PostgreSQL (dalam file env `XUI_DB_TYPE=postgres`), setelah menghapus file panel, skrip juga menanyakan apakah server PostgreSQL beserta semua database-nya juga perlu dihapus: «Also purge PostgreSQL and delete all of its data?». Permintaan ini memerlukan konfirmasi eksplisit (default – menolak) dan disertai peringatan: penghapusan akan memengaruhi **SEMUA** database PostgreSQL di mesin, termasuk yang dimiliki aplikasi lain, dan tidak dapat dibatalkan. Jika ditolak, PostgreSQL dan datanya tetap tidak tersentuh.

Penghapusan tidak dapat dibatalkan: bersama panel, Xray dan semua data (termasuk database) juga dihapus. Jika data mungkin diperlukan, lakukan ekspor database terlebih dahulu (bagian 16.1).

16.9. Perintah `x-ui migrateDB`

Mulai versi 3.3.0, skrip pengelola `x-ui.sh` mendapatkan subperintah `migrateDB` – pembungkus di sekitar biner bawaan `x-ui` (`x-ui migrate-db`) untuk mengonversi database panel SQLite antara dua format: biner `.db` dan dump teks portabel `.dump` (teks SQL biasa).

Apa yang dilakukan perintah ini

Perintah bekerja dalam dua arah, dan arahnya ditentukan **secara otomatis** berdasarkan file input:

Arah	Nama	Yang terjadi
<code>.db → .dump</code>	dump (ekspor)	database SQLite biner diekspor ke file SQL teks
<code>.dump → .db</code>	restore (pemulihan)	database SQLite biner dibangun ulang dari file SQL teks

Di balik layar, skrip memanggil biner panel:

- ekspor: `x-ui migrate-db --src <input> --dump <output>`
- pemulihan: `x-ui migrate-db --restore <input> --out <output>`

Sintaks pemanggilan

```
x-ui migrateDB [file.db|file.dump] [output]
```

- **[file.db|file.dump]** – file input (argumen pertama). Jika tidak ditentukan, database panel yang terinstal secara default digunakan: `/etc/x-ui/x-ui.db`.
- **[output]** – jalur ke file output (argumen kedua). Opsional: jika tidak ada, nama dipilih secara otomatis di sebelah file input (lihat di bawah).

Contoh:

```
x-ui migrateDB                               # ekspor /etc/x-ui/x-ui.db -> /etc/x-ui/x-
ui.dump
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db       # bangun .db dari dump
```

Bagaimana arah ditentukan

Skrip melihat ekstensi file input:

- `*.db`, `*.sqlite`, `*.sqlite3` → mode **dump** (ekspor ke teks);
- `*.dump`, `*.sql` → mode **restore** (membangun database).

Jika ekstensi tidak dikenali, skrip membaca 16 byte pertama file: tanda tangan `SQLite format 3` berarti database biner (mode dump), jika tidak file dianggap sebagai dump (mode restore).

Nama file output, jika argumen kedua tidak ditentukan:

- saat ekspor – nama yang sama dengan input, dengan ekstensi `.dump`;
- saat pemulihan – nama yang sama dengan ekstensi `.db`.

Pemeriksaan perlindungan dan perilaku

- **Keberadaan biner.** Jika biner `x-ui` tidak ditemukan atau tidak dapat dieksekusi – ditampilkan error «x-ui binary not found ... Is the panel installed?».
- **Dukungan fitur dalam build.** Skrip memverifikasi bahwa biner mendukung `migrate-db --dump/--restore` (melalui `x-ui migrate-db -h`). Jika tidak – disarankan untuk memperbarui panel terlebih dahulu dengan perintah `x-ui update`.
- **Keberadaan file input.** Jika file input tidak ada, error dicetak bersama baris sintaks pemanggilan.
- **Penimpaan output.** Jika file output sudah ada, konfirmasi diminta (default «tidak»); tanpa konfirmasi, operasi dibatalkan. Saat pemulihan, file output lama dihapus terlebih dahulu.
- **Perlindungan database aktif.** Saat pemulihan ke database default `/etc/x-ui/x-ui.db`, ketika panel sedang berjalan, operasi ditolak dengan permintaan untuk menghentikan panel terlebih dahulu (`x-ui stop`) atau memilih jalur output lain. Ini mencegah penimpaan database aktif dari layanan yang sedang berjalan.
- Jika pembuatan database gagal, file output yang tidak lengkap dihapus.

Untuk apa ini diperlukan

- **Backup.** File `.dump` teks dapat dibaca manusia, nyaman untuk disimpan dalam sistem kontrol versi dan untuk melihat konten database secara diferensial.
- **Migrasi.** Dump portabel antar mesin dan tahan terhadap perbedaan versi format file SQLite – di server baru, `.db` yang berfungsi dibangun dari dump.
- **Diagnostik.** Dari `.dump`, Anda dapat melihat struktur dan data panel secara visual tanpa memiliki alat SQLite.

Mode interaktif

Selain pemanggilan langsung, konversi juga tersedia dari menu interaktif. Di submenu PostgreSQL (`x-ui` → bagian manajemen PostgreSQL) terdapat poin **9. Convert SQLite .db <-> .dump**: ia menanyakan jalur ke file input (default `/etc/x-ui/x-ui.db`) dan ke file output (dapat dibiarkan kosong untuk penamaan otomatis), sementara arahnya, seperti dalam mode CLI, ditentukan secara otomatis.

Dokumen disiapkan berdasarkan kode sumber 3X-UI. Jika ada poin antarmuka yang berbeda di versi Anda – prioritas diberikan pada perilaku panel dan petunjuk di UI itu sendiri.